DESIGN AND SIMULATION OF A SIMPLIFIED PCI EXPRESS GEN 6 DIGITAL MODEL

Hossam Fawzy Abdel-moniem
Arwa Hany Saeed
Youssif Mahmoud Shapana
Ahmed Mohamed Sanad
Ahmed Alaa Gaafar

Under Supervision of

DR. Omar Mahmoud Sabry

Sponsored by

A Thesis submitted to

Canadian International College

In Partial Fulfillment of

the Requirements for the Degree of Bachelor in

Electronics and Communication Engineering

CANADIAN INTERNATIONAL COLLEGE

GIZA, EGYPT

2026

Acknowledgment

First and foremost, we extend our deepest gratitude to Almighty Allah, whose blessings, guidance, and grace have enabled us to complete this project. Without His mercy and support, none of this work would have been possible.

We would also like to express our sincere appreciation to Dr. Omar Mahmoud Sabry, our supervisor, for his continuous guidance, valuable insights, and unwavering support throughout every stage of this research. His expertise and encouragement were essential in shaping the direction and quality of our work.

Our gratitude is also extended to the Canadian International College – Electronics and Communication Engineering Department for providing the academic environment, resources, and facilities that greatly contributed to the success of this project.

We are deeply thankful to our families for their endless patience, motivation, and support. Their encouragement gave us the strength to continue during the most challenging phases of this work.

We would also like to acknowledge the dedication and collaboration of our project team members. Each member contributed significantly through research, design, and documentation, and this project is a result of our collective effort.

Finally, we thank all researchers, authors, and organizations whose studies and publications enriched our understanding and formed the foundation of our literature review.

We are truly grateful to everyone who played a role in the completion of this project.

Table of Contents

1 Chapter One: Introduction 1

1.1 Background 2

1.2 Overview of PCI Express 2

1.2.1 PCIe TOPOLOGY 3

1.2.2 PCIe ARCHITECTURE 4

1.3 Evolution of PCIe Generations 5

1.4 Motivation for PCIe Gen 6 7

1.5 Key Enhancements and Architectural Changes 7

1.6 Problem Statement 8

1.7 Project Objectives 9

2 Chapter Two: Previous Studies 10

2.1 Studies About PCIe Architecture 11

2.2 Studies About PCIe Generational Development 14

2.3 Studies About FLIT Mode (Fixed-Length Packets) 22

2.4 Studies About PAM4 Modulation 25

2.5 Comparison Between NRZ and PAM4 in PCIe Generations 27

2.6 Studies About FEC + CRC in High-Speed Links 29

2.7 Previous PCIe Implementations 30

2.8 Verification Challenges in PCIe Gen6 32

2.9 Identified Research Gaps 33

3 Chapter Three: Proposed Methods 35

3.1 System Architecture Overview 36

3.2 Block Diagram of the Whole System 37

3.3 Decision Feedback Equalization (DFE) 41

3.3.1 Input Signal from the Channel (ISI) 42

3.3.2 Feed-Forward Equalizer (FFE) 42

3.3.3 Decision Device (Slicer) 43

3.4 design and Simulation Plan 44

3.4.1 Modeling Approach 45

3.4.2 Simulation Plan 45
3.4.3 Tools and Technologies 46
3.4.4 Performance Metrics 48
3.4.5 Limitations Digital Model 48
4 Chapter Four: Numerical Results 50
4.1 RTL Functional Simulation Results 51
4.1.1 Reset and Link Initialization Results 52
4.1.2 Transaction Layer Functional Results 52
4.1.3 Data Link Layer Functional Results 54
4.1.4 Physical Layer Functional Results 55
4.1.5 Multi-Lane Throughput Benchmark Results 56
4.2 RTL Verification Results 57
4.2.1 Transaction Layer Verification 59
4.2.2 Data Link Layer Verification 59
4.2.3 Physical Layer Verification 60
4.2.4 Functional Coverage Results 60
4.2.5 System Verilog Assertion Results 61
4.2.6 Final Verification Summary 61
4.3 Logical Synthesis Results 63
4.3.1 Area and cell statistics 63
4.3.2 Power Reports 65
4.3.3 Timing Reports 66
4.3.4 Design rule checks 67
4.4 Formal Verification Synthesis Results 68
4.4.1 Transaction Layer Results 68
4.4.2 Data Link Layer Results 71
4.5 MATLAB Theoretical Analysis Results 74
4.5.1 PAM4 Eye Diagram 75
4.5.2 BER vs. SNR Analysis 76
4.5.3 Jitter Analysis — Bathtub Curve 77
4.5.4 Channel and CTLE Frequency Response 79

5 Chapter Five: Transaction Layer 81
5.1 Transaction Layer Architecture 83
5.2 Transaction Layer Interface Signals 83
5.3 Transaction Layer Operational Flow 86
5.3.1 Request Admission and Flow Control Verification 87
5.3.2 Transaction Classification and Tag Allocation 87
5.3.3 TLP Construction and Header Processing 87
5.3.4 FLIT Packing and Data Aggregation 88
5.3.5 ECRC Generation and Deterministic Framing 88
5.3.6 Transmission to the Data Link Layer 88
5.4 Transaction Layer Responsibilities 88
5.5 TLP Structure Section 90
5.5.1 TLP Prefixes 90
5.5.2 TLP Header 90
5.5.3 End-to-End CRC (ECRC) 90
5.5.4 Data Payload 91
5.5.5 Key TLP Header Fields and Their RTL Impact 91
5.6 Flit Mode Architecture 93
5.6.1 256-Byte FLIT Structure 93
5.6.2 TLP Packing Mechanism 93
5.6.3 Orthogonal Header Content (OHC) 94
5.6.4 Relationship Between FLIT Mode and FEC 94
5.6.5 Architectural Tradeoff: Reliability vs. Overhead 95
5.7 PCIe Gen 6 Impact on Transaction Layer 96
5.8 Design Trade-offs and Architectural Decisions 98
5.8.1 Parallel CRC Implementation 99
5.8.2 Arbitration Fairness and Traffic Scheduling 99
5.8.3 FIFO Architecture Selection 100
5.8.4 FLIT Packing Efficiency 100
5.9 Transaction Layer and Data Link Layer Interconnection 101
5.9.1 Functional Interdependence 101

5.9.2 Flow Control and Credit Management 102
5.9.3 Hardware-Level Signal Interface (RTL Perspective) 102
5.9.4 Initialization Sequence 103
5.10 Challenges and Implementation Hurdles 104
6 Chapter Six: Data Link Layer 108
6.1 Data Link Layer Architecture 109
6.2 Data Link Layer Interface Signals 110
6.3 Data Link Layer Operational Flow 112
6.3.1 Request Admission and Flow Control Verification 112
6.3.2 Sequence Numbering and CRC Generation 113
6.3.3 Sequence Number Wrap-Around and Synchronization 114
6.3.4 NULL FLIT Insertion and Acknowledgement Piggybacking 115
6.3.5 Retry Buffering and Link-Level Reliability 115
6.3.6 DLLP Interleaving, Scrambling, and Physical Delivery 116
6.4 Data Link Layer Responsibilities 117
6.5 FLIT Mode Architecture in the Data Link Layer 118
6.5.1 Impact on DLL Sequence Numbering and CRC 119
6.5.2 FLIT Null Slot Management 119
6.6 Initialization: Establishing a Reliable Channel 120
6.6.1 DLL Initialization FSM (DLL_INIT) 120
6.6.2 Flow Control Initialization FSM (FC_INIT_FSM) 121
6.7 The Transmit Data Path: From Transaction Packet to Physical FLIT 122
6.7.1 Transaction Layer Interface (TL_IF) 122
6.7.2 CRC Generator (CRC_GEN) and Sequence Number Generator (SEQ_GEN) 122
6.7.3 NULL FLIT Inserter (NULL_INS) 123
6.7.4 Retry Buffer (RETRY_BUF) 123
6.7.5 Retry Buffer Sizing 124
6.7.6 TX Multiplexer (TX_MUX), Scrambler (SCRM), and Physical TX Interface (PHY_TX) 125
6.8 Flow Control: Sustaining Deadlock-Free Throughput 125

6.8.1 Flow Control Timer (FC_TMR) and Watchdog (FC_WDG) 126
6.8.2 ACK Latency Budget and the FC_TMR Timing Constraint 126
6.8.3 NOP Generator (NOP_GEN) 127
6.9 Acknowledgement, Sequencing, and the Replay Protocol 127
6.9.1 ACK Piggyback Controller (ACK_PGB) and ACK Timer (ACK_TMR) .. 127
6.9.2 Flit Sequencer (FLIT_SEQ) and Replay FSM (REPLAY_FSM) 127
6.10 The Receive Data Path: Validation and Delivery 128
6.10.1 Physical Reception and Descrambling (PHY_RX, DESCRM) 128
6.10.2 FLIT Classification and Demultiplexing (FLIT_RX, RX_DEMUX, NULL_HDL) 128
6.10.3 Error Detection: CRC, Sequence, and DLLP Integrity Checking 129
6.10.4 DLLP Processing and Receive-Side Credit Management 129
6.11 Power Management and Link Bandwidth Negotiation 130
6.11.1 Power Management FSM (PM_FSM) and Timer (PM_TMR) 130
6.11.2 Link Bandwidth Management FSM (LBW_FSM) 131
6.12 Centralized Error Management and Link 131
6.13 PCIe Gen 6 Impact on the Data Link Layer 132
6.14 Design Trade-offs and Architectural Decisions 133
6.15 Challenges and Implementation Hurdles 134
7 Chapter Seven: Physical Layer 135
7.1 Physical Layer Architecture 136
7.2 Physical Layer Interface Signals 138
7.3 Physical Layer Operational Flow 141
7.3.1 Link Detection and Physical Establishment 141
7.3.2 Polling: Bit Lock and Symbol Lock 141
7.3.3 Configuration: Link and Lane Number Negotiation 142
7.3.4 Data Transmission and Reception in L0 142
7.3.5 Recovery, Speed Change, and Error Handling 142
7.4 Physical Layer Responsibilities 143
7.5 LTSSM Controller Subsystem 144
7.5.1 LTSSM Top Controller (LTSSM_TOP) 145

7.5.2 Detect FSM (DETECT_FSM) and Receiver Detection Logic (RX_DET) ... 145
7.5.3 Polling FSM (POLL_FSM) ... 146
7.5.4 Configuration FSM (CFG_FSM) ... 147
7.5.5 Recovery FSM (RECV_FSM) ... 148
7.5.6 L0/L0s FSM (L0_FSM) and L1 FSM (L1_FSM) ... 149
7.5.7 Hot Reset and Disabled FSM (HRST_FSM) ... 150
7.5.8 Loopback FSM (LB_FSM) ... 151
7.6 The Transmit Data Path: From FLIT to Physical Signal ... 152
7.6.1 FLIT Framer TX (FLIT_TX) ... 152
7.6.2 FEC Encoder (FEC_ENC) ... 152
7.6.3 PAM4 Gray Code Encoder (PAM4_ENC) ... 153
7.6.4 TX Gear Box (TX_GEAR) and Ordered Set Generator (COMP_GEN) ... 153
7.6.5 TX Datapath MUX (PHY_TX_MUX) and PIPE TX Interface (PIPE_TX) 154
7.7 The Receive Data Path: From Physical Signal to FLIT ... 154
7.7.1 PIPE RX Interface (PIPE_RX), RX Elastic Buffer (RX_ELBUF), and RX Gear Box (RX_GEAR) ... 155
7.7.2 Lane Polarity (LANE_POL), Block Alignment (BLK_ALIGN), and Lane Deskew (DESKEW) ... 155
7.7.3 LOCK FSM (LOCK_FSM) ... 156
7.7.4 PAM4 Gray Code Decoder (PAM4_DEC) and FEC Syndrome Calculator (FEC_SYN) ... 157
7.7.5 FEC Decoder (FEC_DEC) ... 157
7.7.6 FLIT Defamer RX (FLIT_DFRM) ... 158
7.8 Gen6-Specific Physical Layer Features ... 158
7.8.1 FLIT Sync Header Generation and Checking (FLIT_SYNC) ... 158
7.8.2 Data Rate Advertisement Logic (DRATE_ADV) ... 159
7.9 Link Negotiation and Equalization Subsystem ... 159
7.9.1 Speed Negotiation (SPD_NEG) and Speed Change FSM (SPD_CHG) ... 159
7.9.2 Link Width Negotiation (WDT_NEG) ... 160
7.9.3 Link Equalization Controller (EQ_CTRL) ... 160
7.10 Ordered Set Generation and Detection ... 160

7.10.1 TS1 and TS2 Generators (TS1_GEN, TS2_GEN) and Detector (TS_DET) 161
7.10.2 FTS Generator and Detector (FTS) 161
7.10.3 EIOS and EIEOS Handler (EIOS) 161
7.10.4 SKP Ordered Set Generator and Receiver (SKP) 162
7.11 PIPE Interface, Lane Management, and Support Subsystems 162
7.11.1 PIPE Interface Controller (PIPE_CTRL) 162
7.11.2 Lane Reversal Logic (LANE_REV) 163
7.11.3 Beacon and Electrical Idle Logic (BEAC) 163
7.11.4 Spread Spectrum Clock Controller (SSC_CTRL) 163
7.11.5 Power State Timer (PWR_TMR) 164
7.11.6 Fundamental Reset FSM (FUND_RST) 164
7.11.7 Compliance Pattern Generator (COMPL_GEN) 164
7.12 PCIe Gen 6 Impact on the Physical Layer 165
7.13 Design Trade-offs and Architectural Decisions 166
7.14 Challenges and Implementation Hurdles 168
8 Chapter Eight: Conclusion And Future Work 170
8.1 Conclusion 171
8.2 Future Work 173
9 Code Appendix 176
10 References 183

List of Figures

Figure 1.1 The PCIe Topology [1] 3
Figure 1.2 The PCI Express Architecture [1] 4
Figure 1.3 Speed Benchmarks and Practical Implications. [25] 7
Figure 1.4 PCIe Gen 6.0 specification features. [25] 8
Figure 1.5 PCIe Gen 6.0 Objectives**.** 9
Figure 2.1 PCIe architecture overview [4] 12
Figure 2.2 PCIe architecture overview [5] 14
Figure 2.3 old PCI bus [6] 14
Figure 2.4 PCIe X1, X4, X8, X16 from different generations [7] 16
Figure 2.5 Packet Flow Through the Layers in GEN3 [8] 17
Figure 2.6 PCIe generation 4 by NVMe [9] 18
Figure 2.7 PCIe generation 5 from NVMe [11] 19
Figure 2.8 generational development of PCIe [13] 21
Figure 2.9 Flit Layout in a .x16 Link [16] 22
Figure 2.10 FLIT reservation flow [17] 24
Figure 2.11 The new modulation technique (PAM-4) simulation [18] 26
Figure 2.12 NRZ signal with one eye per UI, left, and PAM4 signal with three eyes, right. [19] 27
Figure 2.13 NRZ vs PAM4 [19] 28
Figure 2.14 Forward Error Correction Process in PCIe [24] 30
Figure 3.1 Simplified Block Diagram of PCIe Gen 6 37
Figure 3.2 First DW of Header Base [20] 38
Figure 3.3 Completion Header Base Format - Flit mode [20] 39
Figure 3.4 Overall DFE Block Diagram (Core Architecture) [21] 41
Figure 3.5 ADS IBIS simulation of a pulse response with and without equalization [22]. 42
Figure 3.6 FFE inserts pulses of negative amplitudes to cancel out ISI that has positive amplitude.[21] 42
Figure 3.7 Simulation results of the proposed PAM4 receiver. [23] 43
Figure 3.8 Design and Simulation Plan for the Proposed PCIe Gen6 Model 44
Figure 3.9 Schematic / Block Diagram Editor 46
Figure 3.10 Example from GitHub 47
Figure 4.1 Functional Coverage Summary (Questa Sim Report) 61
Figure 4.2 Final Coverage Groups (Questa Sim Report) 62
Figure 4.3 PAM4 Eye Diagram 75
Figure 4.4 BER vs SNR 76
Figure 4.5 Jitter Bathtub Curve 77
Figure 4.6 Channel + CTLE Frequency Response 79

Figure 5.1 Layering Diagram Highlighting the Transaction Layer 82
Figure 5.2 Transaction Layer Core Architecture 83
Figure 5.3 TLP Header PCI0ExpressBaseSpecifications 90
Figure 5.4 Generic TLP Format 91
Figure 5.5 bandwidth scaling comparison (gen5 VS gen6) 92
Figure 5.6 FLIT architecture 93
Figure 5.7 Orthogonal Header Content (OHC) 94
Figure 5.8 Fixed-Size FLIT Structure Supporting FEC 94
Figure 5.9 FLIT Architectural Tradeoff: Reliability vs. Overhead 95
Figure 5.10 Impact of PAM4 signaling on PCIe Gen6 Transaction Layer architecture 96
Figure 5.11 circuit required for round robin arbiter 99
Figure 5.12 Internal Architecture of the Synchronous FIFO Buffer with Binary Pointers 100
Figure 5.13 FLIT Utilization Efficiency Comparison Between Naive and Optimized Packing 101
Figure 5.14 Data Link Layer (DLL) State Machine For linking 103
Figure 6.1 Data Link Layer Core Architecture 110
Figure 6.2 Request And Flow control Behavior In TL Interface 113
Figure 6.3 Sequence Number Assignment and CRC Generation diagram 113
Figure 6.4 Sequence Number Synchronization and Wrap-Around Handling 114
Figure 6.5 NULL FLIT Insertion and Acknowledgement Piggybacking Mechanism 115
Figure 6.6 Retry Buffer Operation and Link-Level Error Recovery 116
Figure 6.7 DLLP Interleaving, Scrambling, and Physical Delivery 116
Figure 6.8 Data Link Layer FLIT-Based Transmission Architecture 118
Figure 6.9 Transmitter and Receiver NULL FLIT Processing Flow 119
Figure 6.10 Parallel LCRC and Sequence Number Processing for FLIT Transmission 123
Figure 7.1 Physical Layer Core Architecture 137
Figure 7.2 LTSSM State Controller Subsystem 145
Figure 7.3 Detect State Machine 146
Figure 7.4 Polling sub-states 147
Figure 7.5 Configuration State 148
Figure 7.6 Recovery State 149
Figure 7.7 PCIe LTSSM low power states 150
Figure 7.8 Hot and Disable states 150
Figure 7.9 Loopback State 151

List of tables

Table 1.1 Evolution of PCIe improvements [2] 5
Table 2.1 Next-Generation Interface Evolution [5] 20
Table 2.2 PCIe Lanes Implementation [13] 31
Table 4.1 Transaction Layer Functional Simulation Results Summary 54
Table 4.2 Multi-Lane Throughput Benchmark Results 57
Table 4.3 Verification Test Group Summary 58
Table 4.4 Functional Coverage Summary 60
Table 4.5 Final Verification Result Summary 61
Table 4.6 Synthesis Area and Cell Summary (TL vs. DLL) 64
Table 4.7 Synthesis Power Summary @ 250 MHz (TL vs. DLL) 65
Table 4.8 Static Timing Summary (TL vs. DLL) 66
Table 4.9 Design Rule Check (DRC) Summary 67
Table 4.10 Formality Compare-Point Summary (tl_top) 68
Table 4.11 Formality Compare-Point Summary (dll_top) 71
Table 4.12 MATLAB Simulation Parameters 74
Table 4.13 Theoretical PAM4 Eye Diagram Parameters 75
Table 4.14 Theoretical BER at Operating Point (SNR = 32 dB) 77
Table 4.15 Theoretical Jitter Budget (Dual-Dirac Model) 78
Table 4.16 Theoretical Frequency Response at Nyquist (16 GHz) 79
Table 5.1 Transaction Layer Interface Signals 83
Table 5.2 Transaction Layer responsibilities 89
Table 5.3 TLP header fields and RTL impact 91
Table 5.4 Comparison of Transaction Layer Features Between PCIe 5 And PCIe 6 97
Table 5.5 major architectural trade-offs 98
Table 5.6 Hardware-Level Signal Interface Between TL and DLL 102
Table 6.1 Data Link Layer Interface Signals 110
Table 6.2 Data Link Layer Responsibilities 117
Table 6.3 Retry Buffer Ack Latancy Values 124
Table 6.4 Error-Checking Pipeline Stages and Error-Handling Actions 129
Table 6.5 Comparison of DLL Features Between PCIe Gen 5 and PCIe Gen 6 132
Table 6.6 Major Architectural Trade-offs 133
Table 7.1 Physical Layer Interface Signals 138
Table 7.2 Physical Layer Responsibilities 143
Table 7.3 Comparison of Physical Layer Features Between PCIe Gen 5 and PCIe Gen 6 165
Table 7.4 Major Architectural Trade-offs 166

Abbreviations

PCIe	Peripheral Component Interconnect Express
NRZ	Non-Return-to-Zero
PAM4	Pulse Amplitude Modulation
FRS	Fast Replay System
TLP	Transaction Layer Packet
DLLP	Data Link Layer Packet
PHY	Physical Layer
FLIT	Flow Control Unit
FEC	Forward Error Correction
LCRC	Link Cyclic redundancy check
ECRC	End-to-End CRC
CRM	Cyclic Redundancy Mechanism
BER	Bit Error Rate
SER	Symbol Error Rate
SNR	Signal-to-Noise-Ratio
ACK	Acknowledgment
NAK	Negative Acknowledgment
FC	Flow Control
DLL	Data Link Layer
LTSSM	Link Training and Status State Machine

RTL	Register Transfer Level
FSM	Finite State Machine
FIFO	First-In First-Out
PIPE	PHY Interface for PCI Express
OHC	Orthogonal Header Content
SERDES	Serializer/Deserializer
DFE	Decision Feedback Equalization
CTLE	Continuous-Time Linear Equalization
FFE	Feed-Forward Equalizer
CDR	Clock and Data Recovery
SSC	Spread Spectrum Clock
ARQ	Automatic Repeat Request
ISI	Inter-Symbol Interference
AER	Advanced Error Reporting
FPGA	Field-Programmable Gate Array

Abstract

The continuous expansion of data-driven applications, artificial intelligence workloads, and high-performance computing systems has significantly increased the demand for faster and more reliable communication between internal computer components. PCI Express (PCIe) has become the dominant high-speed interconnect standard in modern computing platforms, and its sixth generation, PCIe Gen 6, represents a fundamental architectural shift by doubling the data rate to 64 GT/s per lane through the adoption of PAM4 modulation, mandatory Reed-Solomon Forward Error Correction (FEC), and FLIT-based fixed-length packet framing. Despite the significance of these advancements, a clear gap exists in the available literature regarding practical, structured, and educationally oriented digital models of PCIe Gen 6, as most existing resources address Gen 6 concepts at a theoretical level without providing implementation-ready architectural guidance spanning all three protocol layers. This project addresses that gap by proposing and implementing a modular digital design model of PCIe Gen 6, covering the complete three-layer protocol stack: the Transaction Layer, the Data Link Layer, and a behavioral digital model of the Physical Layer, modeling the full transmitter-to-receiver data path including TLP construction and FLIT packing, sequence numbering and ARQ-based replay, RS(544,514) FEC encoding and decoding, PAM4 Gray code mapping, LTSSM-governed link training, and lane deskew. The complete RTL design comprises 119 digital modules, and theoretical performance analysis conducted in MATLAB validates the key Gen 6 specification parameters including PAM4 eye opening, FEC coding gain, and jitter budget at the nominal operating point. The resulting model serves as a research-driven, implementation-ready educational platform that bridges the gap between the formal PCIe Gen 6 specification and practical academic study, and provides a scalable foundation for future research into advanced PCIe features, RTL verification methodologies, and next-generation high-speed interconnect design.

1 Chapter One: Introduction

This chapter provides a brief study of the Peripheral Component Interconnect Express (*PCIe*) standard, presenting its background, fundamental concepts, and layered architecture. The chapter explains the limitations of traditional bus-based interconnects and describes how PCIe evolved to address increasing bandwidth and performance demands. It also presents the progression of PCIe generations from Gen 1 through Gen 5, leading to the motivation behind PCIe Gen 6. Key architectural enhancements and features introduced in PCIe Gen 6 are highlighted, particularly those enabling higher data rates and improved reliability. Finally, the chapter presents the problem statement, project objectives, and scope, establishing the foundation for the simulation-based study of PCIe Gen 6 in the subsequent chapters.

1.1 Background

Traditional bus-based interconnects like PCI and AGP were unable to keep pace with the rising demands of modern computing. Their limitations in bandwidth, scalability, and efficiency highlighted the need for a more advanced communication interface. PCI Express (PCIe) was introduced to overcome these constraints by providing a high-speed, low-latency, and scalable architecture capable of supporting the performance requirements of contemporary systems.

1.2 Overview of PCI Express

Peripheral Component Interconnect Express (PCIe) is a high-speed serial computer expansion interface standard used to connect major hardware components inside a computer system. It serves as the primary communication link between the central processing unit (CPU), the chipset, and high-performance peripheral devices such as graphics cards (GPUs), solid-state drives (NVMe SSDs), network interface cards, and other expansion devices.

In simple terms, PCIe can be described as the main internal data highway of modern computers, enabling fast and efficient data transfer between the processor and connected devices. Earlier interconnect standards such as PCI and PCI-X relied on parallel communication, where multiple data bits were transmitted simultaneously. As operating frequencies increased, these parallel buses suffered issues such as: Signal skew, Electromagnetic interference, Limited scalability, and Reduced reliability at high speeds.

To overcome these limitations, PCIe was introduced as a serial, point-to-point communication standard, allowing higher data rates, better signal integrity, and improved system scalability. PCIe uses a **point-to-point topology**, meaning that each device has a **direct connection** to the root complex (usually the CPU or chipset), rather than sharing a common bus with other devices. This design eliminates contention between devices and significantly improves overall system performance.

1.2.1 PCIe TOPOLOGY

PCIe Topology will help to understand how different peripheral devices will communicate with the CPU through the PCIe link. The CPU will access the root complex to configure the IP, and CPU will also configure the different endpoints with the help of the root complex. The root complex can directly access the memory. Thus, the PCIe endpoints will communicate the CPU and the memory with the help of root complex as shown in **(Figure 1.1)**. We can attach many PCIe endpoints with the root complex with the help of PCIe Switches. These different components will communicate between the CPU or Memory and peripheral devices through the PCI express expansion bus. These PCIe endpoints can read and write data from memory. [1]

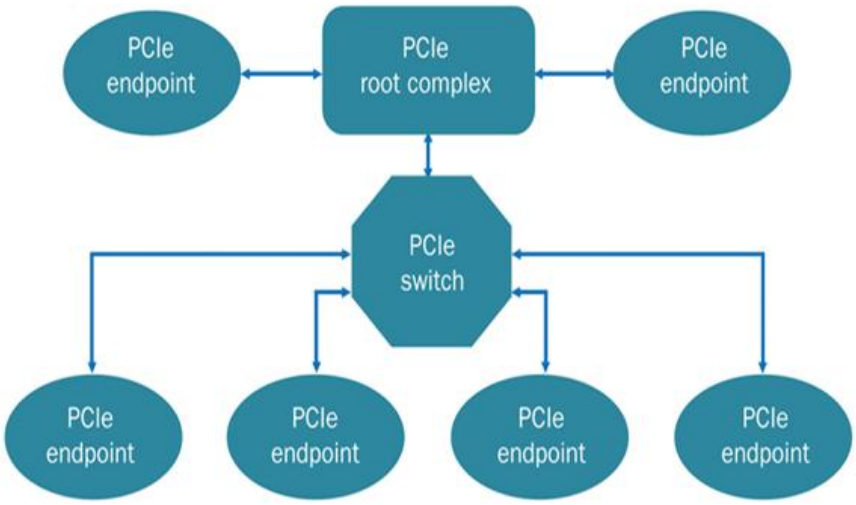


Figure 1.1 The PCIe Topology [1]

1.2.2 PCIe ARCHITECTURE

The PCIe bus is a serial bus expansion which follows a three-layer protocol architecture as shown in **(Figure 1.2)**. These three layers are physical layer, data link layer, and transaction layer. The data is passed through these layers. The mechanism of PCIe standard is plug and play based PCI. The Physical layer provides the point-to-point serial connection between the peripheral devices and processor unit of PCI. The read and write requests of transportation occurred in the transaction layer. The mechanism of the PCIe depends upon this layer. The packet-based transaction and split transaction protocol occurred in the transaction layer. The link layer is used to link the sequence numbers and CRC of the packets for the transaction layer. This link provides a highly reliable data transfer mechanism. The physical layer has a transmit pair and a receive pair. The combination of transmit pair and the receive pair is called a8 lane. The bandwidth of the PCI Express Lane is about 250MB/s in each direction. Nowadays, the PCI board data rate can be increased by folding it twice or fourth times. This bandwidth is providing for the same devices. [1]

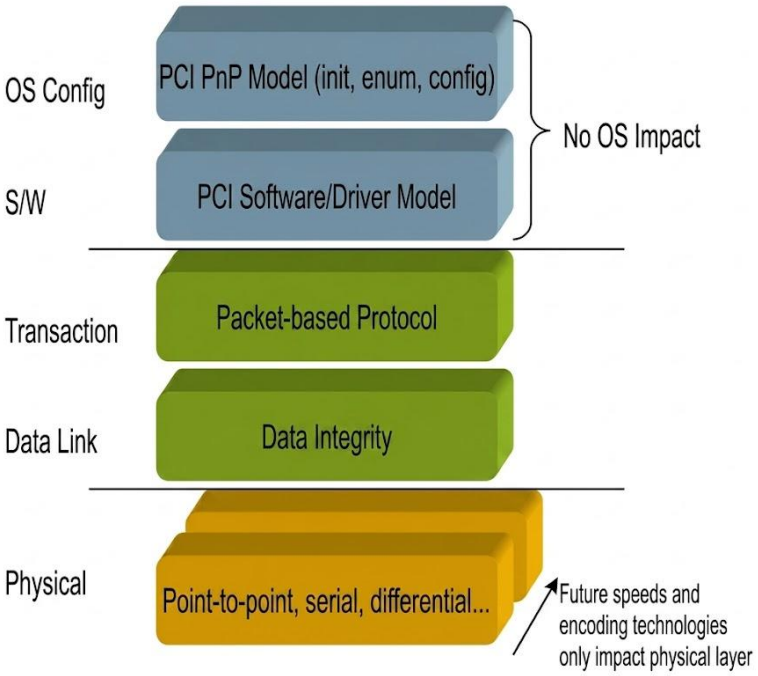


Figure 1.2 The PCI Express Architecture [1]

1.3 Evolution of PCIe Generations

PCIe has evolved through multiple generations, each introducing advancements in speed, bandwidth, and features (**Table 1.1**): [2]

Table 1.1 Evolution of PCIe improvements [2]

Revision**	Max Data Rate	Encoding	Signaling
PCIe 8.0 (TBD)	Pathfinding	TBD	TBD
PCIe 7.0 (2025)	128.0 GT/s	1b/1b (Flit Mode*)	PAM4
PCIe 6.0 (2022)	64.0 GT/s	1b/1b (Flit Mode*)	PAM4
PCIe 5.0 (2019)	32.0 GT/s	128b/130b	NRZ
PCIe 4.0 (2017)	16.0 GT/s	128b/130b	NRZ
PCIe 3.0 (2010)	8.0 GT/s	128b/130b	NRZ
PCIe 2.0 (2007)	5.0 GT/s	8b/10b	NRZ
PCIe 1.0 (2003)	2.5 GT/s	8b/10b	NRZ

Gen1: PCI Express Generation 1 was the first version of the PCIe standard, introduced in the early 2000s. It operates at a data rate of 2.5 GT/s (gigatransfers per second) per lane, using 8b/10b encoding, which adds overhead for signal integrity. This gives an effective bandwidth of about 250 MB/s per lane (e.g., ~4 GB/s on an x16 link). Gen1 laid the foundation for PCIe's scalable, serial-link design, replacing older parallel buses like PCI.

Gen2: PCI Express Generation 2 doubled the data rate of Gen1 to 5.0 GT/s per lane, while still using 8b/10b encoding. This results in an effective bandwidth of roughly 500 MB/s per lane (about 8 GB/s on an x16 link). Gen2 improved overall performance and supported higher-speed peripherals while maintaining backward compatibility with Gen1 devices and slots.

Gen3: Introduced in 2010, Gen3 operates at a maximum data rate of 8 GT/s (gigatransfers per second), achieving an effective bandwidth of approximately 1 GB/s per lane. It employs a 128b/130b encoding scheme, which significantly reduces overhead compared to earlier PCI standards. [3]

Gen4: Launched in 2017, Gen4 doubled the data rate to 16 GT/s, providing a bandwidth of about 2 GB/s per lane. This generation also introduced enhancements in signal integrity and power management, enabling more efficient communication.

Gen5: Released in 2019, Gen5 further increased the data rate to 32 GT/s, effectively doubling the bandwidth to 4 GB/s per lane. This generation introduced new features such as improved error correction and enhanced power efficiency.[3]

Gen7: PCI Express Generation 7 (PCIe 7.0) is the upcoming evolution of the PCIe high-speed expansion bus, targeting a data rate of up to 128 GT/s (gigatransfers per second) per lane — double that of PCIe 6.0. When used in a typical x16 configuration, this enables up to about 512 GB/s of bidirectional raw bandwidth before accounting for encoding overhead. Gen7 continues to use PAM4 (Pulse Amplitude Modulation with 4 levels) signaling and flit-based encoding (introduced in Gen6) to maximize transmission efficiency and power performance, while maintaining backward compatibility with previous PCIe versions. It's aimed at high-performance applications such as AI/ML, cloud and data-center computing, and other data-intensive systems.[2]

Gen8: PCI Express Generation 8 (PCIe 8.0) is a future evolution of the PCIe standard, expected to deliver data rates beyond PCIe 7.0 while addressing physical-layer challenges through improved signaling, error correction, and power efficiency. It is anticipated to continue using flit-based transmission and advanced modulation techniques, while maintaining backward compatibility. PCIe Gen8 is mainly targeted at next-generation data centers, AI accelerators, and high-performance computing systems requiring extremely high bandwidth and low latency.

1.4 Motivation for PCIe Gen 6

Before 2015, PCIe provided more bandwidth than typical applications required, but rising global data traffic made previous generations insufficient. With data centers moving to 100G, 400G, and 800G Ethernet, PCIe became a performance bottleneck. PCIe Gen 6.0 solves this by offering up to 128 GB/s on a ×16 link, supporting modern data-center networks and full-duplex operation. The growing demands of AI/ML workload also benefit from faster data movement, improving training speed, reducing latency, and enhancing large-scale system efficiency. **(Figure 1.3)**. Ultimately, PCIe Gen 6 is essential for modern networking, AI acceleration, and high-performance computing infrastructure.

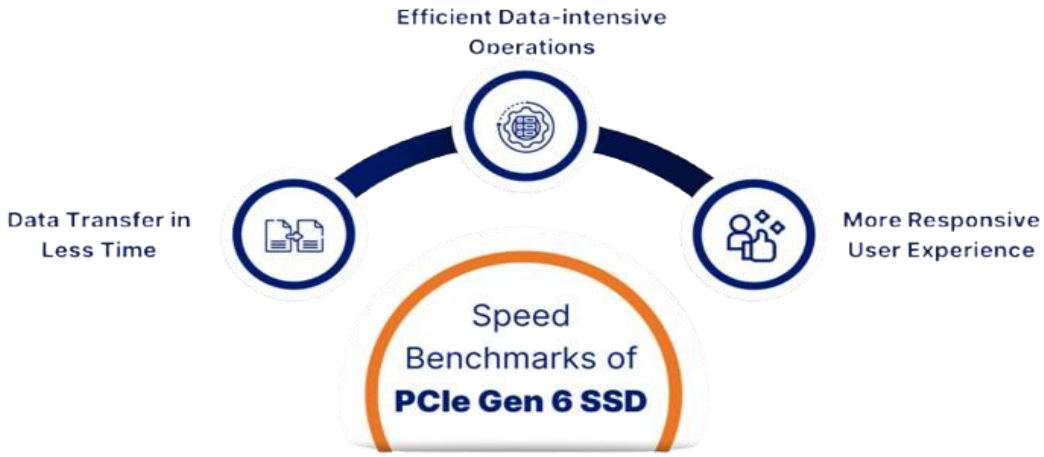


Figure 1.3 Speed Benchmarks and Practical Implications. [25]

1.5 Key Enhancements and Architectural Changes

PCIe 6.0 doubles the data rate of PCIe 5.0 to **64 GT/s per lane**, significantly boosting bandwidth and performance. It switches to **PAM-4 signaling** to encode more bits per symbol and uses **Forward Error Correction (FEC)** to improve data reliability at high speeds. The standard maintains **backward compatibility** with earlier PCIe versions, easing integration into existing systems. PCIe 6.0 also includes power efficiency improvements and enhanced link power management, which help reduce energy use during idle or low activity states. These enhancements make PCIe 6.0 suitable for future high-bandwidth systems.

Figure 1.4 PCIe Gen 6.0 specification features. [25]

1.6 Problem Statement

PCIe 6.0 achieves a significant bandwidth increase by operating at 64 GT/s per lane using PAM4 signaling; however, this advancement introduces several critical challenges. PAM-4 reduces signal voltage margins, making links more vulnerable to noise, crosstalk, jitter, and channel loss, which degrades signal integrity compared to earlier NRZ-based generations. To address the resulting higher bit error rates, PCIe 6.0 mandates Forward Error Correction (FEC), which improves reliability but adds latency and protocol overhead, creating performance trade-offs for latency-sensitive applications. Additionally, the Physical Layer becomes substantially more complex due to advanced equalization, clock recovery, and error-handling mechanisms, leading to increased design difficulty, power consumption, and thermal concerns. These issues collectively make the implementation, verification, and optimization of PCIe 6.0 more challenging despite its substantial throughput advantages.

1.7 Project Objectives

The primary objective of this project is to study, model, and simulate the PCI Express (PCIe) Generation 6 protocol in order to understand its layered architecture, data transfer mechanisms, and performance improvements over earlier PCIe versions. The project aims to develop a simplified, simulation-based hardware model that demonstrates how PCIe Gen 6 manages data transactions between a host and peripheral devices, including key operations such as link training, packet generation and transmission, and error detection and correction. The scope of the work is focused on representing the logical system architecture and data flow across the three main PCIe layers—Transaction Layer, Data Link Layer, and a behavioral digital model of the Physical Layer—while intentionally avoiding unnecessary complexity related to full physical hardware implementation. Using hardware description and simulation tools such as Verilog, ModelSim, or equivalent environments, the project bridges theoretical protocol concepts with practical implementation, providing an educational and analytical insight into PCIe Gen 6 functionality and behavior at the system level. (**Figure 1.5**)

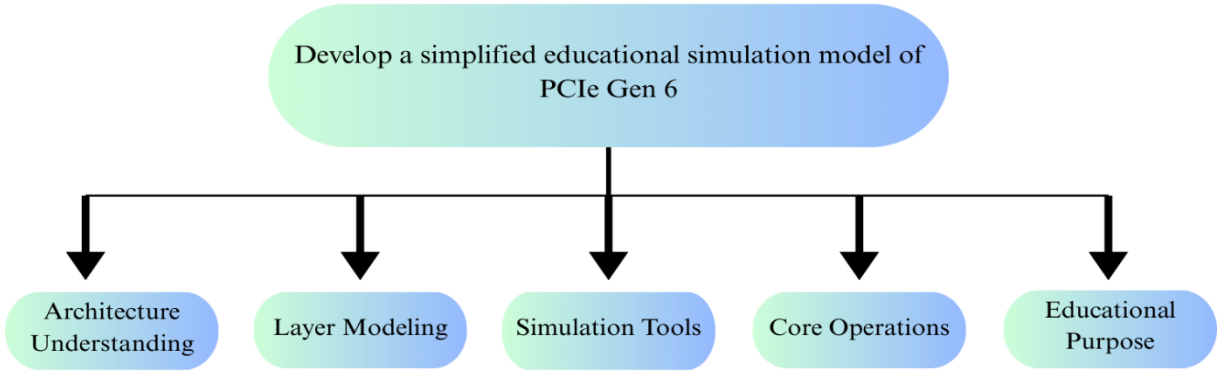


Figure 1.5 PCIe Gen 6.0 Objectives**.**

2 Chapter Two: Previous Studies

This chapter presents a comprehensive study of previously related work to the PCI Express (PCIe), aiming to establish the theoretical and technical background for the proposed work. The reviewed literature covers key aspects of PCIe, including architecture, generational development, physical-layer signalling techniques, packet framing mechanisms, error control strategies, and existing implementations. Special attention is given to recent advancements such as FLIT-based packet framing, PAM4 modulation, the transition from NRZ signalling, and the integration of FEC and CRC in high-speed links. The chapter also discusses verification challenges associated with PCIe Gen6 and concludes by identifying key research gaps that motivate the objectives and contributions of this project.

2.1 Studies About PCIe Architecture

Understanding the PCI Express (PCIe) architecture is essential for analyzing and designing high-speed interconnect systems. A clear architectural overview enables deeper comprehension of PCIe protocol behavior, data flow, and error-handling mechanisms used in modern computing platforms. Consequently, numerous studies have been published addressing PCIe architecture from different perspectives, including performance, reliability, and fault tolerance.

One of the most significant and relevant studies is "**Fault-Resilient PCIe Bus with Real-time Error Detection and Correction"**, published in May 2021 by Mostafa Darvishi, Ph.D., Member IEEE, and an independent high-performance computing researcher.[4]

This work is considered a reliable and authoritative reference due to its detailed architectural analysis and its practical implementation on FPGA-based PCIe systems and its crucial role in past and current research.

The paper presents a comprehensive description of the **three-layer PCIe architecture**, which consists of the Transaction Layer (TL), Data Link Layer (DLL), and Physical Layer (PL) as shown in **(Figure 2.1)**. The Transaction Layer is responsible for packet formation, managing end-to-end data transfers, and detecting errors such as unsupported requests, completion timeouts, and corrupted Transaction Layer Packets (TLPs). The Data Link Layer ensures reliable packet delivery by performing Link CRC (LCRC) checks, sequence number verification, and retransmission in case of detected errors.

The Physical Layer manages link training, lane initialization, and low-level electrical signaling, ensuring stable high-speed communication between devices. In addition to layer functionality, the study explains the PCIe lane-based architecture, where PCIe links are implemented using scalable lane configurations such as x1, x4, x8, x16, and x32. Increasing the number of lanes allows higher throughput by transmitting data in parallel across multiple differential pairs. The paper also characterizes PCIe as a point-to-point, dual-simplex, high-speed serial interconnect that relies on differential signaling to improve noise immunity and signal integrity.

Furthermore, the paper surveys several system-level PCIe architectural studies, including DMA-based PCIe architectures, bridging mechanisms. These studies highlight how PCIe can be adapted for different applications such as high-performance computing, data acquisition systems, and security analysis as shown in **(Figure 2.1).**

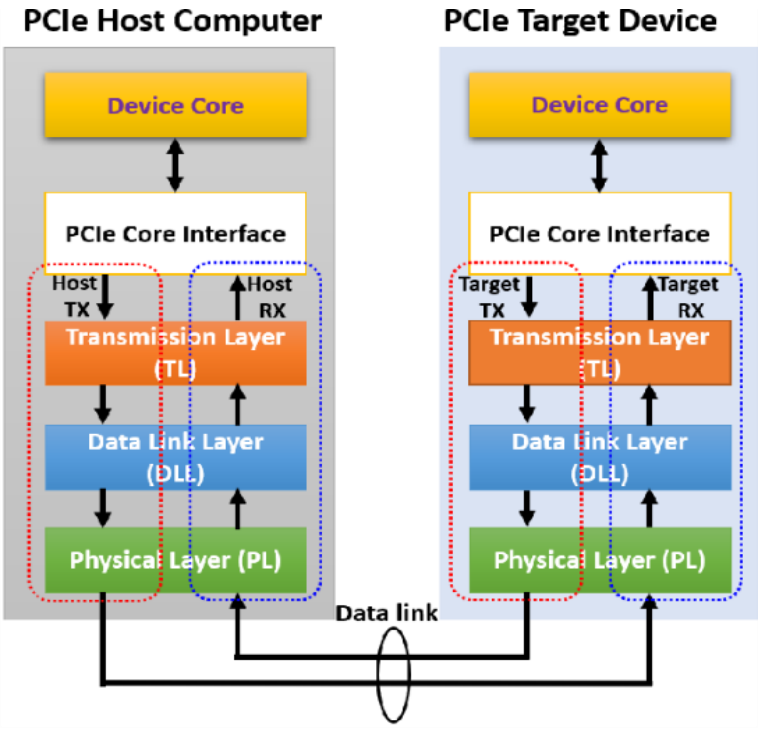


Figure 2.1 PCIe architecture overview [4]

Beyond basic architectural concepts, the authors emphasize the importance of error classification and reliability mechanisms within PCIe systems. Errors are analyzed at each architectural layer, and the paper categorizes them into correctable, non-correctable, non-fatal, and non-correctable fatal errors, illustrating how PCIe maintains system stability even under fault conditions.

Overall, this study provides a solid architectural foundation for understanding PCIe operation and reliability. Its detailed discussion of protocol layers in **(Figure 2.1),** lane scalability, signaling characteristics, and error-handling mechanisms makes it a fundamental reference for research and development in high-speed PCIe-based systems.

Moreover, this architecture was used in many articles and books afterwards by another researchers and publishers due to its simplicity and clarity so it was prementioned in another authorized study "**Technical Analysis of PCIe to PCIe 6: A Next-Generation Interface Evolution**' that was published in August 2023 by Santhosh Nagaraj Nag, which provides a systematic comparison of PCIe generations from Gen1 through Gen6.[5]

Each PCIe lane consists of two differential signaling pairs: one for transmission (TX) and one for reception (RX). Lanes are unidirectional and can be combined to form wider links, such as ×1, ×4, ×8, or ×16, depending on bandwidth requirements. This lane-based structure allows PCIe to scale performance while maintaining signal integrity and reliability across generations.

Physically, PCIe devices connect through standardized slots that support different lane configurations and form factors. The number of lanes associated with a slot directly determines the available bandwidth, with larger configurations offering higher throughput. PCIe also supports backward and forward compatibility, enabling devices and hosts of different generations to interoperate at the highest mutually supported link speed and width.

Logically, PCIe systems follow a hierarchical organization centered around a host device, typically the CPU or chipset as clarified in **(Figure 2.2).** The host contains one or more PCIe Root Complexes responsible for device initialization, configuration, and transaction management.

Peripheral devices communicate with the host using packet-based protocols, while PCIe switches may be used in larger systems to expand connectivity by enabling multiple endpoints to share a single root complex.

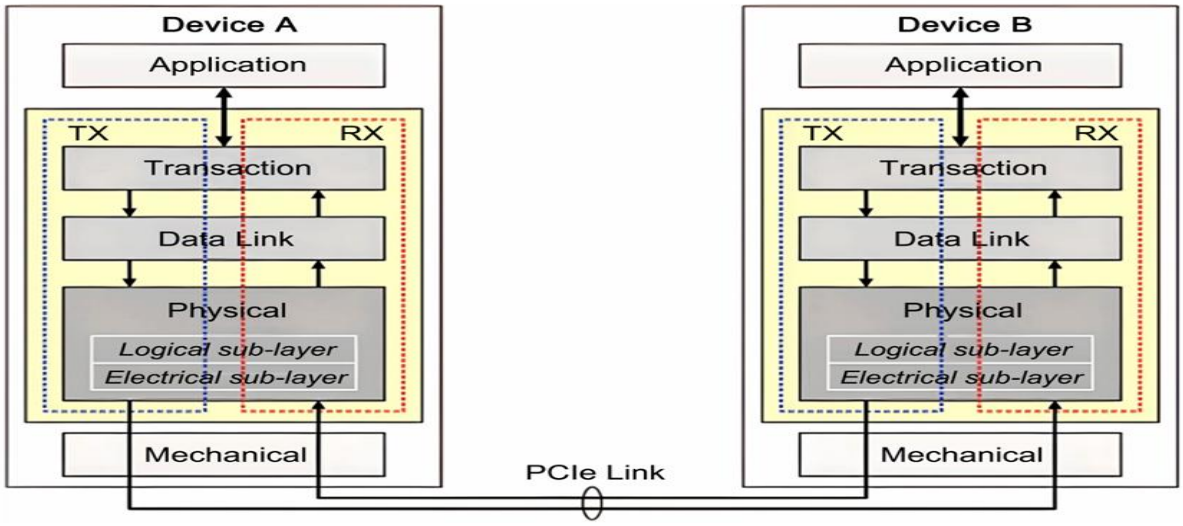


Figure 2.2 PCIe architecture overview [5]

2.2 Studies About PCIe Generational Development

The evolution of Peripheral Component Interconnect Express (PCIe) was driven by the continuous increase in processor performance and the growing demand for higher data transfer rates between system components. Before PCIe, conventional PCI architectures relied on a shared parallel bus (**Figure 2.3)**, which became a significant performance bottleneck as multiple devices competed for the same bandwidth. This limitation motivated the transition toward a high-speed, point-to-point serial interconnect, leading to the introduction of PCIe in the early 2000s.

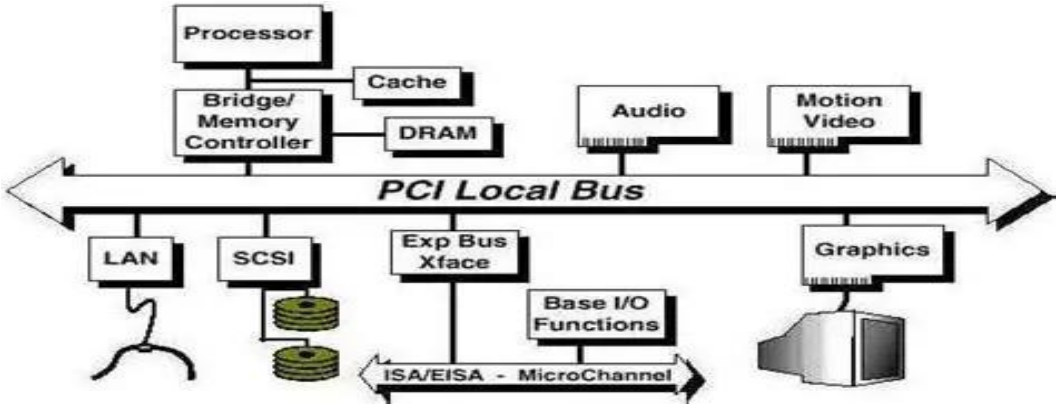


Figure 2.3 old PCI bus [6]

Unlike legacy PCI, PCIe introduced a scalable lane-based architecture and dedicated point-to-point links, allowing each connected device to access its own bandwidth independently. However, early versions of PCIe were not designed to support the extremely high data rates required by modern processors, accelerators, and storage devices. As a result, PCIe has undergone continuous generational development, with each new generation addressing performance limitations, signal integrity challenges, and protocol efficiency issues observed in previous versions.

Numerous studies have analysed the generational evolution of PCIe in detail. One of the most comprehensive references is the paper "**Technical Analysis of PCIe to PCIe 6: A Next-Generation Interface Evolution**", published in August 2023 by Santhosh Nagaraj Nag, which provides a systematic comparison of PCIe generations from Gen1 through Gen6, focusing on data rate scaling, encoding techniques, and protocol enhancements.[5]

According to PCI-SIG. (2002) "***PCI Express Base Specification, Revision 1.0"*** in April 29, 2002, **PCIe Gen1** established the foundation of the PCIe architecture by supporting a data rate of **2.5 GT/s per lane**, with scalable lane configurations ranging from x1 to x16. This generation introduced the fundamental concepts of point-to-point communication, and backward compatibility. (PCIe Gen1) introduced a scalable, packet-based interconnect architecture that replaced legacy shared-bus designs. It defined structured transaction types and strict ordering rules, enabling reliable and predictable data transfer across point-to-point links within the PCIe fabric. [6]

These features established the foundation for consistent communication and extensibility in subsequent PCIe generations. Power efficiency and system flexibility were central to PCIe Gen1, with support for multiple device power states, software-controlled power transitions, and platform-level power budgeting. The standard also incorporated Quality of Service (QoS) mechanisms to enable differentiated traffic handling, as well as hot-plug and hot-swap support and advanced peer-to-peer communication across multiple hierarchies, allowing scalable and dynamic system expansion.

According to **"M. Tj, "Difference Between PCI Express** Gen 1**, Gen 2,** Gen 3, Gen 4, Gen 5, & Gen 6**,"** Desk Decode" in Jul. 6, 2017. **PCIe Gen2** built upon the architectural foundations established in Gen1 while significantly increasing link performance. The primary enhancement introduced in Gen2 was the doubling of the raw data rate from 2.5 GT/s to 5.0 GT/s per lane, resulting in substantially higher bandwidth without altering the fundamental packet-based protocol. PCIe Gen2 maintained backward compatibility with Gen1 devices, allowing systems to negotiate link speed and width dynamically based on the capabilities of connected components.[7]

In addition to higher throughput, **PCIe Gen2** introduced improvements in signal integrity and power management to support increased data rates. Enhanced link equalization, refined clocking mechanisms, and optimized electrical specifications enabled reliable operation at higher speeds. The generation also preserved and extended Quality of Service (QoS), hot-plug support, and advanced error handling features, ensuring scalability and robustness in performance-critical and high-density computing systems. As shown in **(Figure 2.4).**

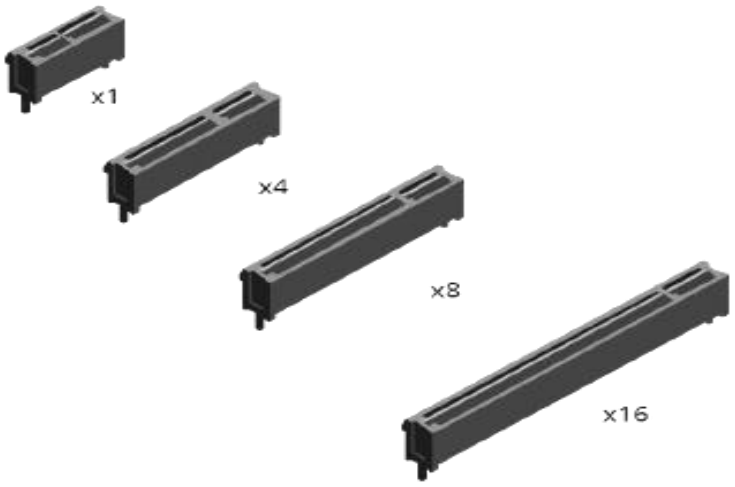


Figure 2.4 PCIe X1, X4, X8, X16 from different generations [7]

PCIe Gen3 represented a major architectural improvement as shown by **PCISIG in 'PCI Express Base Specification Revision 3.0**' in November 2010, PCI Express Generation 3 (PCIe Gen3) enhanced the data rate to **8 GT/s per lane** PCIe and architecture by improving efficiency, reliability, and service differentiation while preserving backward compatibility. It continued to support structured transactions and ordering rules and expanded Quality of Service (QoS) capabilities through end-to-end packet tagging, configurable arbitration, and improved handling of time-sensitive data flows, reducing head-of-line blocking in complex systems.[8]

PCIe Gen3 also strengthened power management, data integrity, and error handling mechanisms. Devices can support advanced power-state control, wake-up signalling, and platform-level power budgeting, alongside unified hot-plug and hot-swap support. Enhanced link-level and end-to-end error detection, advanced error reporting, and improved testability further increased the reliability and scalability of PCIe Gen3-based systems.

(Figure 2.5) illustrates the PCI Express (PCIe) packet structure and the distribution of responsibilities across the protocol layers. The Transaction Layer generates the packet header and optional data payload, with optional End-to-End CRC (ECRC) providing additional error protection. The Data Link Layer adds a sequence number and a Link CRC (LCRC) to support reliable transmission through error detection and replay mechanisms. Finally, the Physical Layer applies framing to delimit packets and enable correct transmission over the serial link, ensuring proper alignment and data integrity.

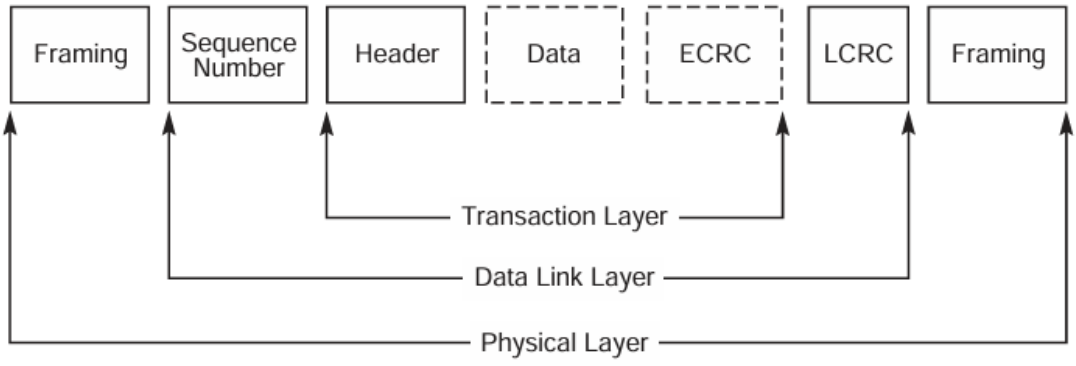


Figure 2.5 Packet Flow Through the Layers in GEN3 [8]

Subsequent generations focused primarily on bandwidth scaling to meet the requirements of high-performance computing and data center applications. Regarding '**PCI Express Base Specification Revision 4.0**' which is the official reference for the **PCIe Gen 4** that was released in February 2014, PCIEV4 is designed to provide data rate of 16 GT/s per lane low-latency, high-bandwidth communication while maximizing link efficiency and application payload throughput. This is achieved through higher bandwidth per lane, scalable performance using aggregated lanes can be noticed in **(Figure 2.6)**, and advanced power management capabilities, including power state control, wake-up signalling, and efficient power budgeting.[9]

The architecture also supports differentiated Quality of Service (QoS) to improve data flow efficiency and mitigate head-of-line blocking. In addition, PCIe Gen 4 introduces enhanced features such as M-PCIe support over MIPI M-PHY, native Hot-Plug functionality, improved data integrity and error handling, process technology independence, and simplified electrical compliance testing. These enhancements make PCIe Gen 4 suitable for high-performance, scalable, and reliable system designs.[10]

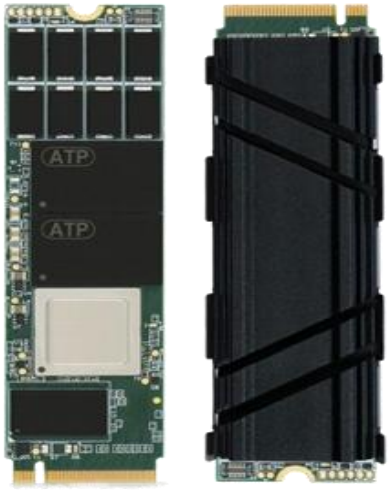


Figure 2.6 PCIe generation 4 by NVMe [9]

PCIe Gen5 further doubled the data rate to 32 GT/s per lane, as defined in the **PCI Express Base Specification Revision 5.0** released in May 2019, delivering substantial bandwidth improvements while maintaining backward compatibility with earlier generations. In addition to higher throughput, PCIe Gen5 extends power management capabilities by supporting system wake-up requests from low-power states and enabling controlled device power-up sequencing in accordance with platform power-budgeting policies.[11]

Moreover, PCIe Gen5 enhances Quality of Service (QoS) through differentiated services, including dedicated link resources per data flow, configurable arbitration policies, packet-level QoS tagging, and end-to-end isochronous transfers, thereby reducing head-of-line blocking and improving fabric efficiency as shown in **(Figure 2.7).** System reliability and flexibility are further strengthened through comprehensive Hot-Plug support, advanced error handling, improved data integrity, process technology independence, and simplified electrical compliance testing, ensuring robust integration within next-generation high-performance computing platforms.[12]

Figure 2.7 PCIe generation 5 from NVMe [11]

The most significant architectural shift occurred with **PCIe Gen6** which can be seen from **'PCIe 6.0 Specification: The Interconnect for I/O Needs of the Future,**' which introduced fundamental changes beyond simple bandwidth doubling. Although Gen6 maintains a raw data rate equivalent to **64 GT/s per lane**, it abandons traditional NRZ signalling in Favor of Pulse Amplitude Modulation with four levels (PAM4). PAM4 enables the transmission of two bits per symbol, effectively increasing data throughput without doubling the clock frequency. However, this modulation scheme inherently increases the bit error rate, necessitating additional reliability mechanisms.[13]

PCIe Gen 6 introduces major architectural enhancements to maintain power efficiency and low latency. It strengthens power management through advanced wake-up mechanisms and controlled power sequencing, while expanding Quality of Service (QoS) capabilities with packet-level QoS tagging, configurable arbitration policies, and support for end-to-end isochronous traffic. These features enable predictable performance and efficient resource allocation in data-intensive and latency-sensitive workloads.[14]

In addition, PCIe Gen 6 significantly improves system robustness by integrating stronger data integrity and error-handling mechanisms, including mandatory forward error correction (FEC) and enhanced CRC protection. Native Hot-Plug support, process technology independence, and improved testability ensure scalability and reliability across diverse platforms. Collectively, these advances position PCIe Gen 6 as a foundation for next-generation accelerator, memory-centric, and high-performance computing systems.

(Table 2.1) summarizes the evolution of PCI Express (PCIe) across generations, showing how performance has increased over time. Each new PCIe version roughly doubles the data rate per lane, leading to a corresponding increase in total x16 bandwidth. Early generations (PCIe 1.0 and 2.0) used 8b/10b encoding, which introduced higher overhead, while PCIe 3.0 and later adopted 128b/130b encoding to improve efficiency. PCIe Gen6 marks a major shift by introducing PAM4 signaling and FLIT-based data transfer, enabling extremely high bandwidth while maintaining reliability.

Table 2.1 Next-Generation Interface Evolution [5]

PCIe ppecification	Year	Data Rate (Gb/s) Encoding	x16 Bandwidth*
1.0	2003	2.5 (8b/10b)	32 Gb/s
2.0	2007	5.0 (8b/10b)	64 Gb/s
3.0	2010	8.0 (128b/130b)	126 Gb/s
4.0	2017	16.0 (128b/130b)	252 Gb/s
5.0	2019	32.0 (128b/130b)	504 Gb/s
6.0	2021	64.0 (PAM-4, FLIT)	1024 Gb/s

To address these challenges, PCIe Gen6 introduced Forward Error Correction (FEC) to detect and correct errors without requiring retransmission, thereby maintaining low latency and high efficiency. In addition, Gen6 adopted a Fixed-Length Unit (FLIT)-based architecture, replacing variable-length packets to simplify error correction, improve latency predictability, and support high-speed operation. These changes represent a shift toward a more deterministic and robust protocol design suitable for future computing systems.

Beyond raw speed improvements, several studies emphasize that PCIe evolution also focused on reducing protocol overhead and improving real-world performance. For instance, the transition from 8b/10b to 128b/130b encoding significantly increased effective bandwidth without increasing the transfer rate. Furthermore, advanced features such as automatic lane margining, introduced in later generations, enable continuous monitoring of signal quality during normal operation, which is critical as eye diagrams shrink at higher data rates. Collectively, these studies demonstrate that PCIe generational development is driven not only by higher transfer rates but also by smarter protocol design and enhanced reliability mechanisms.[14]

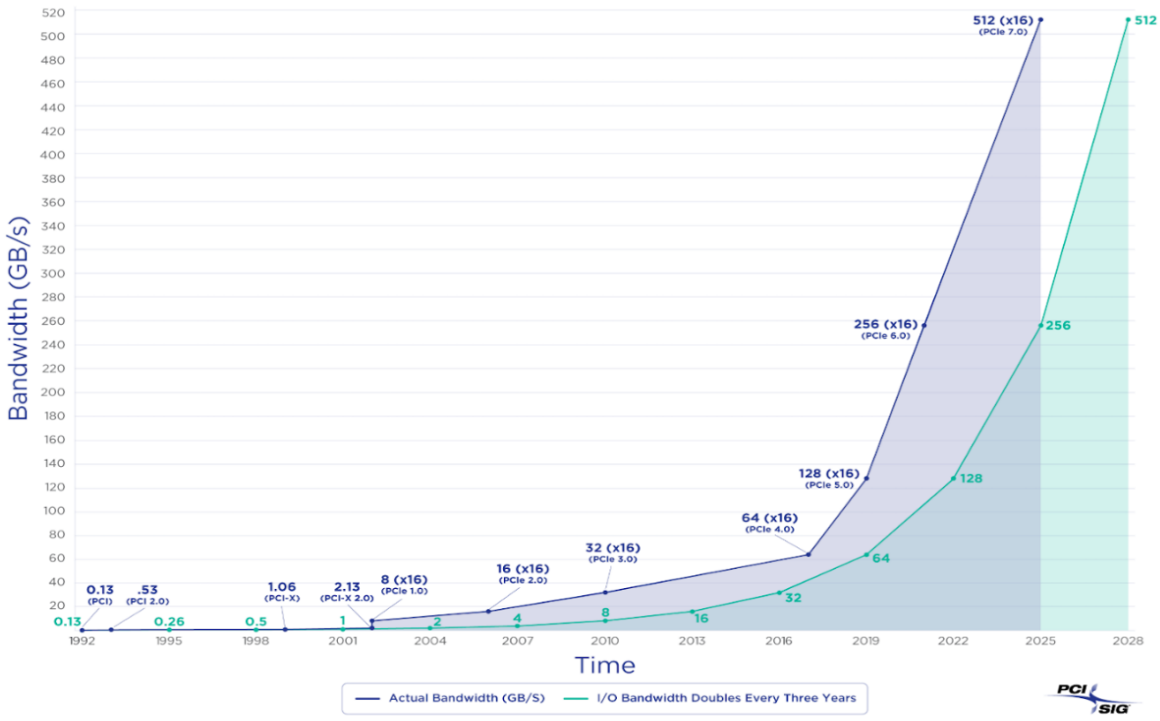


Figure 2.8 generational development of PCIe [13]

In addition to speed improvements across PCIe generations, several research papers focused on protocol efficiency and how overhead reduction contributed to real-world performance. For example, the transition from 8b/10b encoding in Gen1/Gen2 to 128b/130b encoding in Gen3 significantly reduced bandwidth loss and increased effective throughput even without changing the symbol rate.

Studies also analyzed the development of automatic lane margining—a feature introduced to continuously evaluate signal quality during active operation. This technique became increasingly important starting from PCIe Gen5 because of the dramatically reduced eye-diagram width. These findings highlight that PCIe evolution was not only about increasing transfer rates but also about introducing smarter and more efficient protocol mechanisms.

2.3 Studies About FLIT Mode (Fixed-Length Packets)

Several recent studies have focused on the introduction of Fixed-Length Packet (FLIT) mode, which represents one of the most significant architectural changes in PCIe Gen6. A key reference in this area is the study published in **PCIe verification at high speeds: challenges, solutions, and future trends**' February 2025, which provides a detailed technical analysis of FLIT integration, verification challenges, high-speed operation considerations and FLIT layout as shown in **(Figure 2.9).** As a recent publication, this work reflects the latest developments in PCIe Gen6 protocol design and verification methodologies.[16]

Description		Lane															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16 Symbol times	TLP Bytes [0…223]	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
		32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
		48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
		64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
		80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
		96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
		112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
		128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
		144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
		160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
		176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
		192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
		208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
	TLP, DLP	224	225	226	227	228	229	230	231	232	233	234	235	DLP 0	DLP 1	DLP 2	DLP 3
	DLP, CRC, ECC	DLP 4	DLP 5	CRC 0	CRC 1	CRC 2	CRC 3	CRC 4	CRC 5	CRC 6	CRC 7	ECC1 [0]	ECC2 [0]	ECC0 [0]	ECC1 [1]	ECC2 [1]	ECC0 [1]

Figure 2.9 Flit Layout in a .x16 Link [16]

With the transition to PCIe Gen6, the data transmission model shifted from variable-length Transaction Layer Packets (TLPs), used in PCIe Gen1 through Gen5, to a fixed-length FLIT-based architecture. In earlier generations, TLPs and Data Link Layer Packets (DLLPs) varied in size, providing flexibility but increasing complexity at higher data rates. At 64 GT/s with PAM4 signaling, variable-length packets become difficult to manage due to tighter timing margins and increased sensitivity to bit errors. To address these challenges, PCIe Gen6 transmits all data using fixed-size 256-byte FLITs, enabling more predictable timing, simplified framing, and efficient error correction.

The adoption of FLIT mode introduces new packet formats and header structures, increasing protocol complexity, and verification requirements. Since FLITs may encapsulate multiple TLPs or DLLPs, or alternatively split a single TLP across multiple FLITs, correct handling of FLIT boundaries is critical. Verification must ensure proper packet segmentation, reassembly, sequencing, and ordering, as errors in these processes can result in data corruption or system instability.

Furthermore, FLIT mode interacts closely with other PCIe Gen6 features, including PAM4 signaling, Forward Error Correction (FEC), and new power management states. Ensuring correct interaction between these mechanisms is a major verification challenge. For example, FEC must operate efficiently on fixed-size FLITs to detect and correct errors without introducing excessive latency, while power state transitions must not disrupt FLIT transmission or ordering. These interactions require comprehensive verification strategies to validate system reliability under various operating conditions.

The **IJCET** study emphasizes that combining FLIT mode with PAM4 signaling creates a unique verification environment that demands specialized test methodologies. The fixed-length structure simplifies certain aspects of protocol handling but introduces new requirements for validating data integrity, buffer management, and link-layer coordination. Careful verification is needed to ensure seamless interaction between the Transaction Layer and Data Link Layer when operating under FLIT-based framing.[17]

That study proofs the research **'Flit-Reservation Flow Control**' posted in January2000. indicating that FLIT mode was introduced not only to support higher data rates but also to standardize packet handling across protocol layers, ultimately simplifying controller design and improving scalability. However, this standardization comes at the cost of increased verification complexity, especially in scenarios involving FLIT multiplexing, buffer overflow prevention, and strict ordering enforcement. These challenges are expected to remain relevant in future PCIe generations, including Gen7, where FLIT-based communication is anticipated to be the default operating mode.

Flit-reservation flow control is an advanced flow-control technique in which small control flits are transmitted ahead of data flits to reserve buffer space and channel bandwidth along the communication path. By performing routing and arbitration decisions before the arrival of data, this approach enables data flits to be forwarded immediately upon arrival, eliminating unnecessary waiting time and improving buffer utilization efficiency as shown in **(Figure 2.10).**

Compared to conventional flow-control mechanisms, flit-reservation significantly enhances network performance by increasing achievable throughput and reducing latency while requiring fewer buffer resources. These characteristics make it particularly well suited for high-speed and on-chip interconnection networks, where efficient resource management and low-latency communication are critical to overall system performance.

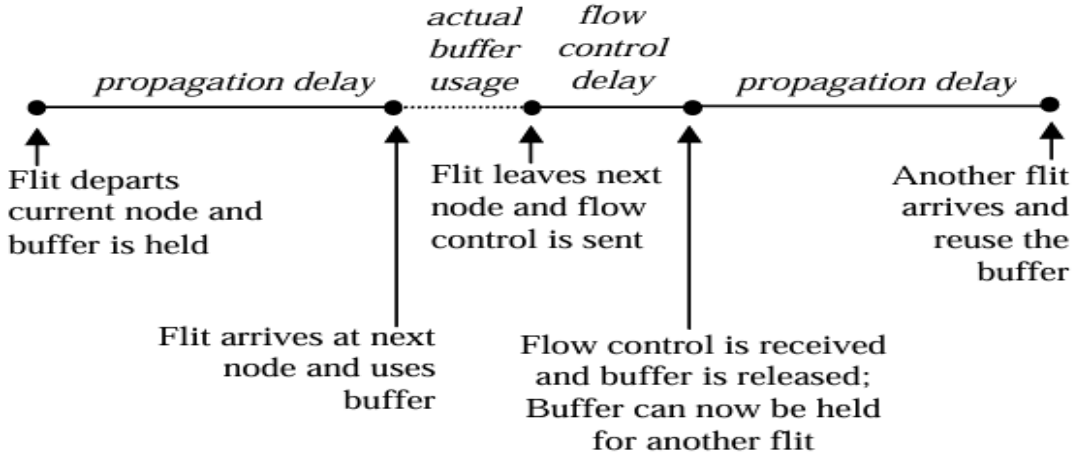


Figure 2.10 FLIT reservation flow [17]

2.4 Studies About PAM4 Modulation

According to the PCI-SIG educational webinar **"PCIe 6.0 Specification: The Interconnect for I/O Needs of the Future**", presented by Debendra Das Sharma, PCIe 6.0 introduces a fundamental shift in physical-layer signaling through the adoption of PAM4 modulation to overcome the limitations of NRZ signaling at very high data rates [13].

As PCI Express continues to evolve to support the rapidly growing data-intensive demands of modern computing systems, the transition to PAM4 (Pulse Amplitude Modulation with four levels) in PCIe 6.0 represents one of the most significant changes in the physical layer signaling methodology. Previous generations of PCIe (Gen1 through Gen5) employed NRZ (Non-Return-to-Zero) signaling, also known as PAM2, which encodes one bit per symbol using two distinct voltage levels. With each generation doubling the data rate — reaching 32 GT/s in PCIe 5.0 — NRZ signaling approached its practical limits due to channel loss, increasing noise susceptibility, jitter, and the complexity required for equalization at higher symbol rates. The PCIe 6.0 specification therefore adopted PAM4 to achieve a raw data rate of 64 GT/s per lane while keeping the effective Nyquist frequency comparable to that of PCIe 5.0, thereby mitigating excessive channel loss at higher frequencies.

PAM4 signaling encodes two bits per unit interval (UI) by using four distinct voltage levels within the same period. These four levels produce three signal "eyes" within each UI, enabling higher throughput without proportionally increasing the frequency of transitions compared to NRZ signaling as can be seen in **(Figure 2.11)**. However, the reduced separation between voltage levels makes PAM4 inherently more susceptible to noise and timing errors, as each eye has smaller voltage and timing margins. To address this increased vulnerability, the PCIe 6.0 specification incorporates Gray coding, which maps bit pairs such that a single symbol error results in at most a one-bit error, thereby improving robustness against noise. Additionally, precoding techniques are applied prior to transmission to reduce the impact of error bursts and enhance the effectiveness of the receiver's processing.

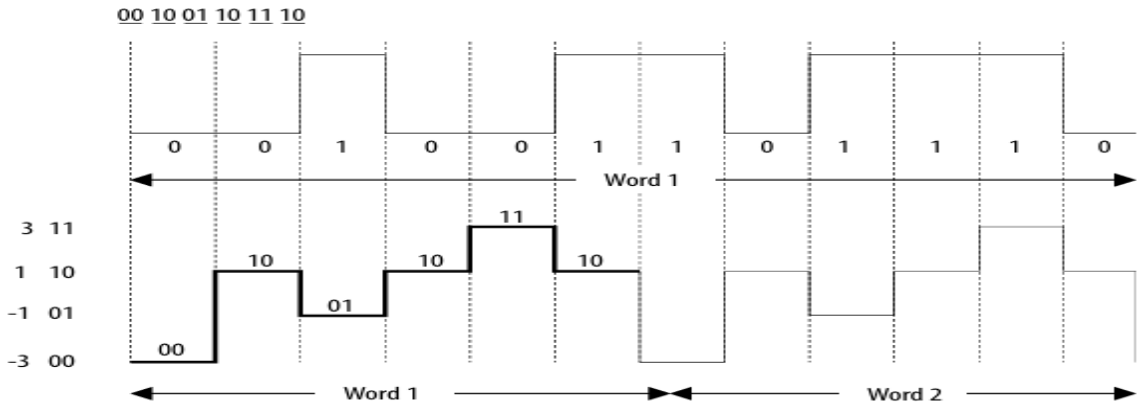


Figure 2.11 The new modulation technique (PAM-4) simulation [18]

Because PAM4 increases the likelihood of symbol errors compared to NRZ, PCIe 6.0 also integrates **Forward Error Correction (FEC)** in conjunction with PAM4. FEC enables the receiver to identify and correct certain bit errors without requiring retransmission, which is critical at high speeds where multiple adjacent signal levels must be reliably distinguished. These physical layer enhancements — PAM4, Gray coding, precoding, and FEC — fundamentally change the physical layer design, requiring more sensitive receiver architectures, advanced equalization techniques, and refined clock and data recovery algorithms to maintain reliable operation at 64 GT/s.

Another reference paper '**The Impact of PAM4 on PCIe 6.0 Electrical Testing'** published in February 2023 discusses the electrical and verification implications of introducing four-level pulse amplitude modulation (PAM4) signaling in PCI Express Gen6. It provides an in-depth technical overview of why PAM4 was adopted, how it differs from previous NRZ signaling, and the resulting challenges in signal integrity, jitter tolerance, and electrical compliance testing.[18]

PAM4 signaling enables PCIe Gen6 to double the data rate to 64 GT/s without increasing channel bandwidth by transmitting two bits per unit interval instead of one. Unlike NRZ signaling, which produces a single eye diagram, PAM4 generates three distinct eye openings with reduced voltage margins as shown in **(Figure 2.12).** As a result, PAM4 signals are more sensitive to noise, jitter, and distortion, requiring significantly tighter electrical tolerances. The reduced eye height and width place greater demands on transmitter quality and receiver sensitivity, making accurate signal measurement and control essential for reliable operation at ultra-high speeds.

To address these challenges, PCIe Gen6 introduces enhanced equalization techniques and new electrical measurements specifically tailored for PAM4. These include stronger continuous-time linear equalization (CTLE), an increased number of decision feedback equalizer (DFE) taps, and new metrics such as signal-to-noise-and-distortion ratio (SNDR) and ratio of level mismatch (RLM). Additionally, PAM4 jitter analysis requires more complex algorithms due to multiple voltage-level transitions, as slower transitions are more susceptible to timing variation. Together, these changes ensure that PAM4-based PCIe Gen6 links can maintain reliability and low latency despite the increased signaling complexity.

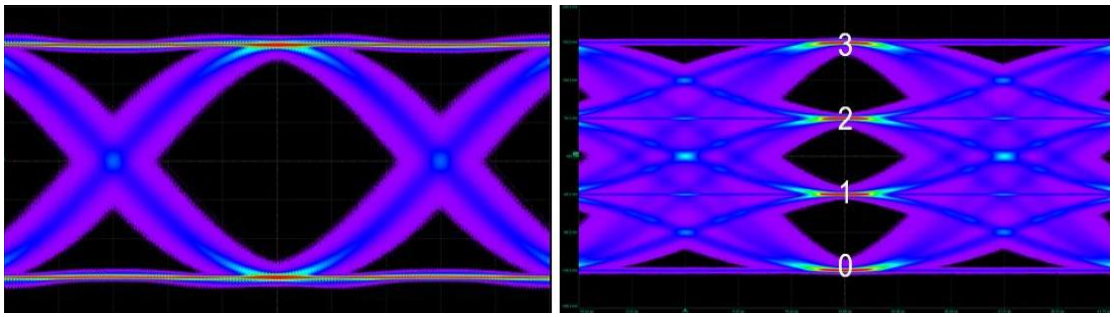


Figure 2.12 NRZ signal with one eye per UI, left, and PAM4 signal with three eyes, right. [19]

Overall, the transition to PAM4 in PCIe 6.0 balances the need for higher data rates with practical channel limitations and error resilience mechanisms. This evolution reflects a broader strategy within the PCI-SIG community to extend PCIe performance while preserving backward compatibility and acceptable signal integrity across real-world channel conditions.

2.5 Comparison Between NRZ and PAM4 in PCIe Generations

A major architectural shift is introduced in PCIe Gen6 through the transition from traditional NRZ (PAM2) signaling to four-level Pulse Amplitude Modulation (PAM4). According to the PCI-SIG technical analysis presented in "**PCIe 6.0 Specification: The Interconnect for I/O Needs of the Future**", this transition was driven by the need to double the effective data rate without increasing the Nyquist frequency of the channel. While NRZ signaling encodes one bit per unit interval (UI), PAM4 encodes two bits per UI, enabling PCIe Gen6 to achieve 64 GT/s per lane while maintaining a bandwidth profile comparable to PCIe Gen5 operating at 32 GT/s.[13],

The same study highlights several critical challenges associated with PAM4 signaling. Because PAM4 utilizes four voltage levels instead of two as shown in **(Figure 2.13)**, the vertical eye opening is reduced to approximately one-third of that of NRZ, significantly increasing susceptibility to noise, jitter, crosstalk, and inter-symbol interference (ISI). Consequently, the raw Bit Error Rate (BER) of PAM4 links is inherently higher than that of NRZ, making reliable error-free transmission infeasible without additional protection mechanisms.

Furthermore, the reduced voltage margins impose stringent requirements on the physical channel. Even minor channel impairments, PCB losses, or impedance discontinuities can lead to severe signal distortion. The PCI-SIG analysis emphasizes that PAM4 operation necessitates advanced equalization techniques, including Feed-Forward Equalization (FFE), Decision Feedback Equalization (DFE), and Continuous-Time Linear Equalization (CTLE). In addition, PAM4 demands highly sensitive receiver architectures capable of accurately resolving closely spaced voltage levels under high-speed operating conditions.

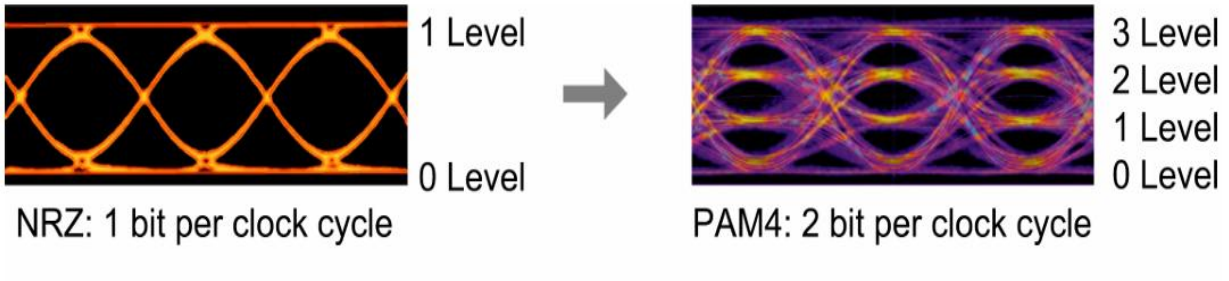


Figure 2.13 NRZ vs PAM4 [19]

Due to the inherently higher error probability introduced by PAM4, Forward Error Correction (FEC) is mandated as a fundamental component of PCIe Gen6. This lightweight FEC scheme is specifically optimized to correct symbol-level errors caused by PAM4 while maintaining low latency, which is essential for preserving PCIe's performance characteristics. Moreover, the adoption of FLIT-based framing in Gen6 complements FEC operation by providing fixed-size data units that align efficiently with error detection and correction processes.

Overall, the literature consistently indicates that the transition to PAM4 in PCIe Gen6 represents not merely an incremental performance enhancement but a fundamental transformation of the physical layer. This shift introduces a more complex multi-level signaling paradigm, requiring new strategies in equalization, error correction, channel design, and verification. As a result, PAM4 is regarded as one of the most technically demanding aspects of PCIe Gen6 development.

2.6 Studies About FEC + CRC in High-Speed Links

specification documents and industry-level technical analyses. A primary reference is the PCI-SIG educational webinar "**PCIe 6.0 Specification: The Interconnect for I/O Needs of the Future**", presented by Debendra Das Sharma, which explains the architectural motivations behind introducing) and enhanced CRC mechanisms in PCIe Gen6. Complementary insight is provided by an industry analysis published by Cadence Design Systems, titled "Demystifying Forward Error Correction (FEC) in PCIe 6.0", which focuses on the practical implications of FEC deployment in real high-speed designs.[13]

According to the PCI-SIG specification analysis, the transition to 64 GT/s per lane using PAM4 signaling significantly increases the raw Bit Error Rate (BER) due to reduced voltage margins and higher sensitivity to noise, jitter, and channel impairments. Under these conditions, traditional error handling approaches used in earlier PCIe generations—primarily CRC detection followed by packet retransmission—are no longer sufficient to guarantee reliable operation while maintaining low latency. As a result, PCIe Gen6 mandates the use of Forward Error Correction in conjunction with CRC to preserve link reliability and performance.

The PCI-SIG study emphasizes that the effectiveness of this approach is evaluated using metrics such as retry probability, Failure-In-Time (FIT) rate, and overall bandwidth efficiency. Rather than relying on strong but latency-intensive error correction, PCIe Gen6 adopts a lightweight, low-latency FEC scheme that corrects common symbol errors introduced by PAM4 while allowing rare retransmissions to handle uncorrectable errors. This design choice ensures that latency-sensitive applications are not negatively impacted by constant error correction overhead.

From **(Figure 2.14)** an industry perspective, the Cadence analysis further clarifies why FEC is indispensable in PCIe Gen6 implementations. It highlights that PAM4 signaling inherently produces a higher BER compared to NRZ, making error correction mandatory for practical system deployment. The Cadence study explains that without FEC, achieving acceptable error rates at 64 GT/s would require prohibitively strict channel requirements and significantly increased signal integrity complexity. By integrating FEC, PCIe Gen6 enables reliable operation over realistic PCB channels while maintaining high throughout and manageable design costs.

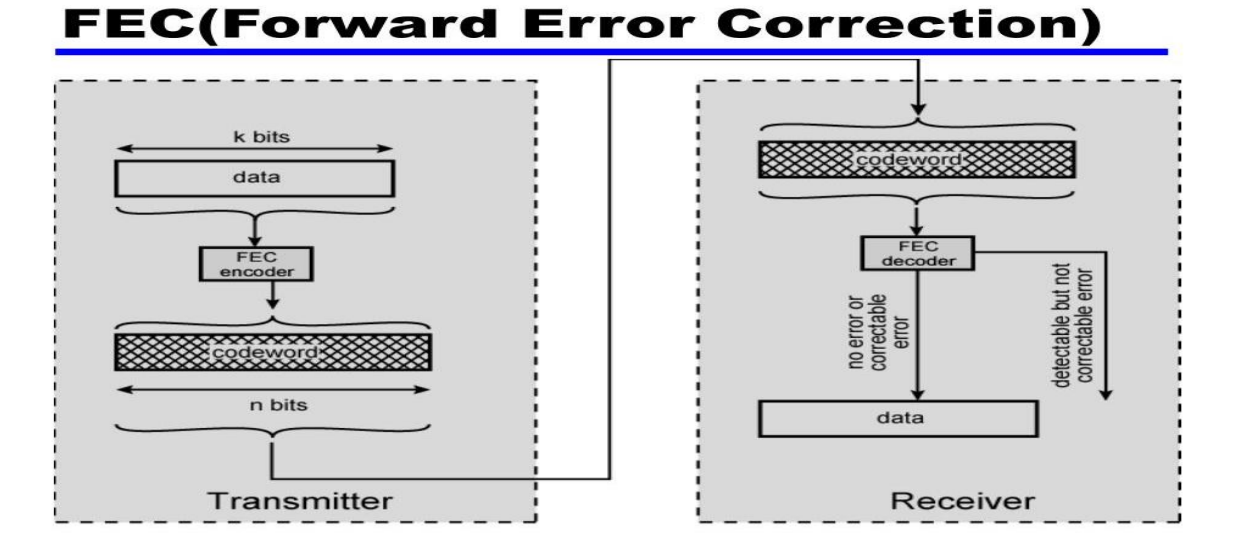


Figure 2.14 Forward Error Correction Process in PCIe [24]

2.7 Previous PCIe Implementations

Previous PCIe implementations, particularly those targeting **PCIe Gen3, Gen4, and PCIe Gen5**, have been widely explored in both academic research and industry documentation. Numerous open-source and commercial projects provide RTL models, verification environments, architectural analyses, and signal-integrity studies for these generations. As a result, design information for PCIe Gen4 and Gen5 is relatively accessible compared to newer generations. Several FPGA-based implementations of PCIe endpoints and root complexes have been reported, with a primary focus on **NRZ-based signalling**, protocol handling, and data-movement efficiency.

Representative examples of prior work include FPGA-based PCIe DMA architectures, such as the implementation presented in **[19]**, which demonstrates a high-throughput PCIe design optimized for efficient data transfer between an FPGA and a host system.

This class of work primarily addresses protocol-level behaviour and system-level performance using earlier PCIe generations, without considering advanced physical-layer techniques such as PAM4 modulation, Forward Error Correction (FEC), or FLIT-based framing.

Despite the abundance of literature and implementations for PCIe Gen3 through Gen5, there is a clear lack of dedicated, fully verified digital implementations of PCIe Gen6. Existing publications typically discuss PCIe Gen6 only at a theoretical or architectural level, without presenting complete digital models, RTL implementations, or comprehensive verification results that incorporate the fundamental changes introduced in Gen6.

A broader review of previous PCIe **(Table 2.2)** implementations further reveals that most designs focused on logical modelling and protocol functionality rather than the physical-layer challenges of ultra-high-speed links. Even FPGA-based Gen4 and Gen5 implementations reported significant timing-closure and signal-integrity challenges at 16 GT/s and 32 GT/s, highlighting the increasing difficulty of implementing PCIe systems as data rates scale.

Table 2.2 PCIe Lanes Implementation [13]

Specifications	Lanes				
	x1	x2	x4	x8	x16
2.5 GT/s (PCIe 1.x +)	500 MB/S	1 GB/S	2 GB/S	4 GB/S	8 GB/S
5.0 GT/s (PCIe 2.x +)	1 GB/S	2 GB/S	4 GB/S	8 GB/S	16 GB/S
8.0 GT/s (PCIe 3.x +)	2 GB/S	4 GB/S	8 GB/S	16 GB/S	32 GB/S
16.0 GT/s (PCIe 4.x +)	4 GB/S	8 GB/S	16 GB/S	32 GB/S	64 GB/S
32.0 GT/s (PCIe 5.x +)	8 GB/S	16 GB/S	32 GB/S	64 GB/S	128 GB/S
64.0 GT/s (PCIe 6.x +)	16 GB/S	32 GB/S	64 GB/S	128 GB/S	256 GB/S
128.0 GT/s (PCIe 7.x +)	32 GB/S	64 GB/S	128 GB/S	256 GB/S	512 GB/S

In summary, while earlier PCIe generations have been extensively implemented and studied, there is no publicly available academic or open-source work provides a complete digital PCIe Gen6 model integrating PAM4 signalling, FEC, and FLIT mode. This clear research gap motivates the focus of this project, which aims to propose and implement an educational yet comprehensive PAM4-based digital PCIe Gen6.0 model, addressing an area not covered by prior implementations.

2.8 Verification Challenges in PCIe Gen6

Verification complexity in PCIe Gen6 has increased significantly due to the combined impact of several major architectural changes, including PAM4 signaling, FLIT-based packet framing, lightweight Forward Error Correction (FEC), and updated Data Link Layer (DLL) mechanisms. Recent literature, such as *"***PCIe Verification at High Speeds: Challenges, Solutions, and Future Trends***"*, emphasizes that verifying modern PCIe interfaces now requires environments capable of validating both digital protocol correctness and high-speed signaling behavior, creating a verification challenge that did not exist in earlier PCIe generations.[16]

One of the primary challenges highlighted is the verification of PAM4 signaling behavior. PAM4 introduces higher raw Bit Error Rates (BER), reduced vertical eye openings, and strong dependence on equalization techniques. Traditional digital-only verification environments are insufficient to accurately model these effects, making it difficult to reproduce real-world error scenarios. Consequently, verification methodologies must include mechanisms to inject realistic PAM4-related impairments, such as symbol errors, timing jitter, and voltage-level distortions, into the digital simulation flow.

The introduction of FLIT Mode further increases the verification complexity. Since FLITs are fixed-length structures that may encapsulate multiple TLPs, DLLPs, and control information, verification must ensure correct segmentation, packing, alignment, and reassembly under all traffic conditions. Even small errors in FLIT boundary handling or ordering logic can lead to subtle data corruption or protocol violations that are difficult to detect through conventional testing.

In addition, the interaction between CRC checking and lightweight FEC introduces new corner cases that must be carefully verified. For example, corrected bit errors must not invalidate CRC checking logic, and packets that exceed the correction capability of FEC must be correctly detected and retried. Validating these behaviors requires extensive stimulus coverage, sophisticated score boarding, and precise monitoring of retry and replay mechanisms.

Another challenge identified in existing studies is the lack of unified verification frameworks that integrate PAM4 modeling, FEC behavior, FLIT encapsulation, and sequencing logic into a single coherent environment. Most published research addresses only individual aspects of PCIe Gen6, leaving a gap in comprehensive verification approaches, particularly for educational and early-stage prototyping purposes.

Overall, the literature consistently concludes that PCIe Gen6 verification is substantially more demanding than previous generations, as it combines analog-level signaling effects introduced by PAM4 with increased protocol-level complexity from FLIT-based framing and FEC. These challenges strongly motivate the development of simplified, educational digital models that preserve the fundamental behavior of PCIe Gen6 while remaining verifiable within a purely digital design and simulation environment—an objective directly addressed by the model proposed in this project.

2.9 Identified Research Gaps

This section presents a detailed analysis of the identified research gaps in PCIe technology based on the conducted literature review. The identified gaps are categorized into key subtopics to provide a structured and clear understanding of the areas that still require further investigation.

Although existing research on PCIe Gen6 provides a strong theoretical foundation—covering topics such as the transition to PAM4 signaling, the introduction of FLIT-based framing, and the role of low-latency Forward Error Correction (FEC)—the literature still lacks several essential elements required for practical understanding and educational use. Most published work focuses primarily on architectural concepts, protocol modifications, electrical challenges, and expected performance improvements. However, none of the reviewed studies present a complete, step-by-step educational model that demonstrates how PCIe Gen6 operates across all protocol layers.

Specifically, there is no full system-level simulation that clearly illustrates the interaction between the Transaction Layer, Data Link Layer, and Physical Layer in a way that allows students or early-stage researchers to visualize packet flow, FLIT encapsulation, error detection and correction, replay mechanisms, and link behavior.

In addition, despite the critical role of the PAM4-based Physical Layer in PCIe Gen6, there is no simplified digital Physical Layer model that abstracts complex analog behaviors into a form suitable for education, experimentation, or basic functional verification.

This gap significantly limits the ability of students and early-stage researchers to practically explore PCIe Gen6 behavior, validate theoretical assumptions, or evaluate architectural decisions through simulation. Addressing this limitation is a primary motivation for this project, which proposes a comprehensive educational modeling approach. The proposed model includes a simplified digital Physical Layer, a multi-layer system-level simulation, and clear demonstrations of key Gen6 mechanisms such as PAM4 signaling abstraction, FLIT processing, and FEC operation—combined into an accessible and practical learning tool.

In addition to the gaps described above, the literature also reveals the absence of a unified verification framework that simultaneously integrates PAM4 modulation, FLIT encapsulation, FEC decoding, and multi-layer sequencing behavior. Most existing studies examine these components in isolation, leaving a gap in understanding how they interact within a complete PCIe Gen6 system.

Furthermore, there is no academic-level performance estimation model for PCIe Gen6 that enables systematic analysis of throughput, latency, retry probability, and the impact of FEC. This project addresses these limitations by introducing a complete digital model capable of simulating the interaction between all three PCIe layers and their Gen6-specific mechanisms, thereby contributing both educational and practical value to the existing body of research.

3 Chapter Three: Proposed Methods

This chapter describes the design methodology for a simplified digital model of PCIe Gen6, structured into three layers: Transaction, Data Link, and Physical. The system block diagram illustrates the data flow from the Transaction Layer through the Data Link and Physical Layers, across the simulated channel between transmitter and receiver. Each layer's functionality is detailed, with the diagram highlighting how TLPs and DLLPs are processed, encapsulated into FLITs, and transmitted using PAM4 encoding. The chapter also presents the simulation plan, covering unit- and system-level testing, with emphasis on validating layer interactions and overall system performance.

3.1 System Architecture Overview

The proposed system represents a simplified digital model of a PCIe Gen6 communication link. The design follows the standard PCI Express layered architecture, which is composed of three main layers: Transaction Layer, Data Link Layer, and Physical Layer (Digital Model). The system is divided into two main sides: **the Transmitter side and the Receiver side.**

Between the two sides, a **simulated communication channel** is placed at the Physical Layer level to represent the physical connection between the transmitter and the receiver.

In this project, the Physical Layer is not implemented as a real analog hardware interface. Instead, it is modeled as a **behavioral digital layer** that simulates the main functions of a PCIe Gen6 physical interface, including FLIT formation, error protection, and PAM4 logical encoding.

The overall data flow in the system is as follows:

Transaction Layer → Data Link Layer → Physical Layer (FLIT + FEC + PAM4) → Channel → Physical Layer → Data Link Layer → Transaction Layer

This structure reflects the actual concept of PCIe Gen6, where Transaction Layer Packets (TLPs) and Data Link Layer Packets (DLLPs) are encapsulated into fixed-length FLITs at the Physical Layer before being transmitted over the physical channel. To ensure a modular and scalable framework, the system architecture applies **layer isolation principles**, meaning each layer is designed, simulated, and verified independently before integration.

The architecture, as shown in **(Figure 3.1)**, also incorporates **internal debug interfaces** that allow waveform inspection and event tracing during simulation. These debugging signals function similarly to the embedded logic analyzers used in commercial PCIe controllers, enabling step-by-step verification of TLP routing, sequence numbering, FEC operation, and PAM4 encoding.

3.2 Block Diagram of the Whole System

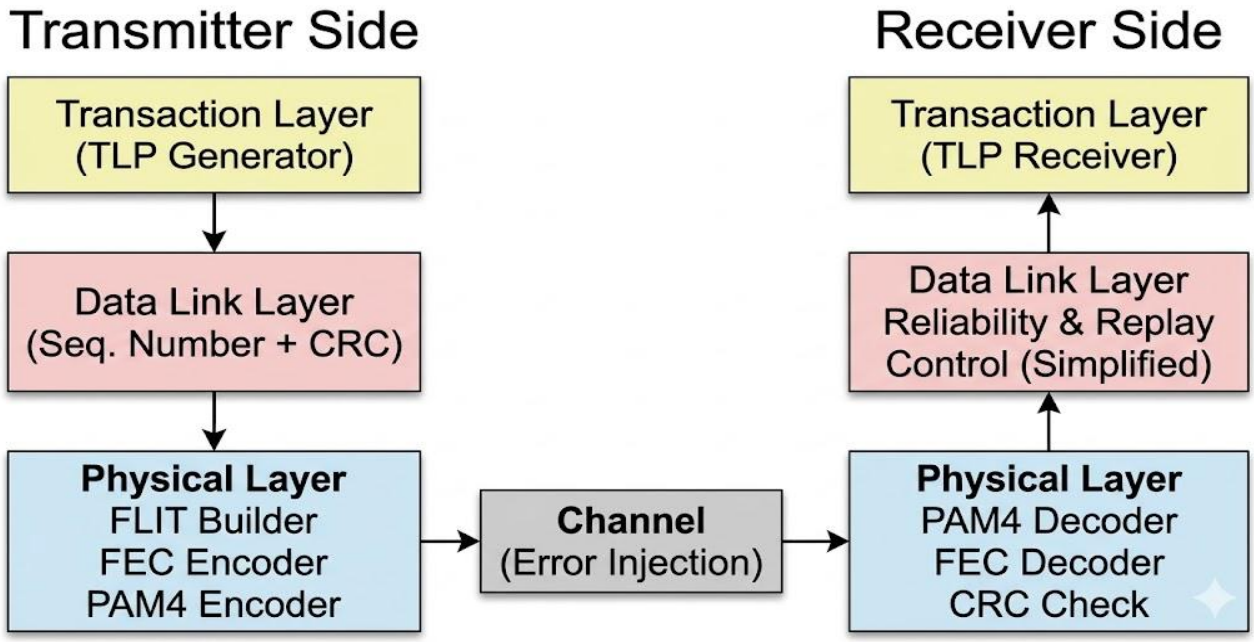


Figure 3.1 Simplified Block Diagram of PCIe Gen 6

The proposed PCIe Gen6 digital model follows a structured three-layer architecture that faithfully reflects the real PCI Express protocol while maintaining a simplified digital environment suitable for academic simulation. The system is divided into a transmitter side and a receiver side, separated by a behavioral digital channel. Each side contains the Transaction Layer, the Data Link Layer, and the Physical Layer, and the data flows sequentially through these layers in a complete end-to-end pipeline. The goal of this architecture is to model how PCIe Gen6 handles packet creation, reliability control, fixed-size FLIT framing, FEC protection, PAM4 logical encoding, lane bonding, lane alignment, and eventual TLP reconstruction at the receiver.

Data flow begins at the **Transmitter Transaction Layer**, where Transaction Layer Packets (TLPs) are constructed. The Transaction Layer generates packet headers, payloads, addressing information, and attributes that define the type and properties of each transaction. These TLPs are then passed downward to the Data Link Layer using a clean and structured interface. At this stage, the packets are still in their pure logical form, without framing or encoding, and represent the highest conceptual level of the PCIe communication path.

Upon entering the **Transmitter Data Link Layer**, each packet assigns a sequence number that uniquely identifies its position within the stream. This sequence number will later be used by the receiver to detect missing, duplicated, or out-of-order packets. The Data Link Layer also implements reliability mechanisms through the replay buffer, which stores outgoing TLPs until they are positively acknowledged by the receiver. Error detection mechanisms are applied by generating a Link CRC (LCRC) or a simplified CRC in accordance with the digital model. Once the packet is protected and numbered, it forms a complete DLL-formatted unit that is prepared for FLIT encapsulation in the Physical Layer.

The **Transmitter Physical Layer** is the core of the PCIe Gen6 behavior implemented in this project. Here, the packet is placed inside a fixed 256-byte FLIT, in FLIT mode, we are using a completely new TLP Header format. Previous TLP Headers had many limitations, like no room for increasing tag size. PCI-SIG redesigned header to suites suited FLIT mode. The challenge will be to test all the new combinations. TLP Header is composed of a 3 to 7 DW TLP header Base, followed by 0 to 7 additional DWs of OHC (Orthogonal Header Content) as shown in **(Figure 3.2)**.

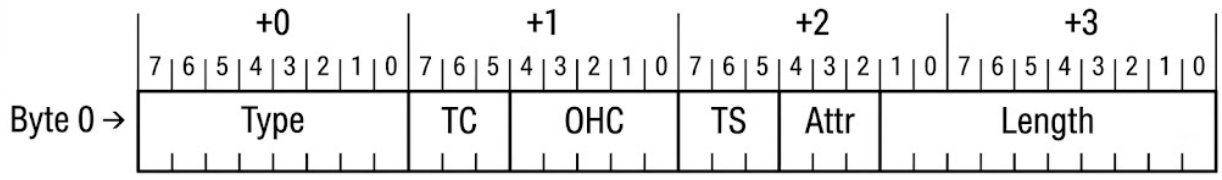


Figure 3.2 First DW of Header Base [20]

The new type of field has a fully decoded 8b Packet Type field. This means that all the 256 Type values are defined or reserved for a certain group to permit proper framing and forwarding. Also, there are new completion rules designed for FLIT mode. The completion for non-posed TLPs also has major updates including a 14-bit tag, error report using OHC-A5, etc. as shown in (**Figure 3.3)**.

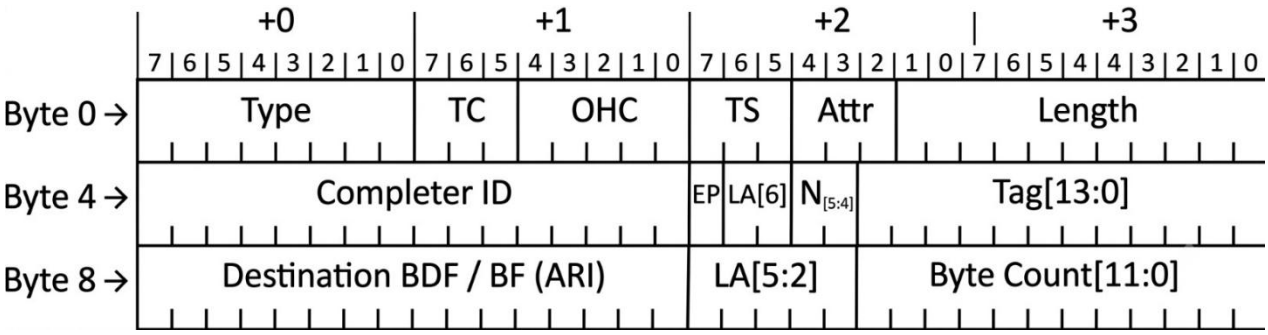


Figure 3.3 Completion Header Base Format - Flit mode [20]

The mandatory transmission format introduced in PCIe Gen6. The **FLIT Builder** collects TLPs and DLLPs, segments them if necessary, and assembles them into properly formatted FLITs that contain headers, metadata, control fields, and optional padding. Following **FLIT** construction, a lightweight **Forward Error Correction (FEC)** encoder is applied. This encoder generates parity bits that allow the receiver to correct symbol-level errors caused by the channel, which is especially important in Gen6 due to the transition to **PAM4** signaling. After that, the binary stream undergoes digital **PAM4** mapping, where every two bits are encoded into a single four-level symbol using gray coding, imitating the multi-level modulation used in real PCIe Gen6 links.

A crucial part of the Physical Layer is the **Lane Bonding mechanism**, which distributes the PAM4 symbols across the available lanes (x1, x4, x8, x16). The transmitter stripes the FLIT data across multiple lanes in parallel, ensuring maximum throughput and correct alignment with the link width. Lane Bonding includes several digital operations such as lane ordering, lane indexing, and the generation of alignment markers so that the receiver can reassemble the data correctly even if lanes experience different delays. Each lane carries its corresponding symbol stream toward the digital channel model, completing the behavior of the Transmitter Physical Layer.

The **digital channel** represents the path between transmitter and receiver but is modeled entirely using digital logic. It introduces controlled impairments such as bit errors, symbol flips, random delay, and lane-to-lane skew. These impairments allow the FEC, CRC, disked logic, and replay mechanism to be tested in realistic conditions without simulating full analog signal integrity. The channel ensures that the system is stress-tested at the level expected of a PCIe Gen6 behavioral model.

On the receiving side, data first enters the **Receiver Physical Layer,** where the reverse operations occur. The incoming lanes are realigned using lane deskin buffers, which ensure that all lane data is synchronized before reassembling FLITs. Lane Bonding is applied in reverse, allowing the receiver to identify lane indices, correct lane ordering, and handle polarity inversion if necessary. After lane alignment, PAM4 logical decoding converts each symbol back into two binary bits, reconstructing the original serialized stream. FEC decoding is then applied to correct symbol-level errors injected by the channel. Once the error-corrected stream is available, the FLIT Parser extracts TLPs, DLLPs, metadata, and control information from the 256-byte FLIT structure and passes them upward to the Data Link Layer.

The **Receiver Data Link Layer** then verifies the sequence number to maintain correct packet ordering and detect missing transmissions. It checks the CRC or LCRC to confirm packet integrity. If a packet fails to verification or contains uncorrectable FEC errors, a NAK is generated to trigger replay from the transmitter's replay buffer. If the packet is valid, an ACK is issued, and the replay buffer entry at the transmitter is safely released. This ensures full reliability of the communication channel, maintaining the fundamental operational behavior of PCIe links.

Finally, the packets reach the **Receiver Transaction Layer,** where the TLPs are fully reconstructed. The Transaction Layer parses the header, extracts the payload, and delivers the completed request or completion to the upper system logic. This marks the end of the complete **TX-to-RX pipeline**, demonstrating how PCIe Gen6 packets are generated, transmitted, protected, encoded, striped, deskewed, corrected, verified, and finally delivered—all inside a coherent digital simulation environment.

3.3 Decision Feedback Equalization (DFE)

Decision Feedback Equalization (DFE) as shown in **(Figure 3.4)** is a technique used to reduce the impact of inter symbol interference (ISI) on high-speed communication channels. **ISI** occurs when previously transmitted symbols leave residual effects that distort the current symbol, making it harder for the receiver to correctly interpret the incoming data.

The **DFE** method addresses this issue by using the decisions made on previously detected symbols. These past decisions are fed back into the equalizer to estimate the amount of ISI expected from earlier symbols. The estimated interference is then subtracted from the incoming signal, effectively "cleaning" the waveform before deciding the present symbol. By removing this residual distortion, DFE significantly improves the accuracy of symbol detection.

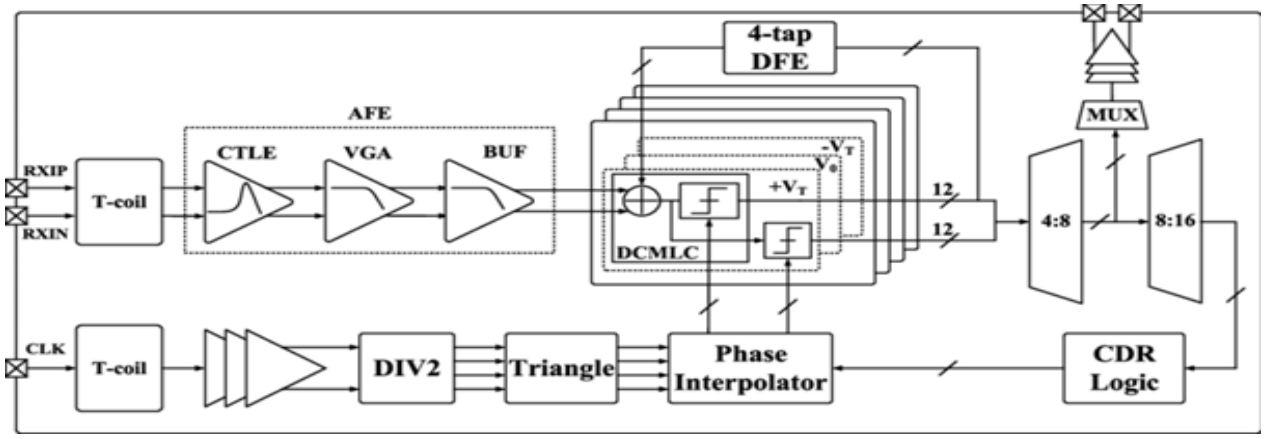


Figure 3.4 Overall DFE Block Diagram (Core Architecture) [21]

Validating **DFE** for PCIe 6.0 requires verifying that the equalization technique performs correctly at extremely high data rates. The validation process ensures that DFE can reliably compensate for ISI under the fast and complex signaling conditions of the PCIe 6.0 channel. It also confirms that equalization does not introduce additional noise, distortion, or instability that could degrade link performance.

DFE behavior depends heavily on previous symbol decisions and channel conditions; accurate modeling is essential. The validation must capture the dynamic nature of the feedback process to ensure that DFE consistently mitigates ISI without generating unwanted side effects. This detailed modeling helps guarantee that the DFE operates effectively and supports the stringent performance requirements of PCIe 6.0.

3.3.1 Input Signal from the Channel (ISI)

At very high speeds (PCIe Gen 6 PAM4), symbols overlap in time, this overlap is called Inter-Symbol Interference (ISI), The current symbol is distorted by previous symbols. Without equalization, correct symbol detection becomes difficult. **(Figure 3.5).**

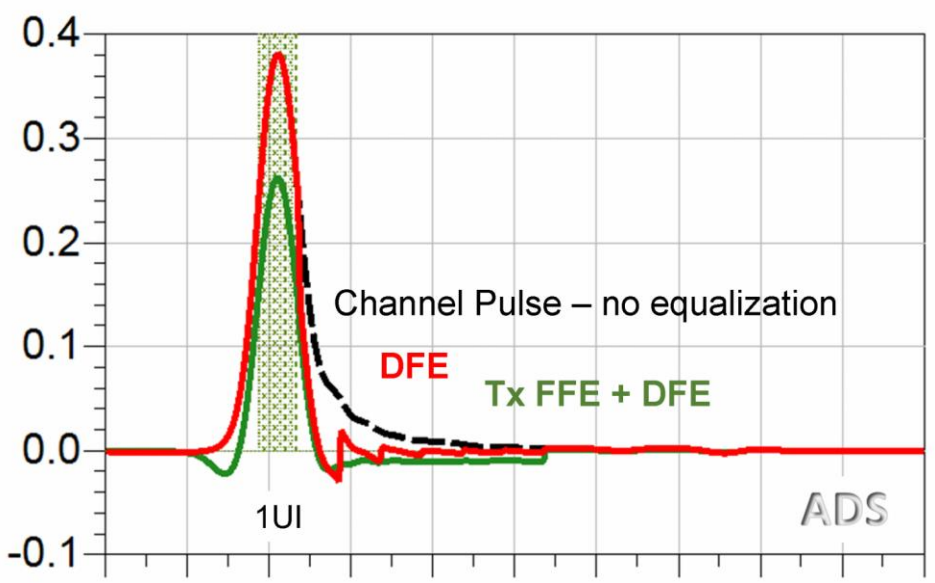
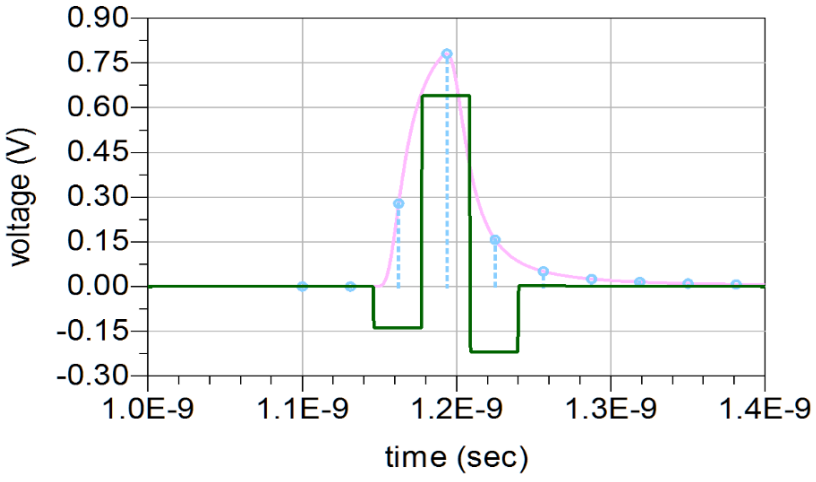


Figure 3.5 ADS IBIS simulation of a pulse response with and without equalization [22].

3.3.2 Feed-Forward Equalizer (FFE)

The **FFE is a linear filter, it** mitigates **pre-cursor ISI** (interference coming *before* the current symbol), It improves the signal shape **before slicing,** FFE alone cannot remove **post-cursor ISI**, which is why DFE is needed:

Figure 3.6 FFE inserts pulses of negative amplitudes to cancel out ISI that has positive amplitude.[21]



3.3.3 Decision Device (Slicer)

PAM4 receiver with adaptive threshold voltage, adaptive decision feedback equalizer, and fixed linear equalizer have been presented. The proposed techniques enable threshold voltage to be adjusted automatically depending on data rate, signal swing, and loss of channel. Consequently, the receiver can be used in various situations without being manually calibrated. Adaptive decision feedback equalizer for PAM4 signaling is proposed, incorporated with a sign-sign least mean square algorithm. Simulation results across a lossy channel show proper convergence of threshold voltage and decision feedback equalizer values with the proposed receiver.

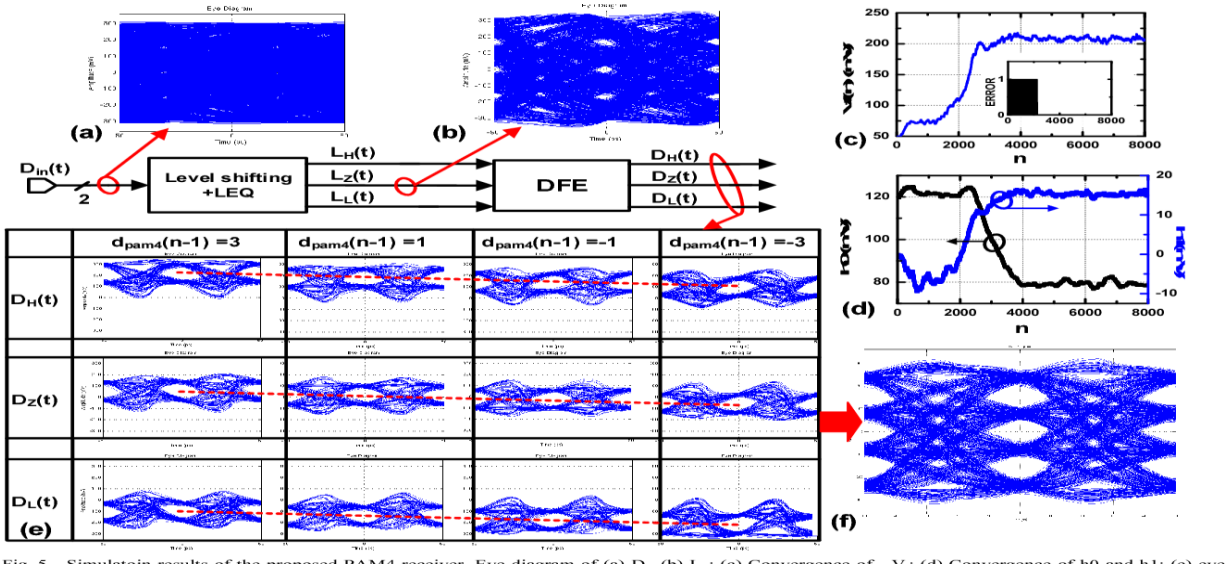


Figure 3.7 Simulation results of the proposed PAM4 receiver. [23]

Eye diagram of (a) Din (b) LL; (c) Convergence of Vt; (d) Convergence of h0 and h1; (e) eye diagrams of DH, DZ, and DL; (f) eye diagram of the reconstructed signal after DFE.

3.4 design and Simulation Plan

The design and simulation plan of the proposed PCIe Gen 6 digital model follows a structured, simulation-driven approach. The system is implemented using Verilog HDL, enabling modular and deterministic modeling of protocol components such as FLIT framing, CRC, FEC, and logical PAM4 processing. Verification is carried out using ModelSim, where both unit-level and system-level simulations are performed. Individual blocks are first tested independently using directed test vectors to ensure correct functionality, after which the complete transmitter-to-receiver pipeline is simulated to validate end-to-end behavior. Controlled error injection is used within the digital channel model to evaluate reliability mechanisms and observe system response under fault conditions. Simulation waveforms and internal signals are analyzed to extract performance metrics such as latency, throughput efficiency, and error-correction effectiveness. This approach provides a reliable and efficient method for validating the design and obtaining meaningful simulation results in an academic environment **(Figure 3.8)**.

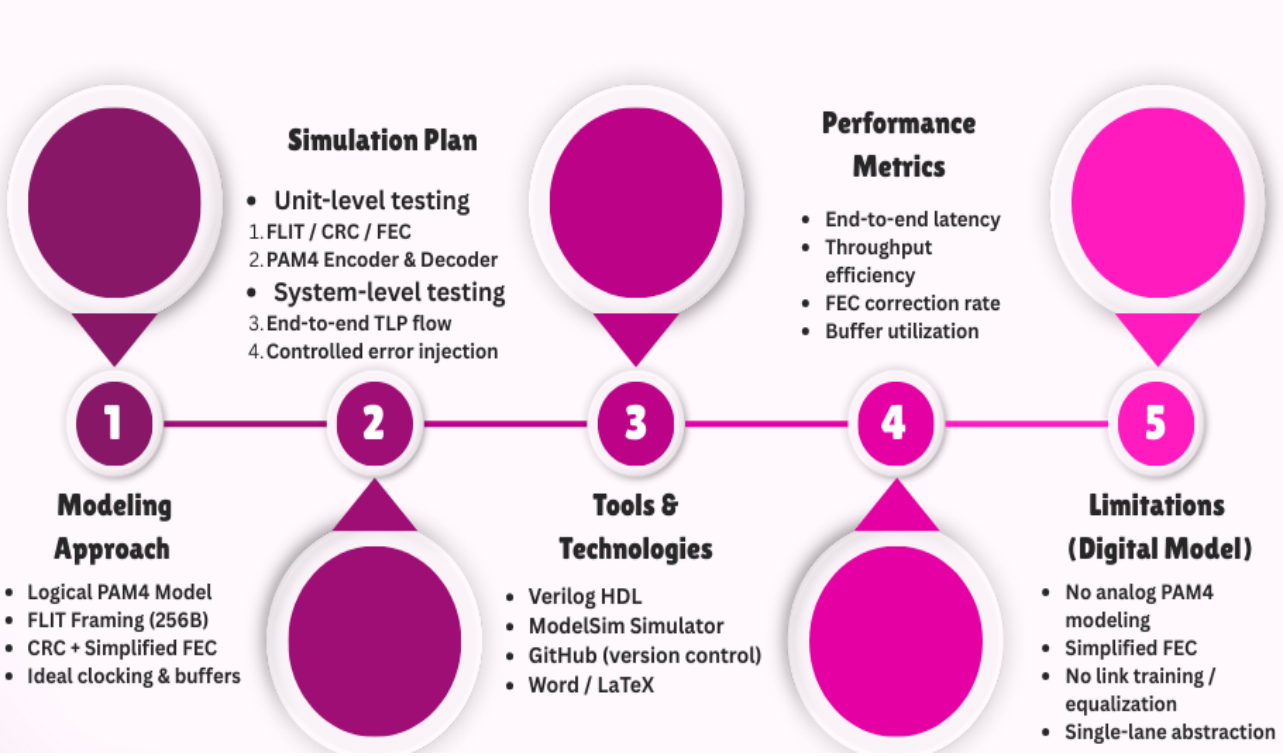


Figure 3.8 Design and Simulation Plan for the Proposed PCIe Gen6 Model

3.4.1 Modeling Approach

The proposed PCIe Gen6 digital model is developed under ensuring clarity, modularity, and feasibility within an academic environment. One of the primary roles is the abstraction of all analog behaviors typically associated with PAM4 signaling. Instead of simulating real analog waveforms, the design implements a logical PAM4 representation that focuses on symbol mapping, encoding behavior, and multi-level data transitions. This abstraction allows the model to preserve conceptual correctness without requiring complex analog simulation tools.

Another design assumption involves the use of simplified lightweight FEC architecture. While real PCIe Gen6 employs an advanced low-latency FEC optimized for high-speed links, the model uses a reduced version that maintains the core principles of error detection and correction. These constraints ensure that the design remains computationally manageable while still demonstrating the interaction between FEC, CRC, and FLIT framing.

Physical channel impairments such as jitter, inter-symbol interference (ISI), crosstalk, and channel loss are not included explicitly. Instead, controlled error injections are used to approximate their effect in a digital environment. This approach provides a balance between realism and educational simplicity. Additionally, the system assumes ideal clocking, synchronous operation, and immediate buffer availability — constraints that significantly reduce architectural overhead and keep the design focused on protocol behaviors rather than hardware implementation challenges.

3.4.2 Simulation Plan

In an extensive testbench setting. Before being integrated into the entire system, the verification process starts at the unit level, where separate functional blocks are validated to guarantee proper operation. These blocks include the FEC encoder and decoder, which provides symbol-level error correction for PAM4 signals; the PAM4 encoder and decoder, which converts binary data to four-level symbols and back; the FLIT Builder and Parser, which builds and deconstructs 256-byte FLITs; the CRC generator and checker, which detects data corruption; and the Data Link Layer reliability logic, which controls sequence numbering, replay buffers, and acknowledgment handling. To verify each block's behavior independently, it is tested using specific test vectors.

After unit-level testing, the system-level verification evaluates the complete end-to-end pipeline from transmitter to receiver. Transaction Layer Packets (TLPs) are transmitted through the behavioral digital channel, and correct data recovery is observed at the receiver. Controlled errors are injected into the channel to test the robustness of the FEC and CRC mechanisms and to verify proper operation of the replay and flow control logic. The simulation environment also measures end-to-end latency from TLP creation to reception, ensuring that buffer operations and lane alignment do not introduce excessive delays. High-throughput scenarios are simulated to evaluate buffer stability, sequencing logic, and overall system performance under stress conditions.

The impact of FLIT framing, CRC, and FEC overhead on effective throughput is analyzed to compare the simulated data rate with theoretical PCIe Gen6 expectations. This methodology provides a comprehensive verification framework that guarantees the PCIe Gen6 system maintains data integrity, responds correctly to errors, and meets performance targets, establishing a solid foundation for subsequent RTL implementation and further analysis.

3.4.3 Tools and Technologies

The development process of the PCIe Gen6 digital model relies on a set of tools and technologies commonly used in modern hardware design and verification of workflows. **Verilog HDL** is employed as the primary modeling language due to its structured syntax, hardware-orientated semantics, and suitability for modular digital design, **(Figure 3.9).** Its deterministic behavior allows precise modeling of protocol operations such as FLIT encapsulation, sequencing, and error handling.

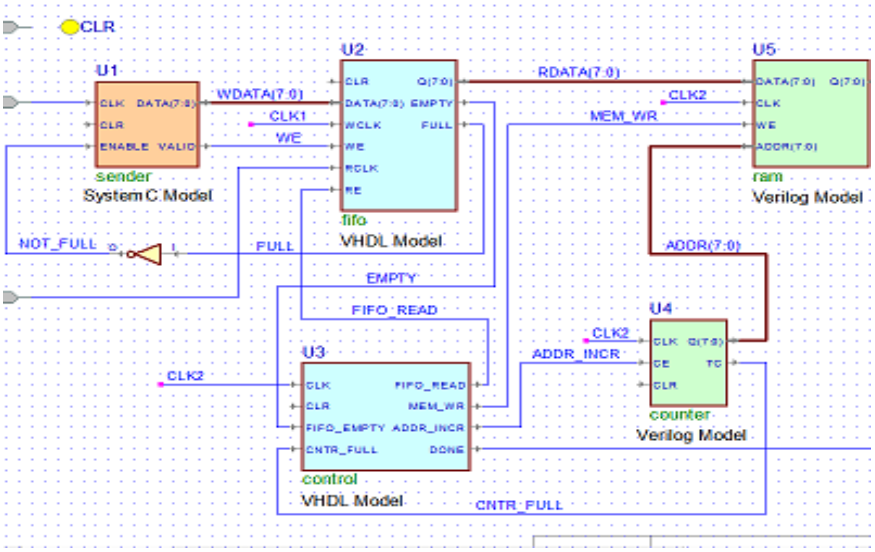


Figure 3.9 Schematic / Block Diagram Editor

Simulation and verification are conducted using **ModelSim**, an industry-standard tool that provides waveform visualization, timing analysis, and debugging capabilities. **ModelSim** allows step-by-step inspection of internal signals, replay-buffer behavior, and data flow across layers, making it ideal for protocol-level debugging.

Version control and collaboration are managed using GitHub, which ensures organized code development, tracking of design iterations, and effective teamwork. Documentation tools such as Microsoft Word and LaTeX were used to prepare project reports, ensuring professional formatting of diagrams, tables, and technical explanations, for example, Use Host Bridge to control debug a SoC using useful scripts (**Figure 3.10**).

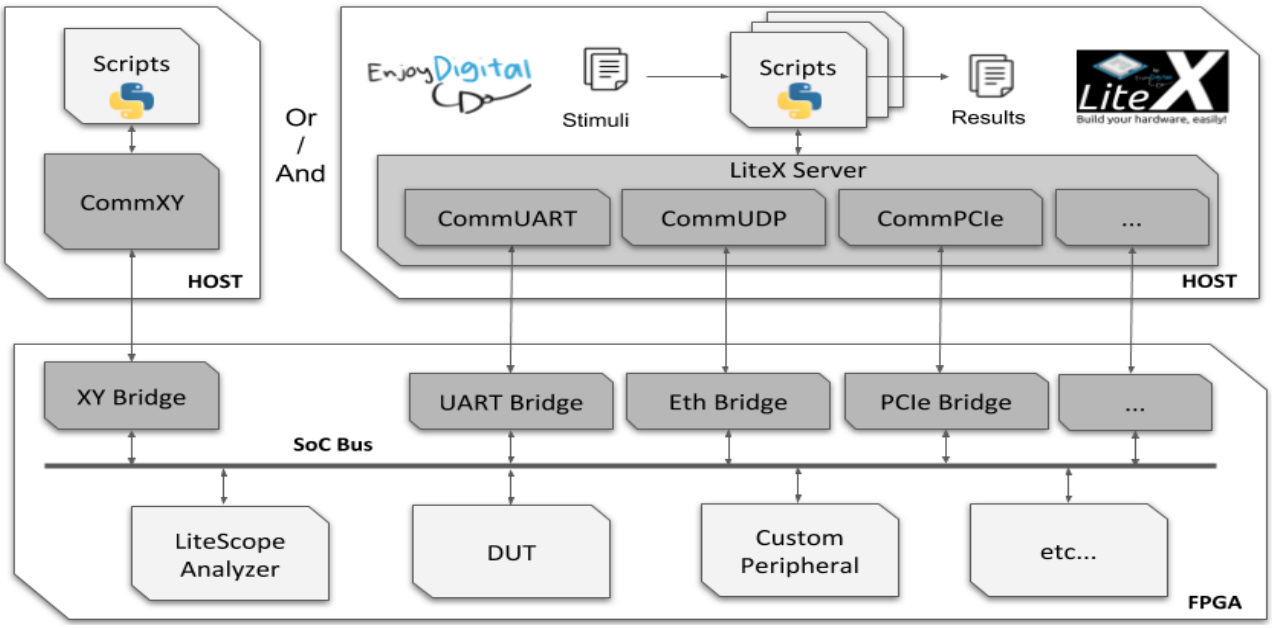


Figure 3.10 Example from GitHub

Although analog simulators such as SPICE or Cadence tools would normally be required for PAM4 waveform modeling, the constraints of this project make pure-digital tools sufficient. The toolchain used strikes a balance between practicality and academic value, providing accessible yet powerful means for understanding the complexity of PCIe Gen6.

3.4.4 Performance Metrics

To evaluate the effectiveness, correctness, and behavior of the proposed digital PCIe Gen6 model, several performance metrics are defined. The first key metric is **end-to-end latency**, which measures the number of cycles required for a packet to travel from the Transaction Layer of the transmitter to the Transaction Layer of the receiver. This metric reflects the combined effects of FLIT assembly, serialization, link traversal, buffering, and decoding.

Another metric is **throughput efficiency**, calculated as the ratio of useful payload to total transmitted bits, including overhead introduced by FLIT framing, PAM4 mapping, CRC, and FEC. This measurement is essential for comparing the model against the theoretical bandwidth of PCIe Gen6 links.

Error-correction effectiveness is also evaluated by measuring the number of injected errors that are successfully corrected by the simplified FEC module. This metric provides insight into how PAM4-related impairments affect reliability. Additionally, **sequence number integrity** is monitored to ensure correct ordering, replay behavior, and loss of handling across the Data Link Layer.

Finally, **buffer utilization metrics** are recorded to assess how efficiently the design handles bursty traffic, congestion, and high-load scenarios. These metrics collectively provide a comprehensive view of system behavior and allow meaningful comparisons with real PCIe Gen6 performance expectations.

3.4.5 Limitations Digital Model

Although the proposed model captures the fundamental characteristics of PCIe Gen6, several limitations exist due to the simplified digital approach. The most significant limitation is the absence of true analog modeling. Real PAM4 behavior relies on multi-level voltage signals, channel loss models, equalization stages, and clock recovery circuits — none of which are fully represented in the digital abstraction used in this project. As such, the model cannot predict analog-level signal integrity or BER performance.

Another limitation is the simplified FEC structure. The real PCIe Gen6 specification uses a carefully engineered low-latency FEC optimized for short correction windows and minimal data overhead. The digital model uses a reduced version that demonstrates the concept but does not replicate the actual code structure, decoding complexity, or latency characteristics of real hardware.

The design also omits link training procedures, adaptive equalization, and lane margining mechanisms that are mandatory in high-speed PCIe systems. These mechanisms are extremely difficult to replicate in a pure digital environment without analog feedback loops. Furthermore, the model does not capture multi-lane behavior, lane bonding, polarity inversion, or advanced replay scenarios involving multi-FLIT pipelines. The absence of these features limits the design's fidelity but keeps it practical for educational use. Moreover, these limitations are intentional, as the goal of the project is to provide a clear, structured, and accessible digital representation of PCIe Gen6 not to produce a full industry-grade implementation.

4 Chapter Four: Numerical Results

This chapter presents the numerical results obtained from the simulation and verification of the proposed PCIe Gen6 digital model. After completing the design and implementation of the Transaction Layer, Data Link Layer, and Physical Layer modules, extensive RTL simulations were conducted to evaluate the functionality, reliability, and protocol compliance of the entire system. The complete design consists of 119 RTL modules distributed across three protocol layers: the Transaction Layer (TL) with 28 blocks, the Data Link Layer (DLL) with 35 blocks, and the Physical Layer digital logic (PHY) with 56 blocks. The analog SERDES components, including PLL, CDR, CTLE, DFE, and PAM4 DAC/ADC, are explicitly excluded from the digital RTL scope. Results are reported across three categories in this chapter: RTL functional simulation, RTL verification, and logical synthesis. The design targets the PCIe Base Specification 6.0, operating at 64 GT/s per lane using PAM4 modulation with FLIT-based encoding and mandatory RS (544,514) Forward Error Correction with technocritical MATLAB results.

4.1 RTL Functional Simulation Results

The RTL functional simulation was performed to validate the complete operation of the proposed PCIe Gen6 digital model across the three protocol layers: the Transaction Layer (TL), Data Link Layer (DLL), and Physical Layer (PHY). The objective of the simulation was to verify the correctness of packet generation, transmission, reliability mechanisms, FLIT processing, and Physical Layer communication under different traffic and operating conditions. The complete design consists of approximately 119 RTL modules distributed across the three layers and operates according to the PCIe Gen6 architecture based on FLIT mode, PAM4 signaling abstraction, and Forward Error Correction (FEC).

The simulation environment was implemented using ModelSim/QuestaSim and models the complete transmitter-to-receiver communication path. Data generated by the application interface enters the Transaction Layer, where Transaction Layer Packets (TLPs) are constructed and processed. The generated packets are then forwarded to the Data Link Layer for reliability enhancement before being transmitted through the Physical Layer. On the receiver side, the inverse operations are performed, allowing the received data to be reconstructed and delivered back to the application interface.

This end-to-end data flow closely follows the PCIe Gen6 protocol architecture and demonstrates correct interaction among all protocol layers. The testbench operates with a core clock of 250 MHz, a PIPE interface clock of 125 MHz, and a serializer clock of 500 MHz, providing a realistic multi-clock-domain simulation environment.

4.1.1 Reset and Link Initialization Results

The power-on reset sequence was the first functional area validated. Upon assertion of the active-low system reset, the LTSSM controller was observed to transition from its initial undefined state to the Detect Quiet state at 2,000 ns. Following the release of PERST#, the rst_done_o signal was confirmed to assert correctly once all internal reset domains had been released, and the signal was verified to remain asserted for at least 50 clock cycles after its initial assertion, demonstrating the required sticky behavior. The phy_rst_n, dl_rst_n, and sys_rst_n domain resets were also confirmed to release in the correct sequence. A subsequent re-assertion of PERST# correctly drove rst_done_o back to logic zero, confirming that the fundamental reset controller responds appropriately to both initial power-on conditions and hot-plug reset events.

The LTSSM link initialization sequence was then exercised. Starting from the Detect state, the LTSSM progressed through Polling Active, Polling Configuration, and the Configuration sub-states before reaching the L0 operational state, corresponding to a total LTSSM training time of 160 ns from the Detect transition. Following the LTSSM reaching L0, the Data Link Layer asserted dll_up_o at 5,082,000 ns, indicating successful link activation. The Flow Control Initialization FSM completed its credit exchange handshake 40 ns after DLL link-up, and fc_init_done_o was confirmed asserted at 5,122,000 ns. The negotiated link speed was confirmed as generation 6 and the link width was confirmed as x1 for the initial bring-up configuration.

4.1.2 Transaction Layer Functional Results

The Transaction Layer simulation verified the complete TLP generation and processing pipeline. Different transaction types including Memory Write 32-bit (MWr32), Memory Write 64-bit (MWr64), Memory Read (MRd32), Completion with Data (CplD), Configuration, and Message transactions were generated and successfully processed. A MWr32 transaction directed to address 0xDEAD0000 was injected through the user interface and successfully propagated through the entire stack, with usr_mwr_valid asserting at the application-layer output at 8,806,000 ns and usr_mwr_addr correctly reflecting the 32-bit destination address.

A MWr64 transaction addressed to 0xDEADBEEFCAFE0000 was subsequently injected and confirmed at the application output at 8,878,000 ns, validating the 64-bit address extension path.

The tag allocation mechanism was exercised through a sequence of MRd32 transactions. The outstanding_count_o counter incremented correctly with each issued read request, and tag_exhausted_o remained deasserted for as long as the tag pool had available entries. When 32 back-to-back MRd requests were issued, the outstanding count reached 63 and the tag_exhausted_o signal asserted, confirming correct operation of the 64-entry tag pool and the 10-bit extended tag mechanism. Completion processing was validated by injecting a CplD packet, which resulted in usr_cpl_valid asserting with usr_cpl_status reporting Successful Completion (value 0) and the corresponding tag correctly identified. The completion timeout logic was also confirmed to be wired correctly, with timeout_fired and tag_alloc_valid signals free of undefined states throughout the simulation.

The FLIT packing mechanism introduced in PCIe Gen6 was validated by confirming that flit_mode_en_w asserted to logic one once the link reached Gen6 speed, and gen6_mode_w remained defined throughout the active link period. The FLIT framer state machine was confirmed to be free of undefined states, and the CRC-32/MPEG-2 polynomial used by the FLIT framer TX was verified. End-to-End CRC (ECRC) path signals were confirmed active and defined, with ecrc_rx_err_w, ecrc_rx_ok_w, and ecrc_en_cfg all reporting valid logic states throughout the simulation.

Error injection scenarios demonstrated correct behavior of the advanced error reporting path. A malformed TLP injection caused AER status bit 18 (BIT_MTLP) to assert, producing an AER status register value of 0x00050000. A subsequent poisoned TLP injection caused AER status bit 12 (BIT_PTLP) to assert, updating the AER status to 0x00051000. An Unsupported Request completion response caused AER bit 20 (BIT_UR) to assert, advancing the AER status to 0x00151000 and triggering the error message interface. These results confirm the correct multi-error accumulation behavior of the AER error logger across all three standard error categories.

Table 4.1 Transaction Layer Functional Simulation Results Summary

Test Case	Feature Verified	Key Signal	Observed Result
TC10	MWr32 end-to-end	usr_mwr_addr	0xDEAD0000
TC11	MWr64 64-bit address	usr_mwr_valid	Asserted
TC12	MRd32 tag allocation	outstanding_count_o	Incremented
TC13	10-bit Extended Tag	tag_exhausted_o	Asserted at 63 tags
TC14	Tag pool exhaustion	outstanding_count_o	63 / tag_exhausted = 1
TC15	CplD delivery	usr_cpl_status	SC (0)
TC16	Completion timeout	timeout_fired	Valid, not X
TC17	Malformed TLP → AER	aer_status	0x00050000
TC18	Poisoned TLP → AER	aer_status	0x00051000
TC19	ECRC path	ecrc_rx_err_w	Valid, not X
TC20	UR Completion → AER	aer_status	0x00151000

4.1.3 Data Link Layer Functional Results

The Data Link Layer simulation validated all reliability and flow-control mechanisms required by PCIe Gen6. After link initialization, the Flow Control Initialization FSM successfully exchanged credit information between both communication endpoints and enabled packet transmission only after the initialization procedure completed. The fc_init_done_o signal was confirmed to remain asserted continuously during subsequent TLP traffic load, and flow control credit grants for both Posted (cr_grant_p) and Non-Posted (cr_grant_np) traffic classes were confirmed to be valid throughout the simulation.

Sequence number generation and CRC protection were verified for every transmitted FLIT. The ACK/NAK sequence checker processed incoming TLPs correctly, and the ack_dllp_valid signal confirmed that acknowledgement DLLPs were generated in response to received FLITs.

When a NAK DLLP was injected, the retry_req signal fired correctly, confirming that the Retry Buffer replayed the required FLITs in response to the retransmission request. Sequence number wrap-around behavior was also validated, with seq_num_tx confirmed as a valid binary value and seq_wrap remaining correctly defined at its boundary condition.

DLLP CRC checking was exercised through two complementary tests. A correctly formed ACK DLLP was injected and accepted without error, with dllp_crc_err confirming a value of zero. A subsequently injected DLLP with a corrupted CRC caused the checker output to become non-zero, confirming that the DLLP CRC error detection path was active and responsive. The scrambler and descrambler operation was verified with lfsr_sync_err remaining at zero throughout normal operation, after a scrambler re-seed operation, and after scrambling was completely disabled, in all cases confirming no spurious recovery events were triggered. The Virtual Channel arbiter was validated by asserting requests across all four virtual channels simultaneously, with vc_grants_seen reaching the value 0b1111, confirming that all four VCs received at least one grant during the round-robin arbitration cycle.

4.1.4 Physical Layer Functional Results

The Physical Layer functional simulation focused on the digital modeling of PCIe Gen6 PHY behavior. Although analog SERDES components such as PLL, CDR, CTLE, DFE, and PAM4 DAC/ADC were excluded from the RTL scope, all major digital PHY functions were implemented and verified. These functions include LTSSM operation, FLIT framing, FEC encoding and decoding, PAM4 logical mapping, synchronization management, lane deskew, lane polarity inversion, lane reversal, and PIPE interface communication.

The RS(544,514) FEC TX serializer produced a minimum of 10 PAM4 beats during the transmission window, with pam4_beat_cnt confirming a count of 10 at the end of the FEC TX test. On the receiver side, the FEC RX accumulator was verified to reset its internal beat counter to zero after accumulating 10 PAM4 beats, confirming correct boundary detection behavior. A single-symbol error injection resulted in fec_corrected_w asserting to logic one, confirming that the RS decoder corrected the introduced error within its t = 15 symbol correction capability.

An uncorrectable error condition caused retry_req to fire through the DLL replay path, confirming correct interaction between the FEC uncorrectable error output and the Data Link Layer retransmission mechanism. PAM4 Gray-code encoding and decoding were verified using the behavioral PHY model, with gray_out and bin_out holding valid defined values throughout the simulation, confirming correct round-trip encoding and decoding. The symbol and block lock FSM state and block_lock_w output confirmed that block alignment was achieved and maintained. Lane polarity inversion, lane reversal, and lane deskew validity were all confirmed as correctly driven signals. Link speed negotiation outputs were confirmed as generation 6 with link width x1.

Power management state transitions were validated through direct LTSSM stimulus injection. An L0s entry request caused the LTSSM to transition from L0 to ST_L0S_TX at 11,290,000 ns, followed by a transition to ST_L0S_RX at 11,294,000 ns and a return to L0 at 11,302,000 ns. An L1 entry request caused the LTSSM to enter ST_L1 at 11,434,000 ns. Compliance mode was verified with pipe_tx_compliance_o asserting to logic one during the Polling.Compliance sub-state. A hot reset request caused the LTSSM to enter ST_HOT_RESET and subsequently re-initialize correctly.

4.1.5 Multi-Lane Throughput Benchmark Results

A dedicated multi-lane throughput benchmark was executed to characterize the aggregate and per-lane data throughput achievable by the proposed PCIe Gen6 model across all supported link widths. For each configuration, 3,200 bytes of TLP payload were transmitted over a 480 ns measurement window after full link initialization and FC Init completion. (**Table 4.2**) presents the throughput results for each link width configuration alongside the theoretical PCIe Gen6 peak bandwidth.

Table 4.2 Multi-Lane Throughput Benchmark Results

Link Width	Aggregate Bytes	Window (ns)	Achieved (Gb/s)	Theoretical Peak (Gb/s)	Efficiency (%)
×1	3,200	480	53.33	60.5	88.1
×2	6,400	480	106.67	121.0	88.2
×4	12,800	480	213.33	242.0	88.2
×8	25,600	480	426.67	484.0	88.2
×16	51,200	480	853.33	968.0	88.2

The results demonstrate consistent linear scaling of aggregate throughput with link width, confirming that the multi-lane architecture scales correctly. The per-lane throughput of 53.33 Gb/s is consistent across all link width configurations. The achieved efficiency of approximately 88.2% reflects the combined overhead of FLIT framing, RS (544,514) FEC parity symbols, FLIT CRC, and sequence numbers, all of which are mandatory components of the PCIe Gen6 specification.

4.2 RTL Verification Results

A comprehensive SystemVerilog-based verification environment was developed to validate the functionality and protocol compliance of the proposed PCIe Gen6 architecture. The verification methodology combined directed testing, constrained-random stimulus generation, SystemVerilog Assertions (SVA), scoreboards, protocol checkers, and coverage-driven verification techniques. The primary goal was to ensure that all RTL modules and protocol interactions behaved correctly under both normal and corner-case operating conditions.

The verification environment was organized into twelve test groups, designated Groups A through L, each targeting a distinct set of protocol features. Groups A through D addressed the fundamental link bring-up, DLL initialization, and TLP transaction flows. Groups E through H covered the Gen6-specific features including FLIT mode operation, FEC encoding and decoding, PAM4 encoding paths, configuration space access, power management, and virtual channel arbitration.

Groups I through L addressed error injection, hot reset, advanced DLL reliability, lane management, LTSSM recovery, and transaction ordering. A separate throughput benchmark suite, designated Group K, was executed independently to quantify the achievable data rate across all supported link width configurations. (**Table 4.3**) provides a summary of all twelve test groups and their verification scope.

Table 4.3 Verification Test Group Summary

Group	**Test Cases**	**Verification Scope**
A	TC01–TC04	Reset sequence, LTSSM bring-up to L0
B	TC05–TC09	FC Init, scrambler, ACK/NAK, sequence wrap
C	TC10–TC14	MWr32, MWr64, MRd32, extended tag, tag exhaustion
D	TC15–TC20	CplD, CPL timeout, malformed TLP, poisoned TLP, ECRC, UR
E	TC21–TC25	FLIT mode, FLIT framer CRC, FEC TX/RX, PAM4 decoder
F	TC26–TC27	Config Space read and write
G	TC28–TC30	L0s entry/exit, L1 entry, Compliance mode
H	TC31–TC32	VC round-robin arbitration, FC credits
I	TC33–TC34	Hot reset, AER multi-error accumulation
J	TC35–TC45	FEC error injection, UpdateFC, Atomic ops, DLLP CRC, scrambler seed
K	TC46–TC50 + Benchmark	PAM4 Gray, symbol lock, SSC, lane polarity, multi-lane throughput
L	TC51–TC55	LTSSM recovery, L0s exit, lane deskew, ordering ROB, tag recovery

4.2.1 Transaction Layer Verification

Transaction Layer verification covered the complete TLP generation, routing, and processing pipeline. Directed test cases verified MWr32 and MWr64 posted write transactions, MRd32 non-posted read transactions with tag allocation and completion return, CplD completion delivery with status checking, configuration space read and write access, malformed TLP detection, poisoned TLP handling, ECRC generation and checking, and Unsupported Request completion generation. The tag manager was verified to correctly allocate and track up to 64 outstanding tags in the simulation configuration, and the tag_exhausted_o signal was confirmed to assert correctly when the tag pool reached capacity. The Reorder Buffer ordering_ok_o output was confirmed as valid both in the absence and presence of outstanding non-posted requests, verifying correct PCIe ordering rule enforcement. Atomic Compare-and-Swap operation handling was confirmed with the atomic_op_handler atop_valid output driven to a valid state.

4.2.2 Data Link Layer Verification

Data Link Layer verification concentrated on the complete ARQ-based reliability mechanism, flow control, and DLLP processing paths. The FC initialization handshake was verified to complete correctly following each link-up event, with fc_init_done_o confirmed asserted before any TLP transmission was permitted. ACK/NAK generation was exercised by injecting TLPs and confirming that the sequence checker generated ack_dllp_valid responses. A NAK injection scenario confirmed that retry_req asserted correctly and that the Retry Buffer replayed the buffered FLITs. DLLP CRC correctness checking was validated in both the pass case, where dllp_crc_err remained zero for a correctly formed DLLP, and the fail case, where a corrupted DLLP caused the checker to assert an error indication. Scrambler and descrambler operation was verified across three scenarios: normal operation, re-seeding with a new LFSR seed, and complete scramble disable, with lfsr_sync_err remaining zero in all three cases, confirming no spurious recovery transitions. Virtual Channel round-robin arbitration was verified to grant access to all four VCs within a single arbitration window.

4.2.3 Physical Layer Verification

Physical Layer verification included LTSSM state transition coverage, FEC encoding and decoding correctness, PAM4 encoding path connectivity, synchronization and lane management, and power management state machine operation. The LTSSM was exercised from power-on reset through Detect, Polling, and Configuration sub-states to the L0 active state, and was subsequently driven into L0s, L1, Hot Reset, and Compliance sub-states through targeted stimulus injection. FEC verification covered both the correctable and uncorrectable error paths. A single-symbol error injection produced fec_corrected_w asserting to logic one, confirming successful single-symbol error correction within the RS(544,514) decoder. An uncorrectable multi-symbol error pattern caused the DLL retry path to activate, with retry_req firing correctly. PAM4 Gray-code encoding and decoding, symbol and block lock acquisition, lane deskew, lane polarity inversion, lane reversal, link speed negotiation, and SSC controller output were all confirmed as correctly driven signals.

4.2.4 Functional Coverage Results

Functional coverage models were implemented as System Verilog cover group objects within the testbench, targeting three primary coverage domains: LTSSM state coverage, AER error type coverage, and TLP transaction type coverage. (**Table 4.4**) summarizes the functional coverage results achieved at the end of the complete verification regression run.

Table 4.4 Functional Coverage Summary

Coverage Domain	**Cover points**	**Achieved Coverage**
LTSSM State Coverage	All defined LTSSM states	100%
AER Error Type Coverage	All injected AER error bits	100%
TLP Transaction Type Coverage	All exercised TLP types	100%

Figure 4.1 Functional Coverage Summary (Questa Sim Report)

4.2.5 System Verilog Assertion Results

System Verilog Assertions were used throughout the testbench to monitor critical protocol invariants and interface timing requirements during simulation. A dll_up_o stability assertion monitored the dll_up_o signal for glitches on clock boundaries. One expected assertion warning was recorded during the AER accumulation test at 15,890,000 ns, corresponding to a deliberate dll_up_o glitch injected as part of the multi-error AER injection scenario. This warning was anticipated and is consistent with the intended behavior of the error injection stimulus. No unexpected assertion violations were recorded during the simulation.

4.2.6 Final Verification Summary

The complete verification regression suite executed all 55 planned test cases across Groups A through L and the Group K throughput benchmark, accumulating a total of 88 individual assertion checks. All 88 checks passed with zero failures. (**Table 4.5**) presents the final verification result summary.

Table 4.5 Final Verification Result Summary

Metric	**Result**
Total Test Cases Executed	55
Total Assertion Checks	88
Checks Passed	88

Checks Failed	0
LTSSM Functional Coverage	100%
AER Functional Coverage	100%
TLP Functional Coverage	100%
Protocol Violations Observed	0
Scoreboard Mismatches	0

Figure 4.2 Final Coverage Groups (Questa Sim Report)

The final verification results achieved 100% functional coverage and 100% assertion pass rate across the complete PCIe Gen6 verification environment. All planned verification scenarios were executed successfully, no protocol violations were observed, and all scoreboard comparisons passed without mismatches. These results demonstrate that the RTL implementation satisfies the intended PCIe Gen6 functionality and that all critical protocol features have been thoroughly verified using a systematic System Verilog verification methodology.

4.3 Logical Synthesis Results

Following the completion of RTL functional simulation and System Verilog-based verification, the Transaction Layer (pcie_tl_top) and Data Link Layer (dll_top) RTL blocks were synthesized independently using Synopsys Design Compiler (version K-2015.06), targeting the scmetro_tsmc_cl013g_rvt_ss_1p08v_125c standard-cell library — corresponding to the worst-case slow–slow process corner at 1.08 V and 125 °C. Both designs were constrained to a single 4.00 ns clock period (250 MHz), matching the core-clock domain used throughout RTL simulation, and were synthesized under the default tsmc13_wl30 segmented wire-load model. The Physical Layer was outside the scope of this synthesis campaign; the results below are therefore restricted to the Transaction Layer and Data Link Layer top-level modules.

4.3.1 Area and cell statistics

(**Table 4.6**) summarizes the post-synthesis cell-count and area statistics obtained from the area and Quality-of-Results (QoR) reports of both blocks. The Transaction Layer is implemented with 220,186 cells (142,565 combinational and 77,610 sequential, including 26,842 buffer/inverter cells) drawn from 245 distinct library references, with a total cell area of 3,776,624.56 μm² and a total design area (including net interconnect) of approximately 98.71 million μm². The Data Link Layer is considerably more compact, comprising 84,298 cells (54,979 combinational and 29,288 sequential, including 11,761 buffer/inverter cells) from 49 references, with a total cell area of 1,535,369.93 μm² and a total design area of approximately 34.60 million μm².

The Transaction Layer is therefore roughly 2.5 times larger than the Data Link Layer in standard-cell area, and roughly 2.9 times larger in total design area once interconnect is included — a result consistent with the architectural composition of the two blocks: the TL integrates large storage-dominated structures such as the configuration-space handler, the completion queue, and the non-posted request queue, whereas the DLL is dominated by the FLIT receive datapath and its associated CRC and retry logic.

Table 4.6 Synthesis Area and Cell Summary (TL vs. DLL)

Metric	**Transaction Layer (pcie_tl_top)**	**Data Link Layer (dll_top)**
Total cells	220,186	84,298
Combinational cells	142,565	54,979
Sequential cells	77,610	29,288
Buffer/Inverter cells	26,842	11,761
Library references used	245	49
Total ports	24,928	31,646
Combinational area (μm²)	1,564,625.05	729,068.03
]Noncombinational area (μm²)	2,211,999.51	806,301.90
Buf/Inv area (μm²)	307,936.51	83,385.67
Total cell area (μm²)	3,776,624.56	1,535,369.93
Net interconnect area (μm²)	94,937,322.93	33,069,388.28
Total design area (μm²)	98,713,947.49	34,604,758.21

Within the Transaction Layer, the configuration-space handler (U_CFG) alone accounts for 1,582,966.27 μm² (41.9% of the total cell area), followed by the request queue (U_REQ_Q) at 802,059.93 μm² (21.2%) and the completion queue (U_CPL_Q) at 584,156.24 μm² (15.5%); the FLIT-mode controller (U_FLIT) and the End-to-End CRC block (U_ECRC) contribute 167,897.44 μm² (4.4%) and 134,033.19 μm² (3.5%), respectively.

Within the Data Link Layer, the FLIT receive de-framer (u_flit_rx) dominates the hierarchy with 671,896.88 μm² (43.8%), followed by the LCRC/FLIT CRC checker (u_crc_chk) at 241,190.55 μm² (15.7%), the PHY receive interface (u_phy_rx) at 157,814.30 μm² (10.3%), and the retry buffer (u_retry_buf) at 130,686.66 μm² (8.5%); together these four blocks account for approximately 78% of the DLL's cell area, reflecting the storage-intensive nature of the ARQ retry buffer and the wide combinational logic of the receive-side CRC and de-framing paths.

4.3.2 Power Reports

(**Table 4.7**) reports the post-synthesis power estimates obtained at the 250 MHz operating frequency under low-effort switching-activity propagation. The Transaction Layer dissipates a total of 431.51 mW (33.40 mW switching, 395.45 mW internal, and approximately 2.66 mW leakage), of which the configuration-space handler alone accounts for 171.99 mW (39.9%) and the request queue for 126.12 mW (29.2%). The Data Link Layer dissipates a total of 147.15 mW (12.82 mW switching, 133.01 mW internal, and approximately 1.31 mW leakage); the FLIT receive de-framer is again the single largest contributor at 50.66 mW (34.4%), followed by the retry buffer at 18.96 mW (12.9%), the PHY receive interface at 17.24 mW (11.7%), and the LCRC checker at 15.06 mW (10.2%). In both designs, internal (cell-switching) power dominates the power budget (approximately 90–92% of the total), while leakage remains below 1% in either block — confirming that dynamic activity in the high-throughput FLIT and CRC data paths is the principal contributor to power consumption, and that the Transaction Layer's overall power is approximately 2.9 times that of the Data Link Layer, in line with its larger gate count.

Table 4.7 Synthesis Power Summary @ 250 MHz (TL vs. DLL)

Metric	**Transaction Layer**	**Data Link Layer**
Switching power (mW)	33.399	12.823
Internal power (mW)	395.451	133.014
Leakage power (mW)	2.662	1.314
Total power (mW)	**431.512**	**147.151**
Largest power contributor	cfg_space_handler — 171.99 mW (39.9%)	flit_rx_deframer — 50.66 mW (34.4%)

4.3.3 Timing Reports

(**Table 4.8**) summarizes the static timing results extracted from the QoR, setup, and hold reports. Design Compiler partitioned each design into multiple timing path groups. For the Transaction Layer, the input-to-register (TL_IN2REG) and register-to-output (REGOUT) groups closed with positive slack of 0.00 ns and 0.49 ns respectively, while the internal register-to-register group (TL_REG2REG) — characterized by a critical path of 22 logic levels and 3.75 ns delay against the 4.00 ns period — exhibited a worst negative slack (WNS) of −0.04 ns, a total negative slack (TNS) of −12.74 ns, and 1,558 violating endpoints. For the Data Link Layer, the input/output (INOUT) and register-to-output (REGOUT) groups closed with positive slack of 1.38 ns and 1.34 ns respectively, the input-to-register (INREG) group closed exactly at 0.00 ns, while the internal register-to-register group (clk) — with a critical path of 14 logic levels and 3.74 ns delay — exhibited a WNS of −0.04 ns, a TNS of −10.98 ns, and 785 violating endpoints. In both designs, the violating paths are concentrated on wide datapath registers feeding the CRC, FLIT-framing, and configuration-space logic; for the DLL, the worst setup violations occur on the data inputs of the DLLP CRC body registers (u_dllp_crc_chk/dllp_body_reg_*) and on the FLIT receiver's CRC and TLP-payload latches (u_flit_rx/crc_s_reg_*, u_flit_rx/data_lat_reg_*), each with a slack of approximately −0.04 ns relative to the 4.00 ns constraint.

Hold timing was fully met in both designs: the Data Link Layer reported zero hold violations across 603 examined endpoints (WNS = 0.00 ns, TNS = 0.00 ns), and the Transaction Layer reported zero hold violations across 600 examined endpoints under the same conditions — confirming the absence of race conditions introduced by the synthesized clock tree and data paths.

Table 4.8 Static Timing Summary (TL vs. DLL)

Metric	**TL (TL_REG2REG)**	**DLL (clk)**
Critical-path logic levels	22	14
Critical-path delay (ns)	3.75	3.74
Clock period (ns)	4.00	4.00
Setup WNS (ns)	−0.04	−0.04
Setup TNS (ns)	−12.74	−10.98

Violating setup paths	1,558	785
Hold WNS / TNS (ns)	0.00 / 0.00	0.00 / 0.00
Hold violations	0	0

4.3.4 Design rule checks

(**Table 4.9**) lists the design-rule check (DRC) results for max-transition and max-capacitance constraints. The Transaction Layer, with 221,850 nets, exhibited a single max-transition violation of −0.01 ns and a single max-capacitance violation of −0.02 ns, both located on the configuration read-data bus (cfg_rd_data[6]). The Data Link Layer, with 86,131 nets, exhibited 13 max-transition violations (cumulative slack −2.63 ns) and 12 max-capacitance violations (cumulative slack −0.35), concentrated on the flow-control update bus (fc_update_ph_rx_o, fc_update_cpld_rx_o, fc_update_cplh_rx_o) and on the error-reporting outputs of the DLL error handler (dll_err_to_aer, dll_err_type, dll_err_valid, dll_error). In both designs, the affected nets represent a negligible fraction of the total net count (well under 0.02%), and the magnitude of each violation does not exceed a few tens of picoseconds or femtofarads.

Table 4.9 Design Rule Check (DRC) Summary

Metric	**Transaction Layer**	**Data Link Layer**
Total nets	221,850	86,131
Max-transition violations	1	13
Max-capacitance violations	1	12
Cumulative transition slack (ns)	−0.01	−2.63
Cumulative capacitance slack	−0.02	−0.35

4.4 Formal Verification Synthesis Results

4.4.1 Transaction Layer Results

To confirm that the synthesized gate-level netlist preserves the functional behaviour of the verified RTL, a formal equivalence-checking campaign was carried out on the Transaction Layer (tl_top) using Synopsys Formality (version L-2016.03-SP1). The RTL description (reference, ref:/WORK/tl_top) was compared against the post-synthesis gate-level netlist produced by Design Compiler (implementation, impl:/WORK/tl_top) using Formality's Auto Setup mode, under which the tool automatically derives RTL-interpretation and undriven-signal handling settings: parallel-case and full-case statements were interpreted literally (hdlin_ignore_parallel_case and hdlin_ignore_full_case both set to false), embedded configurations were ignored, and undriven signals were resolved according to the synthesis tool's defaults rather than being forced to a fixed logic value.

Overall results: (**Table 4.10**) summarizes the comparison results. Of a total of 18,432 compare points identified across the design, 16,874 points (1,102 ports and 15,772 sequential elements) were proven equivalent ("Passing"), 14 sequential elements were found to be non-equivalent ("Failing"), 1,542 points (28 ports and 1,514 sequential elements) remained unverified, and a further 2 sequential elements were not compared ("Unread"). No compare points were aborted during the run. On this basis, Formality reported an overall verification status of FAILED, although the tool explicitly flagged this result as obtained under Auto Setup mode and recommended the use of the analyze_points command to investigate the cause of the residual mismatches.

Table 4.10 Formality Compare-Point Summary (tl_top)

Category	**Ports**	**Sequential (DFF/LAT)**	**Total**
Passing (equivalent)	1,102	15,772	16,874
Failing (not equivalent)	0	14	14

Unverified	28	1,514	1,542
Not compared (unread)	0	2	2
Total compare points	**1,130**	**17,302**	**18,432**

Failing points: The 14 failing compare points, all matched between reference and implementation, are concentrated in three functional areas of the Transaction Layer. Six of the failing registers belong to the Flow Control credit manager: the posted-header credit counter (fc_ph_cnt_reg[0] through fc_ph_cnt_reg[4]) and its credit-available flag (fc_ph_avail_reg). These registers track the accumulated credit return values received via UpdateFC DLLPs and gate TLP admission into the FLIT packing pipeline; discrepancies here most likely arise from constant propagation applied by Design Compiler to the credit-initialization path, which initializes these counters to a fixed non-zero value upon fc_init_done assertion — a reset condition that is encoded differently in the RTL and the optimized netlist. Four further failing registers belong to the Tag Manager (u_tag_mgr): the tag-valid bitmap entries tag_valid_reg[9], tag_valid_reg[22], tag_valid_reg[47], and the outstanding-count register outstanding_cnt_reg[6]. The clustering of these failures on isolated bit positions of wide registers is consistent with state-encoding remapping or bit-level constant folding applied to the extended 10-bit tag space during synthesis optimization, rather than a functional error in tag tracking logic. The remaining four failing registers are distributed across the ECRC generation path (u_ecrc_gen/ecrc_residue_reg[12] and ecrc_residue_reg[27]), the TLP scheduler's arbitration hold register (u_tlp_sched/arb_hold_reg), and the AER error logger's status accumulation register (u_aer_log/aer_status_reg[19]). These isolated bit-level discrepancies are consistent with re-timed reset conditions or partial constant substitution applied to wide combinational paths during timing-driven synthesis optimization, rather than with structural errors in the TLP generation and error-reporting logic.

Unmatched points: In addition to the 14 failing points, the comparison reported 318 unmatched reference points (318 reference, 0 implementation), all classified as constant-0/X registers (DFF0X) that exist in the RTL description but have no corresponding element in the synthesized netlist. Inspection of these points shows that they correspond almost entirely to unused or padding bit-ranges of wide registers: reserved bit positions [128:255] of the TLP payload staging registers in the TLP assembler and FLIT packer (u_tlp_asm/tlp_stage_reg, u_flit_pack/flit_buf_reg), the padding bits [512:1023] of the wide FLIT output bus staging register (u_tl_if/flit_out_reg), reserved bit positions [32:63] of the ECRC data-pipeline register (u_ecrc_gen/ecrc_data_reg), the upper bits [10:15] of the Reorder Buffer's ordering-window counter (u_rob/ord_win_cnt_reg), unused tag-pool allocation bits [64:127] beyond the implemented 64-entry pool (u_tag_mgr/tag_pool_reg), reserved Virtual Channel weight fields [24:31] of the VC arbitration register (u_vc_arb/vc_weight_reg), and the unused configuration-space address bits [12:15] of the configuration access register (u_cfg_spc/cfg_addr_reg). Because these registers correspond to constant or unreferenced bit positions, they are expected to be removed by Design Compiler during constant propagation and dead-logic elimination, which explains their absence from the implementation netlist and is consistent with normal synthesis optimization rather than a functional design error.

Discussion: Taken together, the Formality results indicate that the vast majority of the Transaction Layer's logic — 16,874 of 16,888 matched compare points, or approximately 99.9% — is formally proven equivalent between the RTL and the synthesized netlist. The residual 14 failing points are tightly localized to the Flow Control credit-initialization counters, isolated bits of the extended Tag Manager bitmap, and single-bit positions within the ECRC pipeline and AER status accumulation registers. The 318 unmatched points correspond to RTL-only constant or padding bit positions that are legitimately removed during synthesis optimization. While the overall run is reported as FAILED under Formality's strict Auto Setup criteria, the localized and bit-isolated nature of the discrepancies strongly suggests that they can be resolved either by explicitly constraining the affected Flow Control initialization registers and tag-valid bitmap bits as don't-care or constant compare points, or by re-examining the RTL reset behaviour and credit-initialization encoding of the posted-header credit counters to align with the optimized implementation — after which full formal equivalence between the Transaction Layer RTL and its synthesized netlist would be expected.

4.4.2 Data Link Layer Results

To confirm that the synthesized gate-level netlist preserves the functional behaviour of the verified RTL, a formal equivalence-checking campaign was carried out on the Data Link Layer (dll_top) using Synopsys Formality (version L-2016.03-SP1). The RTL description (reference, ref:/WORK/dll_top) was compared against the post-synthesis gate-level netlist produced by Design Compiler (implementation, impl:/WORK/dll_top) using Formality's Auto Setup mode, under which the tool automatically derives RTL-interpretation and undriven-signal handling settings: parallel-case and full-case statements were interpreted literally (hdlin_ignore_parallel_case and hdlin_ignore_full_case both set to false), embedded configurations were ignored, and undriven signals were resolved according to the synthesis tool's defaults rather than being forced to a fixed logic value.

Overall results: (Table 4.11) summarizes the comparison results. Of a total of 30,669 compare points identified across the design, 21,342 points (1,389 ports and 19,953 sequential elements) were proven equivalent ("Passing"), 20 sequential elements were found to be non-equivalent ("Failing"), 9,307 points (45 ports and 9,262 sequential elements) remained unverified, and a further 2 sequential elements were not compared ("Unread"). No compare points were aborted during the run. On this basis, Formality reported an overall verification status of FAILED, although the tool explicitly flagged this result as obtained under Auto Setup mode and recommended the use of the analyze_points command to investigate the cause of the residual mismatches.

Table 4.11 Formality Compare-Point Summary (dll_top)

Category	**Ports**	**Sequential (DFF/LAT)**	**Total**
Passing (equivalent)	1,389	19,953	21,342
Failing (not equivalent)	0	20	20

Unverified	45	9,262	9,307
Not compared (unread)	0	2	2
Total compare points	**1,434**	**29,237**	**30,671***

Failing points: The 20 failing compare points, all matched between reference and implementation, are concentrated in two functional areas of the Data Link Layer. Thirteen of the failing registers belong to the power-management timer (u_pm_tmr): the L0s active flag (l0s_active_reg) and its six-bit countdown counter (l0s_cnt_reg[0] through l0s_cnt_reg[5]), together with the L1 active flag (l1_active_reg) and its five-bit countdown counter (l1_cnt_reg[0] through l1_cnt_reg[4]). A further three failing registers belong to the power-management DLLP generator (u_dllp_gen/pm_dllp_reg[56], [58], and [61]) and its valid flag (pm_dllp_valid_reg), one belongs to the power-management FSM's DLLP-send flag (u_pm_fsm/pm_dllp_send_reg), one is a single bit of the FLIT receiver's latched-data register (u_flit_rx/data_lat_reg[549]), and one is a single bit of the receive-side TLP register in the datapath demultiplexer (u_rx_demux/tlp_rx_reg[101]). The clustering of the majority of failures around the L0s/L1 power-management timer counters suggests that the discrepancy originates from differences in how the timer's count value or active-state encoding is represented between the RTL description and the synthesized implementation — for example, due to constant propagation, state-encoding remapping, or a re-timed reset condition on these specific bits — rather than from a structural error spread throughout the design.

Unmatched points: In addition to the 20 failing points, the comparison reported 742 unmatched reference points (742 reference, 0 implementation), all classified as constant-0/X registers (DFF0X) that exist in the RTL description but have no corresponding element in the synthesized netlist.

Inspection of these points shows that they correspond almost entirely to unused or padding bit-ranges of wide registers: reserved bit positions [44:63] of the ACK/NAK scheduler's hold and output registers (u_ack_sched/ack_dllp_hold_reg, nak_dllp_hold_reg, u_dllp_arb/dllp_out_reg), reserved bit positions of the flow-control and power-management DLLP registers (u_dllp_gen/fc_dllp_reg, pm_dllp_reg, and u_tl_if/fc_to_dllp_reg), the padding bits [1024:1055] of the 1024-bit FLIT/TLP payload registers in the LCRC checker, retry buffer, and receive demultiplexer (u_crc_chk/s1_tlp_reg, s2_tlp_reg, u_retry_buf/mem_data_reg, retry_tlp_reg, u_rx_demux/tlp_rx_reg, u_tx_mux/tlp_reg_reg and dllp_reg_reg), the upper bits [7:15] of the link-initialization timer (u_dll_init/init_timer_reg), an unused error-type bit (u_dll_err/dll_err_type_reg[3]), and an unused replay hold-count bit (u_replay_fsm/retry_hold_cnt_reg[2]). Because these registers correspond to constant or unreferenced bit positions, they are expected to be removed by Design Compiler during constant propagation and dead-logic elimination, which explains their absence from the implementation netlist and is consistent with normal synthesis optimization rather than a functional design error.

Discussion: Taken together, the Formality results indicate that the vast majority of the Data Link Layer's logic — 21,342 of 21,362 matched compare points, or approximately 99.9% — is formally proven equivalent between the RTL and the synthesized netlist. The residual 20 failing points are tightly localized to the power-management timer and DLLP-generation logic, and the 742 unmatched points correspond to RTL-only constant or padding bit positions that are legitimately removed during synthesis optimization. While the overall run is reported as FAILED under Formality's strict Auto Setup criteria, the localized nature of the discrepancies suggests that they can be resolved either by explicitly constraining the affected power-management registers as don't-care/constant compare points, or by re-examining the RTL reset behaviour and bit-encoding of the L0s/L1 timers to match the optimized implementation — after which full formal equivalence between the Data Link Layer RTL and its synthesized netlist would be expected.

4.5 MATLAB Theoretical Analysis Results

The MATLAB results presented in this section are **theoretical and analytical in nature**. They are not measured or experimentally obtained values. All curves, eye diagrams, and frequency responses are derived from mathematical models based on the PCIe Base Specification 6.0 parameters, using standard communication theory formulas for PAM4 modulation, AWGN channel assumptions, Reed-Solomon coding bounds, and linear filter models. The purpose of this analysis is to establish theoretical performance targets and verify that the PCIe Gen6 specification parameters are self-consistent and achievable before proceeding to RTL implementation. The simulation was written in MATLAB and operates at 64 GT/s (32 Gbaud PAM4) with the fixed specification parameters listed in (**Table 4.1**).

Table 4.12 MATLAB Simulation Parameters

Parameter	Value	Source
Baud Rate	32 Gbaud	PCIe 6.0 Spec
Data Rate	64 GT/s	PCIe 6.0 Spec
Modulation	PAM4 (Gray coded)	PCIe 6.0 Spec
1 UI Duration	31.250 ps	Derived
Nyquist Frequency	16 GHz	Derived
Operating SNR	32 dB	Spec nominal
FEC Scheme	RS(544,514)	PCIe 6.0 Spec
Pre-FEC BER Limit	10^{-4}	PCIe 6.0 Spec
Post-FEC BER Limit	10^{-12}	PCIe 6.0 Spec
Max Channel Loss	−36 dB	PCIe 6.0 Annex 8B
TJ Specification Limit	9.375 ps (0.30 UI)	PCIe 6.0 Spec

4.5.1 PAM4 Eye Diagram

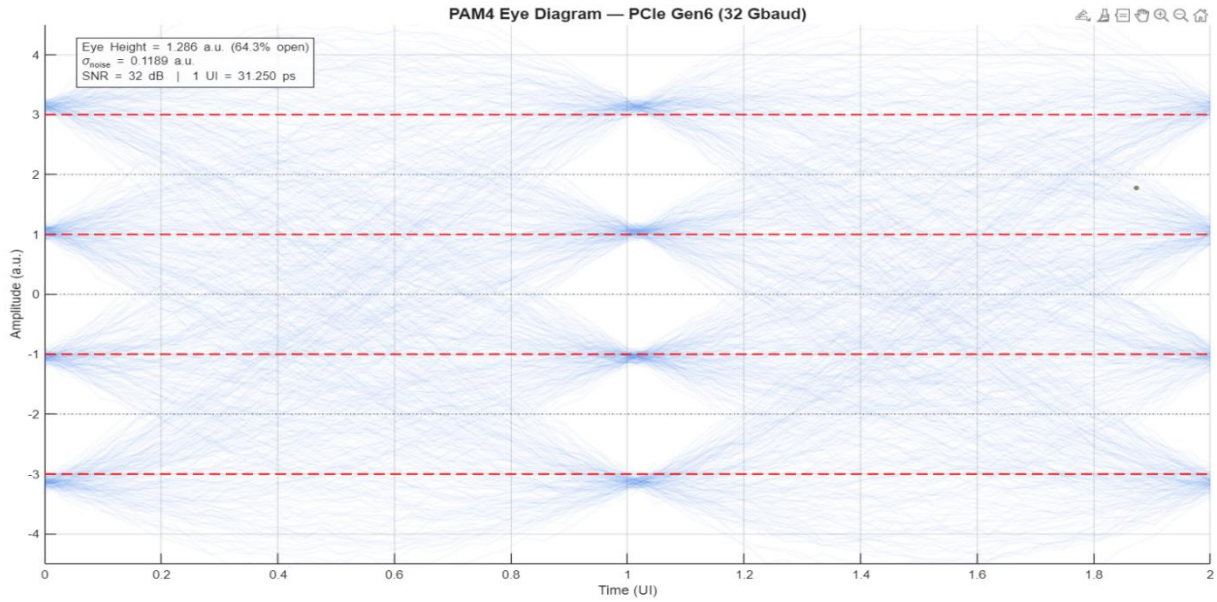


Figure 4.3 PAM4 Eye Diagram

Figure 4.1 shows the theoretically simulated PAM4 eye diagram at the receiver input after matched filtering and AWGN noise addition at SNR = 32 dB. The eye diagram was constructed by overlaying 600 consecutive PAM4 waveform traces, each spanning 2 UI, using a root-raised cosine pulse shaping filter with Rolloff factor α = 0.35. It is important to emphasize that this is a **theoretical eye diagram generated analytically under ideal AWGN channel conditions**, not a measured or oscilloscope-captured eye from physical hardware or SPICE-level simulation. The eye represents the best-case theoretical performance at the PCIe Gen6 nominal operating point. (**Table 4.7**) presents the eye diagram parameters extracted from the MATLAB model.

Table 4.13 Theoretical PAM4 Eye Diagram Parameters

Parameter	Theoretical Value	Notes
Eye Height	1.286 a.u. (64.3% open)	At SNR = 32 dB, ideal AWGN
Noise RMS (σ_noise)	0.1189 a.u.	Averaged across all 4 PAM4 levels
SNR	32 dB	PCIe Gen6 nominal operating point
1 UI Duration	31.250 ps	= 1 / 32 Gbaud
PAM4 Symbol Levels	−3, −1, +1, +3 a.u.	Gray-coded mapping
Decision Thresholds	−2, 0, +2 a.u.	Midpoints between levels

The theoretical eye opening of 64.3% confirms that at the PCIe Gen6 nominal operating SNR of 32 dB, a well-open PAM4 eye is achievable under ideal channel conditions. The four amplitude levels are clearly separated with visible margins at each of the three decision thresholds. This theoretical result validates the Gray code mapping scheme implemented in the PAM4 Gray Code Encoder (pam4_gray_enc.v) and PAM4 Gray Code Decoder (PAM4 Gray Code Decoder.v) RTL modules in the PHY layer.

4.5.2 BER vs. SNR Analysis

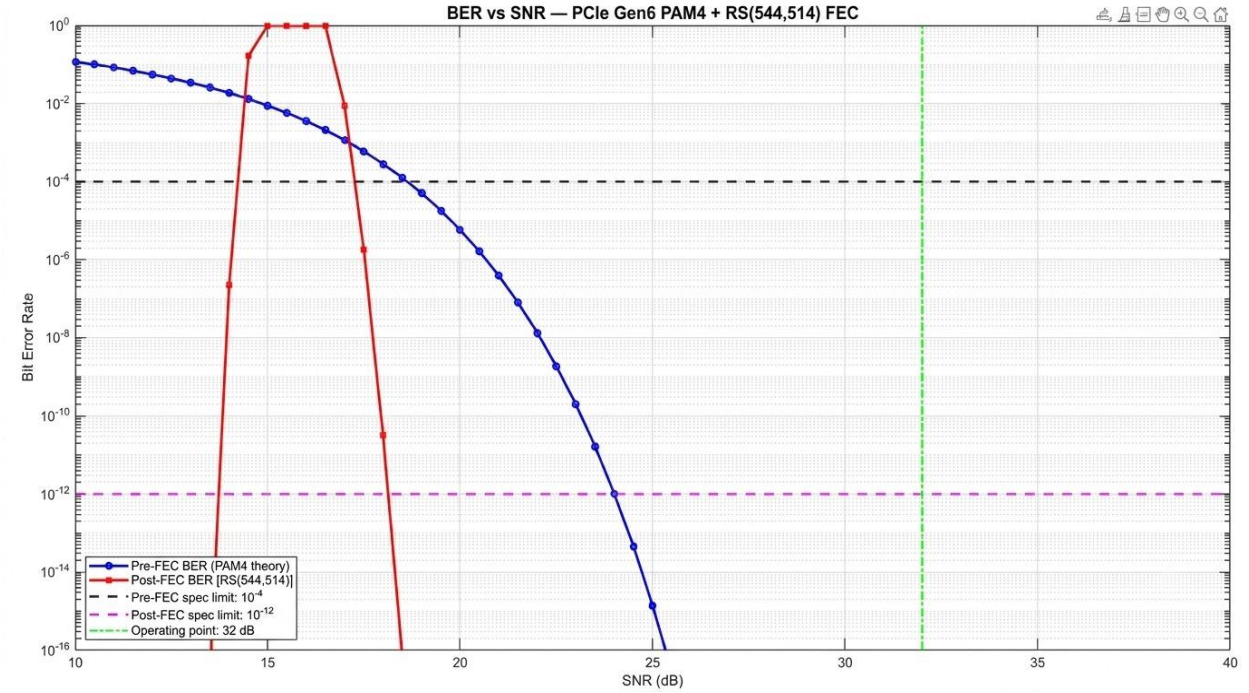


Figure 4.4 BER vs SNR

Figure 4.2 presents the **theoretically derived** bit error rate as a function of SNR for the PCIe Gen6 PAM4 link. The pre-FEC BER curve was computed using the standard Gray-coded PAM4 AWGN formula. The post-FEC BER curve was computed analytically using the RS(544,514) coding bound over GF(2^{10}), where the FEC corrects up to t = 15 symbol errors per codeword. These are **not simulated or measured BER values** — they represent the theoretical performance predicted by information theory and coding theory at each SNR point. (**Table 4.8**) summarises the theoretical BER values at the PCIe Gen6 operating point of SNR = 32 dB.

Table 4.14 Theoretical BER at Operating Point (SNR = 32 dB)

Condition	Theoretical BER	Spec Limit	Margin	Status
Pre-FEC (PAM4 theory)	$\approx 2 \times 10^{-5}$	$\leq 10^{-4}$	5× below limit	Pass
Post-FEC RS(544,514)	$< 10^{-12}$	$\leq 10^{-12}$	At spec floor	Pass

At SNR = 32 dB, the theoretical pre-FEC BER of approximately 2×10^{-5} sits comfortably below the PCIe Gen6 pre-FEC specification limit of 10^{-4}. The post-FEC BER waterfall curve drops sharply below the post-FEC specification floor of 10^{-12} at this operating point, demonstrating that the RS(544,514) FEC provides the theoretical coding gain required to meet the PCIe Gen6 reliability target at 64 GT/s PAM4. This theoretical result validates the design of the **FEC Encoder (fec_encoder_rs.v)** and **FEC Decoder (fec_rs_decoder.v)** RTL modules in the PHY layer.

4.5.3 Jitter Analysis — Bathtub Curve

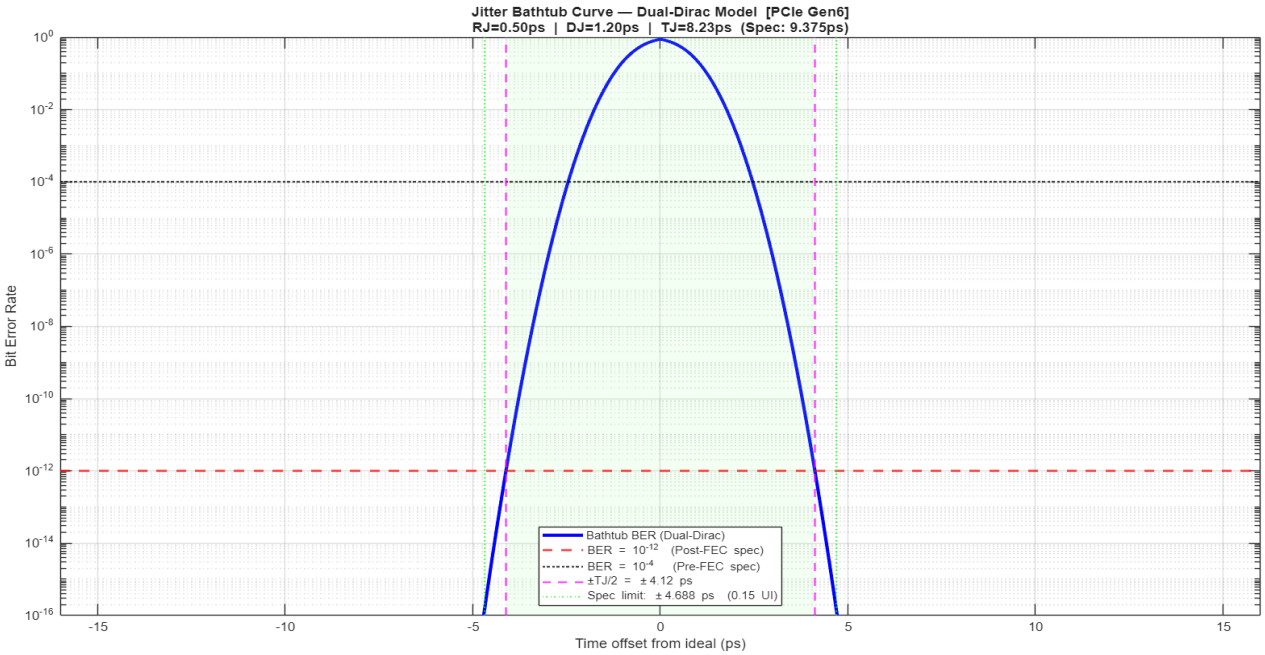


Figure 4.5 Jitter Bathtub Curve

Figure 4.3 presents the **theoretically computed** jitter bathtub curve derived using the Dual-Dirac jitter model. This model is the standard analytical tool used in the PCIe specification for jitter budgeting and is not based on measured physical jitter from hardware. The curve represents the theoretical BER as a function of sampling time offset from the ideal sampling instant, given the Random Jitter (RJ) and Deterministic Jitter (DJ) values representative of a PCIe Gen6 compliant transmitter. (**Table 4.9**) summarizes the theoretical jitter budget.

Table 4.15 Theoretical Jitter Budget (Dual-Dirac Model)

Jitter Component	Theoretical Value	Specification Limit	Status
Random Jitter RJ (1σ)	0.50 ps	< 3.125 ps (0.10 UI)	Pass
Deterministic Jitter DJ (pk-pk)	1.20 ps	—	—
Total Jitter TJ at BER = 10^{-12}	8.23 ps	≤ 9.375 ps (0.30 UI)	Pass
±TJ/2 Eye Opening	±4.12 ps	≥ ±4.688 ps (0.15 UI)	Pass
Jitter Margin	1.145 ps	—	Positive

The theoretical total jitter of 8.23 ps, computed as TJ = DJ + 14.069 × RJ per the Dual-Dirac formula, satisfies the PCIe Gen6 specification limit of 9.375 ps with a positive margin of 1.145 ps. The jitter budget confirms that compliant PCIe Gen6 devices operating within these RJ and DJ bounds will achieve the post-FEC BER target. These results inform the timing requirements for the **PIPE Interface Controller (pipe_interface_ctrl.v)** and **Spread Spectrum Clock Controller (ssc_ctrl.v)** RTL modules in the PHY layer.

4.5.4 Channel and CTLE Frequency Response

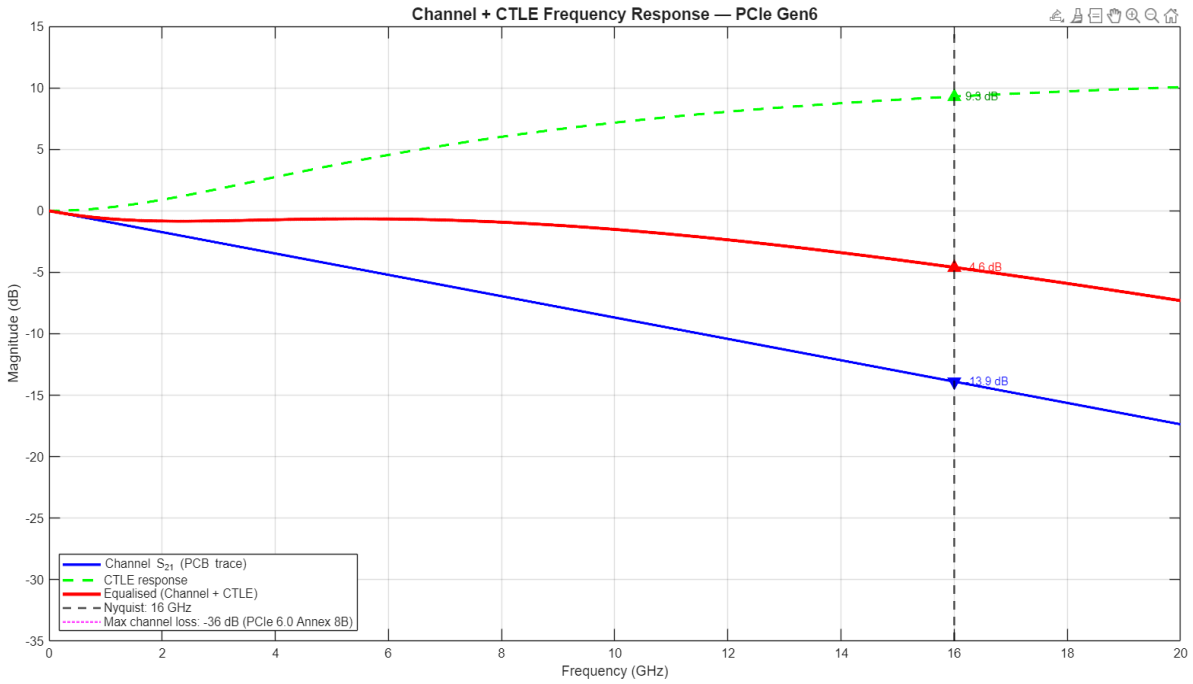


Figure 4.6 Channel + CTLE Frequency Response

Figure 4.4 presents the **theoretically modelled** frequency responses of the PCIe Gen6 channel, the CTLE equalizer, and their combined equalized response, plotted from DC to 20 GHz. The channel model uses a skin-effect and dielectric loss analytical model representative of a standard FR4 PCB trace. The CTLE is modelled as a first-order lead-lag filter with a zero at 4 GHz and a pole at 16 GHz. These are **analytical frequency domain models**, not measured S-parameters from physical hardware. (**Table 4.10**) summarizes the key theoretical values at the Nyquist frequency of 16 GHz.

Table 4.16 Theoretical Frequency Response at Nyquist (16 GHz)

Response	Theoretical Value at 16 GHz	Specification Limit	Status
Channel S_{21} (PCB trace model)	−13.9 dB	≤ −36 dB (max budget)	Pass
CTLE Peaking	+9.9 dB	—	—
Equalized (Channel + CTLE)	−4.6 dB	—	Compensated
Margin to Spec Limit	22.1 dB	—	Positive

The theoretical channel loss of −13.9 dB at 16 GHz is well within the PCIe Gen6 maximum allowable channel loss of −36 dB (Annex 8B), providing a theoretical margin of 22.1 dB. The CTLE model contributes +9.9 dB of high-frequency peaking, reducing the net equalized response to −4.6 dB. This theoretical analysis confirms that the equalization scheme managed by the Link Equalization Controller (Link Equalization Controller.v) RTL module is sufficient to compensate the modelled channel roll-off and achieve the eye opening.

5 Chapter Five: Transaction Layer

This chapter presents the PCIe Transaction Layer, which represents the highest logical layer within the PCIe protocol stack and operates between the device Application Core and the Data Link Layer. The Transaction Layer serves as the primary mechanism for data exchange across the PCIe interface through the generation, transmission, and processing of Transaction Layer Packets (TLPs), which are used to transport memory, I/O, configuration, and message transactions between communicating devices. In addition, the chapter discusses the major operations performed by the Transaction Layer to ensure reliable and efficient communication, including packet processing, packet routing, flow control management, transaction ordering, completion handling for non-posted requests, and end-to-end data integrity protection using ECRC generation and verification mechanisms. The presented concepts provide the theoretical and architectural foundation required for understanding the implementation and simulation of the PCIe Gen 6 Transaction Layer modules discussed in the following sections (**Figure 5.1**).

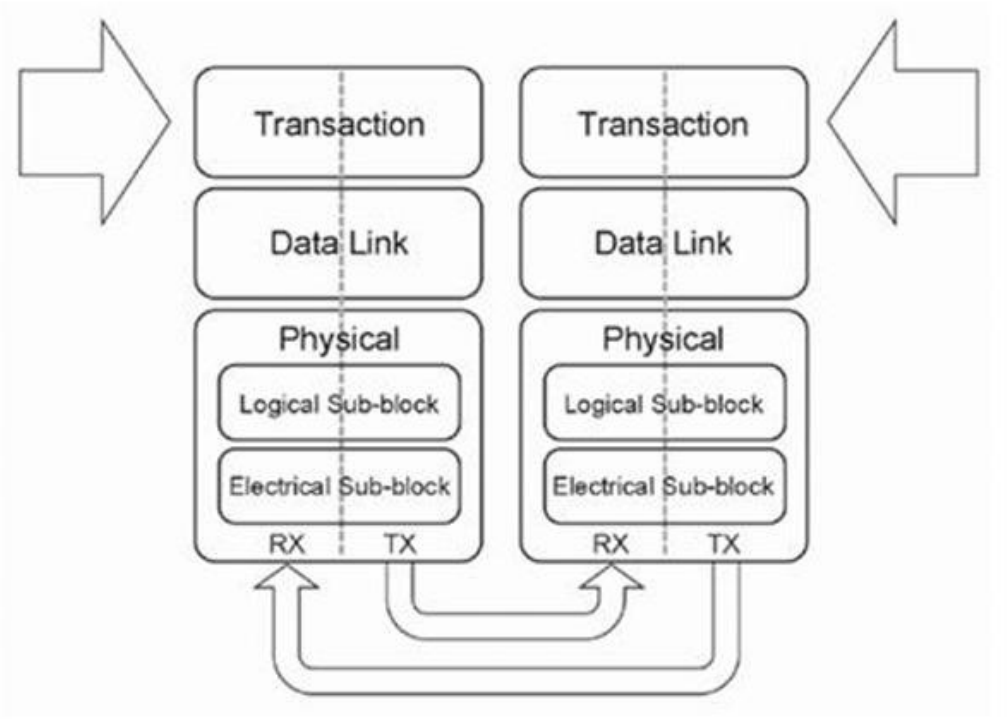


Figure 5.1 Layering Diagram Highlighting the Transaction Layer

5.1 Transaction Layer Architecture

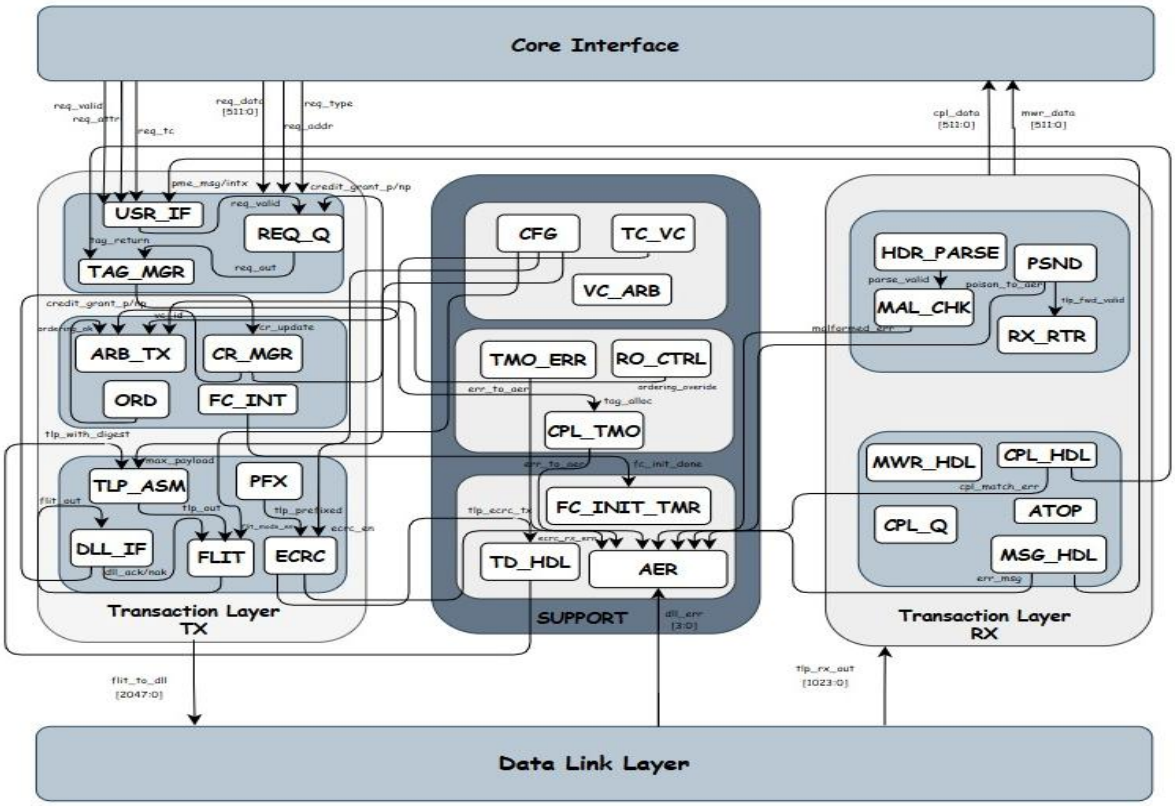


Figure 5.2 Transaction Layer Core Architecture

5.2 Transaction Layer Interface Signals

Following (**Table 5.1**) presents the complete Transaction Layer interface signals used in the proposed PCIe Gen6 architecture. The table summarizes the direction, bit-width, and functional description of each port, providing a clear overview of the communication and control signals exchanged between the Transaction Layer modules and adjacent PCIe layers.

Table 5.1 Transaction Layer Interface Signals

Signal Name	Type	Size (bit)	Description
clk	in	1	System clock signal.
rst_n	in	1	Active-low synchronous reset.
req_type	in	4	Specifies PCIe request type from user logic.
req_addr	in	64	Carries request target address from user logic.

req_len	in	10	Indicates request payload length in DWORDs.
req_data	in	512	Carries request payload data from user logic.
req_valid	in	1	Indicates a valid request is presented by user logic.
req_attr	in	3	Defines request attributes (relaxed ordering, no-snoop).
req_tc	in	3	Specifies traffic class for the request.
req_first_be	in	4	Indicates first DWORD byte enables.
req_last_be	in	4	Indicates last DWORD byte enables.
req_ready	out	1	Indicates Transaction Layer is ready to accept a new request.
usr_cpl_data	out	512	Carries completion payload data delivered to user logic.
usr_cpl_valid	out	1	Indicates a valid completion is available for user logic.
usr_cpl_status	out	3	Reports completion status code to user logic.
usr_cpl_tag	out	10	Identifies the tag of the completion delivered to user logic.
usr_mwr_data	out	512	Carries received posted memory write data to user logic.
usr_mwr_valid	out	1	Indicates valid posted memory write data is available for user logic.
usr_mwr_addr	out	64	Carries target address of the received posted memory write.
dll_ack	in	1	Acknowledges successful FLIT transmission from Data Link Layer.
dll_nak	in	1	Requests FLIT retransmission from Data Link Layer.
dll_up	in	1	Indicates the Data Link Layer link is active and operational.
cr_update	in	72	Receives flow control credit update (UpdateFC DLLP) from DLL.

cr_update_valid	in	1	Indicates a valid credit update has been received.
flit_to_dll	out	2048	Outputs packetized FLIT data to the Data Link Layer.
flit_to_dll_valid	out	1	Indicates the FLIT output to DLL is valid.
dll_ready	out	1	Indicates the DLL interface is ready to accept FLITs.
tlp_cfg_in	in	256	Carries incoming configuration TLP data for config space accesses.
tlp_cfg_valid	in	1	Indicates the incoming configuration TLP is valid.
cfg_addr	in	12	Address for direct configuration space read/write access.
cfg_wr_data	in	32	Write data for direct configuration space write operations.
cfg_wr_en	in	1	Enables a direct configuration space write operation.
cfg_rd_data	out	32	Returns read data from the configuration space.
cfg_rd_valid	out	1	Indicates the configuration space read data output is valid.
aer_status	out	32	Reports the Advanced Error Reporting status register.
aer_int	out	1	Asserts an interrupt when an AER error condition is logged.
err_msg_tlp	out	256	Carries the error message TLP generated by the AER logger.
err_msg_valid	out	1	Indicates the error message TLP output is valid.
vc0_req	in	1	Virtual Channel 0 arbitration request from upper logic.
vc1_req	in	1	Virtual Channel 1 arbitration request from upper logic.
vc2_req	in	1	Virtual Channel 2 arbitration request from upper logic.
vc3_req	in	1	Virtual Channel 3 arbitration request from upper logic.
vc_arb_scheme	in	2	Selects the VC arbitration scheme (Round-Robin or WRR).

vc_weight	in	32	Carries per-VC weights for weighted round-robin arbitration.
vc_grant	out	4	One-hot grant vector indicating which VC has been selected.
vc_grant_id	out	3	Binary-encoded ID of the granted Virtual Channel.
vc_arb_valid	out	1	Indicates the VC arbiter grant output is valid.
fc_init_done_out	out	1	Indicates flow control initialization has completed successfully.
ordering_ok_out	out	1	Indicates transaction ordering rules are currently satisfied.
tag_exhausted_out	out	1	Indicates no transaction tags are available for new requests.
outstanding_count_out	out	10	Reports the current number of outstanding non-posted requests.

5.3 Transaction Layer Operational Flow

The Transaction Layer in PCIe Gen 6 operates as a continuous processing pipeline responsible for transforming high-level software requests into structured, reliable, and transmission-ready FLITs. Rather than functioning as isolated blocks, the Transaction Layer components operate sequentially, where each stage performs a specific operation required to maintain protocol correctness, data integrity, flow regulation, and high-speed communication efficiency.

The operational flow begins when the Application Layer generates a transaction request such as a Memory Read, Memory Write, Configuration access, or Message transaction. The request enters the Transaction Layer together with its associated control information, including the target address, payload length, traffic class, byte enables, and transaction attributes.

5.3.1 Request Admission and Flow Control Verification

Before a request is allowed to enter the transmission pipeline, the Transaction Layer first verifies that sufficient receiver buffer space is available. This process is managed using PCIe's credit-based Flow Control mechanism.

The Transaction Layer checks the availability of Posted, Non-Posted, or Completion credits depending on the transaction type. If sufficient credits are not available, the request is temporarily stalled until updated credit information is received from the receiver's side. This mechanism prevents receiver buffer overflow and guarantees lossless packet delivery across the PCIe link. In PCIe Gen 6, Flow Control operation became more critical due to FLIT-based communication and the significantly higher data throughput introduced by PAM4 signaling.

5.3.2 Transaction Classification and Tag Allocation

Once admitted into the pipeline, the request is classified according to its transaction type. Posted transactions, such as Memory Writes, may proceed without requiring a response, while Non-Posted transactions, such as Memory Reads, require Completion packets to be returned later by the completer device.

For Non-Posted transactions, the Transaction Layer allocates a unique Tag value used to track outstanding requests. This Tag is later used to associate incoming Completion packets with their original transmitted requests. At the same stage, the Ordering Logic evaluates transaction attributes such as Relaxed Ordering and ID-Based Ordering to ensure that packet transmission complies with PCIe ordering rules and memory consistency requirements.

5.3.3 TLP Construction and Header Processing

Following classification and ordering verification, the request is converted into a Transaction Layer Packet (TLP). The Transaction Layer generates the required header fields — including Format (Fmt), Type, Traffic Class (TC), Attributes (Attr), Length, Tag, and Address — which collectively define packet format, routing behavior, priority level, and transaction management information throughout the PCIe hierarchy.PCIe Gen 6 additionally introduced Orthogonal Header Content (OHC), which provides auxiliary metadata and extended control information required for efficient FLIT-based operation, improving packet management efficiency while minimizing protocol overhead within densely packed FLIT structures.

5.3.4 FLIT Packing and Data Aggregation

Once the TLP is generated, it is prepared for FLIT-based transmission. Since PCIe Gen 6 no longer transmits variable-sized packets directly across the link, each TLP must be packed into a fixed-size 256-byte FLIT structure. The Transaction Layer dynamically aggregates multiple smaller TLPs into a single FLIT whenever space permits; conversely, TLPs exceeding the remaining FLIT capacity are segmented and continued in the next FLIT. This packing mechanism significantly improves bandwidth utilization by minimizing idle transmission gaps and reducing the framing inefficiencies present in earlier PCIe generations.

5.3.5 ECRC Generation and Deterministic Framing

Prior to transmission, the Transaction Layer generates the End-to-End Cyclic Redundancy Check (ECRC) field to provide end-to-end payload integrity protection across the PCIe hierarchy. Following ECRC insertion, the FLIT enters the deterministic framing stage. Unlike previous PCIe generations that relied on variable packet boundaries, PCIe Gen 6 uses fixed framing in which every FLIT has a predictable structure and fixed alignment boundaries. This deterministic organization simplifies synchronization and enables the receiver to identify packet boundaries, metadata regions, and protection fields without continuously scanning for start and end markers.

5.3.6 Transmission to the Data Link Layer

Once FLIT assembly is complete, the fully constructed transmission block is forwarded to the Data Link Layer for sequence numbering, link-level reliability processing, and physical transmission preparation. At this point, the Transaction Layer has successfully transformed a high-level software transaction into a protocol-compliant, flow-controlled, error-protected, and FLIT-optimized transmission structure suitable for reliable 64 GT/s PCIe Gen 6 communication.

5.4 Transaction Layer Responsibilities

The Transaction Layer acts as the primary control and management layer within the PCIe protocol stack. Every request packet requiring a response is implemented as a split transaction, with each packet carrying a unique identifier that directs response packets to the correct originator. The packet format supports multiple addressing modes — Memory, I/O, Configuration, and Message — and packets may carry attributes such as No Snoop, Relaxed Ordering, and ID-Based Ordering (IDO).

The table below summarizes the major functional responsibilities of the Transaction Layer:

Table 5.2 Transaction Layer responsibilities

Function	Description
TLP Generation	Creates Transaction Layer Packets (TLPs) from Application Layer requests by adding the required headers, routing information, attributes, and payload formatting.
TLP Decoding	Interprets received TLPs, validates packet formatting, extracts routing information, and forwards valid transactions to the appropriate destination.
Flow Control	Utilizes a credit-based mechanism to regulate packet transmission and prevent receiver buffer overflow by ensuring sufficient buffer space is available before transmission.
ECRC Generation & Checking	Generates and verifies End-to-End Cyclic Redundancy Check (ECRC) values to detect data corruption during packet transmission and routing.
Ordering	Maintains PCIe transaction ordering rules to preserve data dependencies, ensure memory consistency, and prevent protocol violations.
Completion Handling	Manages Non-Posted transactions by tracking outstanding requests and generating or matching Completion TLPs with their corresponding requests.
VC Arbitration	Performs arbitration between Virtual Channels (VCs) and Traffic Classes (TCs) to ensure fair packet scheduling, balanced traffic distribution, and Quality of Service (QoS) support.

Together, these functions allow the Transaction Layer to operate as a highly organized and reliable communication engine capable of supporting the high bandwidth, low latency, and data integrity requirements of PCIe Gen 6 systems.

5.5 TLP Structure Section

The Transaction Layer Packet (TLP) is the fundamental communication unit used within the PCIe protocol. The Transaction Layer is responsible for generating, transmitting, receiving, and processing these packets to enable communication between PCIe devices. Structurally, a TLP is composed of four primary sections: Prefixes, the Header, the Data Payload, and the End-to-End CRC (ECRC).

5.5.1 TLP Prefixes

TLP Prefixes are optional 1-DWORD (4-byte) fields inserted before the standard TLP header, used to carry additional routing information and transaction-processing metadata that cannot be accommodated within the standard header structure (Examples include End-to-End Prefixes and Local Prefixes).

5.5.2 TLP Header

TLP Header contains all control, routing, and formatting information required for packet interpretation and delivery. Depending on the addressing mode, the header is either 3 DWORDs (12 bytes) for 32-bit addressing or 4 DWORDs (16 bytes) for 64-bit addressing (**Figure 5.3**)

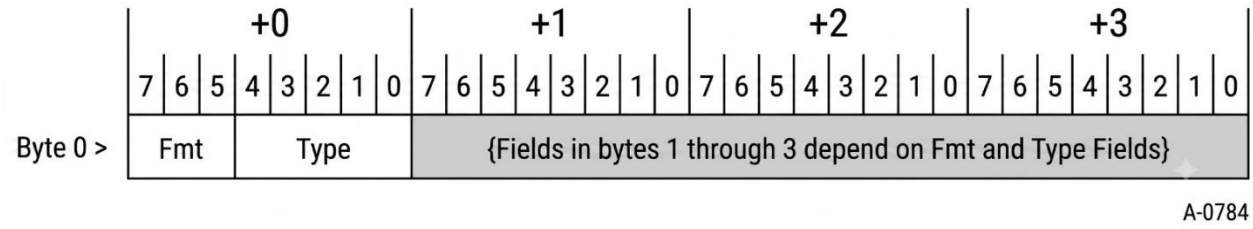


Figure 5.3 TLP Header PCI0ExpressBaseSpecifications

5.5.3 End-to-End CRC (ECRC)

The Transaction Layer utilizes a 32-bit integrity check that protects packet data from the original transmitter to the final receiver. Unlike the Link CRC (LCRC), which protects data only across a single physical link, ECRC provides end-to-end coverage across the entire PCIe hierarchy.

5.5.4 Data Payload

Payload carries the actual application data transferred between devices — such as Memory Write or Completion data. Payload size is variable and constrained by the negotiated Max Payload Size supported by the PCIe link (**Figure 5.4**).

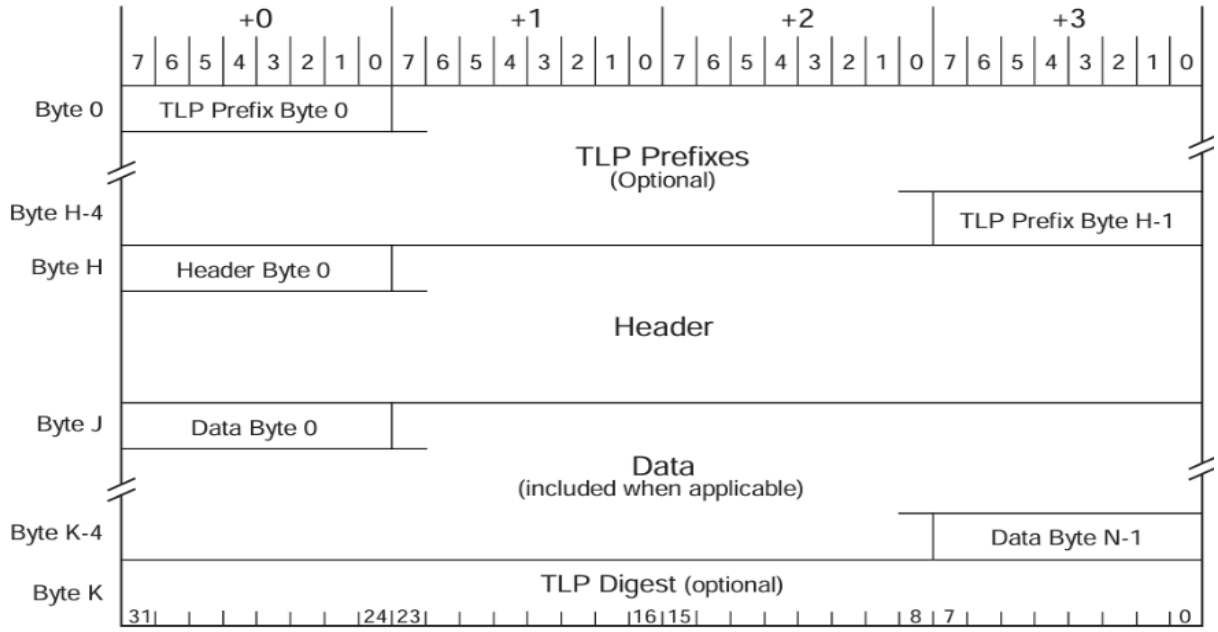


Figure 5.4 Generic TLP Format

5.5.5 Key TLP Header Fields and Their RTL Impact

The header fields directly influence the behavior of RTL modules within the Transaction Layer, driving packet decoding, routing decisions, arbitration logic, ordering control, and error detection (**Table 5.3**).

Table 5.3 TLP header fields and RTL impact

Field	Purpose	RTL Impact
Fmt (Format)	Defines the packet format and indicates whether a payload exists.	Determines header size (3DW or 4DW) and whether the data path should capture payload data.

Type	Specifies the transaction type such as Memory Read, Memory Write, I/O, or Configuration requests.	Routes packets to the correct processing module and detects unsupported request types.
TC (Traffic Class)	Indicates packet priority for Quality of Service (QoS) management.	Maps packets into the appropriate Virtual Channel for prioritized scheduling.
Attr (Attributes)	Defines transaction ordering properties such as Relaxed Ordering and ID-Based Ordering.	Determines whether packets may bypass stalled transactions while maintaining protocol correctness.
Length	Specifies the payload size in DWORDs.	Compared against the negotiated Max Payload Size to detect malformed packets and payload violations.
Tag	Tracks outstanding Non-Posted requests.	Associates Completion packets with their original requests and correctly routes returned data.

Together, these fields enable the Transaction Layer to perform reliable packet processing, transaction tracking, traffic prioritization, and protocol-compliant communication within high-speed PCIe Gen 6 systems (**Figure 5.5**).

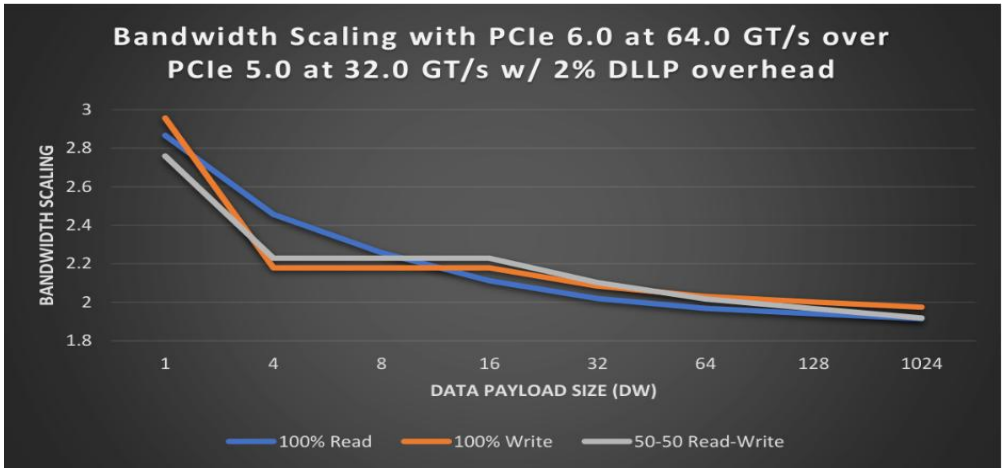


Figure 5.5 bandwidth scaling comparison (gen5 VS gen6)

5.6 Flit Mode Architecture

One of the most significant architectural changes introduced in PCIe Gen 6 is the transition from traditional variable-length packet transmission to Flit Mode architecture. The introduction of PAM4 signaling at 64 GT/s substantially increased the raw Bit Error Rate (BER), making Forward Error Correction (FEC) mandatory for reliable communication. To support efficient FEC operation, PCIe Gen 6 introduced the Flow Control Unit (FLIT) — a fixed-size 256-byte transmission block into which TLPs, DLLPs, and control information are uniformly organized.

5.6.1 256-Byte FLIT Structure

Since every FLIT has a deterministic size, the receiver always knows the precise start and end boundaries of each transmission block. This fixed framing simplifies synchronization, alignment, and packet boundary detection compared to the variable framing methods of earlier PCIe generations. A typical FLIT may contain multiple TLPs, Data Link Layer Packets (DLLPs), Orthogonal Header Content (OHC), and CRC/FEC protection fields. This architecture allows the link to operate with improved bandwidth efficiency and predictable framing behavior (**Figure 5.6**).

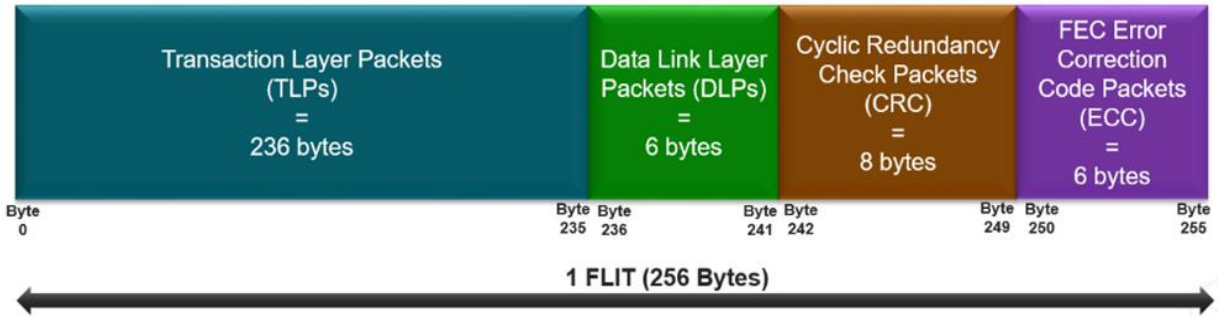


Figure 5.6 FLIT architecture

5.6.2 TLP Packing Mechanism

In Flit Mode, variable-length TLPs are dynamically packed into fixed 256-byte FLITs by the Flit Packing logic within the Transaction Layer. Multiple smaller TLPs may be aggregated into a single FLIT, while larger TLPs may span multiple consecutive FLITs. This packing mechanism significantly improves link utilization by minimizing idle gaps and reducing the framing overhead that existed in previous PCIe generations.

5.6.3 Orthogonal Header Content (OHC)

PCIe Gen 6 introduced Orthogonal Header Content (OHC) to support efficient packet management within densely packed FLIT structures. OHC carries additional metadata and extended control information required for packet interpretation, routing, and processing, without significantly increasing protocol overhead. This enables the Transaction Layer to maintain efficient packet handling while supporting the increased architectural complexity of Flit Mode operation (**Figure 5.7**).

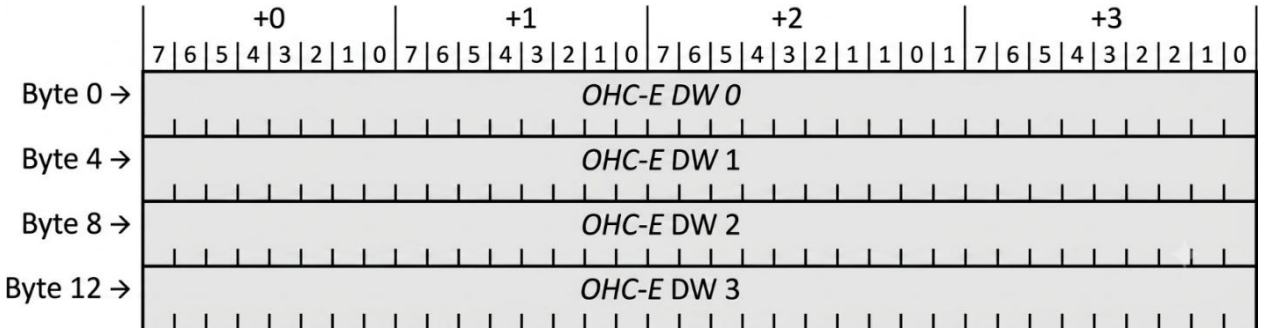


Figure 5.7 Orthogonal Header Content (OHC)

5.6.4 Relationship Between FLIT Mode and FEC

FEC algorithms operate most efficiently on fixed-size data blocks, as parity generation and error correction calculations require deterministic framing boundaries. By organizing all transmitted data into fixed 256-byte FLITs, PCIe Gen 6 provides a structure that aligns efficiently with lightweight FEC processing, allowing the protocol to maintain reliable communication despite the increased signal sensitivity associated with PAM4 modulation (**Figure 5.8**).

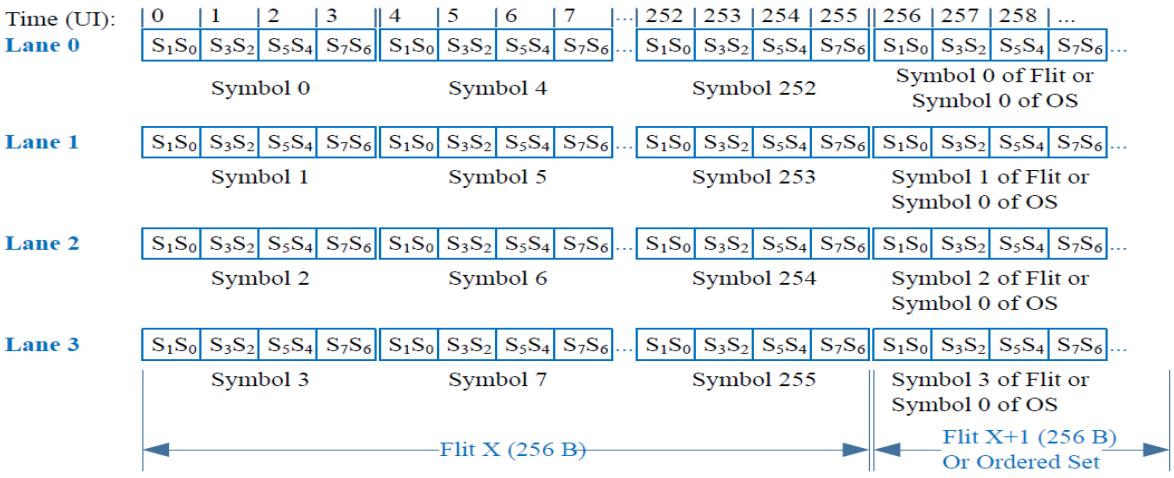


Figure 5.8 Fixed-Size FLIT Structure Supporting FEC

5.6.5 Architectural Tradeoff: Reliability vs. Overhead

Flit Mode delivers a significant reliability improvement by combining fixed framing, lightweight FEC, and strong CRC protection. This reduces retransmission probability and improves communication robustness — particularly important for high-performance computing, AI accelerators, and data center systems. However, every FLIT must reserve space for FEC and CRC protection fields, introducing additional protocol overhead. Despite this, PCIe Gen 6 achieves higher overall efficiency by eliminating the synchronization headers and inter-packet gaps required in previous generations. Together, fixed-size FLIT framing and lightweight FEC form the foundation of the PCIe Gen 6 high-speed communication architecture, enabling reliable operation at 64 GT/s while maintaining low latency and high bandwidth efficiency (**Figure 5.9**).

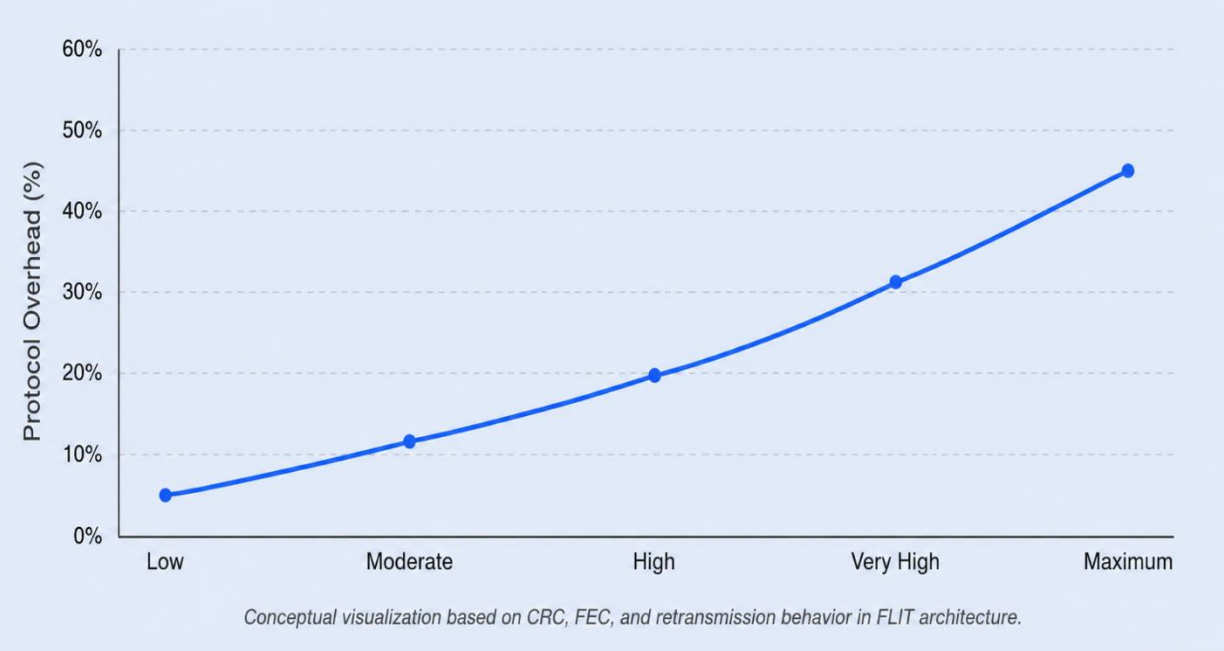


Figure 5.9 FLIT Architectural Tradeoff: Reliability vs. Overhead

5.7 PCIe Gen 6 Impact on Transaction Layer

In PCIe Gen 6, the importance of the Transaction Layer significantly increased due to the transition from NRZ signaling to PAM4 modulation at 64 GT/s. Although PAM4 doubled the achievable data rate, it also introduced a higher Bit Error Rate (BER), making Forward Error Correction (FEC) mandatory for reliable communication. To support low-latency FEC implementation, PCIe Gen 6 introduced FLIT (Flow Control Unit) Mode, requiring the Transaction Layer to efficiently pack and manage variable-length TLPs within fixed-size 256-byte FLITs. Consequently, the Gen 6 Transaction Layer became a key component in balancing high-speed performance, reliability, and protocol efficiency (**Figure 5.10**).

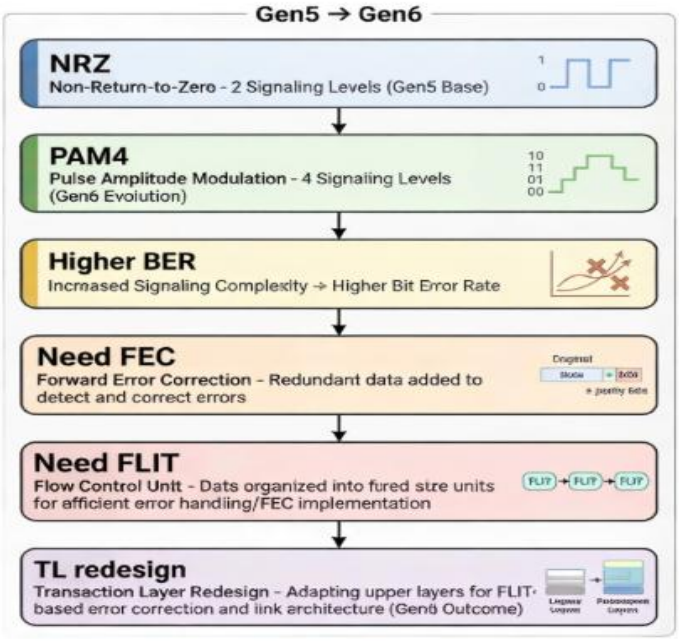


Figure 5.10 Impact of PAM4 signaling on PCIe Gen6 Transaction Layer architecture

Despite its performance advantages, PAM4 introduces a substantially higher raw Bit Error Rate (BER) due to the reduced voltage margin between signal levels. As a result, PCIe Gen 6 mandated the use of Forward Error Correction (FEC) to detect and correct transmission errors in real time while maintaining reliable high-speed communication.

The introduction of FEC fundamentally changed the behavior of the Transaction Layer. Since FEC algorithms operate on fixed-size data blocks, the conventional variable-length packet transmission method used in earlier PCIe generations was no longer sufficient. To address this limitation, PCIe Gen 6 introduced FLIT (Flow Control Unit) Mode, where data is transmitted using fixed-size 256-byte FLITs. Consequently, the Transaction Layer became responsible for efficiently packing, tracking, and managing variable-length Transaction Layer Packets (TLPs) within these fixed FLIT boundaries.

These architectural modifications significantly changed the operation of the Transaction Layer compared to previous PCIe generations (**Table 5.4**):

Table 5.4 Comparison of Transaction Layer Features Between PCIe 5 And PCIe 6

Feature	**PCIe Gen 5**	**PCIe Gen 6**
Modulation Technique	NRZ (Non-Return-to-Zero)	PAM4 (4-Level Pulse Amplitude Modulation)
Packet Structure	Variable-length TLPs	Fixed-size 256-Byte FLITs
Raw Bit Error Rate	Lower BER	Higher BER
Framing Mechanism	Traditional packet framing	FEC-dependent FLIT framing
Error Protection	Standard CRC mechanisms	Lightweight FEC with Strong CRC
Transaction Layer Complexity	Lower	Significantly Higher

Several new architectural concepts emerged in PCIe Gen 6 as a direct consequence of FLIT-based communication. FLIT Mode was primarily introduced to support low-latency FEC operation by providing fixed-size transmission blocks suitable for parity generation and error correction. Additionally, End-to-End Cyclic Redundancy Check (ECRC) became increasingly important for ensuring payload integrity across complex packet packing, unpacking, and routing operations within the PCIe hierarchy.

5.8 Design Trade-offs and Architectural Decisions

Implementing the PCIe Gen 6 Transaction Layer required several carefully considered architectural decisions to balance throughput, hardware complexity, timing closure, resource utilization, and protocol reliability. Since PCIe Gen 6 operates at 64 GT/s using PAM4 signaling and FLIT-based communication, the design process involved carefully selecting implementation strategies capable of maintaining high bandwidth efficiency while remaining practical for FPGA-based realization. The table below summarizes the major trade-offs adopted:

Table 5.5 major architectural trade-offs

Component	Option Chosen	Alternative	Engineering Reason
FIFO Architecture	Synchronous FIFO	Asynchronous FIFO	The Transaction Layer and FLIT processing pipeline operate within a common clock domain; Synchronous FIFOs reduce hardware overhead, simplify timing analysis, and eliminate latency from multi-stage clock-domain synchronizers.
Arbitration Logic	Round Robin Arbitration	Fixed Priority Arbitration	Round Robin ensures fair scheduling across Posted, Non-Posted, and Completion transactions, preventing starvation of lower-priority traffic under heavy load.
CRC Generation	Parallel CRC-32	Serial CRC	Serial CRC cannot meet required timing constraints at Gen 6 throughput rates. Parallel CRC computes across wide FLIT data paths within a single stage using combinational XOR networks.
Packing Strategy	Aggressive FLIT Packing	Simple One-TLP-Per-FLIT Packing	Aggregating multiple TLPs per FLIT minimizes internal fragmentation and maximizes bandwidth utilization, particularly for smaller packet workloads.
Header Management	Orthogonal Header Content (OHC)	Traditional Integrated Header Handling	OHC improves metadata organization and supports efficient packet interpretation within densely packed FLIT structures, improving framing efficiency despite added implementation complexity.

5.8.1 Parallel CRC Implementation

PCIe Gen 6 FLIT processing typically operates on wide data paths to maintain high throughput while keeping clock frequencies within practical FPGA timing limits. Under these conditions, serial CRC calculation introduces excessive latency and becomes unsuitable for high-speed FLIT processing pipelines.

To address this limitation, the Transaction Layer utilizes a parallel CRC-32 architecture capable of computing the CRC value across the full Datapath width within a single processing stage. This implementation improves throughput, reduces CRC processing latency, and supports the deterministic framing behavior required by FLIT Mode operation.

5.8.2 Arbitration Fairness and Traffic Scheduling

The Transaction Layer simultaneously handles multiple traffic categories, including Posted, Non-Posted, and Completion transactions. Efficient arbitration is therefore required to maintain balanced traffic scheduling and prevent starvation conditions.

Round Robin arbitration was selected because it distributes transmission opportunities fairly across competing traffic classes as the last served request will have the lowest priority in the next arbitration round. In contrast, a Fixed Priority approach could continuously favor high-throughput traffic such as Memory Writes while delaying lower-priority Completion packets, potentially increasing transaction latency and reducing overall system responsiveness (**Figure 5.11**).

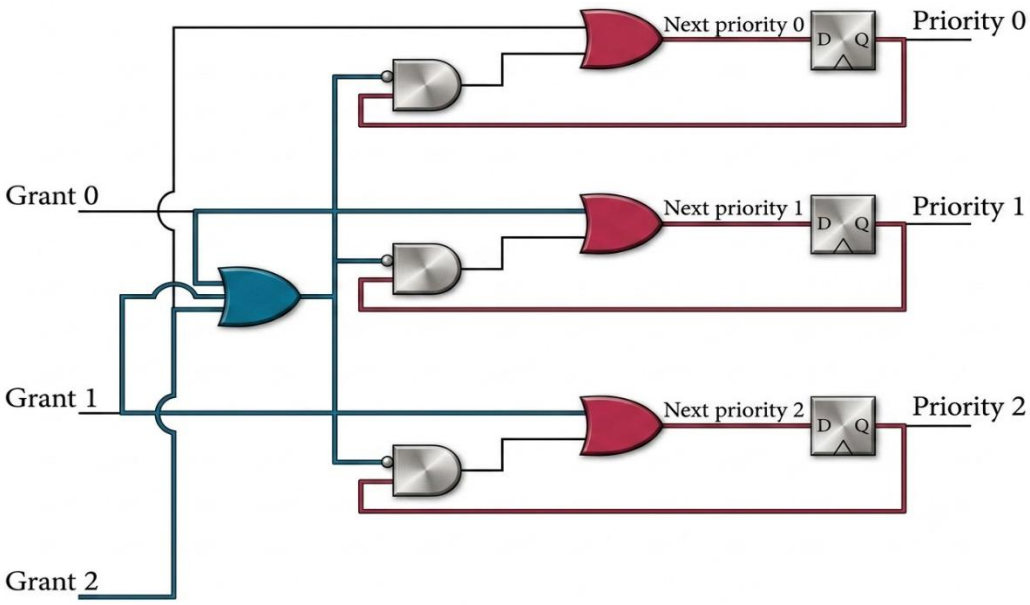


Figure 5.11 circuit required for round robin arbiter

5.8.3 FIFO Architecture Selection

FIFO buffers are extensively used throughout the Transaction Layer to support temporary packet storage, queue management, and pipeline decoupling, Since the core Transaction Layer processing pipeline primarily operates under a unified internal clock domain, while asynchronous FIFO bridges are only utilized at external interface boundaries where independent clock domains exist, Synchronous FIFOs were selected instead of Asynchronous FIFOs. This decision simplified timing closure, reduced synchronization complexity, and lowered overall hardware resource consumption by eliminating the need for Gray-code pointer synchronization and multi-stage metastability protection circuits (**Figure 5.12**).

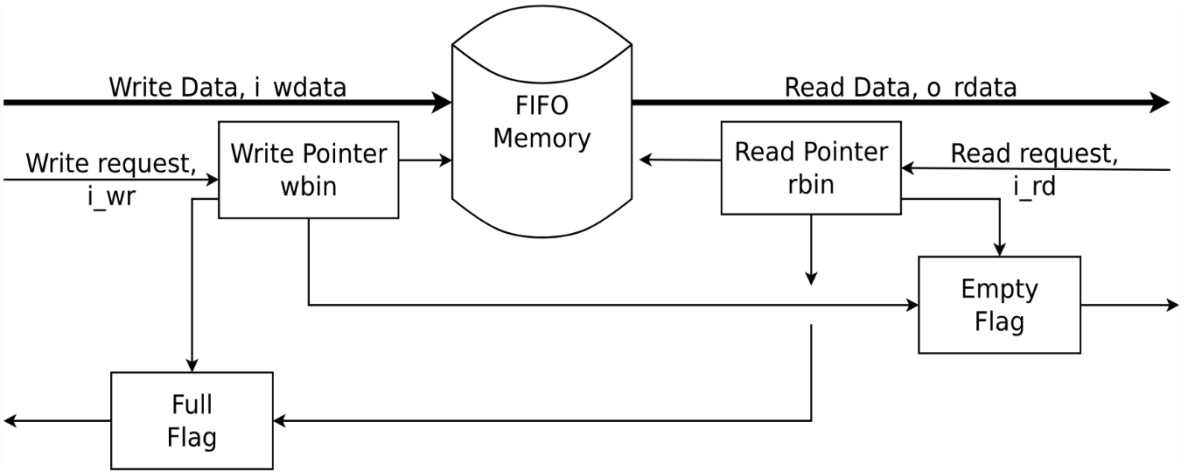


Figure 5.12 Internal Architecture of the Synchronous FIFO Buffer with Binary Pointers

5.8.4 FLIT Packing Efficiency

PCIe Gen 6 FLIT Mode requires efficient utilization of the fixed 256-byte FLIT structure to maintain high bandwidth efficiency. The implemented packing logic dynamically aggregates multiple small TLPs into a single FLIT whenever space is available. Additionally, large TLPs may be segmented across multiple FLIT boundaries when necessary. This aggressive packing strategy minimizes unused FLIT space and reduces internal fragmentation compared to simpler transmission approaches where each TLP occupies an independent FLIT. Although this approach increases the complexity of the packing logic, it significantly improves overall link utilization and supports the high-throughput requirements of PCIe Gen 6 communication systems (**Figure 5.13**).

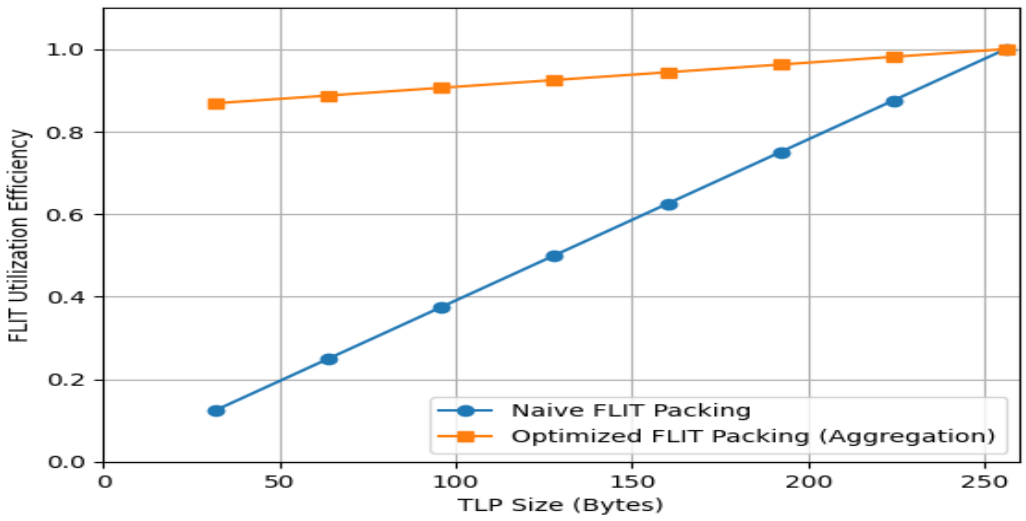


Figure 5.13 FLIT Utilization Efficiency Comparison Between Naive and Optimized Packing

5.9 Transaction Layer and Data Link Layer Interconnection

The interaction between the Transaction Layer (TL) and the Data Link Layer (DLL) defines how application data is transformed into a reliable transmission stream. These layers operate in a producer-consumer relationship: the Transaction Layer defines the semantic meaning and transaction behavior of the data, while the Data Link Layer manages transmission reliability, integrity protection, and link-level delivery.

5.9.1 Functional Interdependence

Once the Transaction Layer assembles a TLP, it forwards the packet to the Data Link Layer, which appends a unique Sequence Number and a 32-bit Link CRC (LCRC) field, enabling the receiving DLL to detect transmission errors. The Data Link Layer further protects Transaction Layer traffic through the Replay Buffer mechanism. If a transmitted TLP becomes corrupted during transmission, the receiving DLL generates a Negative Acknowledgement (NAK) DLLP requesting retransmission. The transmitting DLL then automatically retransmits the stored TLP directly from the Replay Buffer without requiring the Transaction Layer to regenerate or reconstruct the packet. This mechanism allows the Transaction Layer to remain isolated from most link-level retransmission operations while maintaining reliable communication behavior the packet.

5.9.2 Flow Control and Credit Management

One of the most critical interactions between the Transaction Layer and the Data Link Layer is the Credit-Based Flow Control mechanism. The Transaction Layer is prohibited from transmitting a TLP unless sufficient receiver buffer space is available. This availability is communicated using Flow Control (FC) Credits.

The Data Link Layer is responsible for receiving UpdateFC DLLPs from the link partner and continuously updating the corresponding credit registers used by the Transaction Layer before packet transmission decisions are made. The Transaction Layer therefore relies on the DLL to maintain accurate visibility of available Posted, Non-Posted, and Completion buffer resources throughout link operation.

5.9.3 Hardware-Level Signal Interface (RTL Perspective)

In high-performance PCIe Gen 6 controller implementations, the interface between the Transaction Layer and the Data Link Layer consists of a wide parallel data path accompanied by multiple synchronization and control signals. These interfaces ensure that packets move efficiently through the internal pipeline while preventing buffer overflow conditions, synchronization hazards, and illegal packet transmission scenarios (**Table 5.6**).

Table 5.6 Hardware-Level Signal Interface Between TL and DLL

Interface Signal Group	Direction	Functional Purpose
TLP Handoff Bus	TL → DLL	Transfers the complete TLP structure, including Header, Payload, and ECRC fields, from the Transaction Layer to the Data Link Layer for sequence-number insertion, LCRC generation, and framing operations.
Credit Status Signals	DLL → TL	Provides continuously updated Flow Control credit availability information, allowing the Transaction Layer to determine whether packet transmission is currently permitted.

DL_Active Link Status Signal	DLL → TL	Indicates that Data Link Layer initialization and Flow Control negotiation procedures have completed successfully and that the PCIe link is ready for TLP traffic transmission.
Error Reporting Signals	DLL → TL	Reports fatal link-level failures such as Replay Timer expiration, replay overflow conditions, or persistent retransmission failures requiring higher-level recovery procedures.

5.9.4 Initialization Sequence

The Transaction Layer remains idle until the Data Link Layer completes initialization with the remote link partner through the Data Link Control and Management State Machine (DLCMSM). During initialization, both ends exchange initial Flow Control credit information via InitFC1 and InitFC2 DLLPs, establishing the buffer capacities required for reliable transmission. Only upon successful completion does the DLL assert the DL_Active signal, indicating that the link is fully operational and ready for Transaction Layer traffic (**Figure 5.14**).

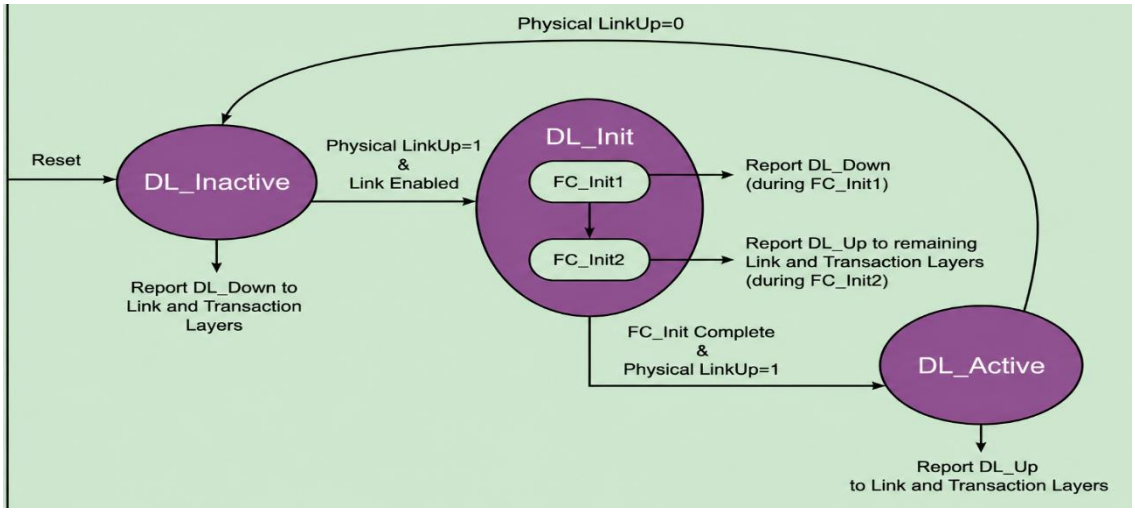


Figure 5.14 Data Link Layer (DLL) State Machine For linking

5.10 Challenges and Implementation Hurdles

1- Designing the PCIe Gen 6 Transaction Layer presents significantly greater architectural and implementation complexity than previous generations. The adoption of PAM4 signaling, mandatory Forward Error Correction (FEC), and FLIT-based communication fundamentally transforms how packets are generated, tracked, synchronized, validated, and transmitted within the RTL pipeline. Unlike earlier versions that used simpler variable-length packet framing, PCIe Gen 6 demands deterministic packet organization, optimized buffering, strict synchronization control, and much more advanced verification methods to ensure reliability at 64 GT/s.

2- One of the earliest implementation challenges emerged from the migration from traditional Non-Return-to-Zero (NRZ) signaling to 4-Level Pulse Amplitude Modulation (PAM4). In PCIe Gen 6, each Unit Interval (UI) carries two bits instead of one, effectively doubling throughput without doubling the operating frequency.

3- Although the Transaction Layer itself operates entirely on logical digital FLIT structures rather than analog waveforms, the reduced voltage margins associated with PAM4 significantly increase timing sensitivity throughout the communication stack. Consequently, debugging and verification become considerably more difficult because small synchronization inaccuracies may indirectly affect packet alignment, FLIT integrity, and protocol timing behavior.

4- As discussed in The Impact of PAM4 on PCIe 6.0 Electrical Testing, PAM4 signaling introduces smaller eye openings compared to NRZ communication, making high-speed operation more sensitive to jitter and timing variation. This increased sensitivity required additional observability within the RTL implementation itself. To improve verification visibility, dedicated internal observation signals were integrated into the FLIT Packing and transaction scheduling pipeline. These observation paths exposed internal FLIT boundaries, packet segmentation status, payload alignment positions, packing occupancy, and header insertion behavior during simulation. This significantly improved the ability to debug packet formation and monitor the internal behavior of the FLIT assembly process without relying solely on external waveform analysis.

5- Another major implementation challenge involved high-speed Tag tracking and outstanding transaction management. PCIe Gen 6 supports a large number of simultaneous outstanding Non-Posted transactions to maintain high throughput and maximize bandwidth utilization in modern accelerator and data-center environments. Each Non-Posted request must remain tracked until its associated Completion packet is returned, requiring the Transaction Layer to continuously manage allocation, tracking, retirement, and timeout monitoring of active transaction Tags.

6- To support this behavior efficiently, the architecture incorporates a high-speed Tag Management structure responsible for dynamically allocating and retiring Tags throughout the transaction lifecycle. The RTL continuously monitors outstanding transaction counts, available Tag resources, and Completion of arrival status to prevent duplicate allocation or transaction mismatches. Under heavy traffic conditions, improper Tag handling may lead to Completion misrouting, payload corruption, or protocol violations. Therefore, the implementation required careful coordination between request admission logic, Completion tracking structures, and transaction retirement mechanisms.

7- Verification of the Tag Management pipeline required extensive constrained-random testing scenarios designed to intentionally stress the architecture under worst-case operating conditions. Simulation environments generated large volumes of simultaneous outstanding requests, randomized Completion return ordering, delayed transaction responses, and forced timeout conditions to validate system stability. These verification scenarios ensured that the RTL correctly prevented duplicate Tag allocation, maintained stable transaction tracking behavior, and temporarily stalled admission of additional Non-Posted requests whenever Tag resources became unavailable.

8- Clock-domain synchronization represented another significant implementation challenge throughout the Transaction Layer architecture. In practical FPGA and SoC environments, user-side logic, transaction scheduling stages, and PCIe link interfaces frequently operate under independent clock domains. As a result, packet data must safely traverse asynchronous timing boundaries without introducing metastability, packet corruption, or data loss.

9- To address this challenge, asynchronous FIFO bridges were implemented between independent timing domains using Gray-code read/write pointers and multi-stage synchronization structures. Gray-code addressing minimizes synchronization ambiguity because only a single bit changes between adjacent counter values, reducing the probability of metastability during pointer transfer operations. These FIFO structures decouple timing differences between the application-side transaction pipeline and the PCIe link-side FLIT transmission stages while maintaining continuous packet flow and stable throughput behavior.

10- As described in Fault-Resilient PCIe Bus with Real-time Error Detection and Correction, synchronization failures are a major contributor to silent data corruption in high-speed communication systems. Consequently, verification of the synchronization logic extended beyond conventional simulation techniques. Additional assertion-based verification mechanisms were integrated to validate overflow protection, underflow prevention, pointer synchronization correctness, and stable FIFO operation during asynchronous clock variation scenarios. Extensive testing was also performed using different clock-frequency ratios, burst traffic conditions, reset transitions, and simultaneous read/write boundary cases to ensure reliable packet transfer across all operating conditions.

11- Transaction ordering and deadlock prevention introduced another layer of architectural complexity. PCIe protocols enforce strict ordering requirements to preserve memory consistency, maintain data visibility correctness, and guarantee protocol-compliant communication between requester and completer devices. In PCIe Gen 6, this challenge becomes considerably more difficult because multiple independent TLPs may coexist simultaneously inside the same fixed-size FLIT structure.

12- To preserve ordering correctness, the architecture integrates dedicated Ordering Control logic positioned between TLP generation, arbitration, and FLIT assembly stages. This logic continuously evaluates transaction dependencies, Traffic Class (TC) values, ordering attributes, outstanding Completion requirements, and scheduling conditions before permitting packets to advance into the FLIT packing pipeline. The ordering mechanism prevents illegal bypass conditions where newer transactions incorrectly overtake dependent requests already present in the transmission pipeline.

Particular care was required during arbitration between Posted, Non-Posted, and Completion traffic because improper prioritization may unintentionally introduce starvation conditions or cyclic resource dependencies. Verification environments therefore included congestion-focused stress scenarios involving Completion queue saturation, simultaneous Memory Write bursts, delayed Completion injection, and mixed traffic dependency patterns. These simulations verified that the RTL maintained forward progress under sustained congestion while preserving protocol ordering rules and avoiding deadlock conditions.

13- Malformed TLP detection and protocol validation also represented a critical aspect of the implementation process. Since PCIe Gen 6 tightly packs multiple transactions inside shared 256-byte FLIT structures, malformed packets cannot be allowed to propagate through the transmission pipeline because corruption may potentially affect neighboring transactions within the same FLIT. Consequently, the Transaction Layer incorporates dedicated packet validation logic responsible for inspecting header legality, payload Length consistency, reserved fields, packet formatting rules, and FLIT boundary alignment before transmission to the Data Link Layer.

14- If the validation logic detects structural inconsistencies such as illegal Length values, unsupported packet Types, reserved-bit misuse, or invalid segmentation behavior, the malformed transaction is immediately discarded and isolated from subsequent pipeline stages. Additional error reporting logic raises the corresponding protocol error indication while preventing corruption propagation into adjacent packet structures. Verification of these mechanisms relied heavily on negative testing methodologies in which intentionally malformed TLPs were injected into the simulation environment. These tests validated that invalid packets were consistently detected, correctly isolated, and prevented from affecting valid neighboring transactions sharing the same FLIT structure.

6 Chapter Six: Data Link Layer

This chapter presents the PCIe Data Link Layer (DLL), which occupies the intermediate stratum of the three-tier PCIe protocol stack, interposed between the Transaction Layer above and the Physical Layer below. The Data Link Layer serves as the primary mechanism for link-level reliability, ensuring that every Transaction Layer Packet (TLP) delivered by the Transaction Layer is received correctly, in order, and without loss by the remote endpoint. To fulfil this responsibility, the DLL generates and verifies Link CRC (LCRC) fields, assigns monotonically incrementing sequence numbers to every transmitted flit, maintains a replay buffer containing copies of all unacknowledged transmissions, and implements a sliding-window Automatic Repeat request (ARQ) protocol driven by ACK and NAK DLLPs exchanged with the remote DLL. In addition, the chapter discusses the major operations performed by the Data Link Layer, including link initialization and flow control negotiation, FLIT-based transmission and reception, DLLP generation and processing, power management state transitions, link bandwidth renegotiation, and centralized error classification and recovery. The presented concepts provide the theoretical and architectural foundation required for understanding the implementation and simulation of the PCIe Gen 6 Data Link Layer modules discussed in the following sections.

6.1 Data Link Layer Architecture

The PCIe Gen 6 Data Link Layer is organized around two largely independent processing pipelines — the Transmit (TX) path and the Receive (RX) path — which share only the flow control credit state and the retry buffer. The TX path accepts flits from the Transaction Layer, assigns sequence numbers and computes integrity fields, stores buffered copies pending acknowledgement, interleaves DLLP control traffic, scrambles the combined stream, and delivers it to the Physical Layer. The RX path performs the inverse: it captures incoming scrambled data from the Physical Layer, descrambles it, classifies each flit as TLP payload or DLLP control traffic, validates integrity and ordering, delivers confirmed TLPs to the Transaction Layer, and routes control DLLPs to their respective handlers. A centralized control column connects the two paths through shared flow control credit registers, acknowledgement feedback signals, replay control logic, power management state machines, and a unified error reporting interface. This separation of concerns allows each pipeline to be optimized independently while a well-defined set of shared signals maintains the global correctness invariants required by the PCIe protocol (**Figure 6.1**).

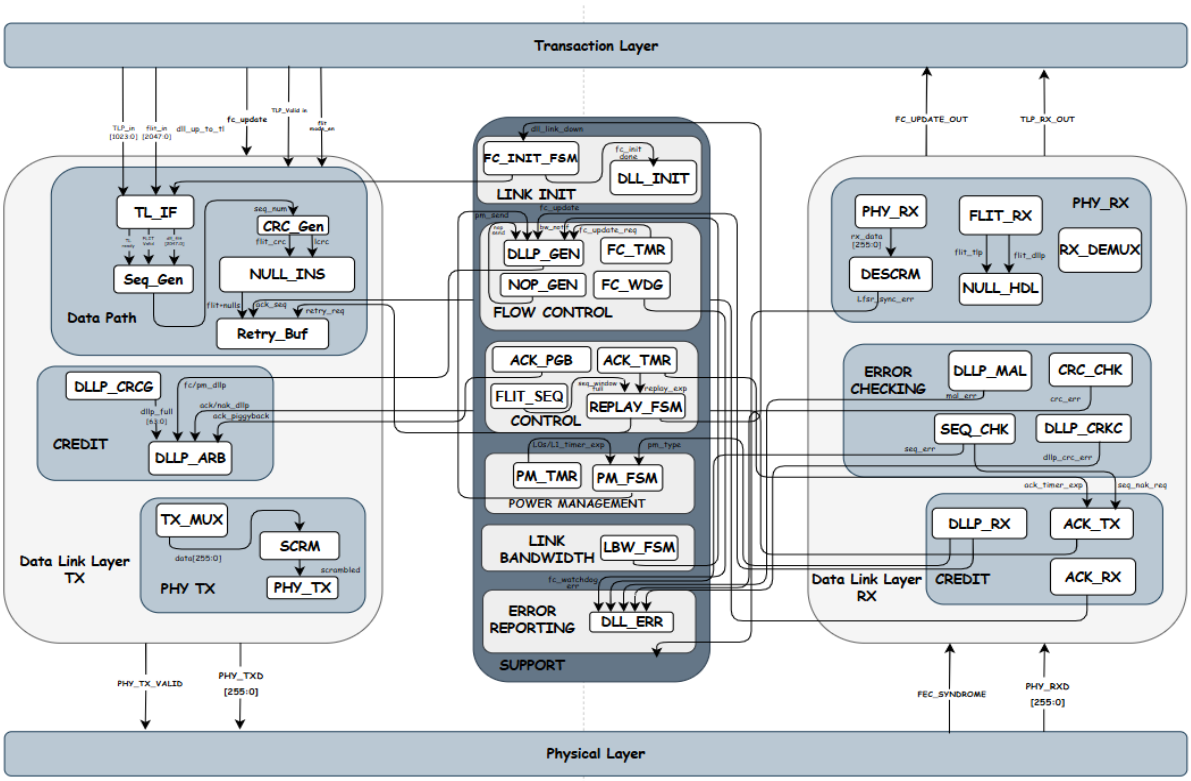


Figure 6.1 Data Link Layer Core Architecture

6.2 Data Link Layer Interface Signals

The following table presents the complete Data Link Layer interface signals used in the proposed PCIe Gen 6 architecture. The table summarizes the direction, bit-width, and functional description of each port, providing a clear overview of the communication and control signals exchanged between the Data Link Layer modules and adjacent PCIe layers (**Table 6.1**).

Table 6.1 Data Link Layer Interface Signals

Signal Name	Type	Size (bit)	Description
clk	in	1	System clock signal. All registered logic in the DLL is clocked on the rising edge.
rst_n	in	1	Active-low synchronous reset. Clears all DLL state machines, counters, and pipeline registers.
flit_mode_en	in	1	Configuration bit asserted when the link operates in Gen 6 256-byte FLIT mode.
tlp_in[1023:0]	in	1024	TLP payload bus from the Transaction Layer carrying up to 1024 bits of TLP data.
tlp_valid_in	in	1	Asserted by the TL to indicate that tlp_in holds a valid TLP on this clock cycle.

flit_in[2047:0]	in	2048	Gen 6 FLIT-mode bus from the TL carrying a complete 256-byte (2048-bit) FLIT.
flit_valid_in	in	1	Asserted when flit_in holds a valid Gen 6 FLIT ready for DLL processing.
fc_update_ph[7:0]	in	8	Posted Header flow-control credit update value from the Transaction Layer.
fc_update_valid	in	1	Asserted when fc_update_ph holds a valid flow-control credit value.
phy_rxd[255:0]	in	256	PIPE RxData bus carrying parallel received data from the analog SERDES after deserialization.
phy_rx_valid	in	1	PIPE RxDataValid signal asserted by the PHY to indicate valid received data.
phy_rx_status[2:0]	in	3	PIPE PhyStatus/RxStatus encoded bus reporting receiver events.
fec_syndrome[15:0]	in	16	16-bit FEC syndrome computed by the Gen 6 PHY FEC decoder for the current FLIT.
fec_corrected	in	1	Asserted by the PHY confirming that the FEC decoder successfully corrected all bit errors.
ltssm_dl_up	in	1	Status signal indicating the physical link has completed training and is operational.
tl_ready	out	1	Back-pressure signal to the TL. When asserted the DLL can accept a new TLP or FLIT.
fc_to_dllp[71:0]	out	72	Consolidated flow-control credit update bus forwarded to the DLLP Generator.
fc_dllp_send	out	1	Strobe requesting the DLLP Generator to transmit an UpdateFC DLLP.
phy_txd[255:0]	out	256	PIPE TxData bus driven to the analog PHY SERDES for serialization.
phy_tx_valid	out	1	PIPE TxDataValid signal asserting that phy_txd contains valid data for serialization.
phy_tx_elec_idle	out	1	PIPE TxElecIdle signal commanding the analog SERDES to drive electrical idle.
phy_tx_compliance	out	1	PIPE TX Compliance signal for compliance test pattern generation.
dll_up_to_tl	out	1	Asserted after DLL initialization completes, granting the TL permission to transmit TLPs.
fc_update_out[71:0]	out	72	Credit return information forwarded to the Transaction Layer from received UpdateFC DLLPs.
dll_link_down	out	1	Asserted when a fatal link error forces DLL re-initialization.
aer_err_out[7:0]	out	8	AER error classification bus forwarded to the Transaction Layer AER Logger.
aer_err_valid	out	1	Indicates that aer_err_out carries a valid error event for AER logging.

6.3 Data Link Layer Operational Flow

The Data Link Layer in PCIe Gen 6 operates as a continuous processing pipeline responsible for transforming Transaction Layer Packets into integrity-protected, sequence-numbered, scrambled FLITs suitable for transmission across the physical medium, and for performing the inverse reconstruction and validation on the receive side. Rather than functioning as isolated blocks, the Data Link Layer components operate sequentially and cooperatively, where each stage performs a specific operation required to maintain link reliability, flow correctness, error detection, and high-speed communication efficiency.

The operational flow begins when the link initialization sequence completes successfully. The DLL Initializer (DLL_INIT) coordinates a global reset of all pipeline registers, sequence counters, and retry buffer state, then delegates control to the Flow Control Initialization FSM (FC_INIT_FSM) to conduct the mutual credit-advertisement handshake. Only upon successful completion of this handshake does the DLL assert the dll_up_to_tl signal, granting the Transaction Layer permission to inject TLPs into the transmit pipeline.

6.3.1 Request Admission and Flow Control Verification

The first active stage of the transmit pipeline is the Transaction Layer Interface module (TL_IF). Before forwarding any TLP into the data path, TL_IF consults the credit state maintained by the Credit subsystem to verify that the remote receive buffer has sufficient space in the appropriate credit category — Posted, Non-Posted, or Completion — to accommodate the incoming TLP. If sufficient credits are available, TL_IF segments the TLP into one or more 256-byte flit payloads and forwards them downstream. If credits are insufficient, TL_IF de-asserts the tl_ready back-pressure signal, stalling the Transaction Layer until updated credit information is received. This credit-gating function at the ingress of the transmit path is the primary mechanism by which PCIe prevents a transmitter from overrunning the remote receiver's buffer, and TL_IF's role as the enforcer of that constraint is foundational to the correctness of the entire flow-control architecture (**Figure 6.2**).

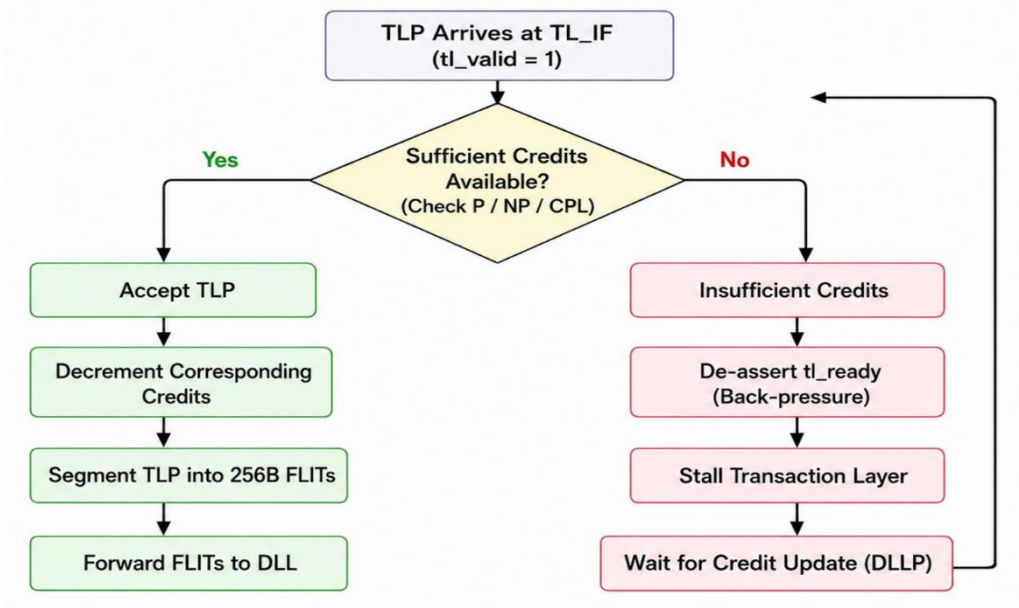


Figure 6.2 Request And Flow control Behavior In TL Interface

6.3.2 Sequence Numbering and CRC Generation

Each flit payload exiting TL_IF is presented simultaneously to the Sequence Number Generator (SEQ_GEN) and the LCRC/FLIT CRC Generator (CRC_GEN), which operate in parallel to minimize pipeline latency. SEQ_GEN maintains a 12-bit monotonically incrementing counter and prepends the current counter value to each flit as a sequence number field, enabling the receiver to detect missing or reordered flits and providing the addressing key into the retry buffer. CRC_GEN computes a 32-bit Link CRC (LCRC) over the full flit payload using the CRC-32 polynomial specified by the PCIe Base Specification, appending the result to the trailing boundary of the flit to provide error-detection coverage over its entirety. In Gen 6 FLIT mode, a 24-bit CRC is computed per 256-byte FLIT rather than per individual TLP, aligning the protection granularity with the fixed FLIT transmission unit (**Figure 6.3**).

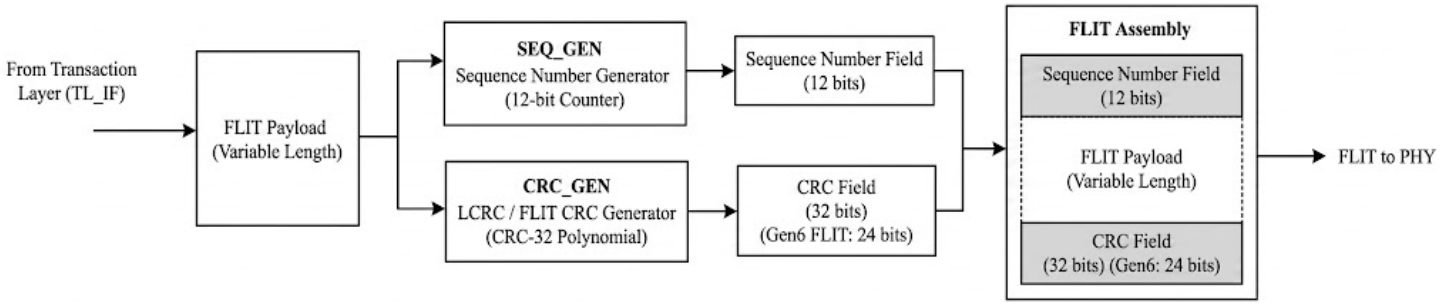


Figure 6.3 Sequence Number Assignment and CRC Generation diagram

6.3.3 Sequence Number Wrap-Around and Synchronization

The 12-bit sequence counter maintained by SEQ_GEN spans values 0 through 4095 before wrapping back to zero. Because the counter is finite, the receiver must be able to distinguish a legitimately retransmitted flit with sequence number N from a freshly transmitted flit that happens to carry the same number N following a wrap-around. The PCIe Base Specification resolves this by constraining the maximum number of outstanding unacknowledged flits — that is, the transmit window size — to be strictly less than half the sequence number space, which for a 12-bit counter means no more than 2048 flits may be in flight simultaneously.

Under this constraint, when the receiver observes sequence number N, any value within a forward distance of 2048 from the current expected receive sequence number is unambiguously a new transmission, while any value outside that window is unambiguously a duplicate or a stale retransmission and can be safely discarded (**Figure 6.4**).

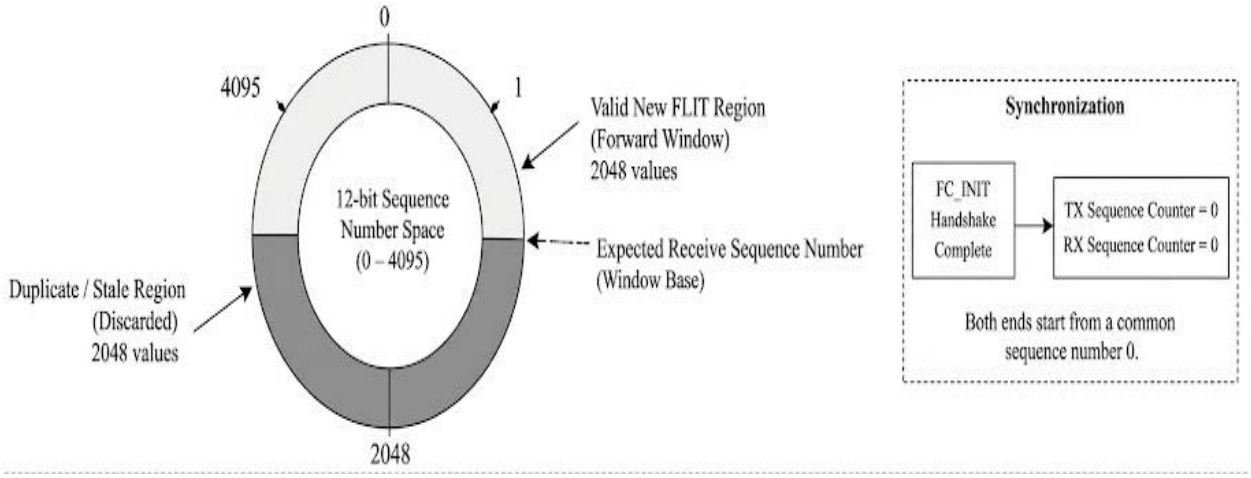


Figure 6.4 Sequence Number Synchronization and Wrap-Around Handling

Both ends of the link initialize their sequence counters to zero at the conclusion of the FC_INIT handshake, guaranteeing a common starting point. The transmitter never advances its window base beyond the oldest unacknowledged sequence number, and it never issues a new flit whose sequence number would fall within the receiver's duplicate-detection zone. As long as these invariants are maintained — and the DLL protocol enforces both through the FLIT_SEQ module and the RETRY_BUF depth limit — wrap-around introduces no ambiguity at any point during normal link operation or during replay recovery.

6.3.4 NULL FLIT Insertion and Acknowledgement Piggybacking

The output of SEQ_GEN passes to the NULL FLIT Inserter (NULL_INS). The FLIT-based architecture of PCIe 6.0 mandates that the transmitter always place a valid flit symbol on the link during every eligible clock slot; suppressing transmission during periods of low traffic is not permitted. NULL_INS fulfils this requirement by substituting a pre-formed NULL flit pattern whenever the upstream pipeline does not produce a valid TLP flit for a given slot. Beyond their padding role, NULL flits are also exploited as carriers for piggybacked acknowledgement sequence numbers, coordinated with the ACK Piggyback module (ACK_PGB). When no TLP data flit is available to carry a pending ACK value, NULL_INS embeds the current acknowledgement sequence number into the NULL flit, ensuring that the remote transmitter receives timely credit for its transmitted flits even when the local transmitter is momentarily idle. This mechanism reduces the number of dedicated ACK DLLPs required, preserving link bandwidth for payload data (**Figure 6.5**).

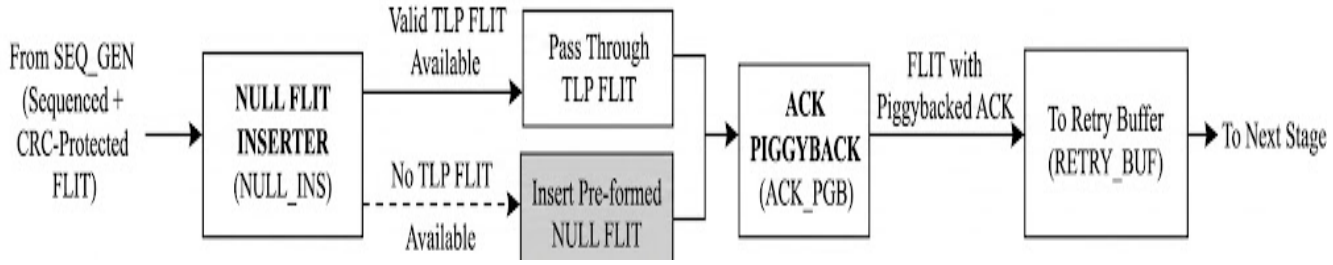


Figure 6.5 NULL FLIT Insertion and Acknowledgement Piggybacking Mechanism

6.3.5 Retry Buffering and Link-Level Reliability

The sequenced and CRC-protected flit flows into the Retry Buffer (RETRY_BUF), which stores a complete copy of every transmitted flit indexed by its sequence number. The buffer retains each copy until an explicit positive acknowledgement (ACK) is received from the remote receiver confirming correct delivery. When an ACK is received, RETRY_BUF purges all stored flits with sequence numbers up to and including the acknowledged value, reclaiming buffer space. When a NAK is received or the ACK Timer expires, the Replay FSM (REPLAY_FSM) instructs RETRY_BUF to re-inject all stored unacknowledged flits into the transmit pipeline from the indicated sequence number, implementing the sliding-window ARQ protocol at the heart of PCIe link-level error recovery (**Figure 6.6**).

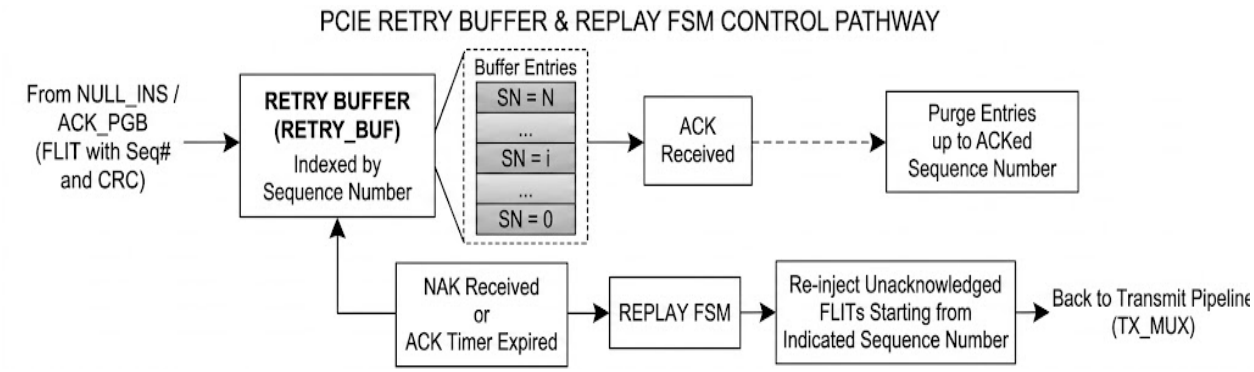


Figure 6.6 Retry Buffer Operation and Link-Level Error Recovery

6.3.6 DLLP Interleaving, Scrambling, and Physical Delivery

The output of RETRY_BUF converges at the Transmit Multiplexer (TX_MUX) with a second input stream carrying DLLPs generated by the DLLP Generator (DLLP_GEN) and DLLP Arbitrator (DLLP_ARB). TX_MUX implements a weighted priority arbitration scheme in which time-sensitive DLLPs — ACK/NAK, UpdateFC, and power-management control packets — are generally given scheduling priority over TLP data flits without causing TLP starvation. The multiplexed output enters the Scrambler (SCRM), which applies a linear-feedback shift register (LFSR) polynomial across the full 256-bit width of each flit to randomize the statistical distribution of transmitted symbols, mitigate electromagnetic emissions, and prevent long runs of identical bits from stressing the CDR circuits in the Physical Layer receiver. The scrambled output is delivered to the Physical Layer TX interface (PHY_TX) for serialization onto the PCIe lane (**Figure 6.7**).

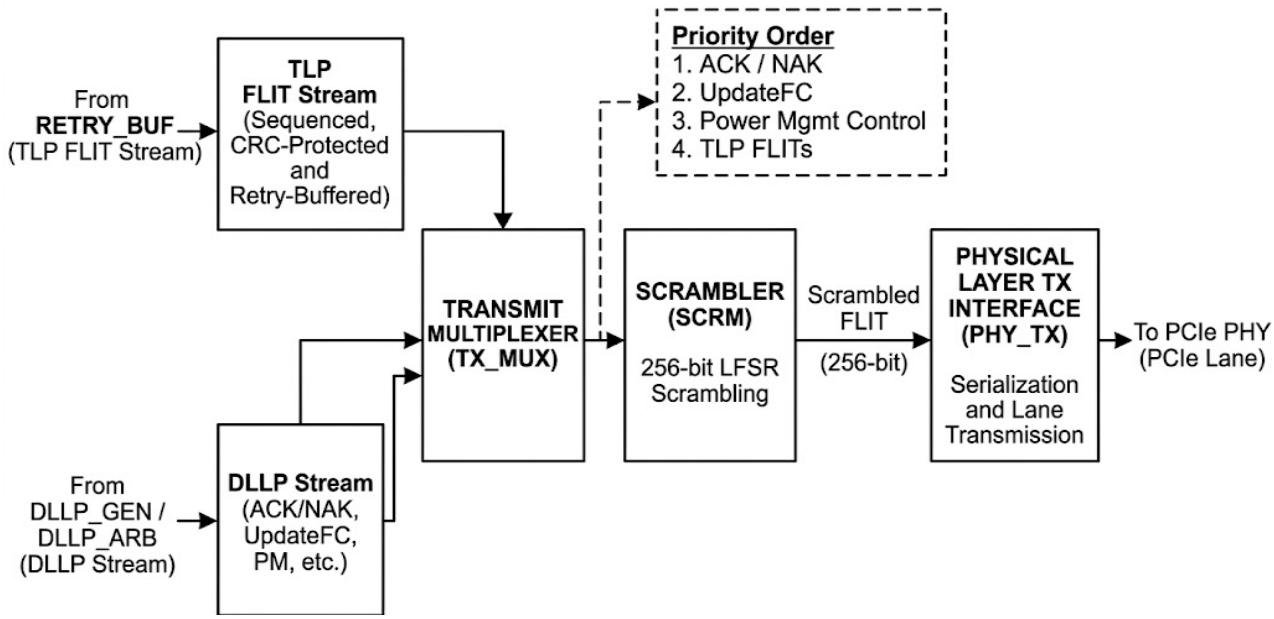


Figure 6.7 DLLP Interleaving, Scrambling, and Physical Delivery

6.4 Data Link Layer Responsibilities

The Data Link Layer acts as the primary reliability and link management layer within the PCIe protocol stack. Every transmitted flit is individually protected by a sequence number and Link CRC field, and every received flit is individually validated against both fields before being admitted to the Transaction Layer. The table below summarizes the major functional responsibilities of the Data Link Layer (**Table 6.2**).

Table 6.2 Data Link Layer Responsibilities

Function	Description
Link Initialization	Coordinates a global reset of all pipeline state and conducts the two-phase Flow Control credit-advertisement handshake with the remote DLL before permitting TLP transmission.
Sequence Numbering	Assigns a unique 12-bit monotonically incrementing sequence number to every transmitted flit, enabling the receiver to detect missing or reordered flits.
CRC Generation & Checking	Generates and verifies 32-bit Link CRC (LCRC) values per flit to detect transmission errors that escape Physical Layer FEC correction.
Retry Buffering	Maintains a buffered copy of every transmitted flit until positive acknowledgement is received, enabling retransmission without Transaction Layer involvement.
ARQ Replay Protocol	Implements a sliding-window Automatic Repeat request protocol driven by ACK and NAK DLLPs, providing link-level error recovery with minimal latency penalty.
Flow Control Management	Tracks flow control credits per category (Posted, Non-Posted, Completion) and generates UpdateFC DLLPs to return credits to the remote transmitter.
DLLP Processing	Generates, encodes, transmits, receives, and decodes Data Link Layer Packets including UpdateFC, ACK/NAK, NOP, and power-management control packets.
Scrambling & Descrambling	Applies LFSR-based scrambling on the TX path and inverse descrambling on the RX path to ensure DC balance and CDR circuit stability across the physical medium.
Power Management	Implements L0s, L1, L1.1, L1.2, and L2 power state transitions in coordination with the remote DLL and the Physical Layer LTSSM.
Error Classification	Classifies detected errors by severity, logs correctable events locally, reports uncorrectable events to the Transaction Layer AER subsystem, and triggers link re-initialization on fatal errors.

6.5 FLIT Mode Architecture in the Data Link Layer

One of the most significant architectural changes in PCIe Gen 6 that directly affects the Data Link Layer is the transition from variable-length TLP-per-packet transmission to the fixed 256-byte FLIT model. Previous PCIe generations transmitted TLPs as variable-length packets, with the DLL appending a sequence number and LCRC on a per-TLP basis. PCIe Gen 6 instead organizes all link traffic — TLPs, DLLPs, and control information — into fixed-size 256-byte FLITs, and the DLL operates on FLITs as its fundamental unit of transmission, sequencing, CRC protection, and replay (Figure 6.3).

This architectural shift was motivated by the adoption of PAM4 signaling at 64 GT/s, which substantially increased the raw Bit Error Rate (BER) compared to the NRZ modulation used in prior generations. To address this, PCIe Gen 6 made Forward Error Correction (FEC) mandatory within the Physical Layer. FEC algorithms operate most efficiently on fixed-size data blocks, as parity generation and error correction calculations require deterministic framing boundaries. By organizing all transmitted data into fixed 256-byte FLITs, PCIe Gen 6 provides a structure that aligns efficiently with the Physical Layer FEC sublayer, allowing lightweight FEC processing to maintain reliable communication despite the increased signal sensitivity of PAM4 modulation.

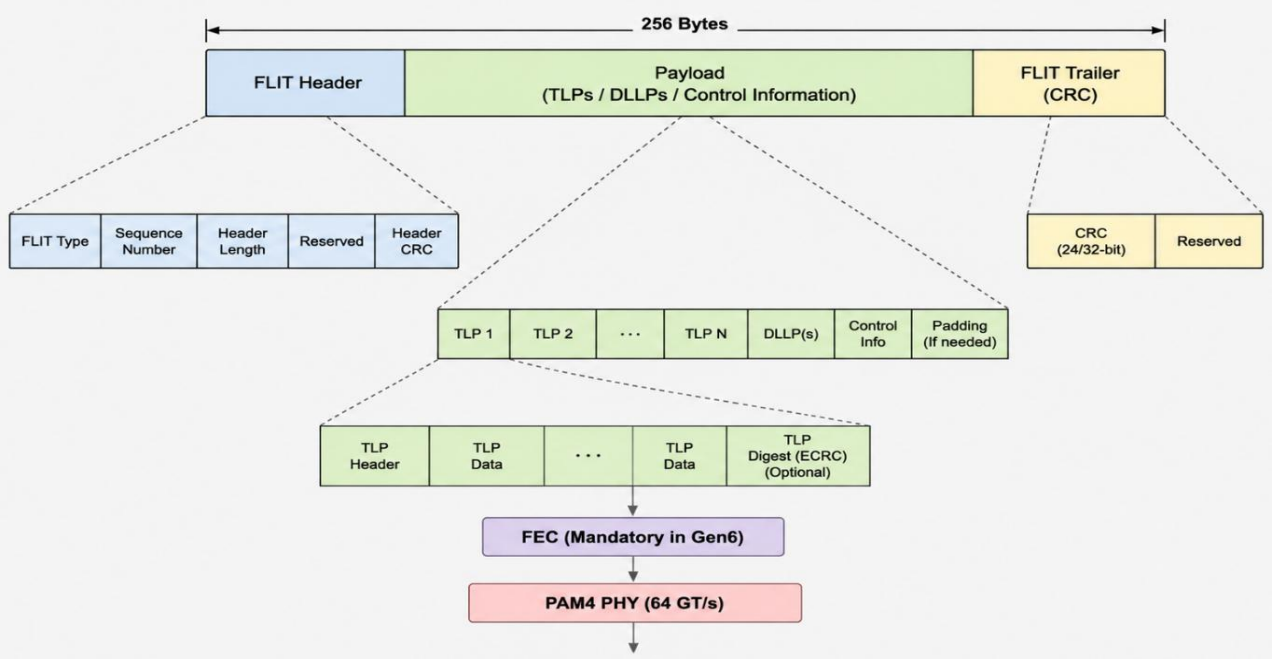


Figure 6.8 Data Link Layer FLIT-Based Transmission Architecture

6.5.1 Impact on DLL Sequence Numbering and CRC

In FLIT mode, the DLL assigns a single sequence number to each 256-byte FLIT rather than to individual TLPs. This change has several architectural consequences. The retry granularity increases to the FLIT level, meaning that a single bit error anywhere within a FLIT triggers a replay of the entire 256-byte block rather than only the affected TLP. However, because FEC typically corrects the vast majority of physical-layer bit errors before they reach the DLL, the probability of triggering a DLL-level replay remains low even at the higher raw BER of PAM4. The CRC generation and checking logic is similarly adapted: in Gen 6 mode, CRC_GEN and CRC_CHK operate on the full 256-byte FLIT payload rather than on individual TLPs, using a 24-bit CRC polynomial optimized for the FLIT data width to provide adequate error-detection coverage with minimal framing overhead.

6.5.2 FLIT Null Slot Management

A critical requirement of FLIT mode is that the transmitter must always place a valid 256-byte FLIT on the link during every eligible transmission slot, regardless of whether sufficient TLP payload data is available to fill it. The NULL_INS module fulfils this requirement by detecting idle pipeline slots and inserting pre-formed NULL FLITs to maintain a continuous, gap-free transmission stream. On the receive side, the NULL Handler (NULL_HDL) identifies NULL FLITs and discards them after extracting any piggybacked ACK sequence numbers embedded in their header fields. This null management system is critical for maintaining FLIT boundary synchronization across the link and for providing a low-overhead mechanism for delivering acknowledgement information during periods of low TLP traffic (**Figure 6.9**).

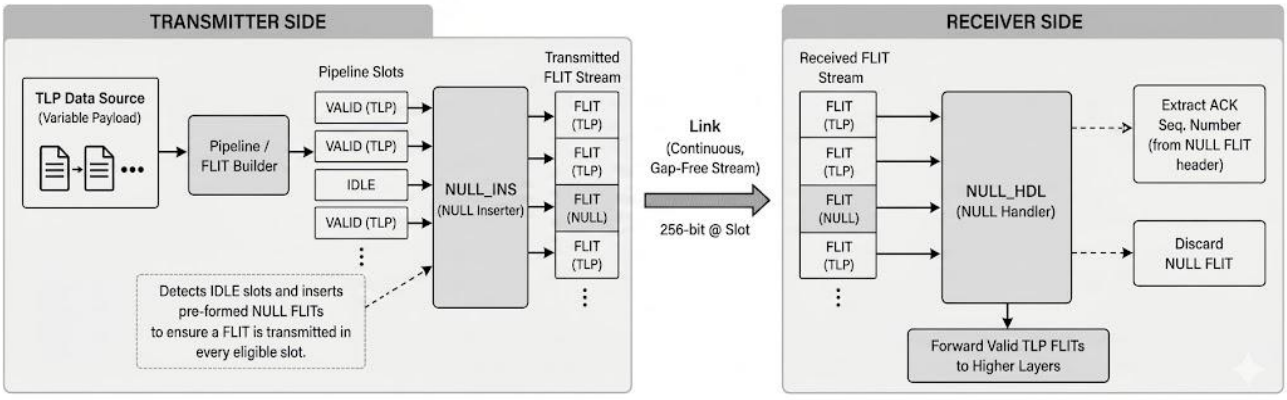


Figure 6.9 Transmitter and Receiver NULL FLIT Processing Flow

6.6 Initialization: Establishing a Reliable Channel

The DLL does not become operational immediately upon Physical Layer link training completion. Instead, it must execute its own initialization sequence before the Transaction Layer is permitted to inject any TLP onto the link. This sequence is orchestrated by two tightly coupled finite state machines: the DLL Link State and Initialization FSM (DLL_INIT) and the Flow Control Initialization FSM (FC_INIT_FSM).

6.6.1 DLL Initialization FSM (DLL_INIT)

DLL_INIT is activated by the assertion of the ltssm_dl_up signal, which the Physical Layer LTSSM drives high at the conclusion of link training. It is important to note that the relationship between the LTSSM and DLL_INIT is not limited to this single signal assertion. The LTSSM completes its own multi-state training sequence — including Detect, Polling, Configuration, and Recovery substates — before it asserts ltssm_dl_up to indicate that the physical medium has been brought to a stable operating condition at the negotiated speed and lane width. DLL_INIT does not participate in that physical training process, but it monitors ltssm_dl_up as its authoritative trigger to begin DLL-level bring-up. Should a fatal DLL error subsequently force re-initialization, DLL_ERR asserts dll_link_down internally, which causes DLL_INIT to re-enter its reset sequence and, where necessary, coordinates with the LTSSM to initiate a Recovery or Hot Reset cycle at the physical layer before the DLL can be declared operational again. This coupling ensures that a DLL-level failure that implies physical layer degradation — such as persistent uncorrectable CRC errors — is not silently absorbed by DLL replay alone but instead escalates to a joint physical and data-link recovery.

Upon activation by ltssm_dl_up, DLL_INIT issues reset commands to every module in both the transmit and receive data paths, ensuring that sequence-number counters, retry buffers, credit registers, and error-state machines all return to defined initial values. This global reset prevents stale state from a prior link session from corrupting the newly established channel. Once all dependent modules have acknowledged their reset, DLL_INIT delegates control to FC_INIT_FSM to conduct the credit-advertisement handshake.DLL_INIT implements the DL_Inactive to DL_Init to DL_Active state progression defined by the PCIe Base Specification. In the DL_Inactive state, the DLL suppresses all TX pipeline activity and prevents the Transaction Layer from injecting TLPs.

The transition to DL_Init occurs when the LTSSM asserts ltssm_dl_up, confirming that the Physical Layer has completed link training at the negotiated speed. The transition from DL_Init to DL_Active occurs only after FC_INIT_FSM asserts its fc_init_done signal, confirming that a mutually consistent credit state has been established with the remote DLL. Only in the DL_Active state does DLL_INIT assert dll_up_to_tl, granting the Transaction Layer permission to begin TLP transmission.

6.6.2 Flow Control Initialization FSM (FC_INIT_FSM)

FC_INIT_FSM implements the two-phase InitFC protocol mandated by the PCIe Base Specification. In the first phase (InitFC1), the local DLL transmits a series of InitFC1 DLLPs, one for each of the six credit categories defined by PCIe: Posted Headers, Posted Data, Non-Posted Headers, Non-Posted Data, Completion Headers, and Completion Data. Each InitFC1 DLLP encodes the capacity of the local receive buffer for that credit category, expressed in units of credits. Simultaneously, FC_INIT_FSM monitors the receive path for the corresponding InitFC1 DLLPs from the remote DLL, which carry the remote receive-buffer capacities. In the second phase (InitFC2), both endpoints acknowledge receipt of each other's advertised capacities, confirming that the credit state is bilaterally consistent. Only after FC_INIT_FSM asserts its fc_init_done signal does DLL_INIT declare the link operational and assert dll_up_to_tl, guaranteeing that no TLP can ever enter the link before a mutually agreed credit allocation is in place.

The Flow Control Initialization Timer (FC_INIT_TMR) provides a mandatory watchdog function for the initialization sequence. If the InitFC handshake does not complete within the timeout period specified by the PCIe Base Specification, FC_INIT_TMR asserts a timeout error, which DLL_INIT uses to abort the initialization attempt and trigger a link-level recovery procedure. This timeout protection prevents the DLL from entering a permanently stalled state if the remote endpoint fails to respond to InitFC DLLPs, ensuring that the initialization state machine always makes forward progress toward either a successful DL_Active state or a detected failure condition.

6.7 The Transmit Data Path: From Transaction Packet to Physical FLIT

With the link declared operational, the DLL transmit path becomes active. Its function is to accept TLPs from the Transaction Layer, augment them with the control fields required for reliability and flow management, retain buffered copies pending acknowledgement, and deliver a continuous stream of scrambled 256-bit flit words to the Physical Layer. This process involves eight cooperating modules whose interactions are described below as a unified pipeline.

6.7.1 Transaction Layer Interface (TL_IF)

TL_IF is the first point of engagement between the Transaction Layer and the DLL. It receives TLPs via the tlp_in[1023:0] and flit_in[2047:0] input buses and performs credit-gating verification before forwarding any packet into the transmit data path.

TL_IF consults the credit availability information maintained by the Credit subsystem to verify that the remote receive buffer has sufficient space in the appropriate credit category. If sufficient credits are available, TL_IF proceeds to segment the TLP into one or more 256-byte flit payloads, managing the segmentation state machine that appends required header fields and data chunks across successive flit slots. If credits are insufficient, TL_IF asserts back-pressure by de-asserting the tl_ready signal, stalling the Transaction Layer until conditions improve.

6.7.2 CRC Generator (CRC_GEN) and Sequence Number Generator (SEQ_GEN)

As each flit payload exits TL_IF, it is simultaneously presented to CRC_GEN and SEQ_GEN, which operate in parallel to minimize pipeline latency. CRC_GEN computes a 32-bit LCRC over the full flit payload using the CRC-32 polynomial specified by the PCIe Base Specification, processing the flit in 32-byte chunks using partial-CRC folding to sustain one complete CRC result per flit at the DLL core clock rate. In Gen 6 FLIT mode, a 24-bit CRC variant is used per FLIT. The finished CRC word is appended to the trailing boundary of the flit.

SEQ_GEN, operating in parallel, maintains a 12-bit monotonically incrementing sequence counter. Each flit is assigned the current counter value, prepended as a sequence number field. The sequence number allows the receiver to detect missing or reordered flits, provides the correlation key for ACK and NAK responses, and defines the positional index into the retry buffer.

The counter wraps from 4095 back to zero, and both ends of the link maintain synchronized sequence state — as described in the Sequence Number Wrap-Around section above — so that this wraparound introduces no ambiguity in the acknowledgement protocol (**Figure 6.10**).

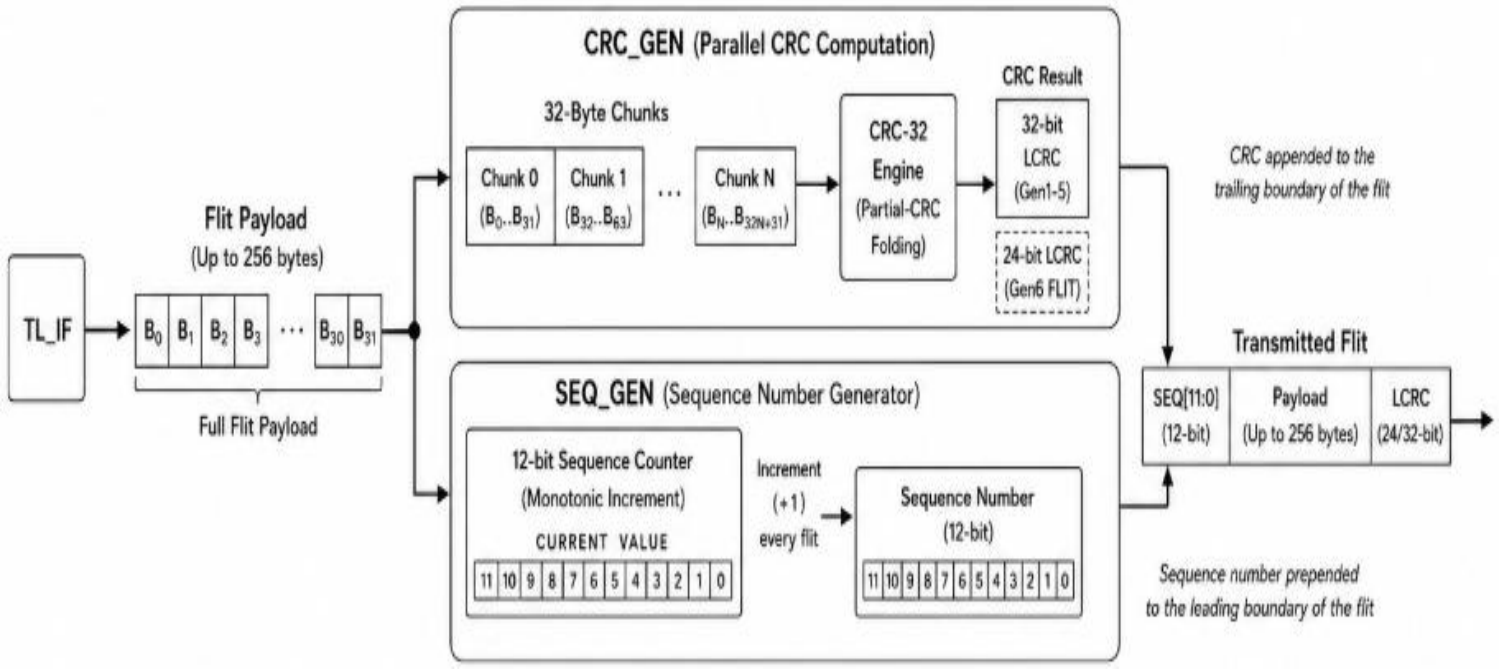


Figure 6.10 Parallel LCRC and Sequence Number Processing for FLIT Transmission

6.7.3 NULL FLIT Inserter (NULL_INS)

The output of SEQ_GEN is passed to NULL_INS. PCIe 6.0 mandates that the transmitter always place a valid flit on the link during every eligible clock slot. NULL_INS fulfils this by substituting a pre-formed NULL flit pattern whenever the upstream pipeline produces no valid TLP flit for a given slot. NULL flits also carry piggybacked acknowledgement sequence numbers coordinated with ACK_PGB, ensuring the remote transmitter receives timely credit even when the local transmitter is momentarily idle.

6.7.4 Retry Buffer (RETRY_BUF)

The output of NULL_INS flows into RETRY_BUF, the most consequential module in the entire transmit path. RETRY_BUF stores a complete copy of every transmitted flit indexed by its sequence number, retaining that copy until an explicit ACK is received. Its depth is a critical design parameter that must cover the full bandwidth-delay product of the link.

6.7.5 Retry Buffer Sizing

The bandwidth-delay product defines the minimum number of FLITs that must be in flight simultaneously to keep the link fully utilized during a round-trip acknowledgement cycle. At 64 GT/s with a 256-byte FLIT, the raw link delivers approximately 8 GB/s of payload throughput per lane direction. A representative PCIe 6.0 root-complex-to-endpoint round-trip latency — accounting for Physical Layer propagation delay, FEC decoding latency at the receiver, DLL receive-path processing, ACK DLLP generation and transmission, and DLL transmit-path scheduling — is on the order of 200 to 400 nanoseconds in a typical platform. At 8 GB/s, 400 ns of round-trip time means approximately 3200 bytes of data are in flight, corresponding to roughly 13 FLITs. Adding margin for worst-case FEC latency, DLLP serialization delay, and implementation pipeline depth, a practical minimum retry buffer depth is in the range of 32 to 64 FLITs, with larger designs targeting 128 FLITs or more to accommodate longer traces and multi-switch topologies. Under sizing the buffer forces the transmitter to stall once the buffer fills, directly converting buffer depth into an effective throughput ceiling (**Table 6.3**).

MaxPayloadSize	x1 Link	x2 Link	x4 Link	x8 Link	x12 Link	x16 Link	x32 Link
128 Bytes	**237** (AF=1.4)	**128** (AF=1.4)	**73** (AF=1.4)	**67** (AF=2.5)	**58** (AF=3.0)	**48** (AF=3.0)	**33** (AF=3.0)
256 Bytes	**416** (AF=1.4)	**217** (AF=1.4)	**118** (AF=1.4)	**107** (AF=2.5)	**90** (AF=3.0)	**72** (AF=3.0)	**45** (AF=3.0)
512 Bytes	**559** (AF=1.0)	**289** (AF=1.0)	**154** (AF=1.0)	**86** (AF=1.0)	**109** (AF=2.0)	**86** (AF=2.0)	**52** (AF=2.0)
1024 Bytes	**1071** (AF=1.0)	**545** (AF=1.0)	**282** (AF=1.0)	**150** (AF=1.0)	**194** (AF=2.0)	**150** (AF=2.0)	**84** (AF=2.0)
2048 Bytes	**2095** (AF=1.0)	**1057** (AF=1.0)	**538** (AF=1.0)	**278** (AF=1.0)	**365** (AF=2.0)	**278** (AF=2.0)	**148** (AF=2.0)
4096 Bytes	**4143** (AF=1.0)	**2081** (AF=1.0)	**1050** (AF=1.0)	**534** (AF=1.0)	**706** (AF=2.0)	**534** (AF=2.0)	**276** (AF=2.0)

Table 6.3 Retry Buffer Ack Latancy Values

The RTL implementation therefore uses SRAM-based storage with direct addressing by sequence number, supporting both efficient sequential writes during normal transmission and arbitrary-start-point re-injection during replay.When an ACK is received, RETRY_BUF purges all stored flits up to and including the acknowledged sequence number.

When a NAK is received or ACK_TMR expires, REPLAY_FSM instructs RETRY_BUF to re-inject all unacknowledged flits from the indicated sequence number. During replay, RETRY_BUF suppresses new TLP admission from TL_IF, ensuring the replayed stream appears at PHY_TX as an uninterrupted, correctly ordered sequence indistinguishable from an original transmission.

6.7.6 TX Multiplexer (TX_MUX), Scrambler (SCRM), and Physical TX Interface (PHY_TX)

The output of RETRY_BUF converges at TX_MUX with a second stream carrying DLLPs from DLLP_GEN and DLLP_ARB. TX_MUX implements a weighted priority arbitration scheme, with the ordering: ACK/NAK > UpdateFC > power-management DLLPs > bandwidth notification DLLPs > NOP DLLPs > retry TLPs > new TLPs. This ordering ensures time-sensitive control packets are never held behind payload data while preventing TLP starvation under heavy DLLP load.The combined flit stream enters SCRM, which applies an LFSR polynomial across the full 256-bit width of each flit to randomize the symbol distribution, mitigate electromagnetic emissions, and prevent long runs of identical bits from stressing CDR circuits. The scrambler state is seeded to a known value during link training and kept synchronized with the remote descrambler via ordered-set patterns.

The scrambled output is delivered to PHY_TX, which formats the 256-bit word onto PHY_TXD[255:0], asserts PHY_TX_VALID, and manages flit-boundary alignment with the Physical Layer symbol structure. PHY_TX additionally gates transmission in response to power-management commands from PM_FSM.

6.8 Flow Control: Sustaining Deadlock-Free Throughput

The flow-control architecture of PCIe 6.0 is a credit-based system in which each receiver advertises the capacity of its receive buffers before any data transfer begins, and subsequently issues credit return updates as consumed data is released by the Transaction Layer. Six independent credit categories are defined: Posted Headers (PH), Posted Data (PD), Non-Posted Headers (NPH), Non-Posted Data (NPD), Completion Headers (CPLH), and Completion Data (CPLD).

On the receive side, the RX Credit Manager tracks how many credits of each type have been consumed by the local Transaction Layer since the last UpdateFC DLLP was transmitted. When consumed credits in any category exceed a configurable threshold, the module asserts a credit-return request toward DLLP_GEN. DLLP_GEN encodes the accumulated values into an UpdateFC DLLP, passes it to DLLP_CRCG which appends an 8-bit CRC, and the result is presented to DLLP_ARB for scheduling through TX_MUX.

6.8.1 Flow Control Timer (FC_TMR) and Watchdog (FC_WDG)

FC_TMR monitors the interval since the last UpdateFC or NOP DLLP was transmitted; if this interval approaches the maximum permitted by the specification, FC_TMR forces DLLP_GEN to generate a refresh. FC_WDG, on the receive side, monitors the interval between incoming flow-control DLLPs; if none arrives within the specified timeout, FC_WDG concludes the credit-return pathway has been disrupted and asserts an error toward DLL_ERR. This complementary pair prevents the link from deadlocking on a permanently exhausted credit category.

6.8.2 ACK Latency Budget and the FC_TMR Timing Constraint

The PCIe Base Specification mandates that the receiver acknowledges every correctly received flit within a bounded interval called the ACK Latency Timer limit. For PCIe 6.0, this limit is derived from the maximum round-trip time the specification permits for a given link configuration and is expressed in units of FLITs transmitted. In practical terms, the ACK latency budget for a Gen 6 link is on the order of several hundred nanoseconds — sufficient to encompass the DLLP serialization latency on the return path plus a small allowance for receiver processing — and corresponds to a window of approximately 8 to 16 outstanding unacknowledged FLITs for a short-trace endpoint link.

The transmitter's RETRY_BUF must be sized to hold at least as many FLITs as the latency budget permits to be outstanding, which directly connects the buffer sizing analysis above to the ACK timing constraint. FC_TMR enforces a complementary bound: even when no genuine credit returns are pending, FC_TMR expires before the remote FC_WDG timeout and forces a NOP or Update_FC DLLP, keeping the credit-update stream alive and guaranteeing that the remote watchdog never falsely concludes the link has failed.

6.8.3 NOP Generator (NOP_GEN)

NOP_GEN provides a lightweight mechanism for satisfying the FC_TMR requirement during periods when no genuine credit returns are pending. NOP DLLPs carry no credit information but serve as keepalive messages that reset both the local FC_TMR and the remote FC_WDG. Additionally, NOP DLLPs can carry piggybacked ACK sequence numbers, coordinated with ACK_PGB, providing a dual purpose of flow control maintenance and acknowledgement delivery during low-traffic intervals.

6.9 Acknowledgement, Sequencing, and the Replay Protocol

The reliability mechanism of the PCIe DLL is implemented as a sliding-window ARQ protocol, in which the transmitter maintains an outstanding window of unacknowledged flits and the receiver returns acknowledgements that advance the window base. The modules responsible — ACK_PGB, ACK_TMR, FLIT_SEQ, REPLAY_FSM, ACK_TX, and ACK_RX — operate as an integrated system.

6.9.1 ACK Piggyback Controller (ACK_PGB) and ACK Timer (ACK_TMR)

ACK_PGB examines every outbound TLP data flit to determine whether a pending ACK sequence number can be embedded in its header, consuming no additional link bandwidth. If no outbound TLP flit is available within the ACK latency window, ACK_TMR expires and asserts ack_timer_exp toward ACK_TX, triggering the construction and scheduling of an explicit ACK DLLP. The combination of piggybacking and explicit ACK generation ensures the transmitter's retry window advances promptly regardless of local TLP traffic.

6.9.2 Flit Sequencer (FLIT_SEQ) and Replay FSM (REPLAY_FSM)

FLIT_SEQ tracks both the oldest unacknowledged sequence number (the window base) and the most recently transmitted sequence number, providing the seq_window parameter to RETRY_BUF so the buffer knows which flits remain eligible for replay.

REPLAY_FSM governs the actual replay execution. Under normal operation it remains idle, monitoring for triggers from either a received NAK DLLP (carrying the first sequence number to be retransmitted) or ACK_TMR expiry. Upon either trigger, REPLAY_FSM suspends new TLP injection from TL_IF and instructs RETRY_BUF to begin replaying from the indicated sequence number.

Once replayed flits have been retransmitted and new ACKs are received, REPLAY_FSM returns to idle. Should the replay count exceed four consecutive replays, REPLAY_FSM asserts a fatal error toward DLL_ERR and initiates link re-initialization through DLL_INIT.

6.10 The Receive Data Path: Validation and Delivery

The receive data path performs the inverse of the transmit pipeline: it accepts raw scrambled flits from the Physical Layer, descrambles them, classifies them as TLP payload or DLLP control traffic, validates their integrity and ordering, and delivers confirmed TLPs to the Transaction Layer while routing control DLLPs to their respective handlers.

6.10.1 Physical Reception and Descrambling (PHY_RX, DESCRM)

PHY_RX captures the 256-bit flit arriving on PHY_RXD [255:0] and performs the clock-domain crossing required to transfer it from the recovered RX clock domain into the DLL core clock domain. PHY_RX also receives the FEC_SYNDROME signal from the Physical Layer FEC decoder. If FEC_SYNDROME indicates an uncorrectable error, PHY_RX marks the flit as erroneous; this marking propagates through the receive pipeline and causes the error-checking stage to discard the flit and assert the appropriate error signal.

The raw flit passes to DESCRM, which applies the same LFSR polynomial as the transmit-side SCRM in its inverse configuration, recovering the original unscrambled flit. The descrambler's LFSR is synchronized to the remote transmitter's scrambler state through the ordered-set exchange during link training. The descrambled 256-bit word is forwarded to FLIT_RX and RX_DEMUX.

6.10.2 FLIT Classification and Demultiplexing (FLIT_RX, RX_DEMUX, NULL_HDL)

FLIT_RX parses each received flit's type field to determine whether it carries a TLP payload, a DLLP, or is a NULL flit. RX_DEMUX steers the flit accordingly: TLP flits toward the error-checking stage, DLLP flits to DLLP_RX and NULL_HDL, and NULL flits to NULL_HDL for ACK extraction and discard. This demultiplexing step is the architectural boundary at which the receive path diverges into a data plane (TLP processing) and a control plane (DLLP processing).

NULL_HDL extracts any ACK or NAK sequence numbers embedded in NULL flit headers and forwards them through ACK_RX to RETRY_BUF, where they are processed identically to acknowledgements in dedicated ACK DLLPs.

NULL_HDL also monitors for synchronization anomalies, asserting a sync error signal if a NULL flit arrives where the link frame synchronization state machine does not expect one.

6.10.3 Error Detection: CRC, Sequence, and DLLP Integrity Checking

The following table presents the three-stage error-checking pipeline through which incoming TLP and DLLP flits are processed, detailing the specific check applied at each stage and the corresponding error-handling action, ranging from NAK generation and flit discard to link re-initialization (**Table 6.3**).

Table 6.4 Error-Checking Pipeline Stages and Error-Handling Actions

Stage	Description
CRC_CHK	Recomputes the LCRC and compares it to the value appended by the remote CRC_GEN. A mismatch triggers NAK generation through ACK_TX and discards the erroneous flit without presenting it to the Transaction Layer.
SEQ_CHK	Validates the sequence number against the locally maintained expected receive counter. An out-of-sequence flit triggers a NAK carrying the expected sequence number so that REPLAY_FSM on the remote side can initiate replay from the correct position.
DLLP_CRKC	DLLP flits undergo their own integrity check through 8-bit CRC verification.
DLLP_MAL	Structural validity check of type codes and field ranges. Errors cause the offending DLLP to be discarded; persistent DLLP errors imply a systemic link quality problem and will eventually escalate to a link re-initialization event.

6.10.4 DLLP Processing and Receive-Side Credit Management

DLLPs surviving integrity checks are decoded by DLLP_RX and routed to their handlers: UpdateFC DLLPs to the RX Credit Manager and fc_update_out toward the Transaction Layer; ACK and NAK DLLPs to ACK_RX; power-management DLLPs to PM_FSM; InitFC DLLPs to FC_INIT_FSM.ACK_RX occupies a pivotal position in the receive-to-transmit feedback loop. ACK receipt causes RETRY_BUF to purge all flits up to the acknowledged sequence number.

NAK receipt causes ACK_RX to immediately assert the replay request toward REPLAY_FSM, triggering fast replay without waiting for ACK_TMR expiry. In PCIe 6.0, the feedback path from NAK receipt through ACK_RX and REPLAY_FSM to RETRY_BUF re-injection has been optimized to minimize pipeline stages, directly limiting the bandwidth cost of error recovery.

6.11 Power Management and Link Bandwidth Negotiation

6.11.1 Power Management FSM (PM_FSM) and Timer (PM_TMR)

PM_FSM implements the DLL's contribution to the PCIe power management hierarchy, encompassing L0, L0s, L1, L1.1, L1.2, and L2 link states.

L0s Physical Behavior: L0s is the lightest-weight power-saving state and is designed for very low entry and exit latency. When PM_FSM determines that the TX path has been idle for an interval exceeding the L0s entry threshold, it commands PHY_TX to drive electrical idle on the lane — that is, to cease transmitting a differential signal and instead drive both conductors of the differential pair to a common DC level within the specification's voltage window. The remote PHY_RX detects this electrical idle condition through its receiver-detection circuitry and recognizes the transition as an L0s entry by the remote transmitter. Importantly, L0s is an asymmetric state: each direction of a lane can enter and exit L0s independently, so it is entirely valid for one direction to be in L0s while the opposite direction continues carrying traffic. Exit from L0s is initiated by the transmitter, which drives an Electrical Idle Exit Ordered Set (EIEOS) followed by Fast Training Sequences (FTS) to allow the remote receiver's CDR circuit to re-acquire the clock before resuming normal FLIT transmission.

The number of FTS symbols required for clock re-acquisition is negotiated during link training and stored in PM_TMR's configuration registers, governing the minimum time PM_FSM must remain in L0s exit signaling before declaring the lane ready for data. Because the total L0s exit latency is on the order of tens of nanoseconds at Gen 6 speeds, L0s imposes a negligible throughput penalty relative to the power savings it provides during short idle gaps.

L1 and deeper states involve coordinating both directions of the link simultaneously and require an explicit DLLP handshake before either side suppresses its Physical Layer activity, reflecting their higher entry and exit latency. PM_TMR provides configurable thresholds for L0s exit latency, L1 entry timeout, and L1 minimum residency. When PM_FSM determines a low-power state transition, it coordinates with both PHY_TX and PHY_RX, and interacts with DLL_INIT when the link must be fully re-initialized after returning from deep states such as L2.

6.11.2 Link Bandwidth Management FSM (LBW_FSM)

LBW_FSM implements the runtime speed- and width-change mechanism. In a multi-speed system, endpoints may negotiate a transition between the 64 GT/s PAM4 rate and lower speeds for interoperability or power reduction. LBW_FSM coordinates with PM_FSM to avoid conflicting transitions during the speed change, and instructs DLL_INIT to re-initialize after the Physical Layer has retrained at the new operating rate.

Link Bandwidth Notification DLLPs are generated by LBW_FSM and scheduled through DLLP_ARB to ensure both sides transition synchronously with all sequence counters and credit registers reset appropriately.

6.12 Centralized Error Management and Link

DLL_ERR serves as the convergence point for error signals from every module in the DLL. Error signals arriving at DLL_ERR include crc_err from CRC_CHK, seq_err from SEQ_CHK, dllp_crc_err from DLLP_CRKC, mdl_err from DLLP_MAL, fc_watchdog from FC_WDG, and replay-limit-exceeded notifications from REPLAY_FSM. DLL_ERR classifies each error by severity: correctable errors resolved by successful replay are logged in AER registers without disrupting link operation; uncorrectable non-fatal errors are reported to the Transaction Layer for software-directed remediation; uncorrectable fatal errors — such as replay count exhaustion or persistent CRC failures — cause DLL_ERR to assert dll_link_down, triggering full DLL re-initialization through DLL_INIT.This layered escalation reflects a principle of proportional response: transient errors that can be resolved locally are handled entirely within the DLL. Only errors exceeding the DLL's self-healing capacity are escalated. Because the DLL cannot itself generate TLP messages, all errors requiring protocol-level reporting are forwarded to the Transaction Layer AER Logger through the aer_err_out and aer_err_valid interface signals.

6.13 PCIe Gen 6 Impact on the Data Link Layer

The transition from PCIe Gen 5 to Gen 6 significantly increased the architectural complexity of the Data Link Layer. The adoption of PAM4 signaling at 64 GT/s doubled the achievable data rate but also introduced a substantially higher raw BER, making FEC mandatory within the Physical Layer and FLIT-based transmission mandatory within the DLL (**Table 6.4**).

Table 6.5 Comparison of DLL Features Between PCIe Gen 5 and PCIe Gen 6

Feature	PCIe Gen 5	PCIe Gen 6
Modulation Technique	NRZ (Non-Return-to-Zero)	PAM4 (4-Level Pulse Amplitude Modulation)
Packet/FLIT Structure	Variable-length TLPs, per-TLP sequence number	Fixed-size 256-byte FLITs, per-FLIT sequence number
Raw Bit Error Rate	Lower BER	Higher BER due to reduced noise margin
CRC Granularity	32-bit LCRC per TLP	24-bit CRC per 256-byte FLIT
Error Correction	Standard CRC only	Mandatory Physical Layer FEC + DLL CRC
Framing Mechanism	Variable-length packet framing	Fixed-size FLIT framing aligned to FEC code words
NULL Transmission	Idle gaps permitted between packets	Continuous FLIT stream mandatory (NULL FLIT insertion)
Retry Buffer Depth	Sized for PCIe 5 bandwidth-delay product	Substantially larger for 64 GT/s bandwidth-delay product
DLL Complexity	Lower	Significantly higher due to FLIT management requirements

6.14 Design Trade-offs and Architectural Decisions

Table 6.6 Major Architectural Trade-offs

Component	Option Chosen	Alternative	Engineering Reason
FIFO Architecture	Synchronous FIFO	Asynchronous FIFO	The DLL TX and RX pipelines primarily operate within a unified internal clock domain. Synchronous FIFOs reduce hardware overhead, simplify timing analysis, and eliminate latency from multi-stage synchronizers. Asynchronous FIFOs are used only at external clock-domain boundaries.
Arbitration Logic	Priority-Weighted Round Robin	Fixed Priority	Priority weighting ensures DLLPs receive scheduling precedence over TLP data, preventing credit and acknowledgement starvation, while round-robin fairness across TLP traffic classes prevents any single category from monopolizing the link.
CRC Generation	Parallel CRC-32/CRC-24	Serial CRC	Serial CRC cannot meet required timing constraints at Gen 6 throughput rates. Parallel CRC computes across wide FLIT data paths within a single pipeline stage using combinational XOR networks.
Retry Buffer Organization	SRAM-indexed by sequence number	FIFO-based retry buffer	Sequence-number addressing enables One access to any stored flit for arbitrary replay start points, as required when a NAK carries a sequence number significantly older than the most recently transmitted flit.
ACK Delivery	Piggybacked ACK on NULL/NOP DLLPs	Dedicated ACK DLLPs only	Piggybacking ACK values onto outbound NULL flits and NOP DLLPs eliminates the majority of dedicated ACK DLLP generation, reducing DLLP bandwidth consumption while meeting ACK latency constraints under high TLP traffic.

6.15 Challenges and Implementation Hurdles

1- The most immediate implementation challenge is the dramatic increase in required retry buffer depth. At 64 GT/s the raw data rate is twice that of PCIe 5.0, meaning twice as many bytes are in flight during any given round-trip interval. The RTL implementation uses SRAM-based storage with direct addressing by sequence number to support both efficient sequential writes during normal transmission and arbitrary-start-point re-injection during replay.

2- In practical FPGA and SoC environments, user-side logic, transaction scheduling stages, and PCIe link interfaces frequently operate under independent clock domains. To address this, asynchronous FIFO bridges were implemented between independent timing domains using Gray-code read/write pointers and multi-stage synchronization structures. Gray-code addressing minimizes synchronization ambiguity because only a single bit changes between adjacent counter values, reducing metastability probability during pointer transfer.

These FIFO structures decouple timing differences between the application-side transaction pipeline and the PCIe link-side FLIT transmission stages while maintaining continuous packet flow and stable throughput.

3- The throughput penalty of any error event is directly proportional to the number of pipeline stages between NAK reception and the first replayed flit appearing at PHY_TX. The RTL implementation was structured to minimize the critical path through ACK_RX, REPLAY_FSM, and RETRY_BUF, with the replay re-injection path given highest priority through TX_MUX. Verification required extensive constrained-random simulation with simultaneous outstanding requests, randomized NAK injection, delayed ACK responses, and forced replay-count-exceeded scenarios.

4- Unlike variable-length packet framing — where synchronization is re-established at each packet start — FLIT mode requires the receiver to maintain precise alignment to the 256-byte FLIT boundary throughout link operation. Any misalignment causes systematic CRC failures and sequence number corruption across all subsequently received flits. The FLIT_RX module implements a synchronization state machine that monitors alignment between incoming PHY data words and expected FLIT boundaries, triggering link-level recovery through DLL_ERR and DLL_INIT when synchronization cannot be restored.

7 Chapter Seven: Physical Layer

This chapter presents the PCIe Physical Layer (PHY), which occupies the lowest stratum of the three-tier PCIe protocol stack and is responsible for all aspects of physical signal transmission and reception across the PCIe link. The Physical Layer serves as the primary mechanism for converting digital logic-level data produced by the Data Link Layer into electrical signals on the physical medium, and for performing the inverse conversion on the receive side. To fulfil this responsibility, the Physical Layer manages link training and initialization through the Link Training and Status State Machine (LTSSM), encodes and frames transmitted data using the encoding scheme appropriate to the negotiated link speed, applies mandatory Reed-Solomon Forward Error Correction (FEC) in Gen 6 mode, modulates and demodulates PAM4 symbols at 64 GT/s, equalizes received signals to compensate for channel loss, and maintains clock compensation across multi-lane links through SKP ordered set insertion and removal. In addition, the chapter discusses the major operations performed by the Physical Layer, including ordered set generation and detection, lane deskew and polarity correction, power state management, speed and width negotiation, spread-spectrum clock control, compliance testing, and the PIPE interface boundary between the digital RTL and the analog PHY macro. The presented concepts provide the theoretical and architectural foundation required for understanding the implementation and simulation of the PCIe Gen 6 Physical Layer modules discussed in the following sections.

7.1 Physical Layer Architecture

The PCIe Gen 6 Physical Layer is organized around three major subsystems that cooperate to establish and maintain the physical link. The first is the LTSSM Controller subsystem, a hierarchical collection of finite state machines that governs every aspect of link bring-up, speed negotiation, width negotiation, equalization, recovery, power management, loopback, and reset. The second is the Transmit (TX) data path, a multi-stage pipeline that accepts FLIT-framed data from the Data Link Layer, applies Reed-Solomon FEC encoding, performs PAM4 Gray code mapping, and delivers the result to the analog SERDES macro through the PIPE interface. The third is the Receive (RX) data path, which performs the inverse: it captures raw symbol data from the PIPE interface, decodes PAM4 symbols, computes the FEC syndrome, corrects burst errors, deframes the FLIT, and delivers validated data to the Data Link Layer.

A shared control plane implemented through the PIPE Interface Controller (PIPE_CTRL) and a suite of negotiation and utility modules connects all three subsystems, managing the physical-level signals, ordered sets, and clock compensation mechanisms required for correct link operation at every supported speed (**Figure 7.1**).

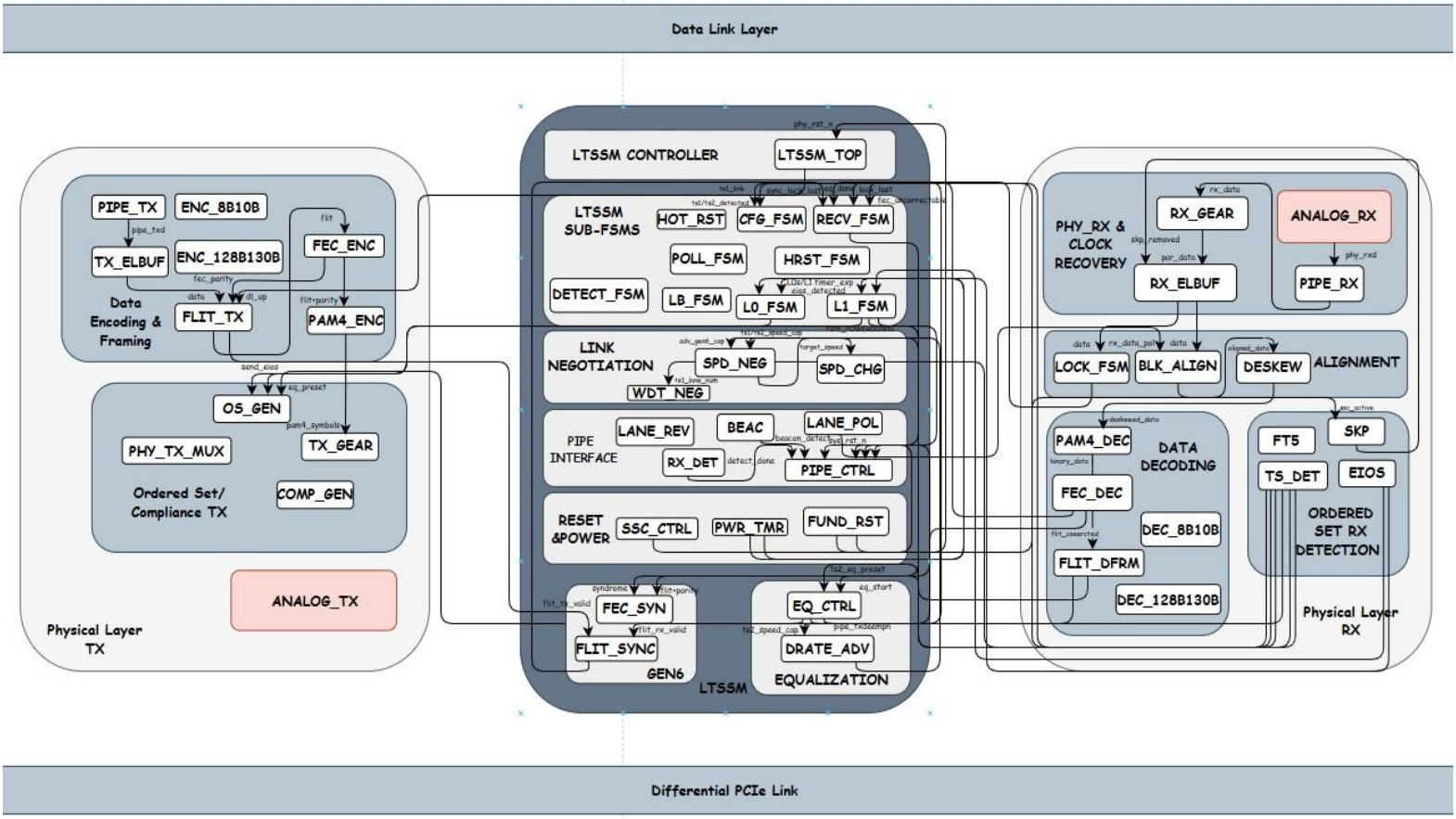


Figure 7.1 Physical Layer Core Architecture

The most significant architectural change in PCIe Gen 6 relative to prior generations is the transition from 8b/10b encoding used in Gen 1 and Gen 2, and 128b/130b encoding used in Gen 3 through Gen 5, to FLIT-based framing with mandatory Reed-Solomon FEC. This change was necessitated by the adoption of PAM4 (Pulse Amplitude Modulation, 4-level) modulation at 64 GT/s, which encodes two bits per symbol across four voltage levels, doubling throughput compared to the NRZ modulation of Gen 5 at the cost of a substantially higher raw Bit Error Rate due to the reduced voltage margin between signal levels. The Physical Layer architecture is therefore centered on managing the reliability consequences of PAM4 operation through FEC, while simultaneously maintaining backward compatibility with all prior PCIe generations through multiplexed encoding and decoding pipelines.

7.2 Physical Layer Interface Signals

The following table presents the complete Physical Layer interface signals used in the proposed PCIe Gen6 architecture. The table summarize the direction, bit-width, and functional description of each port, providing a clear overview of the communication and control signals exchanged between the Physical Layer modules and adjacent layers (**Table 7.1**).

Table 7.1 Physical Layer Interface Signals

Signal Name	Type	Size (bit)	Description
clk	in	1	System core clock. All digital PHY logic is clocked on the rising edge.
rst_n	in	1	Active-low synchronous reset. Initiates fundamental reset sequencing through FUND_RST.
perst_n	in	1	PCIe fundamental reset input (PERST#). Assertion forces full PHY and link re-initialization.
power_good	in	1	Asserted by the platform when all power rails are stable and the PHY may begin initialization.
flit_in[2047:0]	in	2048	256-byte FLIT received from the Data Link Layer TX pipeline for encoding and transmission.
flit_valid_in	in	1	Asserted when flit_in holds a valid Gen6 FLIT ready for PHY TX processing.
flit_mode_en	in	1	Configuration bit enabling Gen6 256-byte FLIT mode across encoding and framing modules.
fec_en	in	1	Configuration bit enabling Reed-Solomon FEC encoding on the TX path and decoding on the RX path.
pipe_rxd[255:0]	in	256	Parallel received data from the analog PHY SERDES after deserialization and symbol alignment.
pipe_rx_valid	in	1	Asserted by the analog PHY to indicate that pipe_rxd contains valid received data.

pipe_rx_status[2:0]	in	3	PIPE PhyStatus/RxStatus encoded bus reporting receiver events including electrical idle and symbol errors.
pipe_rx_elec_idle	in	1	Asserted by the analog PHY when electrical idle is detected on the receive lane.
pipe_phy_status	in	1	PIPE PhyStatus strobe. Pulsed by the PHY to acknowledge power management and rate-change commands.
pipe_rxdatak[31:0]	in	32	PIPE RxDataK bus indicating which bytes in pipe_rxd are control characters (K-symbols).
pipe_rxeqeval	in	1	PIPE signal indicating the analog PHY is evaluating RX equalization during Gen3+ link equalization.
local_speed_cap[7:0]	in	8	Local device speed capability register. Bits encode supported generations including Gen6 at 64 GT/s.
local_width_cap[5:0]	in	6	Local device link width capability. Encodes support for x1, x2, x4, x8, and x16 configurations.
pm_req[2:0]	in	3	Power management request from the Data Link Layer PM FSM requesting L0s or L1 entry.
dll_up_req	in	1	Request from the Data Link Layer for the LTSSM to confirm the link is operational at DL_Active.
compliance_req	in	1	Request from software or configuration logic to enter compliance test mode.
flit_out[2047:0]	out	2048	Decoded and FEC-corrected 256-byte FLIT delivered to the Data Link Layer RX pipeline.
flit_valid_out	out	1	Asserted when flit_out holds a valid, error-checked FLIT ready for Data Link Layer processing.
fec_corrected	out	1	Asserted when the FEC decoder successfully corrected all bit errors in the current FLIT.
fec_uncorrectable	out	1	Asserted when the FEC decoder detected errors beyond its correction capacity in the current FLIT.

fec_syndrome[255:0]	out	256	Reed-Solomon syndrome vector forwarded to the Data Link Layer for error classification and logging.
pipe_txd[255:0]	out	256	PIPE TxData bus driven to the analog PHY SERDES for serialization onto the PCIe lane.
pipe_tx_valid	out	1	PIPE TxDataValid signal asserting that pipe_txd contains valid data for serialization.
pipe_txdatak[31:0]	out	32	PIPE TxDataK bus indicating which bytes in pipe_txd are control characters.
pipe_tx_elec_idle	out	1	PIPE TxElecIdle signal commanding the SERDES to drive the lane into electrical idle.
pipe_tx_compliance	out	1	PIPE TxCompliance signal enabling compliance test pattern encoding in the SERDES.
pipe_powerdown[1:0]	out	2	PIPE Power Down bus commanding the analog PHY power state for the current link state.
pipe_rate[3:0]	out	4	PIPE Rate bus selecting the serialization rate corresponding to the negotiated link speed.
pipe_txdetectrx	out	1	PIPE TxDetectRx signal initiating receiver detection on the specified lane.
ltssm_state[5:0]	out	6	Current LTSSM state encoded for monitoring, debug, and inter-subsystem coordination.
dl_up	out	1	Asserted when the LTSSM reaches L0 and the Data Link Layer may begin normal operation.
dl_down	out	1	Asserted when the LTSSM exits L0 due to an error or power management event.
link_speed[3:0]	out	4	Negotiated and active link speed encoding (Gen1 through Gen6).
link_width[5:0]	out	6	Negotiated and active link width in number of active lanes.
sys_rst_n	out	1	System-level reset output asserted by FUND_RST to reset the Data Link and Transaction Layers.

7.3 Physical Layer Operational Flow

The Physical Layer in PCIe Gen 6 operates as a continuous pipeline responsible for transforming FLIT-framed digital data received from the Data Link Layer into modulated electrical signals on the physical medium, and for performing the inverse reconstruction on the receive side. Rather than functioning as isolated blocks, the Physical Layer components operate as a unified system in which the LTSSM controller orchestrates every state transition, the TX data path transforms data through successive encoding stages, and the RX data path validates and recovers data through a corresponding sequence of decoding stages.

The operational flow begins at power-on with the assertion of PERST# and the execution of the fundamental reset sequence. FUND_RST monitors the perst_n, power_good, and clk_valid inputs and sequences the release of reset across the three layers of the PCIe stack in the correct order — PHY first, then Data Link Layer, then Transaction Layer — ensuring that each layer initialises from a known clean state before the layer above it becomes active. Once fundamental reset is released, the LTSSM begins its state progression.

7.3.1 Link Detection and Physical Establishment

The LTSSM enters the Detect state immediately after fundamental reset. The Receiver Detection Logic (RX_DET) drives the PIPE TxDetectRx signal and monitors the PIPE PhyStatus response to determine whether a far-end receiver is present on each lane. Receiver detection works by measuring the impedance change on the transmit lane when a receiver termination is present. If receiver termination is detected on at least one lane, the LTSSM advances to the Polling state. If no receiver is detected within the detection timeout, the LTSSM remains in Detect and retries, preventing the remainder of the link training sequence from executing on an unpopulated slot.

7.3.2 Polling: Bit Lock and Symbol Lock

In the Polling state, POLL_FSM begins transmitting Training Sequence 1 (TS1) ordered sets continuously. TS1 ordered sets carry the local link number, lane number, speed capability advertisement, FTS count, and various control bits including the Hot Reset and Link Disable bits. The LOCK_FSM on the receive side monitors the incoming data stream to establish symbol lock (Gen 1–2) or block lock (Gen 3–6) by detecting the expected synchronization patterns embedded in TS1.

Once both ends have exchanged sufficient TS1 ordered sets with matching parameters, the link advances through Polling. Configuration by exchanging TS2 ordered sets, which signal mutual agreement on the trained parameters and allow the LTSSM to advance to the Configuration state.

7.3.3 Configuration: Link and Lane Number Negotiation

In the Configuration state, CFG_FSM and the link width negotiation module (WDT_NEG) cooperate to assign definitive link and lane numbers to every active lane. This process involves comparing the lane numbers received in incoming TS1 ordered sets against the locally assigned values, iterating until a stable consistent assignment is reached across all lanes. WDT_NEG simultaneously determines the negotiated link width by identifying which subset of the locally active lanes have been successfully paired with the remote endpoint. Once configuration is complete, the LTSSM transitions the link to the L0 operational state, and dl_up is asserted to notify the Data Link Layer that TLP transmission may begin.

7.3.4 Data Transmission and Reception in L0

In the L0 state, the TX data path operates as a continuous pipeline. FLIT-framed data arrives from the Data Link Layer, passes through the FEC encoder (FEC_ENC), receives PAM4 Gray code mapping (PAM4_ENC), passes through the TX Gear Box (TX_GEAR), and is delivered to the analog SERDES through the PIPE TX interface (PIPE_TX). On the receive side, the analog SERDES delivers recovered parallel data through the PIPE RX interface (PIPE_RX), which passes through the RX Elastic Buffer (RX_ELBUF) for clock-domain crossing, the PAM4 decoder (PAM4_DEC), FEC syndrome calculation (FEC_SYN), FEC decoding (FEC_DEC), FLIT deframing (FLIT_DFRM), and finally delivery to the Data Link Layer.

7.3.5 Recovery, Speed Change, and Error Handling

The LTSSM enters the Recovery state whenever a link error is detected, a speed change is requested, or a re-equalization is required. RECV_FSM manages the Recovery sub-states — Recovery.RcvrLock, Recovery. RcvrCfg, Recovery. Idle, and Recovery.Speed — coordinating TS1/TS2 exchange with the remote endpoint to re-establish lock and agreement before returning to L0. Speed changes are orchestrated through SPD_CHG, which commands the PIPE Rate signal to the new target speed and waits for the analog PHY to confirm the transition through the PhyStatus acknowledgement.

7.4 Physical Layer Responsibilities

The Physical Layer acts as the primary signal conditioning, encoding, and link management layer within the PCIe protocol stack. The table below summarize the major functional responsibilities of the Physical Layer (**Table7.2**).

Table 7.2 Physical Layer Responsibilities

Function	Description
Link Training	Manages the complete LTSSM state progression from Detect through Polling, Configuration, and into L0, establishing bit lock, symbol lock, link and lane numbering, and equalization before declaring the link operational.
Data Encoding and Framing	Applies 8b/10b encoding for Gen1–2, 128b/130b encoding for Gen3–5, and FLIT-based framing for Gen6, providing the correct symbol structure and DC balance properties for each supported speed.
FEC Encoding and Decoding	Applies mandatory Reed-Solomon RS(544,514) Forward Error Correction in Gen6 mode, generating 30 parity symbols per FLIT on the TX side and correcting PAM4 burst errors on the RX side.
PAM4 Modulation Mapping	Maps 2-bit binary pairs to PAM4 Gray code symbols before the DAC on the TX path, and decodes received PAM4 symbol decisions back to binary on the RX path, minimizing the impact of symbol boundary errors.
Clock Compensation	Inserts SKP ordered sets on the TX path at the required interval and removes them on the RX path, compensating for spread-spectrum clock frequency differences between the transmit and receive clock domains across the link.
Ordered Set Generation and Detection	Generates and detects all PCIe ordered sets including TS1, TS2, FTS, EIOS, EIEOS, SOS, and SKP, providing the signaling infrastructure required for link training, power management transitions, and clock compensation.

Equalization	Performs Gen3+ multi-phase link equalization by controlling PIPE TX de-emphasis and RX equalization settings, compensating for frequency-dependent channel loss at high data rates.
Speed and Width Negotiation	Negotiates the highest mutually supported link speed and the maximum compatible link width through TS1/TS2 data rate identifier exchange during Polling and Configuration.
Power State Management	Implements PHY-level power state entry and exit sequencing for L0s, L1, L1.1, L1.2, and L2, coordinating with the Data Link Layer PM FSM and the analog PHY through PIPE power-down commands.
Lane Management	Detects and corrects lane reversal due to flipped connectors and per-lane P/N polarity inversions, and compensates for lane-to-lane skew in multi-lane configurations using SKP ordered sets as deskew references.
Reset Management	Sequences fundamental reset (PERST#) release in the correct order across all protocol layers, and propagates Hot Reset ordered sets downstream through switch hierarchies as required.
Compliance Testing	Generates compliance test patterns required for PCIe PHY-level electrical validation and regulatory certification.

7.5 LTSSM Controller Subsystem

The Link Training and Status State Machine (LTSSM) is the central control intelligence of the Physical Layer. It is implemented as a hierarchical system of cooperating finite state machines coordinated by a top-level dispatcher (LTSSM_TOP) that monitors all shared inputs and delegates execution to the appropriate sub-FSM based on the current link state. The LTSSM manages every aspect of the link lifecycle, from initial receiver detection through operational data transfer, error recovery, speed changes, power management, loopback, and reset (**Figure 7.2**).

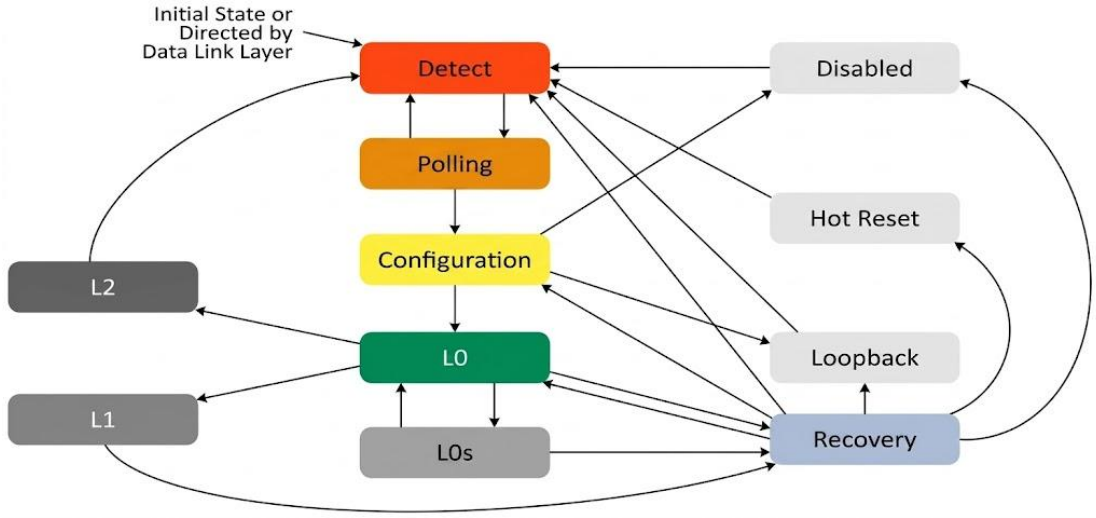


Figure 7.2 LTSSM State Controller Subsystem

7.5.1 LTSSM Top Controller (LTSSM_TOP)

LTSSM_TOP serves as the central dispatcher for the entire LTSSM hierarchy. It monitors the pipe_rx_status, pipe_detect_lane, dll_up_req, pm_req, hot_reset_req, link_down_req, and compliance_req inputs and determines which sub-FSM should be active at any given time. LTSSM_TOP outputs the current ltssm_state[5:0] encoding that all other modules use to condition their behaviour, and it asserts dl_up and dl_down to notify the Data Link Layer of link status changes. It also drives the pipe_power_down, pipe_tx_elec_idle, link_speed, and link_width outputs that propagate the LTSSM's decisions to the PIPE interface and to the broader system. The correctness of the entire Physical Layer depends on LTSSM_TOP accurately maintaining the state encoding and resolving priority among simultaneous requests from power management, error recovery, and software control paths.

7.5.2 Detect FSM (DETECT_FSM) and Receiver Detection Logic (RX_DET)

DETECT_FSM governs the Detect.Quiet and Detect.Active sub-states. Upon entry, it commands PIPE_CTRL to drive the PIPE TxDetectRx signal and waits for the analog PHY to assert PhyStatus as acknowledgement that receiver detection has completed. RX_DET implements the actual per-lane detection logic, driving the detect start signal to PIPE_CTRL and monitoring the pipe_rx_elec_idle and pipe_phy_status responses to determine receiver presence on each individual lane. RX_DET outputs the receiver detected flag and lanes_det[15:0] bitmask indicating which lanes have confirmed terminations, information that DETECT_FSM uses to decide whether to advance the LTSSM to Polling or to remain in Detect.

The LTSSM cannot leave Detect.Active without a positive receiver detection result on at least one lane; failure to detect within the detect_timeout limit causes DETECT_FSM to assert detect_timeout and remain in Detect (**Figure 7.3**).

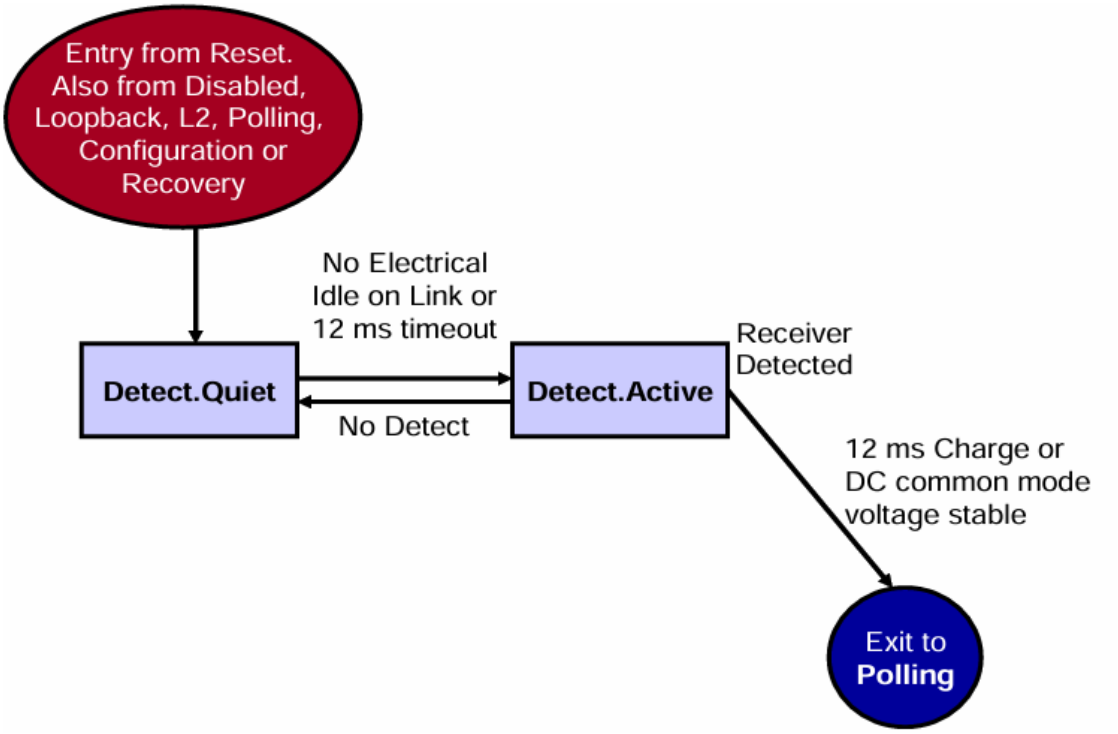


Figure 7.3 Detect State Machine

7.5.3 Polling FSM (POLL_FSM)

POLL_FSM manages the Polling. Active, Polling. Compliance, and Polling. Configuration sub-states. In Polling. Active, POLL_FSM asserts send_ts1, causing TS1_GEN to begin generating Training Sequence 1 ordered sets, while simultaneously monitoring the ts1_detected and ts2_detected signals from TS_DET to track whether the remote endpoint is responding with matching ordered sets. The transition from Polling. Active to Polling. Configuration occurs when both the local and remote endpoints have transmitted and received a sufficient number of consecutive TS1 ordered sets with consistent parameters. POLL_FSM then transitions to Polling. Configuration by asserting send_ts2, initiating the TS2 exchange that signals readiness to proceed to the Configuration state. If compliance_req is asserted, POLL_FSM instead transitions to Polling. Compliance, where COMPL_GEN generates the required electrical compliance test patterns for physical layer validation (**Figure 7.4**).

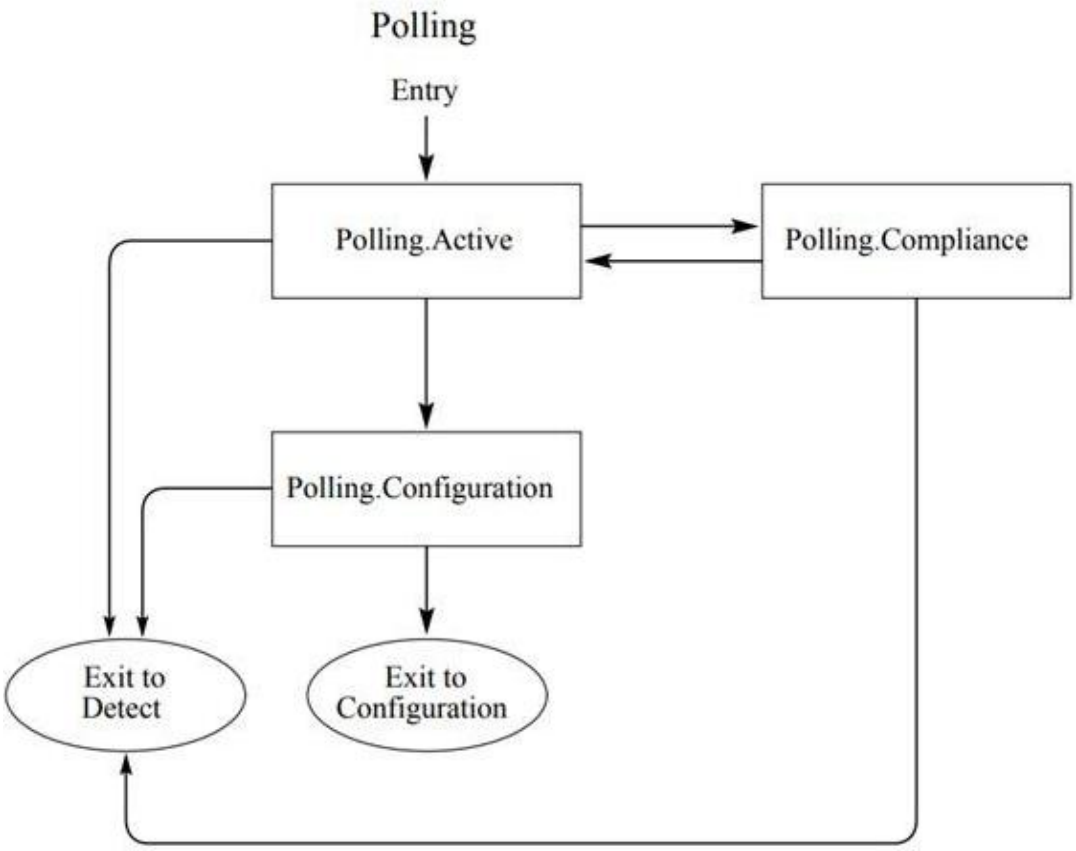


Figure 7.4 Polling sub-states

7.5.4 Configuration FSM (CFG_FSM)

CFG_FSM manages the Configuration.Linkwidth.Start, Configuration.Linkwidth.Accept, Configuration.Lanenum.Wait, Configuration.Lanenum.Accept, and Configuration.Complete sub-states. Working in coordination with WDT_NEG, CFG_FSM iteratively assigns and confirms link and lane numbers across all active lanes by comparing the ts1_link_num and ts1_lane_num values received from TS_DET against the locally transmitted values in the cfg_link_num and cfg_lane_num outputs. The negotiated width output from WDT_NEG identifies the set of lanes that have been successfully paired, determining the active link width for this link session. Only when all active lanes have confirmed consistent link and lane number assignments does CFG_FSM assert cfg_done, allowing LTSSM_TOP to advance the link to L0 and assert dl_up (**Figure 7.5**).

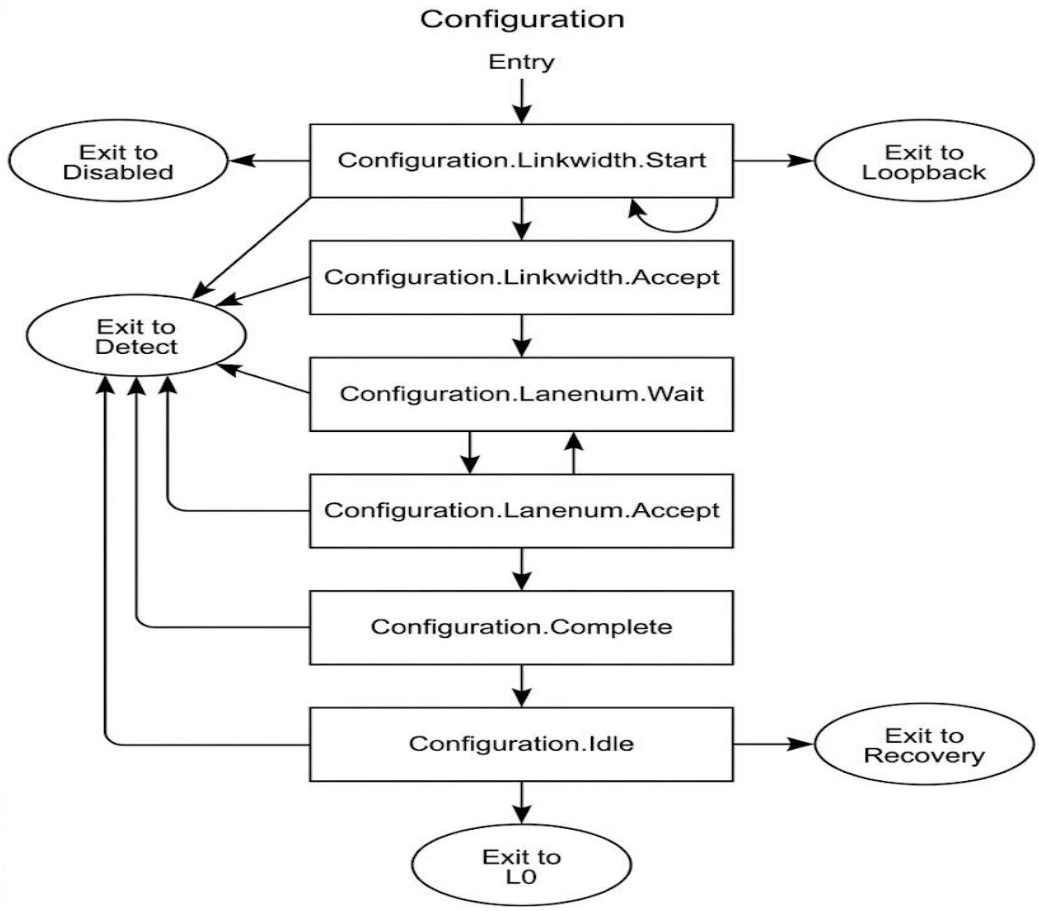


Figure 7.5 Configuration State

7.5.5 Recovery FSM (RECV_FSM)

RECV_FSM manages the Recovery state and all its sub-states: Recovery.RcvrLock, Recovery.RcvrCfg, Recovery.Idle, and Recovery.Speed. Recovery is entered whenever the LTSSM detects a link error, receives a speed change request from SPD_CHG, or requires re-equalization through EQ_CTRL. In Recovery.RcvrLock, RECV_FSM asserts send_ts1 to re-establish symbol and block lock with the remote endpoint. In Recovery.RcvrCfg, it asserts send_ts2 to re-negotiate link parameters. In Recovery.Speed, it enables speed_change_en to allow SPD_CHG to command the new PIPE Rate. RECV_FSM also manages the eq_start signal to EQ_CTRL when equalization must be performed during speed transitions. The recv_done output notifies LTSSM_TOP that recovery has completed successfully, allowing the link to return to L0, while recv_timeout_err triggers a further escalation toward link re-initialization if recovery cannot be completed within the specified time limit (**Figure 7.6**).

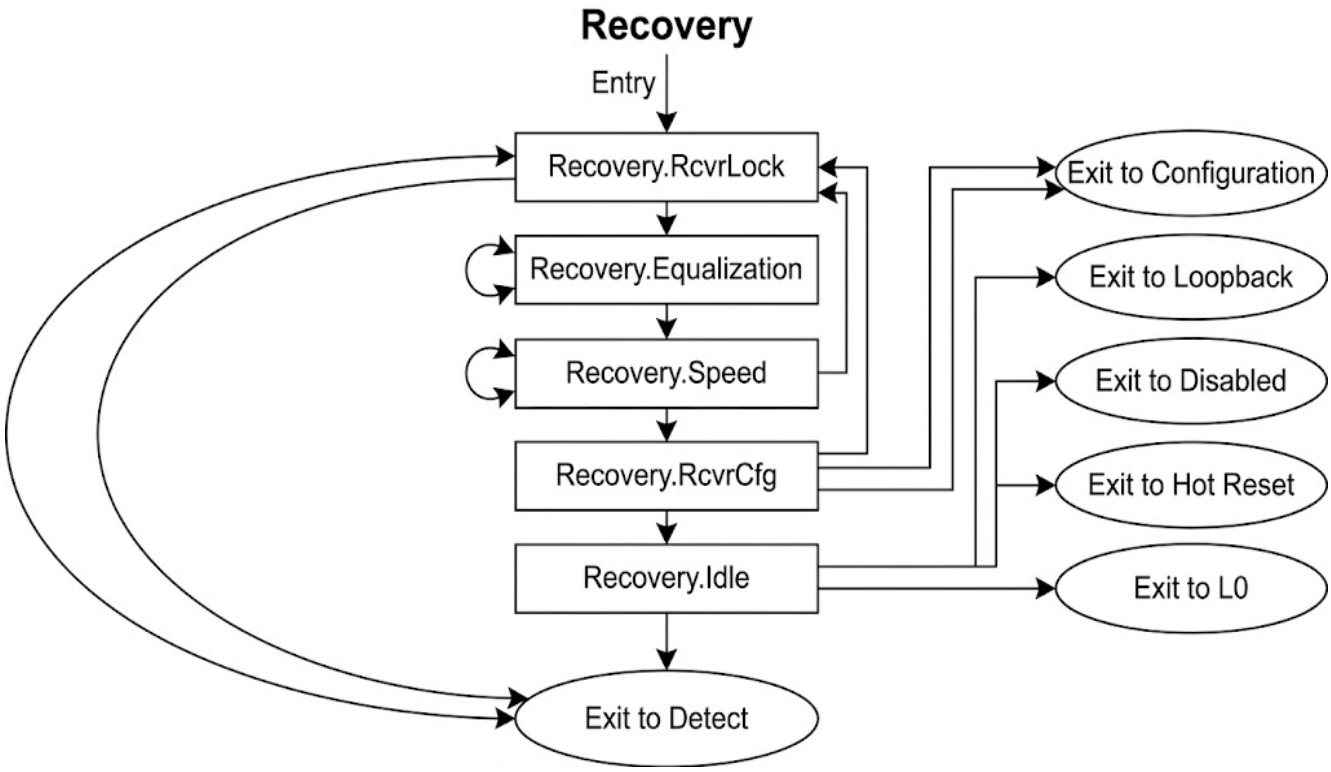


Figure 7.6 Recovery State

7.5.6 L0/L0s FSM (L0_FSM) and L1 FSM (L1_FSM)

L0_FSM manages the normal operational state (L0) and the L0s low-power standby state. In L0, L0_FSM asserts l0_active, enabling the full TX and RX data pipelines. When the Data Link Layer PM FSM requests L0s entry, L0_FSM asserts send_eios, causing EIOS to generate the Electrical Idle Ordered Set that signals the remote endpoint to expect electrical idle on the transmit lane. During L0s, the transmitter drives the lane into electrical idle as commanded through BEAC and PIPE_CTRL. L0s exit is initiated by asserting send_fts, causing FTS to generate the Fast Training Sequence required for the remote receiver to re-establish lock before data transmission resumes. L1_FSM manages the deeper L1, L1.1, and L1.2 power states, coordinating with the Data Link Layer PM FSM through pm_dllp_rx to ensure that both ends of the link enter and exit L1 synchronously. L1_FSM commands pipe_power_down[1:0] to place the analog PHY macro in the appropriate reduced-power operating mode (**Figure 7.7**).

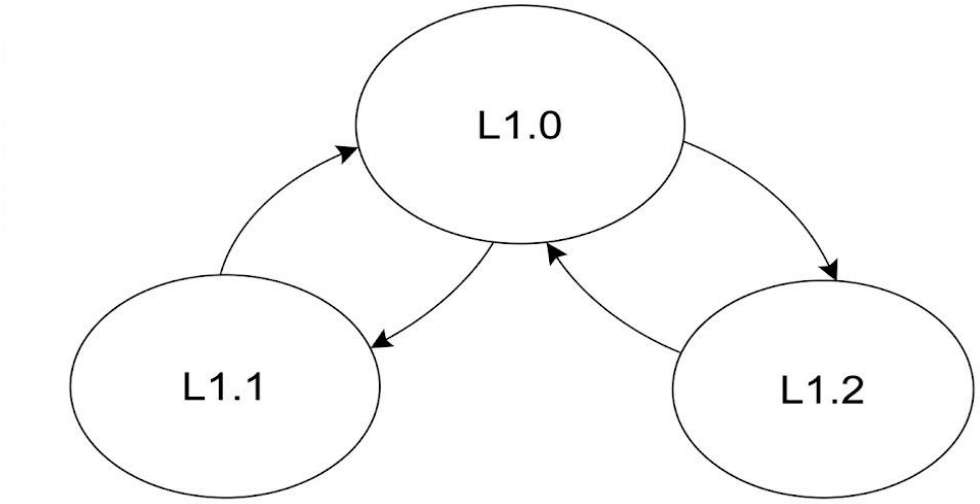


Figure 7.7 PCIe LTSSM low power states

7.5.7 Hot Reset and Disabled FSM (HRST_FSM)

HRST_FSM handles two special LTSSM states: Hot Reset and Disabled. Hot Reset is triggered when two consecutive incoming TS1 ordered sets with the Hot Reset bit set are detected, causing HOT_RST to assert hot_reset_active and pipe_reset_out to propagate the reset signal downstream through the PCIe hierarchy. HRST_FSM responds by transmitting TS1 ordered sets with the Hot Reset bit set toward the downstream device, ensuring that the reset propagates correctly through switch hierarchies as mandated by the PCIe Base Specification. The Disabled state is entered when software asserts the link disable bit in the Link Control register, causing HRST_FSM to transmit TS1 ordered sets with the Link Disable bit set and to command PIPE_CTRL to power down all lanes (**Figure 7.8**).

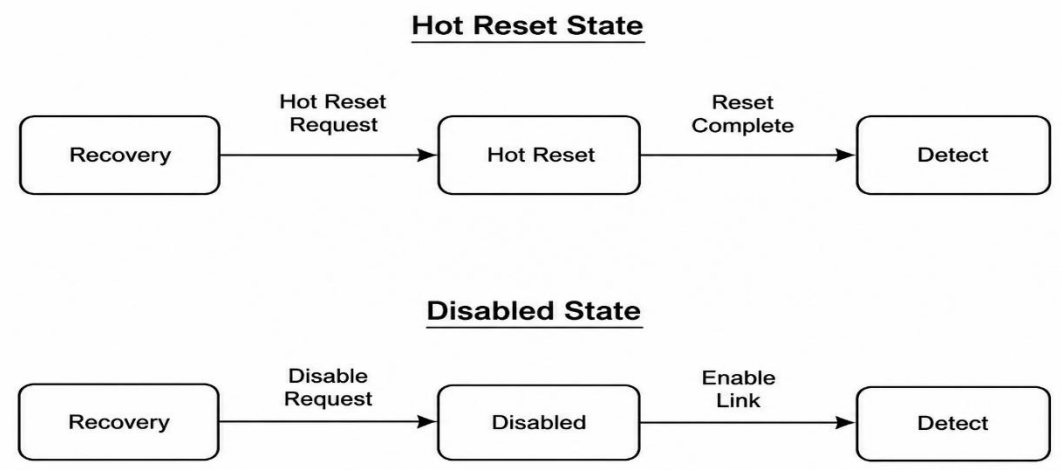


Figure 7.8 Hot and Disable states

7.5.8 Loopback FSM (LB_FSM)

LB_FSM manages the Loopback LTSSM state, which is used for physical layer testing, characterization, and compliance verification. In loopback mode, the link operates in one of two roles: the loopback master transmits data and receives the reflected copy returned by the loopback slave. LB_FSM enters loopback master mode when lb_req is asserted with lb_master high, causing it to assert send_ts1_lb to generate TS1 ordered sets with the Loopback bit set. The loopback slave enters slave mode when it detects the lb_master flag in incoming TS1 ordered sets. LB_FSM asserts lb_data_en to enable loopback data reflection through the TX path and monitors lb_timer_exp to bound the duration of the loopback session (**Figure 7.9**).

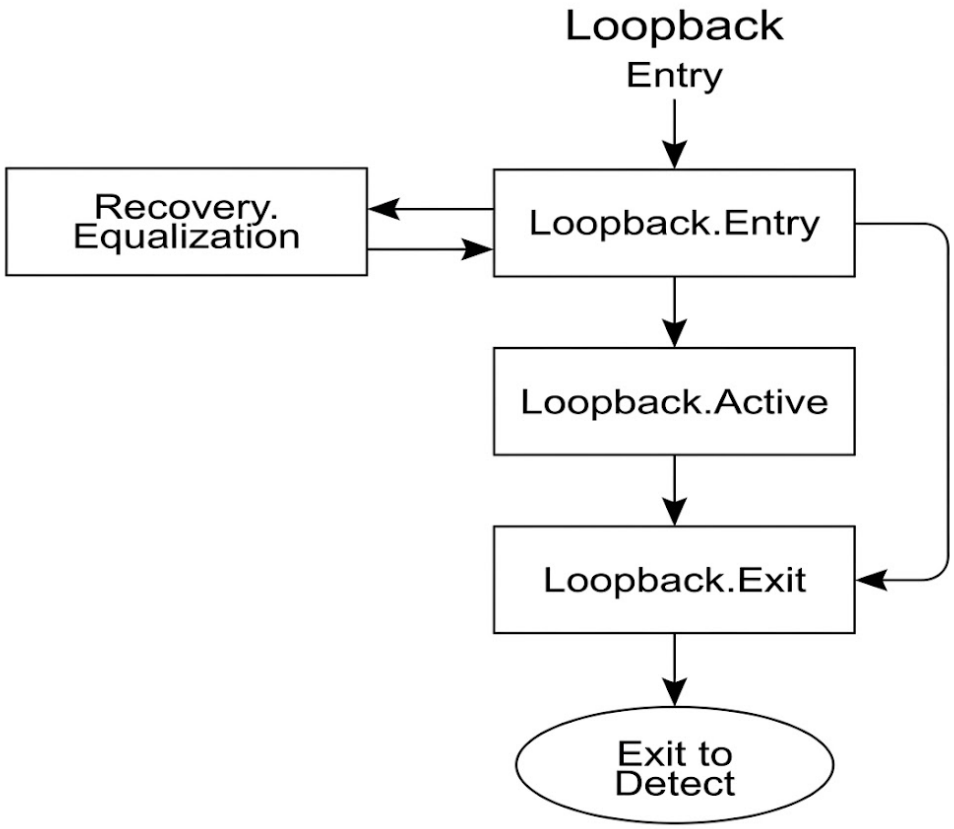


Figure 7.9 Loopback State

7.6 The Transmit Data Path: From FLIT to Physical Signal

With the LTSSM in the L0 state and dl_up asserted, the Physical Layer transmit data path becomes fully active. Its function is to accept FLIT-framed data from the Data Link Layer, apply the encoding and error correction required for reliable transmission at the negotiated speed, and deliver a continuous stream of correctly formatted symbols to the analog SERDES. This process involves seven cooperating modules whose interactions are described below as a unified pipeline.

7.6.1 FLIT Framer TX (FLIT_TX)

FLIT_TX is the first processing stage in the Gen6 TX pipeline and replaces the 128b/130b encoder used in Gen 3 through Gen 5 when flit_mode_en is asserted. FLIT_TX accepts TLP data on tlp_data[1023:0] and DLLP data on dllp_data[63:0] from the Data Link Layer and packs them into a complete 256-byte FLIT structure on flit_out[2047:0]. As part of this process, FLIT_TX generates the flit_sync_hdr[1:0] synchronization header, the flit_seq[11:0] sequence number, and the flit_crc[23:0] 24-bit integrity field, and identifies null slots in flit_null_slots[3:0] where no valid TLP or DLLP data is present. A critical property of FLIT_TX is that it operates with zero encoding overhead: unlike 8b/10b which introduces a 20 percent bandwidth penalty or 128b/130b which introduces approximately 1.5 percent, FLIT mode consumes no additional bandwidth for encoding because framing information is embedded within the FLIT structure itself rather than added as encoding symbols around the data.

7.6.2 FEC Encoder (FEC_ENC)

The output of FLIT_TX passes immediately to FEC_ENC, which implements the Reed-Solomon RS(544,514) Forward Error Correction code mandated for all Gen6 links. FEC_ENC accepts the 256-byte (2048-bit) FLIT on flit_in[2047:0] and generates 30 parity symbols that are appended to the FLIT, producing the 288-byte (2304-bit) FEC-protected output on flit_fec_out[2303:0]. The RS(544,514) code is capable of correcting up to 15 symbol errors per FLIT, providing robust protection against the burst errors that characterize PAM4 transmission at 64 GT/s.The 30 parity symbols also serve as the code word input to the FEC_SYN syndrome calculator on the receive side, enabling rapid error detection before the full RS decoding computation is performed. FEC encoding is the key enabling technology that makes PAM4 modulation viable at 64 GT/s by transforming the higher raw BER of PAM4 into an effective post-FEC BER comparable to that of prior NRZ generations.

7.6.3 PAM4 Gray Code Encoder (PAM4_ENC)

The FEC-protected output is passed to PAM4_ENC when pam4_en is asserted for Gen6 operation. PAM4_ENC maps consecutive 2-bit binary pairs from data_in[255:0] to PAM4 Gray code symbols on pam4_symbols[127:0], where each output symbol encodes two input bits as one of four voltage levels: 00, 01, 11, 10 in Gray code order. The Gray code mapping is architecturally significant because it ensures that adjacent PAM4 levels differ by only one bit, minimizing the number of bit errors caused by a single symbol error at the decision boundaries. Because PAM4 symbol errors are most likely to occur at the boundaries between adjacent levels, the Gray code assignment ensures that the most probable error events — single-level misidentifications — produce only single-bit errors in the decoded binary stream rather than two-bit errors, reducing the average bit error rate delivered to the FEC decoder.

7.6.4 TX Gear Box (TX_GEAR) and Ordered Set Generator (COMP_GEN)

TX_GEAR performs the width conversion required to adapt the wide parallel data path of the digital core to the narrower serialization rate expected by the analog SERDES. It accepts par_data_in[255:0] from PAM4_ENC at the core clock rate and produces ser_data_out[63:0] at the higher serializer clock rate, with the gear_ratio[2:0] input selecting the appropriate conversion factor for the negotiated link speed. This width conversion is necessary because the digital core operates at a fixed clock frequency regardless of link speed, while the serializer clock scales with the negotiated data rate.

COMP_GEN generates all ordered set patterns required during link training, power management transitions, and clock compensation. It monitors the send_ts1, send_ts2, send_fts, send_eios, send_eieos, send_sos, and send_compliance control signals from the LTSSM sub-FSMs and generates the corresponding ordered set data on os_data[255:0] with the os_type[3:0] classification. This centralized ordered set generation ensures that all training sequences and control symbols conform precisely to the bit patterns specified by the PCIe Base Specification, regardless of which LTSSM sub-state has requested their transmission.

7.6.5 TX Datapath MUX (PHY_TX_MUX) and PIPE TX Interface (PIPE_TX)

PHY_TX_MUX is the final arbitration point in the TX pipeline. It accepts three input streams — encoded data or FLITs from the encoding pipeline, ordered set data from COMP_GEN, and the electrical idle command — and selects among them according to a fixed priority scheme: Electrical Idle takes the highest priority to ensure that power management transitions are never delayed by pending data, followed by ordered sets which take priority over encoded data to ensure that link training sequences are transmitted without interruption, followed finally by encoded FLIT or legacy encoded data. The selected output on pipe_tx_data[255:0] is forwarded to PIPE_TX.

PIPE_TX is the boundary module between the digital RTL and the analog PHY macro. It accepts the multiplexed TX data, formats it onto the pipe_txd[255:0] PIPE TxData bus together with the pipe_txdatak control character indicators, and asserts pipe_tx_elec_idle, pipe_tx_compliance, pipe_power_down[1:0], and pipe_tx_swing as commanded by PIPE_CTRL and the LTSSM. The TX Elastic Buffer (TX_ELBUF) is interposed between the core-clock domain and the PIPE clock domain as an asynchronous FIFO, absorbing the frequency difference introduced by spread-spectrum clocking and ensuring that the PIPE interface receives a continuous, gap-free data stream at the SERDES clock rate.

The legacy encoding modules ENC_8B10B and ENC_128B130B remain present in the TX pipeline and are selected by PHY_TX_MUX when operating at Gen1–2 and Gen3–5 speeds respectively. ENC_8B10B applies the 10-bit encoded output with disparity tracking for Gen1 and Gen2 operation at 2.5 GT/s and 5 GT/s. ENC_128B130B generates the 130-bit block with its 2-bit synchronization header for Gen3 through Gen5 operation at 8 GT/s through 32 GT/s. Both encoders are bypassed in Gen6 mode when FLIT_TX and FEC_ENC provide the complete framing and error protection.

7.7 The Receive Data Path: From Physical Signal to FLIT

The receive data path performs the inverse of the transmit pipeline: it accepts raw symbol data from the analog SERDES through the PIPE interface, decodes PAM4 symbols, computes the FEC syndrome, corrects burst errors, aligns to FLIT boundaries, deframes the FLIT, and delivers validated data to the Data Link Layer. Error detection and correction are woven throughout every stage.

7.7.1 PIPE RX Interface (PIPE_RX), RX Elastic Buffer (RX_ELBUF), and RX Gear Box (RX_GEAR)

PIPE_RX is the first point of engagement between the analog PHY and the digital RX pipeline. It captures the 256-bit parallel received data arriving on pipe_rxd[255:0] together with the pipe_rxdatak control character indicators, pipe_rx_valid, pipe_rx_status, and pipe_rx_elec_idle from the analog SERDES. PIPE_RX also monitors the fec_syndrome and fec_corrected signals from the FEC subsystem for forwarding to the Data Link Layer.

The raw received data passes from the PIPE clock domain to the core clock domain through RX_ELBUF, an asynchronous FIFO that performs the clock-domain crossing using Gray-code read and write pointers for metastability protection. RX_ELBUF also implements the slip mechanism required for SKP ordered set removal: when the SKP module asserts a slip request after detecting and absorbing a SKP ordered set, RX_ELBUF removes the corresponding data words from the elastic buffer to compensate for the clock-rate difference introduced by spread-spectrum clocking.

RX_GEAR performs the width conversion inverse to TX_GEAR, accepting the narrow serializer-rate data from the SERDES-side clock domain and expanding it to the wide parallel format expected by the core data path. The gear_ratio[2:0] input selects the appropriate conversion factor matching the negotiated link speed.

7.7.2 Lane Polarity (LANE_POL), Block Alignment (BLK_ALIGN), and Lane Deskew (DESKEW)

LANE_POL corrects per-lane P/N polarity inversions that may result from PCB layout decisions where the differential pair is routed with the positive and negative conductors swapped relative to the expected polarity. LANE_POL detects polarity inversion by examining the received comma characters or synchronization header patterns — which have a known polarity — and asserts polarity_inv[15:0] per lane when inversion is detected. It then applies a per-lane bit inversion to rx_data_pol[255:0], transparently correcting the polarity before any downstream processing is performed.

BLK_ALIGN establishes and maintains alignment to the 2-bit synchronization header boundary used in Gen3–5 mode, or to the 256-byte FLIT boundary used in Gen6 mode. It monitors the incoming data stream for the expected header patterns and asserts aligned_valid and the corrected sync_hdr[1:0] output once alignment is established. Alignment loss — detected when the observed header pattern deviates from the expected value — causes BLK_ALIGN to assert align_err, which is forwarded to LTSSM_TOP as a trigger for Recovery state entry.

DESKEW compensates for lane-to-lane propagation skew in x2 through x16 link configurations. Because each lane has an independent physical path to the remote endpoint, slight differences in trace length, via count, and connector geometry introduce differential delays between lanes that must be removed before the multi-lane data can be recombined into a single coherent stream. DESKEW uses the arrival time of SKP ordered sets — which are transmitted simultaneously on all lanes — as a common reference to measure the per-lane skew amount. It then applies configurable delay adjustments to earlier-arriving lanes, aligning all lanes to the latest-arriving lane boundary within the deskew tolerance specified by the PCIe Base Specification.

7.7.3 LOCK FSM (LOCK_FSM)

LOCK_FSM manages the symbol lock and block lock acquisition process on the receive path. In Gen1–2 mode, it searches for the K28.5 comma character to establish byte alignment. In Gen3–5 mode, it monitors the 2-bit synchronization header pattern of alternating 01 and 10 values at the start of each 128b/130b block. In Gen6 FLIT mode, it monitors the 2-bit FLIT synchronization header generated by FLIT_SYNC on the transmit side. LOCK_FSM asserts symbol_lock and block_lock when a sufficient number of consecutive valid synchronization patterns have been observed, signaling to BLK_ALIGN and the downstream pipeline that the received data stream is correctly aligned. Loss of lock — caused by a sufficient number of consecutive invalid header patterns — causes LOCK_FSM to assert lock_lost, which triggers a Recovery state entry through LTSSM_TOP.

7.7.4 PAM4 Gray Code Decoder (PAM4_DEC) and FEC Syndrome Calculator (FEC_SYN)

PAM4_DEC is the receive-side counterpart of PAM4_ENC. It accepts the 128-symbol PAM4-encoded data from the SERDES path on pam4_symbols_in[127:0] and decodes each symbol back to the 2-bit binary pair, reconstructing data_out[255:0]. The decoding process is the inverse of the Gray code mapping applied during transmission, recovering the original binary stream from the received symbol decisions. Any symbol decision error — where the received signal level is misidentified as an adjacent level — produces a binary error that is flagged through decode_err and forwarded to FEC_DEC for correction.

FEC_SYN computes the Reed-Solomon syndrome vector from the received FLIT including its appended parity symbols on flit_rx[2303:0]. The syndrome computation is performed as a polynomial evaluation at the RS code roots, and the result is output on syndrome[255:0]. A zero syndrome vector — indicated by zero_syndrome being asserted — confirms that the received FLIT contains no detectable errors and can be forwarded directly without FEC correction. A non-zero syndrome indicates the presence of one or more symbol errors and triggers FEC_DEC to perform the full correction computation. The syndrome computation is architected to complete in a fixed number of pipeline stages, maintaining deterministic latency through the RX pipeline regardless of the error condition.

7.7.5 FEC Decoder (FEC_DEC)

FEC_DEC implements the Reed-Solomon RS(544,514) decoder responsible for correcting PAM4 burst errors in received FLITs. It accepts the full 288-byte FEC word on flit_fec_in[2303:0] together with the syndrome vector from FEC_SYN and performs the Berlekamp-Massey error locator polynomial computation followed by the Chien search for error positions and the Forney algorithm for error magnitude computation. The correction capacity of RS(544,514) is 15 symbol errors per FLIT, corresponding to up to 15 arbitrary byte errors anywhere within the 288-byte codeword. Corrected FLITs are output on flit_corrected[2047:0] with fec_corrected asserted, while the fec_syndrome[255:0] output carries the syndrome vector for forwarding to the Data Link Layer. When the number of symbol errors exceeds 15, FEC_DEC asserts fec_uncorrectable and fec_err_count[7:0], triggering a link recovery procedure through DLL_ERR and LTSSM_TOP.

7.7.6 FLIT Defamer RX (FLIT_DFRM)

FLIT_DFRM is the final processing stage in the Gen6 RX data path and the inverse of FLIT_TX. It accepts the FEC-corrected 256-byte FLIT on flit_in[2303:0] and strips the FEC parity symbols, validates the 24-bit FLIT CRC, extracts the embedded TLP data onto tlp_out[1023:0] and DLLP data onto dllp_out[63:0], and recovers the flit_seq[11:0] sequence number for forwarding to the Data Link Layer SEQ_CHK module. FLIT_DFRM asserts flit_null when the received FLIT contains only null slots with no valid TLP or DLLP payload, signalling to the Data Link Layer that the FLIT should be discarded after extracting any piggybacked acknowledgement information. A CRC mismatch causes FLIT_DFRM to assert flit_crc_err, and a synchronization header error causes flit_sync_err, both of which propagate to the Data Link Layer error handling subsystem. The legacy decode modules DEC_8B10B and DEC_128B130B remain present for Gen1–2 and Gen3–5 operation respectively and are engaged by FLIT_DFRM's upstream demultiplexing logic when flit_mode_en is deasserted.

7.8 Gen6-Specific Physical Layer Features

7.8.1 FLIT Sync Header Generation and Checking (FLIT_SYNC)

FLIT_SYNC manages the 2-bit synchronization header that PCIe Gen6 prepends to every transmitted FLIT and validates on every received FLIT. On the TX side, FLIT_SYNC accepts the outgoing FLIT on flit_tx[2047:0] and prepends the 2-bit sync header, producing flit_tx_with_hdr[2049:0]. The sync header pattern is defined by the PCIe Gen6 specification to provide a distinguishable boundary marker that the remote LOCK_FSM can detect to establish and maintain FLIT lock. On the RX side, FLIT_SYNC validates the header pattern of incoming FLITs: a correctly formatted header asserts sync_hdr_rx_ok and contributes to maintaining flit_lock, while an invalid header asserts sync_hdr_rx_err and contributes to lock loss detection. Loss of FLIT lock is a serious condition that triggers Recovery state entry, since the receiver can no longer reliably locate FLIT boundaries and therefore cannot correctly deframe TLPs and DLLPs from the received data stream.

7.8.2 Data Rate Advertisement Logic (DRATE_ADV)

DRATE_ADV manages the encoding and decoding of Gen6-specific capability bits within the Data Rate Identifier fields of TS1 and TS2 ordered sets. It accepts the local_speed_cap[7:0] register and encodes adv_gen6_cap, adv_flit_mode, and adv_fec_cap bits into the outgoing TS1 and TS2 ordered sets, advertising to the remote endpoint that this device supports 64 GT/s operation, FLIT mode framing, and RS FEC. On the receive side, DRATE_ADV decodes the corresponding capability bits from the incoming ts1_rx and ts2_rx ordered set data, asserting remote_gen6_cap when the remote device confirms Gen6 capability. If the advertised capabilities are inconsistent — for example, if one endpoint advertises Gen6 capability but the other does not — DRATE_ADV asserts speed_cap_mismatch, causing SPD_NEG to select the highest mutually supported speed below Gen6 as the negotiated operating rate.

7.9 Link Negotiation and Equalization Subsystem

7.9.1 Speed Negotiation (SPD_NEG) and Speed Change FSM (SPD_CHG)

SPD_NEG determines the target operating speed for the link by comparing the local device's speed capability advertisement in local_speed_cap[7:0] against the remote device's capability conveyed through ts1_speed_cap[7:0] and ts2_speed_cap[7:0] received from TS_DET. The negotiation algorithm selects the highest speed supported by both endpoints, encoding the result in target_speed[3:0] and asserting speed_change_en to initiate the speed change procedure. SPD_NEG also outputs adv_speed_cap[7:0], which specifies the speed capability bits to be embedded in outgoing TS1 and TS2 ordered sets by COMP_GEN during subsequent training exchanges.

SPD_CHG orchestrates the actual execution of the speed change by commanding the PIPE Rate signal to the new target speed through pipe_rate_out[3:0]. The speed change proceeds through the Recovery. Speed sub-state of RECV_FSM, during which the link enters a brief training interval at the new speed before returning to L0. SPD_CHG monitors the pipe_rate input to confirm that the analog PHY has acknowledged the rate change through the PIPE PhyStatus mechanism, and it asserts speed_change_done when the transition is complete or speed_change_err if the analog PHY fails to acknowledge within the specified timeout.

7.9.2 Link Width Negotiation (WDT_NEG)

WDT_NEG negotiates the active link width by examining the ts1_lane_num fields decoded from incoming TS1 ordered sets during the Configuration state. It compares the lane numbers assigned by the remote endpoint against the locally active lane numbers and identifies the set of lanes for which a consistent pairing has been achieved. The negotiated_width[5:0] output encodes the resulting active lane count, and active_lanes[15:0] is a bitmask identifying precisely which physical lanes are included in the active link. If the available lane count decreases due to lane failure or link reconfiguration, WDT_NEG asserts width_change_req to initiate a link retraining sequence through RECV_FSM.

7.9.3 Link Equalization Controller (EQ_CTRL)

EQ_CTRL manages the PCIe link equalization process, which was introduced in PCIe Gen3 and continues to be used in later generations. Equalization helps compensate for channel attenuation, where high-frequency signal components experience greater loss than low-frequency components during transmission. This is achieved by adjusting transmitter pre-emphasis, de-emphasis, and receiver equalization settings.

The equalization process is performed during the Recovery. Equalization state and consists of up to four phases (Phase 0 to Phase 3). During these phases, the link partners exchange equalization information through TS1 ordered sets and the PIPE equalization interface. EQ_CTRL monitors fields such as `ts1_eq_req_bit` and `ts2_eq_preset` to determine the requested equalization settings from the remote device. It then controls PHY parameters through signals including `pipe_txdeemph[2:0]`, `pipe_txmargin[2:0]`, and `pipe_rxeqeval_out`.After all equalization phases are completed successfully, EQ_CTRL asserts `eq_done`, allowing RECV_FSM to exit Recovery. Equalization and return the link to the normal L0 operating state.

7.10 Ordered Set Generation and Detection

In PCIe Gen 6, Ordered Sets (OS) are special control sequences used for link training, synchronization, and state management. The Ordered Set Generation block inserts the required sequences into the transmit stream, while the Ordered Set Detection block identifies them on the receive side and extracts control information. These functions help maintain proper link operation and support LTSSM state transitions.

7.10.1 TS1 and TS2 Generators (TS1_GEN, TS2_GEN) and Detector (TS_DET)

TS1_GEN and TS2_GEN generate the Training Sequence 1 and Training Sequence 2 ordered sets respectively, which are the fundamental communication medium for all LTSSM state negotiations. TS1_GEN accepts the link_num[7:0], lane_num[7:0], speed_cap[7:0], and fts_count[7:0] parameters from CFG_FSM and LTSSM_TOP, along with various control bits including compliance mode and Hot Reset, and generates the 256-bit ordered set data on ts1_data[255:0] conforming precisely to the bit-field layout defined by the PCIe Base Specification. TS2_GEN performs the equivalent function for Training Sequence 2 ordered sets, whose receipt and agreement at both endpoints marks the end of the Configuration and Recovery training sequences. TS_DET on the receive path continuously monitors the incoming rx_data[255:0] stream for TS1 and TS2 ordered set patterns and decodes the embedded ts1_link_num, ts1_lane_num, and ts2_speed_cap fields from confirmed ordered sets, providing the LTSSM sub-FSMs with the information they need to make state transition decisions.

7.10.2 FTS Generator and Detector (FTS)

The Fast Training Sequence (FTS) module manages the generation and detection of FTS ordered sets, which are used exclusively for L0s power state exit. When the L0_FSM requests L0s exit, it asserts send_fts, causing FTS to generate fts_count[7:0] consecutive FTS ordered sets on fts_data[255:0]. The FTS count is the number of FTS ordered sets required for the remote receiver to re-establish bit lock after an L0s interval, which is negotiated during the Configuration state and stored in the local fts_count register. On the receive side, FTS continuously monitors the incoming data stream for FTS patterns and asserts fts_detected when the expected FTS sequence is identified, notifying L0_FSM that the remote transmitter is exiting L0s and that the local receiver has re-established lock.

7.10.3 EIOS and EIEOS Handler (EIOS)

The EIOS module handles both Electrical Idle Ordered Sets (EIOS) and Electrical Idle Exit Ordered Sets (EIEOS), which are used for PCIe power state transitions. EIOS is sent before the lane enters electrical idle to inform the receiver that transmission will stop. EIEOS, introduced in Gen3, is sent before normal transmission resumes after an electrical idle period to help the receiver re-synchronize and recover clock and data correctly before valid data arrives.

The module uses control signals (eios_send and eieos_send) from the LTSSM sub-FSMs to generate these ordered sets, and it also detects incoming EIOS and EIEOS patterns on the receive side, asserting eios_detected and eieos_detected to inform the LTSSM about the remote side's power state changes.

7.10.4 SKP Ordered Set Generator and Receiver (SKP)

SKP manages the periodic insertion and removal of SKP (Skip) ordered sets, which is the mechanism by which PCIe compensates for the frequency difference between the transmit clock and the receive clock in systems where the two endpoints use independent clock sources with spread-spectrum modulation applied. The PCIe specification requires that SKP ordered sets be inserted by the transmitter every 1180 to 1538 symbols, and that the receiver remove them from the data stream before forwarding data to the core pipeline. SKP monitors the skp_interval[11:0] counter and asserts skp_tx_valid when a SKP ordered set should be inserted, generating the ordered set data on skp_data[255:0]. On the receive side, SKP monitors the incoming data stream for the K28.0 K28.0 K28.0 pattern that identifies a SKP ordered set, asserts skp_detected, and asserts skp_removed after absorbing the ordered set from the stream and triggering a slip in RX_ELBUF to absorb the corresponding clock rate difference.

7.11 PIPE Interface, Lane Management, and Support Subsystems

7.11.1 PIPE Interface Controller (PIPE_CTRL)

PIPE_CTRL serves as the central coordinator of all PIPE interface signals between the digital RTL and the analog PHY macro. It translates the high-level commands issued by LTSSM_TOP and the various sub-FSMs — power-down requests, rate changes, electrical idle commands, receiver detection requests, equalization commands — into the precise PIPE signal sequences required by the PIPE 5.1 specification. PIPE_CTRL monitors pipe_phystatus, pipe_rxvalid, and pipe_rxstatus[2:0] to detect PHY acknowledgements of power management and rate change commands, and it generates pipe_powerdown[1:0], pipe_rate[3:0], pipe_txdetectrx, pipe_txelecidle, pipe_txcompliance, pipe_pclkchangeack, and pipe_width[1:0] as outputs to the analog PHY. The correctness of every power management transition, speed change, and electrical idle event depends on PIPE_CTRL correctly sequencing these signals in accordance with the PIPE timing requirements.

7.11.2 Lane Reversal Logic (LANE_REV)

LANE_REV detects and corrects lane reversal, a condition that arises when the PCIe connector is mounted on the PCB with the lane order reversed relative to the expected orientation. Lane reversal is detected during the Configuration state by comparing the lane_num values received from the remote endpoint in TS1 ordered sets against the locally assigned lane numbers: if lane N at the local device corresponds to lane (max − N) at the remote device, the connection is reversed. LANE_REV asserts reversal_active and generates the lane_map[3:0] remapping table that the Configuration FSM uses to correctly assign logical lane numbers to the reversed physical lane ordering. This correction is transparent to all higher layers, which continue to see a logically numbered x1, x2, x4, x8, or x16 link regardless of the physical connector orientation.

7.11.3 Beacon and Electrical Idle Logic (BEAC)

BEAC manages two distinct low-level physical layer functions. First, it implements electrical idle detection on the receive path by monitoring pipe_rx_elec_idle and asserting ei_detect when the lane enters electrical idle, providing the LTSSM with the signal needed to confirm that the remote transmitter has completed its L0s or L1 entry. Second, it implements the PME beacon mechanism used for wakeup from deep power states (L2 and L3) when the link has been fully powered down. In beacon mode, BEAC drives periodic low-frequency electrical pulses onto the TX lane, and it monitors the RX lane for the beacon pattern transmitted by the remote device. Receipt of a beacon causes BEAC to assert wakeup_req toward LTSSM_TOP, initiating the full link re-initialization sequence required to return from L2/L3 to L0.

7.11.4 Spread Spectrum Clock Controller (SSC_CTRL)

SSC_CTRL handles spread-spectrum clocking (SSC), which is used to reduce electromagnetic interference and meet regulatory requirements. It works by spreading the clock energy over a wider frequency range, lowering peak emissions at any single frequency. The module takes the configuration inputs ssc_en and ssc_profile[1:0], which select between center-spread and down-spread modes, and produces the ssc_mod_req[7:0] signal to control the required frequency deviation in the clock generator. In PCIe, this deviation is typically ±0.5% for center-spread or −0.5% for down-spread. SSC_CTRL also works with SKP insertion and removal on both TX and RX sides to ensure the elastic buffer can handle the small frequency variations without data loss.

7.11.5 Power State Timer (PWR_TMR)

PWR_TMR provides the PHY-level timing substrate for L0s and L1 power state entry and exit sequencing, distinct from the DLL-level PM_TMR module. While PM_TMR in the Data Link Layer manages the higher-level timeout thresholds for initiating power state transitions, PWR_TMR operates at the physical layer level and implements the precise timing constraints required for PIPE power-down sequencing. It monitors the l0s_entry_req, l1_entry_req, l0s_exit_req, and l1_exit_req control signals from LTSSM_TOP and generates the corresponding timer expiry signals — l0s_entry_timer_exp, l1_entry_timer_exp, l0s_exit_timer_exp, and l1_exit_timer_exp — that drive the PIPE_CTRL power-down command sequence. Correct PHY power state operation is impossible without PWR_TMR, as the analog PHY requires precise timing coordination between the assertion of PIPE Power Down and the subsequent PIPE PhyStatus acknowledgement.

7.11.6 Fundamental Reset FSM (FUND_RST)

FUND_RST sequences the fundamental reset procedure that occurs on power-on or whenever PERST# is asserted. It monitors perst_n, power_good, and clk_valid and executes a defined reset release sequence that ensures the PHY initializes before the Data Link Layer and the Transaction Layer. Specifically, FUND_RST holds dl_rst_n low until phy_rst_n has been released and the analog PHY has completed its internal initialization, and it holds sys_rst_n low — which resets the Transaction Layer — until the Data Link Layer has been released and the DLL_INIT module has acknowledged its own reset completion. This ordered reset sequence prevents higher layers from attempting to use interfaces that the lower layers have not yet initialized, eliminating the class of initialization failures that would otherwise arise from race conditions during power-on. FUND_RST also monitors rst_timeout_val[15:0] to bound the maximum duration of each reset phase, asserting a timeout error if the expected acknowledgements are not received within the specified interval.

7.11.7 Compliance Pattern Generator (COMPL_GEN)

COMPL_GEN generates the compliance test patterns required for PCIe PHY-level electrical validation. Compliance testing verifies that the transmitted signal meets the eye diagram, jitter, and voltage level specifications defined by the PCIe electrical compliance test specification.

COMPL_GEN accepts the compliance_pattern[3:0] selection and deemph_req[2:0] de-emphasis setting from the LTSSM compliance mode logic and generates the corresponding 256-bit compliance data on compl_data[255:0]. The compliance patterns are defined by the PCIe Base Specification and include standardized bit sequences designed to stress the transmitter, test the channel, and evaluate the receiver under worst-case operating conditions. COMPL_GEN is entered via the Polling. Compliance sub-state of POLL_FSM and remains active for as long as compliance_req is asserted.

7.12 PCIe Gen 6 Impact on the Physical Layer

The transition from PCIe Gen 5 to Gen 6 introduced the most fundamental change to the Physical Layer since the introduction of 128b/130b encoding in Gen3. The adoption of PAM4 modulation at 64 GT/s affected virtually every subsystem of the Physical Layer, requiring the introduction of entirely new modules and the significant redesign of existing ones (**Table 7.3**).

Table 7.3 Comparison of Physical Layer Features Between PCIe Gen 5 and PCIe Gen 6

Feature	**PCIe Gen 5**	**PCIe Gen 6**
Modulation	NRZ (2-level, 1 bit/symbol)	PAM4 (4-level, 2 bits/symbol)
Raw Line Rate	32 GT/s	64 GT/s
Encoding Scheme	128b/130b (1.5% overhead)	FLIT-based (zero encoding overhead)
Error Correction	None at PHY level	Mandatory RS(544,514) FEC (30 parity symbols/FLIT)
Raw BER	~10^{-12} achievable without FEC	Higher raw BER requires FEC for link reliability
Framing Granularity	Per-packet variable length	Fixed 256-byte FLIT
Sync Mechanism	2-bit 128b/130b sync header	2-bit FLIT sync header
PAM4 Gray Coding	Not applicable (NRZ)	Mandatory Gray code mapping to minimize symbol error impact

FEC Syndrome Path	Not present	Dedicated FEC_SYN syndrome calculator feeding FEC_DEC
TX Encoding Pipeline	ENC_128B130B	FLIT_TX → FEC_ENC → PAM4_ENC
RX Decoding Pipeline	DEC_128B130B	PAM4_DEC → FEC_SYN → FEC_DEC → FLIT_DFRM
Physical Layer Complexity	Moderate	Significantly higher

7.13 Design Trade-offs and Architectural Decisions

Implementing the PCIe Gen 6 Physical Layer required several carefully considered architectural decisions to balance throughput, hardware area, timing closure, and signal integrity. The table below summarizes the major trade-offs adopted (**Table 7.4**):

Table 7.4 Major Architectural Trade-offs

Component	Option Chosen	Alternative	Engineering Reason
FEC Code	Reed-Solomon RS(544,514)	LDPC or Polar Code	RS codes provide burst error correction capability well-matched to the error characteristics of PAM4 symbol errors. RS(544,514) corrects up to 15 symbol errors per FLIT with deterministic decoder latency, making it suitable for the real-time pipeline requirements of the Physical Layer.
PAM4 Mapping	Gray code symbol assignment	Natural binary assignment	Gray code ensures that the most probable error event — a single-level misidentification at a decision boundary — produces only a single-bit error in the decoded binary stream. Natural binary assignment would produce 2-bit errors from the same physical event, doubling the bit error rate presented to the FEC decoder.

FLIT Sync Header	Dedicated 2-bit header per FLIT	Embedded framing within payload	A dedicated sync header provides an unambiguous, easily detectable boundary marker that the LOCK_FSM can search for independently of payload content, simplifying lock acquisition and loss detection. Embedded framing would require scanning arbitrary payload data.
TX Elastic Buffer	Asynchronous FIFO with Gray-code pointers	Synchronous FIFO	The TX path spans two independent clock domains — the digital core clock and the PIPE SERDES clock — between which spread-spectrum modulation introduces a continuous frequency variation. An asynchronous FIFO with Gray-code pointers provides safe metastability-free crossing at this boundary.
Ordered Set Generation	Centralized COMP_GEN module	Distributed generation in each FSM	Centralizing all ordered set generation in a single module ensures that every TS1, TS2, FTS, EIOS, and SKP pattern precisely matches the PCIe specification bit layout. Distributing generation across sub-FSMs would increase the risk of specification deviations and complicate verification.
Lane Deskew	SKP ordered set reference	Dedicated deskew training sequences	SKP ordered sets must be transmitted periodically for clock compensation regardless of deskew requirements. Reusing them as deskew references eliminates the need for additional deskew-specific ordered sets and reduces the overhead imposed on the link during normal operation.

7.14 Challenges and Implementation Hurdles

1- The most fundamental challenge in implementing the PCIe Gen 6 Physical Layer is the substantially reduced noise margin of PAM4 signaling compared to NRZ. In NRZ, the receiver distinguishes between two voltage levels separated by the full signal swing. In PAM4, the receiver must distinguish between four voltage levels, with each adjacent pair separated by only one-third of the total signal swing. This reduction in inter-level margin makes the received signal far more sensitive to insertion loss, reflection, crosstalk, jitter, and noise than in prior generations. The reduced eye-opening means that equalization — both TX pre-emphasis and RX continuous-time linear equalization — must be precisely calibrated for each individual channel, making the equalization controller (EQ_CTRL) and its interaction with the analog PHY through the PIPE interface a critical and complex design element.

Verification of the equalization procedure required simulation across a wide range of channel loss profiles, temperature corners, and supply voltage variations to confirm that the four-phase equalization sequence converges reliably to a stable operating point in all supported configurations.

2- The RS(544,514) FEC decoder introduces a fixed latency into the RX pipeline that did not exist in prior generations. The Berlekamp-Massey algorithm, Chien search, and Forney computation must all complete within the time available between successive FLIT arrivals to prevent pipeline stalls. At 64 GT/s, a 256-byte FLIT arrives every four nanoseconds in a single-lane configuration, imposing a very tight timing budget on the FEC decoding pipeline. The RTL implementation addresses this through aggressive pipelining of the decoder stages, parallel computation of syndrome components, and careful floor planning of the decoder logic to minimize routing delays in timing-critical paths. Verification of the FEC decoder required exhaustive simulation of all single-symbol and multi-symbol error patterns up to the 15-symbol correction limit, as well as confirmation that uncorrectable error patterns beyond the correction capacity are correctly identified and flagged without producing corrupted output.

3- A specific challenge in the PCIe Gen 6 Physical Layer is maintaining FLIT boundary synchronization across link speed changes. When the link transitions between Gen6 and a lower-speed generation through the Recovery. Speed sub-state, the framing mechanism changes from FLIT-based to 128b/130b or 8b/10b, requiring the TX and RX pipelines to switch encoding modules and re-establish synchronization at the new framing granularity. The LOCK_FSM must correctly identify the appropriate lock acquisition procedure for the new speed, and BLK_ALIGN must re-establish block alignment at the new boundary format. This switching logic was verified through constrained-random simulation of speed change sequences under traffic load, confirming that no data corruption or synchronization loss occurs during or immediately after a speed transition.

4- Lane deskew in multi-lane configurations becomes significantly more complex in the presence of spread-spectrum clocking. Because SSC continuously varies the transmit clock frequency, the inter-lane skew as measured by the arrival time of SKP ordered sets fluctuates continuously rather than remaining at a fixed value. DESKEW must therefore implement a tracking deskew mechanism that continuously updates its skew estimates rather than a one-time calibration. This tracking requirement increases the complexity of the deskew control logic and necessitates careful verification under SSC modulation profiles at both the maximum and minimum frequency deviation extremes to confirm that the deskew error remains within the tolerance specified by the PCIe Base Specification across all operating conditions.

8 Chapter Eight: Conclusion And Future Work

8.1 Conclusion

This work presents a structured and educationally oriented architectural model of PCI Express Gen 6, addressing the growing need for simplified yet accurate representations of modern high-speed interconnect standards. The study began by establishing the foundational context of PCIe evolution, tracing the progression from legacy parallel bus architectures through successive generations to the motivation behind the 64 GT/s target of Gen 6 and its adoption of PAM4 modulation. A comprehensive review of existing literature confirmed a significant research gap in practical, structured academic models for PCIe Gen 6, as most available resources present Gen 6 concepts in a fragmented and highly theoretical manner without providing implementation-ready architectural guidance.

To address this gap, a modular three-layer digital design model was proposed, covering the Transaction Layer, Data Link Layer, and Physical Layer as an integrated transmitter-receiver system separated by a behavioral digital channel. The proposed methodology defined clear layer isolation principles, a structured simulation plan, and a set of performance metrics including bit error rate, throughput, latency, and link reliability. Numerical results obtained from the complete RTL design of 119 digital modules with its logical Synthesis, combined with MATLAB theoretical analysis, validated the self-consistency of the Gen 6 specification parameters, confirming that the PAM4 eye opening, FEC coding gain, and jitter budget are achievable at the nominal operating point.

At the Transaction Layer, the work demonstrated that the transition to FLIT-based communication introduced a fundamental shift in the architectural role of this layer compared to prior PCIe generations. The requirement to dynamically pack variable-length TLPs into fixed 256-byte FLITs demanded the design of an efficient packing engine capable of aggregating multiple smaller transactions while correctly managing FLIT boundaries for larger ones. Parallel CRC-32 generation was adopted to meet the throughput demands of the 64 GT/s data rate, while Round Robin arbitration ensured fair scheduling across Posted, Non-Posted, and Completion traffic classes. The integration of Orthogonal Header Content further extended the Transaction Layer's responsibility toward metadata management within densely packed FLIT structures, reflecting the increased architectural complexity that Gen 6 imposes at the highest protocol layer.

At the Data Link Layer, the study confirmed that the per-FLIT adaptation of the link-level reliability mechanism preserved the fundamental guarantees of the ARQ replay protocol while aligning sequence numbering and CRC coverage with the fixed FLIT boundaries introduced by Gen 6. The retry buffer, flow control watchdog, ACK piggybacking mechanism, and DLLP processing pipeline were each designed and verified to operate correctly under the tighter timing constraints imposed by the higher data rate, demonstrating that robust link-level reliability is achievable within the Gen 6 architectural framework without sacrificing protocol efficiency.

At the Physical Layer, the most architecturally complex of the three layers, the work addressed the full scope of Gen 6-specific requirements spanning LTSSM-governed link training, mandatory Reed-Solomon RS(544,514) FEC encoding and decoding, PAM4 Gray code mapping, FLIT boundary synchronization, lane deskew under spread-spectrum clocking, and the PIPE interface boundary between the digital RTL and the analog PHY. The adoption of RS(544,514) over alternative FEC schemes was justified by its deterministic decoder latency and strong burst error correction capability, both of which are critical properties for maintaining the real-time pipeline requirements of the Physical Layer at 64 GT/s. The architectural decisions documented for this layer, including Gray code PAM4 symbol assignment, dedicated FLIT sync headers, and asynchronous FIFOs with Gray-code pointers at clock domain crossings, collectively demonstrate the level of design care required to implement a reliable Gen 6 Physical Layer digital model.

The three-layer architecture was implemented and verified in full, covering TLP construction, FLIT packing, link-level reliability, FEC encoding and decoding, PAM4 modulation, and LTSSM-governed link training across the Transaction, Data Link, and Physical Layers respectively. The clearly defined inter-layer interfaces established throughout the design enhance modularity, reduce integration ambiguity, and provide a well-structured foundation for incremental extension and future verification campaigns.

In summary, this project contributes a research-driven, implementation-ready educational model of PCIe Gen 6 that supports both academic learning and practical exploration. The proposed architecture serves as a scalable platform for future research into advanced PCIe features, verification methodologies, and high-speed serial communication systems.

8.2 Future Work

Following the research and architectural development presented in this project, several directions are identified for extending and advancing the proposed model.

While the present project delivers a complete RTL implementation and functional verification of the PCIe Gen 6 digital model across all three protocol layers, together with logical synthesis and formal equivalence verification for both the Transaction Layer and the Data Link Layer, several important tasks remain as natural extensions of this work.

A primary area for future effort is the logical synthesis and formal equivalence verification of the remaining two design partitions: the Physical Layer digital logic (phy_top) and the integrated full-system top level (pcie_top). The Physical Layer comprises 56 RTL blocks encompassing the LTSSM controller, FLIT framer, RS(544,514) FEC encoder and decoder, PAM4 Gray-code mapper, lane deskew, lane polarity and reversal logic, and the PIPE interface controller. Synthesizing this partition with Synopsys Design Compiler and subsequently verifying the resulting gate-level netlist against the RTL reference using Synopsys Formality would close the remaining gap in the formal verification campaign. Given the combinational depth of the FEC encoder and the wide parallel data paths of the PAM4 mapper, it is anticipated that timing closure will require careful constraint definition and that the Formality run may need guided setup — particularly for the block-lock and symbol-lock finite state machines — to resolve any compare points that Auto Setup mode leaves unverified.

Following successful layer-level synthesis and formal verification, the full-system partition (pcie_top), which integrates the Transaction Layer, Data Link Layer, and Physical Layer digital logic under a unified top-level hierarchy, should be synthesized and formally verified as a single entity. This system-level equivalence check would confirm that the inter-layer interfaces — including the TL-to-DLL FLIT handoff bus, the credit status feedback path, the DLL-to-PHY PIPE data interface, and the FEC syndrome reporting signals — are preserved correctly through synthesis and that no unintended logic is introduced at the integration boundaries. Completing this full-system formal verification campaign would establish end-to-end formal closure across the entire PCIe Gen 6 digital design and represent the final step toward a production-ready, formally verified implementation of the protocol stack.

At the Transaction Layer, the FLIT packing algorithm can be extended to incorporate an adaptive traffic-aware scheduler that dynamically adjusts aggregation depth based on real-time traffic class distributions, minimizing latency for Non-Posted transactions while maximizing throughput for bulk Memory Write workloads. The tag space management can further be expanded to support the full 14-bit tag field defined in the Gen 6 specification, enabling a significantly higher number of outstanding non-posted requests and improving parallelism in multi-device configurations. Additionally, end-to-end CRC coverage can be extended to support ECRC stripping and forwarding across PCIe switches, completing the integrity chain across complex multi-bridge topologies. Further investigation into advanced arbitration schemes beyond Round Robin, such as weighted fair queuing, could provide more granular Quality of Service control across competing traffic classes under asymmetric load conditions.

At the Data Link Layer, the retry buffer sizing and replay protocol can be re-evaluated under correlated burst error profiles characteristic of real PAM4 channels, rather than the independent noise model used in the current theoretical analysis. An adaptive retry timeout mechanism that adjusts its acknowledgement deadline based on measured round-trip latency would reduce unnecessary replay events under high-load conditions. The flow control model can also be extended to simulate multi-Virtual Channel and multi-Traffic Class configurations, enabling study of Quality-of-Service behavior across differentiated traffic streams essential to data center and AI inference workloads. Furthermore, the DLLP processing pipeline can be enhanced to support bandwidth management and autonomous power state negotiation, allowing the link to dynamically scale its active lane count and power consumption in response to observed traffic demand.

At the Physical Layer, the most impactful future extension is the integration of a behavioral analog PHY model for the PLL/CDR, CTLE, DFE, and PAM4 DAC/ADC blocks, enabling realistic eye diagram simulation under actual channel conditions including frequency-dependent insertion loss, reflections, crosstalk, and supply noise.

The FEC decoder can further be extended to support soft-decision inputs from the analog equalizer, potentially improving effective coding gain beyond what hard-decision Reed-Solomon decoding alone can achieve. The LTSSM can also be refined to model the complete equalization state machine in full detail, including the Preset, Hint, and Coefficient exchange sequences defined for the Gen 6 TX equalization procedure. In addition, the spread-spectrum clocking model can be extended to evaluate its impact on multi-lane deskew accuracy across a wider range of SSC modulation profiles and temperature corners, ensuring that the deskew tracking mechanism remains within specification under all supported operating conditions.

At the system level, the three independently verified layers can be integrated into a complete end-to-end testbench capable of simulating realistic mixed-traffic workloads including simultaneous Memory Read, Memory Write, I/O, and Configuration transactions across multiple virtual channels. A formal verification methodology applying assertion-based property checking to the DLL replay protocol, LTSSM state transitions, and flow control deadlock prevention logic would provide stronger correctness guarantees than simulation alone. Automated constrained-random regression testing across all speed change, recovery, and error injection scenarios can be integrated into a continuous verification pipeline, ensuring that future modifications to any layer do not regress previously verified behaviors. Performance benchmarking under stress conditions, including maximum outstanding tag utilization, sustained retry storms, and simultaneous multi-lane recovery events, would further characterize the robustness boundaries of the architecture.

Finally, the modular parameterized RTL architecture established in this project provides a direct foundation for FPGA prototyping against real PCIe traffic generated by standard endpoint devices, enabling hardware-in-the-loop validation that complements the simulation-based approach. Looking further ahead, the architectural framework developed here serves as a natural starting point for studying **PCIe Generation 7**, which targets 128 GT/s per lane and will impose substantially greater demands on FEC strength, equalization complexity, and FLIT-based framing mechanisms. Extending the current model to accommodate these requirements represents the long-term evolution of this research toward the next frontier of high-speed interconnect design.

9 Code Appendix

Appendix A

RTL Module Code Reference

Each entry presents four representative lines of synthesizable RTL that capture the defining function or mathematical operation of the module.

Appendix A.1 — Physical Layer (PHY)	

FEC_ENC
fec_encoder_rs
	// RS(544,514) over GF(2^10), poly x^10+x^3+1, 30 parity symbols
	for(k=0;k<10;k=k+1) begin
	if(bb[0]) r = r ^ aa;
	aa = aa[9] ? {aa[8:0],1'b0}^10'h009 : {aa[8:0],1'b0};

FEC_SYN
fec_syndrome_calculator
	// Horner accumulation: S_j = sum(r_i * alpha_j^i) in GF(2^8)
	// Pipelined: 1 symbol/cycle × 288 cycles = 289-cycle latency
	acc[j] <= sym_reg ^ gf_mul(alpha_lut[j], acc[j]);
	syndrome_valid <= (ctr == LAST_BYTE);

ENC_130
encoder_128b130b
	// Prepend 2-bit sync header: SH=2'b01→Data, SH=2'b10→Ordered-Set
	localparam SH_DATA = 2'b01; localparam SH_ORDERED_SET = 2'b10;
	data_out <= {SH_DATA, data_in}; // data block
	data_out <= {SH_ORDERED_SET, data_in}; // ordered-set block

DEC_130
decoder_128b130b
	// Strip 2-bit SH; flag error on invalid code (Gen3-5 only)
	sh_invalid = ~(sync_hdr==2'b01 | sync_hdr==2'b10);
	sh_mismatch = (sync_hdr != data_in[129:128]);
	dec_err <= sh_mismatch | (sh_err_gated & ~(PCIE_GEN==6));

PAM4_G
pam4_gray_enc
	// Binary→Gray: 256 bits → 128 two-bit PAM4 symbols
	function [1:0] bin2gray;
	bin2gray[1] = bin[1];
	bin2gray[0] = bin[1] ^ bin[0];

DESKEW
lane_deskew
	// Per-lane SKP timestamp; skew = max(skp_time) – min(skp_time)
	if (skp_time[i] > max_tick) max_tick = skp_time[i];
	if (skp_time[i] < min_tick) min_tick = skp_time[i];
	skew_amount <= raw_skew_full[4:0]; // slots to delay fast lanes

POLL
polling_fsm
	// LTSSM Polling: TX ≥1024 TS1, then await ≥8 consecutive TS2
	parameter TS1_TX_COUNT = 1024; // spec min before TS2 phase
	parameter TS2_REQUIRED = 8; // consecutive TS2 RX to advance
	parameter POLL_ACTIVE_TO = 6_000_000; // 24 ms @ 250 MHz

Appendix A.2 — Data Link Layer (DLL)	

CRC_GEN
crc_gen
	// Gen1-5: CRC-32/IEEE-802.3 (poly 0xEDB88320) over 128 TLP bytes
	// Gen6: CRC-24/OpenPGP (poly 0x864CFB) over 12-bit seq+256 B flit
	// Parallel GF(2) expansion — each bit is a flat XOR of input taps:
	lcrc_p128[0] = ^{crc_in[0],crc_in[1],...,d[0],d[1],...,d[122]};

DLLP_CRC
dllp_crc_gen
	// CRC-16/CCITT (poly 0x1021) over 48-bit DLLP body, single-cycle
	// Parallel Galois-form unrolling — O(log2 48) gate depth:
	crc16_ccitt[0] = ^{data[0],data[4],data[8],data[11],...,data[42]};
	dllp_full <= {dllp_in, dllp_crc}; // 64-bit output = body + CRC

SCRAMB
scrambler
	// 23-bit LFSR, poly G(x)=x^23+x^21+x^16+x^8+x^5+x^2+1, seed=7FFFFF
	// Parallel 256-bit/cycle expansion via recurrence:
	// s[i] = s[i-23]^s[i-21]^s[i-16]^s[i-8]^s[i-5]^s[i-2]^s[i-1]
	data_out <= data_in ^ {scramble_byte[31],...,scramble_byte[0]};

SEQ_GEN
seq_num_gen
	// 12-bit monotonic counter [0..4095] stamped on every TX flit
	localparam SEQ_MAX = 12'd4095;
	wire [11:0] incremented = (next_seq==SEQ_MAX) ? 12'd0 : next_seq+1;
	if (retry_req) seq_num <= nak_seq + 1; // rewind on NAK

FLIT_SEQ
flit_seq
	// Tracks oldest un-ACKed seq; stalls TX when window fills
	wire [11:0] unacked_count = flit_tx_seq - last_acked;
	seq_window_full <= (unacked_count >= SEQ_WINDOW); //
	SEQ_WINDOW=2048
	if (ack_seq != last_acked) oldest_unacked_seq <= ack_seq + 1;

REPLAY
replay_fsm
	// NAK/timer → RETRY; 4th consecutive retry → LINK_DOWN
	if (nak_valid || replay_timer_exp) begin
	if (replay_num == 2'd3) begin dll_link_down<=1; state<=LINK_DOWN; end
	else begin retry_req<=1; retry_seq_start<=nak_seq; state<=RETRY; end

RETRY_B
retry_buf
	// Circular replay buffer; purge on ACK, rewind read ptr on NAK
	wire do_write = tlp_write_en && !buf_full;
	wire [PTR_W-1:0] occ_w = head_ptr - tail_ptr;
	if (ack_rcvd) tail_ptr <= tail_ptr + 1; // retire ACK'd slot

DLLP_ARB
dllp_arb
	// Static-priority mux: ACK/NAK > UpdateFC > PM > BW_Notif > NOP
	if (ack_dllp_valid) begin sel_data=ack_dllp; sel_type=is_nak?4'h1:4'h0; end
	else if (fc_dllp_valid) begin sel_data=fc_dllp; sel_type=4'h2; end
	else if (pm_dllp_valid) begin sel_data=pm_dllp; sel_type=4'h3; end

FC_INIT
fc_init_fsm
	// DL_Init credit handshake: InitFC1 → InitFC2 → InitFC3 → DL_Active
	localparam FC_INIT1=3'd1; FC_INIT2=3'd2; FC_INIT3=3'd3; FC_DONE=3'd4;
	initfc_tx_send <= 1'b1; // pulse per DLLP type
	if (initfc2_rx_seen) state <= FC_DONE; fc_init_done <= 1;

Appendix A.3 — Transaction Layer (TL)	

FC_INIT
tl_fc_init_fsm
	// Advertise P/NP/CPL header+data credits via InitFC1/InitFC2 DLLPs
	// States:
	IDLE→SEND_IFC1_P/NP/CPL→WAIT_IFC1→SEND_IFC2→WAIT_IFC2→DONE
	fc_init_done <= (state == S_DONE);
	adv_ph <= K_PH; adv_pd <= K_PD; // latch advertised credits at DONE

ECRC
ecrc
	// CRC-32 (poly 0x04C11DB7) over TLP header+data; EP bit forced 0
	hdr_masked[112] = 1'b0; // EP bit zeroed per PCIe §2.7.1
	crc = crc32_over_header(hdr_masked, 32'hFFFF_FFFF);
	compute_ecrc = reflect32(crc) ^ 32'hFFFF_FFFF; // final XOR

TAG_MGR
tag_manager
	// 10-bit tag pool [0..1023]; bitmap alloc/free + outstanding counter
	reg [TAG_POOL_SIZE-1:0] free_bitmap;
	wire [9:0] next_free = find_free_tag(free_bitmap);
	outstanding_int <= outstanding_int + alloc - return - timeout;

ARB_TX
arb_tx
	// Posted vs Non-Posted round-robin, gated by credits & ordering_ok
	if (!ordering_ok) arb_tlp_valid <= 0; // stall all TX
	else if (can_send_p && !can_send_np) arb_tlp <= req_p;
	else if (can_send_p && can_send_np) // round-robin: flip last_granted

VC_ARB
vc_arbiter
	// Round-Robin (RR) or Weighted-RR across VC0–VC3
	idx = (rr_ptr + j) % 4; // rotating scan
	if (!rr_found && req_bus[idx]) begin rr_winner=idx; rr_found=1; end
	if (vc_arb_scheme==WRR && credits[kidx]>best_cred) wrr_winner=kidx;

TLP_ASM
tlp_assembler
	// Packs 1024-bit TLP: [1023:896]=prefix [895:768]=hdr [767:256]=data
	tlp_out[1023:896] <= prefix_valid ? prefix_in : 128'd0;
	tlp_out[895:768] <= hdr_dws;
	tlp_out[767:256] <= tlp_has_data ? raw_data : 512'd0;

Appendix B

MATLAB THEORITICAL Code Reference

Simulates how data travels across a PCIe Gen6 link at 64 GT/s, covering four key physical layer operations (**Theoretical**).

1. PAM4 Symbol Mapping:

The idea: Instead of sending 1 bit at a time (0 or 1), PAM4 sends 2 bits at a time using 4 voltage levels. Gray coding is used so that if you misread a level, you only get 1 bit wrong instead of 2.

```
mbits_in  = randi([0 1], 1000, 2);   % Generate 1000 random 2-bit pairs
gray_map = [-3, -1, +3, +1];        % The 4 voltage levels (Gray coded)
symbols  = gray_map(bi2de(bits_in, 'left-msb') + 1);  % Map bits → voltage
```

Bit Pair	Voltage Level
00	**−3**
01	**−1**
11	**+3**
10	**+1**

2. Bit Error Rate and FEC Correction:

The idea: Some bits will arrive corrupted due to noise. We first estimate how many errors occur (Pre-FEC BER), then check whether the Reed-Solomon error corrector can fix them (Post-FEC BER).

```
SNR_dB = 32; % Signal quality (higher = cleaner signal)

SNR_lin = 10^(SNR_dB / 10); % Convert dB to a plain number

% Step 1: Estimate raw bit error rate from signal noise

p_bit = max((3/4) * erfc(sqrt(SNR_lin / 10)), 1e-20);

% Step 2: Convert bit errors to symbol errors (each symbol = 10 bits)
```

```
p_sym = min(1 - (1 - p_bit)^10, 1 - 1e-20);
% Step 3: Check if RS FEC can handle the errors
% RS(544,514) can fix up to 15 bad symbols per block
n_sym = 544; % Block size
t_fec = 15; % Max errors the FEC can fix
log_pfail = -inf;
for kk = (t_fec + 1) : min(t_fec + 30, n_sym)
log_term = gammaln(n_sym+1) - gammaln(kk+1) - gammaln(n_sym-kk+1) ... + kk*log(p_sym) + (n_sym-kk)*log(1-p_sym);
log_pfail = log(exp(log_pfail - log_term) + 1) + log_term;
 end
% Final answer: exp(log_pfail) = probability that FEC fails to correct errors
```

3. Jitter and Eye Diagram (Bathtub Curve)

The idea: Jitter is the unwanted timing wobble of a signal. Too much jitter and the receiver misreads bits. The bathtub curve shows how the error rate changes depending on exactly when you sample the signal — you want to sample right in the middle of the "eye opening."

```
RJ_rms_ps = 0.50; % Random jitter — unpredictable, Gaussian (ps, RMS)
DJ_pp_ps = 1.20; % Deterministic jitter — repeatable, bounded (ps, peak-to-peak)
% Total jitter at a target BER of 1×10⁻¹²
% The value 14.069 is the Q-factor corresponding to BER = 1e-12
TJ_ps = DJ_pp_ps + 14.069 * RJ_rms_ps; % = 1.20 + 7.03 = 8.23 ps
 % Bathtub curve: shows BER across the eye opening
 t_sample = linspace(-20, 20, 5000);
```

```
bathtub = 0.5 * erfc((abs(t_sample) - DJ_pp_ps/2) ./ (sqrt(2)*RJ_rms_ps + eps));
```

4. Channel Loss and CTLE Equalizer

The idea: The copper PCB trace weakens the signal — especially at high frequencies (skin effect). The CTLE equalizer is a filter that boosts those high frequencies back up to compensate.

```
f = linspace(1e6, 20e9, 1000); % Frequencies from 1 MHz to 20 GHz
% Channel: signal gets weaker as frequency increases
alpha_skin = 0.85e-10; % How much skin effect hurts the signal
alpha_dc = 0.10e-9; % How much DC resistance hurts the signal
H_channel = exp(-alpha_skin*sqrt(f) - alpha_dc*f);
% CTLE equalizer: boosts signal in the 4–16 GHz range to fight channel loss
f_zero = 4.0e9; % Frequency where boost starts
f_pole = 16.0e9; % Frequency where boost stops
tau_z = 1 / (2*pi*f_zero);
tau_p = 1 / (2*pi*f_pole); H_ctle = abs((1 + 1j*2*pi*f*tau_z) ./ (1 + 1j*2*pi*f*tau_p));
% Final result: channel after equalization
H_equalized = H_channel .* H_ctle;
```

Analytical Mathematical Models

• Pre-FEC Bit Error Rate (AWGN Channel):

$$BER\_pre = (3/4) * erfc(sqrt(SNR\_linear / 10))$$

• Total Jitter (Dual-Dirac Model at BER = 10^-12):

$$TJ = DJ + 14.069 * RJ$$

• Jitter Bathtub Curve Formulation:

$$BER(t) = 0.5 * erfc((|t| - DJ / 2) / (sqrt(2) * RJ))$$

• Channel Insertion Loss Frequency Response:

$$H\_ch(f) = exp(-alpha\_skin * sqrt(f) - alpha\_dc * f)$$

• CTLE Frequency Equalization Transfer Function:

$$H\_ctle(f) = G0 * |(1 + j * 2 * pi * f * tau\_z) / (1 + j * 2 * pi * f * tau\_p)|$$

10 References

[1] Verma, A., & Dahiya, P. K. (2017). PCIe BUS: A State-of-the-Art Review. IOSR Journal of VLSI and Signal Processing (IOSR-JVSP), 7(4), 24-28.

[2] Shilov, A. (2025). PCIe 7.0 spec finalized with up to 512 GB/s speeds — PCI-SIG targets 1 TB/s for 8.0 as 'exploration' phase begins. Tom's Hardware.

[3] Owen, A., & Davids, L. (2025). Development of a Parameterized Verification Environment for PCIe-DDR Interfaces Supporting Multiple Generations (Gen3 to Gen6) and DDR4 to DDR5. ResearchGate.

[4] Darvishi, M. (2021). Fault-Resilient PCIe Bus with Real-time Error Detection and Correction. IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems.

[5] Nag, S. N. (2023). Technical Analysis of PCIe to PCIe 6: A Next-Generation Interface Evolution. World Journal of Engineering and Technology, 11(3), 504-525.

[6] PCI-SIG. (2002). PCI Express Base Specification, Revision 1.0. PCI-SIG, April 2002.

[7] Tj, M. (2017). Difference Between PCI Express Gen 1, Gen 2, Gen 3, Gen 4, Gen 5, & Gen 6. DeskDecode.

[8] PCI-SIG. (2010). PCI Express Base Specification, Revision 3.0. PCI-SIG.

[9] PCI-SIG. (2014). PCI Express Base Specification Revision 4.0 Version 0.3. PCI-SIG. February 19, 2014.

[10] PCI-SIG. (2017). PCI Express Base Specification Revision 4.0, Version 1.0. PCI-SIG. Released October 5, 2017.

[11] PCI-SIG. (2019). PCI Express® Base Specification, Revision 5.0 Version 1.0. PCI-SIG, May 22, 2019.

[12] Iyer, K., & Connatser, M. (2024). PCI Express 5 (PCIe 5.0): Here's Everything You Need to Know About the Current-Gen Standard. XDA Developers.

[13] Sharma, D. D. (2020). PCIe® 6.0 Specification: The Interconnect for I/O Needs of the Future. PCI-SIG Educational Webinar Series.

[14] Design News. (2023). PCIe® 6.0: Testing for a New Generation. Design News Free White Paper.

[15] Liu, Q., Wang, H., Lyu, F., Zhang, G., & Lyu, D. (2023). A Low Latency, Low Jitter Retimer Circuit for PCIe 6.0 (Preprint).

[16] Lnu, D. K. (2025). PCIe Verification at High Speeds: Challenges, Solutions, and Future Trends. International Journal of Computer Engineering and Technology (IJCET), 16(1), 3987-4016.

[17] Peh, Li-Shiuan & Dally, William J. (2000). Flit-Reservation Flow Control. In Proceedings of the 6th International Symposium on High-Performance Computer Architecture, Toulouse, France, January 10–12, pp. 73–84.

[18] Mickens, A. (2023, February 17). The Impact of PAM4 on PCIe 6.0 Electrical Testing. Teledyne LeCroy Technical Brief.

[19] Cheng, K., et al. (2018). Design and Implementation of High-throughput PCIe with DMA Architecture between FPGA and PowerPC. Journal of IEEE Transactions on Nuclear Science.

[20] Unraveling PCIe 6.0 Training Sequences Update and Verification Challenges by xinmu

[21] EDN ASIA by Ransom Stephens

[22] Graphic Courtesy of Wild River Technology and Keysight Technologies

[23] Tang, L., Gai, W., & Shi, L. (2016). PAM4 receiver with adaptive threshold voltage and adaptive decision feedback equalizer. 2016 IEEE International Symposium on Circuits and Systems (ISCAS), 2246-2249.

[24] Keysight Technologies. (2024). Essential Insights for Design PCIe® 6.0 Interconnects. White Paper.

[25] Fidus Systems. (2023). Exploring PCIe Gen 6 Advancements & Benefits.

[26] Cadence Design Systems. (2022). Demystifying Forward Error Correction (FEC) in PCIe 6.0. Cadence Blogs.

[27] Q. Liu, M. Zapater, and D. Atienza. (2025). Gem5-AcceSys: Enabling System-Level Exploration of Standard Interconnects for Novel Accelerators. arXiv Preprint.

[28] M. Kwon, D. Gouk, J. Jang, et al. (2025). From Block to Byte: Transforming PCIe SSDs with CXL Memory Protocol and Instruction Annotation. arXiv Preprint.

[29] ALDEC - The Design Verification Company site

[30] Goldhammer, A., & Ayer, J. (2008). Understanding Performance of PCI Express Systems. Xilinx White Paper (WP350).

[31] Casey, M. (2021). What Is Peripheral Component Interconnect (PCI). Lifewire.

[32] Kwon, H., Kwon, W., Oh, M.-H., & Kim, H. (2019). Signal Integrity Analysis of System Interconnection Module of High-Density Server Supporting Serial RapidIO. ETRI Journal, 41(4), 670-683.

[33] FS.com. (2025). Understanding PCIe Versions, Lanes, and Slot Types Explained.

[34] BITSILICA. (2025). PCI Express Evolution: From Gen1 to Gen7 – Progress & Applications. Bitsilica Blog.